Universidade Estadual de Campinas

Enemy Leo used Appeal! It's super effective! Pikachu fainted!

CSES Problem Set

Pedro Assunção, Pedro Mesquita, Yvens Porto

2026-08-18

Contents

1 Introductory Problems **2**
1.1 Apple Division . . . 2
1.2 Bit Strings . . . 2
1.3 Chessboardand Queens . . . 2
1.4 Coin Piles . . . 3
1.5 Creating Strings . . . 3
1.6 Digit Queries . . . 3
1.7 Gray Code . . . 4
1.8 Grid Coloring I . . . 4
1.9 Grid Path Description . . . 4
1.10 Increasing Array . . . 5
1.11 Knight Moves Grid . . . 5
1.12 Mex Grid Construction . . . 5
1.13 Missing Number . . . 6
1.14 Number Spiral . . . 6
1.15 Palindrome Reorder . . . 6
1.16 Permutations . . . 7
1.17 Raab Game I . . . 7
1.18 Repetitions . . . 8
1.19 String Reorder . . . 8
1.20 Towerof Hanoi . . . 9
1.21 Trailing Zeros . . . 9
1.22 Two Knights . . . 9
1.23 Two Sets . . . 9
1.24 Weird Algorithm . . . 10

2 Dynamic Programming **11**
2.1 Array Description . . . 11
2.2 Book Shop . . . 11
2.3 Coin Combinations I . . . 11
2.4 Coin Combinations II . . . 12
2.5 Counting Numbers . . . 12
2.6 Counting Tiling . . . 13
2.7 Counting Towers . . . 13
2.8 Dice Combinations . . . 14
2.9 Edit Distance . . . 14
2.10 Elevator Rides . . . 14
2.11 Grid Paths I . . . 15
2.12 Increasing Subsequence . . . 15
2.13 Increasing Subsequence II . . . 16
2.14 Longest Common Subsequence . . . 16
2.15 Minimal Grid Path . . . 16
2.16 Minimizing Coins . . . 17
2.17 Money Sums . . . 17
2.18 Mountain Range . . . 17
2.19 Projects . . . 18
2.20 Rectangle Cutting . . . 18
2.21 Removal Game . . . 19
2.22 Removing Digits . . . 19
2.23 Two Sets II . . . 19

3 Graph Algorithms **21**
3.1 Building Roads . . . 21
3.2 Building Teams . . . 21
3.3 Coin Collector . . . 21
3.4 Counting Rooms . . . 22
3.5 Course Schedule . . . 23
3.6 Cycle Finding . . . 23
3.7 De Bruijn Sequence . . . 24
3.8 Distinct Routes . . . 24
3.9 Download Speed . . . 25
3.10 Flight Discount . . . 25
3.11 Flight Routes . . . 26
3.12 Flight Routes Check . . . 26
3.13 Game Routes . . . 27
3.14 Giant Pizza . . . 27
3.15 Hamiltonian Flights . . . 28
3.16 High Score . . . 28
3.17 Investigation . . . 29
3.18 Knights Tour . . . 29
3.19 Labyrinth . . . 29
3.20 Longest Flight Route . . . 30
3.21 Mail Delivery . . . 31
3.22 Message Route . . . 31
3.23 Monsters . . . 32
3.24 Planets Cycles . . . 32
3.25 Planets Queries I . . . 33
3.26 Planets Queries II . . . 33
3.27 Planetsand Kingdoms . . . 34
3.28 Police Chase . . . 35
3.29 Road Construction . . . 35
3.30 Road Reparation . . . 36
3.31 Round Trip . . . 36
3.32 Round Trip II . . . 37
3.33 School Dance . . . 37
3.34 Shortest Routes I . . . 38
3.35 Shortest Routes II . . . 38
3.36 Teleporters Path . . . 39

4 Range Queries **40**
4.1 Distinct Values Queries . . . 40
4.2 Distinct Values Queries II . . . 40
4.3 Dynamic Range Minimum Queries . . . 41
4.4 Dynamic Range Sum Queries . . . 41
4.5 Forest Queries . . . 42
4.6 Hotel Queries . . . 42
4.7 List Removals . . . 42
4.8 Pizzeria Queries . . . 43
4.9 Polynomial Queries . . . 44
4.10 Prefix Sum Queries . . . 44
4.11 Range Update Queries . . . 45
4.12 Range Xor Queries . . . 46
4.13 Salary Queries . . . 46
4.14 Static Range Minimum Queries . . . 46
4.15 Static Range Sum Queries . . . 47
4.16 Subarray Sum Queries . . . 47
4.17 Subarray Sum Queries II . . . 48
4.18 Visible Buildings Queries . . . 48

5 Tree Algorithms **50**
5.1 Company Queries I . . . 50
5.2 Company Queries II . . . 50
5.3 Counting Paths . . . 51
5.4 Distance Queries . . . 51
5.5 Distinct Colors . . . 52
5.6 Findinga Centroid . . . 53
5.7 Fixed-Length Paths I . . . 53
5.8 Path Queries . . . 54
5.9 Path Queries II . . . 54
5.10 Subordinates . . . 55
5.11 Subtree Queries . . . 55
5.12 Tree Diameter . . . 56
5.13 Tree Distances I . . . 56
5.14 Tree Distances II . . . 57
5.15 Tree Matching . . . 57

6 Mathematics **59**
6.1 Candy Lottery . . . 59

7 String Algorithms **60**
7.1 Counting Patterns . . . 60
7.2 Distinct Substrings . . . 60
7.3 Finding Borders . . . 60
7.4 Repeating Substring . . . 61
7.5 String Functions . . . 61
7.6 String Matching . . . 62
7.7 String Transform . . . 62
7.8 Substring Order I . . . 62
7.9 Substring Order II . . . 63

8 Advanced Techniques **64**
8.1 Distinct Routes II . . . 64
8.2 Eulerian Subgraphs . . . 64
8.3 Housesand Schools . . . 65
8.4 Longest Palindrome . . . 65
8.5 Monster Game . . . 66
8.6 Necessary Cities . . . 66
8.7 Necessary Roads . . . 67
8.8 Parcel Delivery . . . 68
8.9 Signal Processing . . . 68
8.10 Subarray Squares . . . 69
8.11 Substring Reversals . . . 69

9 Additional Problems I **71**
9.1 Beautiful Permutation II . . . 71
9.2 Distinct Values Sum . . . 71
9.3 Maximum Average Subarrays . . . 72
9.4 Maximum Building I . . . 72
9.5 Nearest Campsites I . . . 73
9.6 Nearest Campsites II . . . 74
9.7 Sorting Methods . . . 74
9.8 Swap Game . . . 75
9.9 Water Containers Moves . . . 75
9.10 Writing Numbers . . . 76

10 Additional Problems II **78**
10.1 Book Shop II . . . 78
10.2 Food Division . . . 78
10.3 GCDSubsets . . . 78
10.4 Grid Coloring II . . . 78
10.5 Increasing Array II . . . 79
10.6 KSubset Sums I . . . 80
10.7 KSubset Sums II . . . 80
10.8 Knight Moves Queries . . . 80
10.9 Maximum Building II . . . 81
10.10 Mex Grid Queries . . . 82
10.11 Minimum Cost Pairs . . . 82
10.12 Programmersand Artists . . . 82
10.13 School Excursion . . . 83

11 Advanced Graph Problems **85**
11.1 Acyclic Graph Edges . . . 85
11.2 Course Schedule II . . . 85
11.3 Even Outdegree Edges . . . 85
11.4 Graph Coloring . . . 86
11.5 Split Into Two Paths . . . 86
11.6 Strongly Connected Edges . . . 87
11.7 Tree Coin Collecting I . . . 88
11.8 Tree Coin Collecting II . . . 89
11.9 Tree Traversals . . . 89

12 Bitwise Operations **91**
12.1 All Subarray Xors . . . 91
12.2 And Subset Count . . . 91
12.3 Counting Bits . . . 92
12.4 KSubset Xors . . . 92
12.5 Maximum Xor Subarray . . . 93
12.6 Maximum Xor Subset . . . 94
12.7 Numberof Subset Xors . . . 95
12.8 SOSBit Problem . . . 95
12.9 Xor Pyramid Diagonal . . . 95
12.10 Xor Pyramid Peak . . . 96
12.11 Xor Pyramid Row . . . 96

13 Construction Problems **97**
13.1 Chess Tournament . . . 97
13.2 Inverse Inversions . . . 97

14 Counting Problems **98**
14.1 All Letter Subgrid Count I . . . 98
14.2 All Letter Subgrid Count II . . . 98
14.3 Counting Sequences . . . 99
14.4 Empty String . . . 99
14.5 Grid Paths II . . . 99
14.6 Permutation Inversions . . . 100

15 Sliding Window Problems **101**
15.1 Sliding Window Cost . . . 101
15.2 Sliding Window Median . . . 101

16 Sorting And Searching **103**
16.1 Apartments . . . 103
16.2 Array Division . . . 103
16.3 Collecting Numbers . . . 103
16.4 Collecting Numbers II . . . 104
16.5 Concert Tickets . . . 104
16.6 Distinct Numbers . . . 104
16.7 Distinct Values Subarrays . . . 105
16.8 Distinct Values Subarrays II . . . 105
16.9 Distinct Values Subsequences . . . 105
16.10 Factory Machines . . . 106
16.11 Ferris Wheel . . . 106
16.12 Josephus Problem I . . . 106
16.13 Josephus Problem II . . . 107
16.14 Maximum Subarray Sum . . . 107
16.15 Maximum Subarray Sum II . . . 107
16.16 Missing Coin Sum . . . 107
16.17 Movie Festival . . . 108
16.18 Movie Festival II . . . 108
16.19 Nearest Smaller Values . . . 108
16.20 Nested Ranges Check . . . 109
16.21 Nested Ranges Count . . . 109
16.22 Playlist . . . 110
16.23 Reading Books . . . 110
16.24 Restaurant Customers . . . 110
16.25 Room Alocation . . . 111
16.26 Stick Lengths . . . 111
16.27 Subarray Divisibility . . . 111
16.28 Subarray Sums I . . . 112
16.29 Subarray Sums II . . . 112
16.30 Sumof Four Values . . . 112
16.31 Sumof Three Values . . . 113
16.32 Sumof Two Values . . . 113
16.33 Tasksand Deadlines . . . 114
16.34 Towers . . . 114
16.35 Traffic Lights . . . 114

1 Introductory Problems

1.1 Apple Division

There are n apples with known weights. Your task is to divide the apples into two groups so that the difference between the weights of the groups is minimal.

Input
The first input line has an integer n: the number of apples. The next line has n integers $p_1, p_2, \ldots, p_n$: the weight of each apple.

Output
Print one integer: the minimum difference between the weights of the groups.

Constraints
• $1 \leq n \leq 20$ • $1 \leq p_i \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

void solve(){
    ll n; cin >> n;
    ll p[n], pot = 1, s = 0, dif = 20LL * inf;

    for(int i=0; i<n; i++){
        cin >> p[i];
        s += p[i];
        pot *= 2;
    }

    for(int i = 0; i < pot; i++){
        ll k = i;
        ll soma = 0, num = 0;
        while(k > 0){
            if(k%2 == 1){
                soma += p[num];
            }
            //cout << "num: " << num << '\n';
            k /= 2;
            num ++;
        }
        dif = min(dif, abs(2 * soma - s) );
    }

    cout << dif << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

1.2 Bit Strings

Your task is to calculate the number of bit strings of length n. For example, if $n = 3$, the correct answer is 8, because the possible bit strings are 000, 001, 010, 011, 100, 101, 110, and 111.

Input
The only input line has an integer n.

Output
Print the result modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 10^6$

Código-fonte:

```cpp
#include <bits/stdc++.h>

using namespace std;

#define ll long long int

const ll mod = 1000000007;

ll pot(ll b, ll n){
    if(n == 0){
        return 1;
    }
    ll  k = pot(b, n/2);

    k = (k * k) % mod;

    if(n % 2 == 0){
        return k % mod;
    }

    else {
        return (b * k) % mod;
    }
}

int main(){
    ll a;
    cin >> a;
    cout << pot(2, a) << endl;
}
```

1.3 Chessboardand Queens

Your task is to place eight queens on a chessboard so that no two queens are attacking each other. As an additional challenge, each square is either free or reserved, and you can only place queens on the free squares. However, the reserved squares do not prevent queens from attacking each other. How many possible ways are there to place the queens?

Input
The input has eight lines, and each of them has eight characters. Each square is either free (.) or reserved (*).

Output
Print one integer: the number of ways you can place the queens.

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;
char tab[10][10];

void damas(int n) {
    int k=0, m = 0, v[n];

    for(int i=0; i<n; i++){
        v[i] = -1;
    }

    while(1){
        v[m]++;

        if(v[m] == n){
            if(m == 0) break;
            v[m] = -1;
            m--;
            continue;
        }
        if(tab[v[m]][m] == '*') continue;

        char a = 0;
        for(int i=0; i<m; i++){
            a = v[m] == v[i];
            a = a || (v[m] == m - i + v[i]);
            a = a || (v[m] == i - m + v[i]);
            if(a){
                break;
            }
        }
        if(a) continue;
        m++;

        if(m == n){
            k++;
            m--;
        }
    }

    printf("%d\n", k);
}

void solve(){
    int n = 8;

    for(int i = 0; i < n; i++){
        for(int j = 0; j < n; j++){
            cin >> tab[i][j];
        }
    }

    damas(n);
}
```

```
int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

1.4 Coin Piles

You have two coin piles containing a and b coins. On each move, you can either remove one coin from the left pile and two coins from the right pile, or two coins from the left pile and one coin from the right pile. Your task is to efficiently find out if you can empty both the piles.

Input
The first input line has an integer t: the number of tests. After this, there are t lines, each of which has two integers a and b: the numbers of coins in the piles.

Output
For each test, print "YES" if you can empty the piles and "NO" otherwise.

Constraints
• $1 \leq t \leq 10^5$ • $0 \leq a, b \leq 10^9$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

void printv(vector<int> v){
    for(int i=0; i < v.size(); i++){
        cout << v[i] << ' ';
    }
    cout << '\n';
}

void solve(){
    int a, b;
    cin >> a >> b;
    if((2*b - a) % 3 == 0 && 2*b - a >= 0 && 2*a - b >= 0){
        cout << "YES\n";
        return;
    }
    cout << "NO\n";
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

1.5 Creating Strings

Given a string, your task is to generate all different strings that can be created using its characters.

Input
The only input line has a string of length n. Each character is between a–z.

Output
First print an integer k: the number of strings. Then print k lines: the strings in alphabetical order.

Constraints
• $1 \leq n \leq 8$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

vector<string> palavras;

void perm(string s, string r){
    if(s == ""){
        palavras.pb(r);
        return;
    }
    int letras[26] = {0};

    for(int i = 0; i < s.size(); i++){
        if(letras[s[i] - 'a'] == 0){
            string copia = s;
            copia.erase(i, 1);
            string dig = "";
            dig += s[i];
            perm(copia, dig + r);
        }
        letras[s[i] - 'a'] ++;
    }
}

void solve(){
    string s; cin >> s;
    perm(s, "");
    cout << palavras.size() << '\n';
    sort(all(palavras));
    for(string s : palavras){
        cout << s << '\n';
    }
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

1.6 Digit Queries

Consider an infinite string that consists of all positive integers in increasing order: 12345678910111213141516171819202122232425... Your task is to process q queries of the form: what is the digit at position k in the string?

Input
The first input line has an integer q: the number of queries. After this, there are q lines that describe the queries. Each line has an integer k: a 1-indexed position in the string.

Output
For each query, print the corresponding digit.

Constraints
• $1 \leq q \leq 1000$ • $1 \leq k \leq 10^{18}$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;
char tab[10][10];

void solve(){
    ll k, soma = 0;
    cin >> k;
    ll cont = 1, pot = 1, ant = 0;
    while(soma < k){
        ant = soma;
        soma += 9 * pot * cont;
        pot *= 10;
        cont ++;
    }
    ll copia = k;
    ll dig = cont - 1;
    pot /= 10;
    copia -= ant + 1;

    //cout << dig << '\n';
    //cout << pot << '\n';
    //cout << copia << '\n';

    //cout << copia + pot << '\n';

    string s = to_string(copia/dig + pot);

    cout << s[copia % dig] << '\n';

}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

1.7 Gray Code

A Gray code is a list of all 2^n bit strings of length n, where any two successive strings differ in exactly one bit (i.e., their Hamming distance is one). Your task is to create a Gray code for a given length n.

Input
The only input line has an integer n.

Output
Print 2^n lines that describe the Gray code. You can print any valid solution.

Constraints
- $1 \le n \le 16$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;
int v[26];

void printv(vector<int> v){
    for(int i=0; i < v.size(); i++){
        cout << v[i] << ' ';
    }
    cout << '\n';
}

void solve(){
    int n; cin >> n;
    vector<int> v; v.pb(0); v.pb(1);
    int pot = 2;

    for(int i = 0; i < n - 1; i++){
        int tam = v.size();
        for(int j = tam - 1; j >= 0; j--){
            v.pb(v[j] + pot);
        }
        pot *= 2;
    }

    for(int i = 0; i < v.size(); i++){
        string s;
        for(int j = 0; j < n; j++){
            if(v[i] % 2 == 0) s.pb('0');
            if(v[i] % 2 == 1) s.pb('1');
            v[i] /= 2;
        }
        cout << s << '\n';
    }
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

1.8 Grid Coloring I

You are given an $n \times m$ grid where each cell contains one character A, B, C or D. For each cell, you must change the character to A, B, C or D. The new character must be different from the old one. Your task is to change the characters in every cell such that no two adjacent cells have the same character.

Input
The first line has two integers n and m: the number of rows and columns. The next n lines each have m characters: the description of the grid.

Output
Print n lines each with m characters: the description of the final grid. You may print any valid solution. If no solution exists, just print IMPOSSIBLE.

Constraints
- $1 \le n, m \le 500$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

void solve(){
    int n, m; cin >> n >> m;
    char M[n + 2][m + 2];

    for(int i = 0; i <= n + 1; i++)
        for(int j = 0; j <= m + 1; j++)
            M[i][j] = 'A';

    for(int i = 1; i <= n; i++)
        for(int j = 1; j <= m; j++)
            cin >> M[i][j];

    for(int sum = 2; sum <= m + n; sum++)
    {
        for(int i = max(1, sum - m); i <= min(n, sum - 1); i
            ++)
        {
            int j = sum - i;
            for(int k = 0; k < 4; k++)
            {
                if(M[i][j] - 'A' == k || M[i - 1][j] - 'A' ==
                    k || M[i][j - 1] - 'A' == k) continue;
                M[i][j] = 'A' + k;
                break;
            }
        }
    }

    for(int i = 1; i <= n; i++)
    {
        for(int j = 1; j <= m; j++)
        {
            cout << M[i][j];
        }
        cout << '\n';
    }
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

1.9 Grid Path Description

There are 88418 paths in a 7×7 grid from the upper-left square to the lower-left square. Each path corresponds to a 48-character description consisting of characters D (down), U (up), L (left) and R (right). For example, the path corresponds to the description DRURRRRDDDLUULDDDLDRRURDDLLLLLURULURRUULDLLDDDD. You are given a description of a path which may also contain characters ? (any direction). Your task is to calculate the number of paths that match the description.

Input
The only input line has a 48-character string of characters ?, D, U, L and R.

Output
Print one integer: the total number of paths.

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int n = 7, n2 = 49, inf = 2e9, M = 1e9 + 7;
vector<string> seqs;
string s;

int tab[n+2][n+2], total = 0;

void completar(int x, int y, int soma, char ant){
    if(soma != 1 && s[soma-2] != '?' && s[soma-2] != ant)
        return;

    int fimh = (tab[x-1][y] == 0) + (tab[x+1][y] == 0);
    int fimv = (tab[x][y-1] == 0) + (tab[x][y+1] == 0);

    if(fimh == 0 && fimv == 2 || fimv == 0 && fimh == 2)
        return;

    if(x == 1 && y == n && soma != n2) return;
    if(soma == n2){
```

```
        total ++;
        //seqs.pb(s);
        return;
    }


    if(fimv + fimh == 0) return;

    if(tab[x-1][y] == 0){
        tab[x-1][y] = 1;
        completar(x-1, y, soma+1, 'L');
        tab[x-1][y] = 0;
    }
    if(tab[x+1][y] == 0){
        tab[x+1][y] = 1;
        completar(x+1, y, soma+1, 'R');
        tab[x+1][y] = 0;
    }
    if(tab[x][y-1] == 0){
        tab[x][y-1] = 1;
        completar(x, y-1, soma+1, 'U');
        tab[x][y-1] = 0;
    }
    if(tab[x][y+1] == 0){
        tab[x][y+1] = 1;
        completar(x, y+1, soma+1, 'D');
        tab[x][y+1] = 0;
    }
}

void solve(){
    tab[1][1] = 1;
    for(int i = 0; i <= n + 1; i ++){
        tab[i][0] = 1;
        tab[i][n+1] = 1;
        tab[0][i] = 1;
        tab[n+1][i] = 1;
    }

    cin >> s;

    completar(1, 1, 1, 'S');


    //sort(all(seqs));
    cout << total << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

1.10 Increasing Array

You are given an array of n integers. You want to modify the array so that it is increasing, i.e., every element is at least as large as the previous element. On each move, you may increase the value of any element by one. What is the minimum number of moves required?

Input
The first input line contains an integer n: the size of the array. Then, the second line contains n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print the minimum number of moves.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    // int t; cin >> t; while (t--)
    ll n; cin >> n;
    ll a = 0, total = 0;

    for(int i = 0; i < n; i++){
        ll x; cin >> x;
        total += max(0LL, a - x);
        a = max(a, x);
    }

    cout << total << '\n';

    return 0;
}
```

1.11 Knight Moves Grid

There is a knight on an $n \times n$ chessboard. For each square, print the minimum number of moves the knight needs to do to reach the top-left corner.

Input
The only line has an integer n.

Output
Print the number of moves for each square.

Constraints
• $4 \leq n \leq 1000$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;
const int MAX = 1000;


int n;
int M[MAX][MAX];
vector<pair<int, int>> horse = {{+1, +2}, {-1, +2}, {+1, -2}, {-1, -2},
                                {+2, +1}, {-2, +1}, {+2, -1}, {-2, -1}};

bool check(int x)
{
    return (x >= 0 && x < n);
}

void bfs(int a, int b)
{
    M[a][b] = 0;
    queue<pair<int, int>> q;
    q.push({a, b});
    while(!q.empty())
    {
        pair<int, int> p = q.front(); q.pop();
        for(auto x : horse)
        {
            int x1 = p.first + x.first, y1 = p.second + x.second;
            if(check(x1) && check(y1) && M[x1][y1] == -1)
            {
                q.push({x1, y1});
                M[x1][y1] = M[p.first][p.second] + 1;
            }
        }
    }
}

void solve(){
    cin >> n;

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            M[i][j] = -1;
        }
    }

    bfs(0,0);

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            cout << M[i][j] << ' ';
        }
        cout << '\n';
    }
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

1.12 Mex Grid Construction

Your task is to construct an $n \times n$ grid where each square has the smallest nonnegative integer that does not appear to the left on the same row or above on the same column.

Input

The only line has an integer n.

Output
Print the grid according to the example.

Constraints
• $1 \leq n \leq 100$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

void solve(){
    int n; cin >> n;
    int M[n][n];

    vector<vector<bool>> col(n);
    for(int i = 0; i < n; i++)
    {
        M[i][0] = M[0][i] = i;
        col[i].resize(2 * n, 0);
        col[i][i] = 1;
    }


    for(int i = 1; i < n; i++)
    {
        vector<bool> linha(2 * n, 0);
        linha[i] = 1;
        for(int j = 1; j < n; j++)
        {
            for(int k = 0; k < 2 * n; k++)
            {
                if(linha[k] == 0 && col[j][k] == 0)
                {
                    M[i][j] = k;
                    break;
                }
            }
            col[j][M[i][j]] = true;
            linha[M[i][j]] = true;
        }
    }

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            cout << M[i][j] << ' ';
        }
        cout << '\n';
    }
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

1.13 Missing Number

You are given all numbers between $1, 2, \ldots, n$ except one. Your task is to find the missing number.

Input
The first input line contains an integer n. The second line contains $n - 1$ numbers. Each number is distinct and between 1 and n (inclusive).

Output
Print the missing number.

Constraints
• $2 \leq n \leq 2 \cdot 10^5$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

int v[maxn];

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    // int t; cin >> t; while (t--)
    int n; cin >> n;
    v[0] = 1;
    for(int i = 0; i < n; i++){
        int x; cin >> x;
        v[x] = 1;
    }

    for(int i = 0; i <= n; i++){
        if(v[i] != 1) cout << i << '\n';
    }

    return 0;
}
```

1.14 Number Spiral

A number spiral is an infinite grid whose upper-left square has number 1. Here are the first five layers of the spiral:
Your task is to find out the number in row y and column x.

Input
The first input line contains an integer t: the number of tests. After this, there are t lines, each containing integers y and x.

Output
For each test, print the number in row y and column x.

Constraints
• $1 \leq t \leq 10^5$ • $1 \leq y, x \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

void solve(){
    ll x, y; cin >> y >> x;

    ll a = max(x, y);

    ll resp = (a-1)*(a-1);

    if(a % 2 == 0){
        resp += y + a - x;
    }
    else{
        resp += x + a - y;
    }

    cout << resp << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

1.15 Palindrome Reorder

Given a string, your task is to reorder its letters in such a way that it becomes a palindrome (i.e., it reads the same forwards and backwards).

Input
The only input line has a string of length n consisting of characters A–Z.

Output
Print a palindrome consisting of the characters of the original string. You may print any valid solution. If there are no solutions, print "NO SOLUTION".

Constraints

- $1 \le n \le 10^6$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;
int v[26];

void printv(vector<int> v){
    for(int i=0; i < v.size(); i++){
        cout << v[i] << ' ';
    }
    cout << '\n';
}

void solve(){
    string s;
    cin >> s;
    int a = 0, meio = -1;

    for(char c : s){
        v[c - 'A'] ++;
    }

    for(int i=0; i<26; i++){
        if(v[i] % 2 == 1){
            a++;
            meio = i;
        }
    }

    s = "";

    if(a >= 2){
        cout << "NO SOLUTION\n";
        return;
    }

    for(int i=0; i<26; i++){
        for(int j = 0; j < v[i]/2; j++){
            s.pb((char) ('A' + i));
        }
    }

    cout << s;
    if(meio != -1) cout << (char) ('A' + meio);
    reverse(s.begin(), s.end());
    cout << s << '\n';

}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

1.16 Permutations

A permutation of integers $1, 2, \ldots, n$ is called beautiful if there are no adjacent elements whose difference is 1. Given n, construct a beautiful permutation if such a permutation exists.

Input
The only input line contains an integer n.

Output
Print a beautiful permutation of integers $1, 2, \ldots, n$. If there are several solutions, you may print any of them. If there are no solutions, print "NO SOLUTION".

Constraints
- $1 \le n \le 10^6$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    // int t; cin >> t; while (t--)
    ll n; cin >> n;
    if(n == 1){
        cout << "1\n";
        return 0;
    }
    if(n < 4){
        cout << "NO SOLUTION\n";
        return 0;
    }
    if(n == 4){
        cout << "2 4 1 3\n";
        return 0;
    }

    vector<int> v1;
    vector<int> v2;

    for(int i = 1; i <= n/2; i++){
        v1.pb(i);
    }

    for(int i = n/2 + 1; i <= n; i++){
        v2.pb(i);
    }

    for(int i = 0; i < n; i++){
        if(i % 2 == 1){
            cout << v1.back() << ' ';
            v1.pop_back();
        }
        else{
            cout << v2.back() << ' ';
            v2.pop_back();
        }
    }

    cout << '\n';

    return 0;
}
```

1.17 Raab Game I

Consider a two player game where each player has n cards numbered $1, 2, \ldots, n$. On each turn both players place one of their cards on the table. The player who placed the higher card gets one point. If the cards are equal, neither player gets a point. The game continues until all cards have been played. You are given the number of cards n and the players' scores at the end of the game, a and b. Your task is to give an example of how the game could have played out.

Input
The first line contains one integer t: the number of tests. Then there are t lines, each with three integers n, a and b.

Output
For each test case print YES if there is a game with the given outcome and NO otherwise. If the answer is YES, print an example of one possible game. Print two lines representing the order in which the players place their cards. You can give any valid example.

Constraints
- $1 \le t \le 1000$ • $1 \le n \le 100$ • $0 \le a, b \le n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
#define all(v) (v).begin(), (v).end()

bool comparator(int a, int b)
{
    cout << "? " << a << ' ' << b << endl;
    string ans;
    cin >> ans;
    if(ans == "YES")
        return true;
```

```
    else
        return false;
}

void solve() {
    int n; cin >> n;
    int a, b; cin >> a >> b;

    if((a == 0 && b != 0) || (b == 0 && a != 0)){
        cout << "NO\n";
        return;
    }
    if(a == 0 && b == 0){
        cout << "YES\n";
        for(int i = 1; i <= n; i++) cout << i << ' ';
        cout << '\n';
        for(int i = 1; i <= n; i++) cout << i << ' ';
        cout << '\n';
        return;
    }
    if(a + b > n){
        cout << "NO\n";
        return;
    }
    if(b == 1){
        cout << "YES\n";
        for(int i = 1; i <= n; i++) cout << i << ' ';
        cout << '\n';
        cout << a + 1 << ' ';
        for(int i = 1; i <= a; i++) cout << i << ' ';
        for(int i = a + 2; i <= n; i++) cout << i << ' ';
        cout << '\n';
        return;
    }
    if(a == 1){
        swap(a, b);
        cout << "YES\n";
        cout << a + 1 << ' ';
        for(int i = 1; i <= a; i++) cout << i << ' ';
        for(int i = a + 2; i <= n; i++) cout << i << ' ';
        cout << '\n';
        for(int i = 1; i <= n; i++) cout << i << ' ';
        cout << '\n';
        return;
    }

    cout << "YES\n";
    for(int i = 1; i <= a; i++) cout << i << ' ';
    cout << a + b << ' ';
    for(int i = 1; i <= b - 1; i++) cout << i + a << ' ';
    for(int i = a + b + 1; i <= n; i++) cout << i << ' ';
    cout << '\n';

    cout << a << ' ';
    for(int i = 1; i <= a - 1; i++) cout << i << ' ';
    for(int i = 1; i <= b; i++) cout << i + a << ' ';
    for(int i = a + b + 1; i <= n; i++) cout << i << ' ';
    cout << '\n';
}

signed main() { _
    int t; cin >> t; while(t--)
    solve();
}
```

1.18 Repetitions

You are given a DNA sequence: a string consisting of characters A, C, G, and T. Your task is to find the longest repetition in the sequence. This is a maximum-length substring containing only one type of character.

Input
The only input line contains a string of n characters.

Output
Print one integer: the length of the longest repetition.

Constraints
- $1 \le n \le 10^6$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

int v[maxn];

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    // int t; cin >> t; while (t--)
    string s; cin >> s;
    s.pb('Z');
    int atual = 1, maxi = 0;
    char ant = 'z';

    for(char c : s){
        if(c == ant){
            atual ++;
        }
        else{
            ant = c;
            maxi = max(maxi, atual);
            atual = 1;
        }
    }

    cout << maxi << '\n';

    return 0;
}
```

1.19 String Reorder

Your task is to reorder the characters of a string so that no two adjacent characters are the same. What is the lexicographically minimal such string?

Input
The only line has a string of length n consisting of characters A–Z.

Output
Print the lexicographically minimal reordered string where no two adjacent characters are the same. If it is not possible to create such a string, print -1.

Constraints
- $1 \le n \le 10^6$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

bool check(vector<int> &v) {
    int sum = 0;
    for(int i = 0; i < 26; i++) {
        sum += v[i];
    }
    for(int i = 0; i < 26; i++) {
        if(v[i] > (sum + 1) / 2) {
            return false;
        }
    }
    return true;
}

int find_best(char pst, vector<int> &v) {
    int best = -1;
    for(int i = 0; i < 26; i++) {
        if(pst == 'A' + i || v[i] == 0) continue;
        v[i]--;
        if(check(v)) {
            best = i;
            break;
        }
        v[i]++;
    }
    return best;
}

void solve(){
    string s; cin >> s;
    vector<int> v(26);
    for(int i = 0; i < s.size(); i++){
        v[s[i] - 'A']++;
    }
    string ans = "";
    char pst = '$';

    while(ans.size() < s.size()) {
```

```
        int novo = find_best(pst, v);
        if(novo == -1) {
            cout << -1 << '\n';
            return;
        }
        ans += 'A' + novo;
        pst = 'A' + novo;
    }

    cout << ans << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

1.20 Towerof Hanoi

The Tower of Hanoi game consists of three stacks (left, middle and right) and n round disks of different sizes. Initially, the left stack has all the disks, in increasing order of size from top to bottom. The goal is to move all the disks to the right stack using the middle stack. On each move you can move the uppermost disk from a stack to another stack. In addition, it is not allowed to place a larger disk on a smaller disk. Your task is to find a solution that minimizes the number of moves.

Input
The only input line has an integer n: the number of disks.

Output
First print an integer k: the minimum number of moves. After this, print k lines that describe the moves. Each line has two integers a and b: you move a disk from stack a to stack b.

Constraints
- $1 \leq n \leq 16$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

void hanoi(int n, int ini, int fim){
    if(n == 0) return;
    int mid = 6 - ini - fim;

    hanoi(n - 1, ini, mid);
    cout << ini << ' ' << fim << '\n';
    hanoi(n - 1, mid, fim);
}

void solve(){
    int n; cin >> n;
    int total = 1;
    for(int i = 0; i < n; i++) total *= 2;
    total --;
    cout << total << '\n';
    hanoi(n, 1, 3);
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

1.21 Trailing Zeros

Your task is to calculate the number of trailing zeros in the factorial $n!$. For example, $20! = 2432902008176640000$ and it has 4 trailing zeros.

Input
The only input line has an integer n.

Output
Print the number of trailing zeros in $n!$.

Constraints
- $1 \leq n \leq 10^9$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

void printv(vector<int> v){
    for(int i=0; i < v.size(); i++){
        cout << v[i] << ' ';
    }
    cout << '\n';
}

void solve(){
    int n;
    cin >> n;
    int total = 0;
    for(int i = 5; i < M; i *= 5){
        total += n / i;
    }
    cout << total << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

1.22 Two Knights

Your task is to count for $k = 1, 2, \ldots, n$ the number of ways two knights can be placed on a $k \times k$ chessboard so that they do not attack each other.

Input
The only input line contains an integer n.

Output
Print n integers: the results.

Constraints
- $1 \leq n \leq 10000$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

void solve(ll n){
    ll total = (n * n) * (n * n - 1);

    total -= 4 * 2;
    total -= 8 * 3;
    total -= (4 * (n - 3)) * 4;
    total -= (4 * (n - 4)) * 6;
    total -= ((n - 4) * (n - 4)) * 8;

    cout << total / 2 << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int t; cin >> t; for(int i = 1; i <= t; i++)
    solve(i);
    return 0;
}
```

1.23 Two Sets

Your task is to divide the numbers $1, 2, \ldots, n$ into two sets of equal sum.

Input
The only input line contains an integer n.

Output
Print "YES", if the division is possible, and "NO" otherwise. After this, if the division is possible, print an example of how to create the sets. First, print the number of elements in the

first set followed by the elements themselves in a separate line, and then, print the second set in a similar way.

Constraints
• $1 \leq n \leq 10^6$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

void printv(vector<int> v){
    for(int i=0; i < v.size(); i++){
        cout << v[i] << ' ';
    }
    cout << '\n';
}

void solve(){
    int n;
    cin >> n;
    if(n % 4 == 2 || n % 4 == 1){
        cout << "NO\n";
        return;
    }
    cout << "YES\n";
    vector<int> v1;
    vector<int> v2;
    if(n % 4 == 3){
        v1.pb(1); v1.pb(n-1); v2.pb(n);
        for(int i = 2; i <= (n - 1) / 2; i++){
            if(i%2 == 1){
                v1.pb(i); v1.pb(n-i);
            }
            else{
                v2.pb(i); v2.pb(n-i);
            }
        }
    }

    if(n % 4 == 0){
        for(int i = 1; i <= (n) / 2; i++){
            if(i%2 == 1){
                v1.pb(i); v1.pb(n+1-i);
            }
            else{
                v2.pb(i); v2.pb(n+1-i);
            }
        }
    }

    cout << v1.size() << '\n'; printv(v1);
    cout << v2.size() << '\n'; printv(v2);
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

1.24 Weird Algorithm

Consider an algorithm that takes as input a positive integer n. If n is even, the algorithm divides it by two, and if n is odd, the algorithm multiplies it by three and adds one. The algorithm repeats this, until n is one. For example, the sequence for $n = 3$ is as follows: $3 \to 10 \to 5 \to 16 \to 8 \to 4 \to 2 \to 1$ Your task is to simulate the execution of the algorithm for a given value of n.

Input
The only input line contains an integer n.

Output
Print a line that contains all values of n during the algorithm.

Constraints
• $1 \leq n \leq 10^6$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    // int t; cin >> t; while (t--)
    ll n; cin >> n;
    while(n != 1){
        cout << n << ' ';
        if(n % 2 == 0) n = n/2;
        else n = 3*n + 1;
    }
    cout << "1\n";
    return 0;
}
```

2 Dynamic Programming

2.1 Array Description

You know that an array has n integers between 1 and m, and the absolute difference between two adjacent values is at most 1. Given a description of the array where some values may be unknown, your task is to count the number of arrays that match the description.

Input
The first input line has two integers n and m: the array size and the upper bound for each value. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array. Value 0 denotes an unknown value.

Output
Print one integer: the number of arrays modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 10^5$ • $1 \leq m \leq 100$ • $0 \leq x_i \leq m$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1e5 + 5, maxm = 1e2 + 5, inf = 2e9, M = 1e9 + 7;

// dp[i][j] = numero de maneiras ate a posicao i e terminando em j
int dp[maxn][maxm];
vector<int> x(maxn);

int calc(int n, int m){
    if(x[0] != 0) dp[0][x[0]] = 1;
    else{
        for(int j = 1; j <= m; j++) dp[0][j] = 1;
    }

    for(int i = 1; i < n; i++){
        if(x[i] == 0){
            for(int j = 1; j <= m; j++){
                dp[i][j] = ( (ll) dp[i - 1][j - 1] + dp[i - 1][j] + dp[i - 1][j + 1]) % M;
            }
        }
        else{
            dp[i][x[i]] = ( (ll) dp[i - 1][x[i] - 1] + dp[i - 1][x[i]] + dp[i - 1][x[i] + 1]) % M;
        }
    }

    int tot = 0;
    for(int j = 1; j <= m; j++) tot = (tot + dp[n - 1][j]) % M;

    return tot;
}

void solve() {
    int n, m; cin >> n >> m;

    for(int i = 0; i < n; i++) cin >> x[i];

    cout << calc(n, m) << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

2.2 Book Shop

You are in a book shop which sells n different books. You know the price and number of pages of each book. You have decided that the total price of your purchases will be at most x. What is the maximum number of pages you can buy? You can buy each book at most once.

Input
The first input line contains two integers n and x: the number of books and the maximum total price. The next line contains n integers $h_1, h_2, \ldots, h_n$: the price of each book. The last line contains n integers $s_1, s_2, \ldots, s_n$: the number of pages of each book.

Output
Print one integer: the maximum number of pages.

Constraints
• $1 \leq n \leq 1000$ • $1 \leq x \leq 10^5$ • $1 \leq h_i, s_i \leq 1000$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1e3 + 5, maxx = 1e5 + 5, inf = 2e9, M = 1e9 + 7;

// dp[j] = melhor que consigo usando j dinheiros
int dp[maxx];
vector<int> h(maxn), s(maxn);

int calc(int n, int x){
    // vai usando um livro por vez
    for(int i = 1; i <= n; i++){
        for(int j = x; j >= h[i]; j--){
            dp[j] = max(dp[j - h[i]] + s[i], dp[j]);
        }
    }

    return dp[x];
}

void solve() {
    int n, x; cin >> n >> x;

    for(int i = 1; i <= n; i++) cin >> h[i];
    for(int i = 1; i <= n; i++) cin >> s[i];

    cout << calc(n, x) << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

2.3 Coin Combinations I

Consider a money system consisting of n coins. Each coin has a positive integer value. Your task is to calculate the number of distinct ways you can produce a money sum x using the available coins. For example, if the coins are $\{2, 3, 5\}$ and the desired sum is 9, there are 8 ways:
• $2+2+5$ • $2+5+2$ • $5+2+2$ • $3+3+3$ • $2+2+2+3$ • $2+2+3+2$ • $2+3+2+2$ • $3+2+2+2$

Input
The first input line has two integers n and x: the number of coins and the desired sum of money. The second line has n distinct integers $c_1, c_2, \ldots, c_n$: the value of each coin.

Output
Print one integer: the number of ways modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 100$ • $1 \leq x \leq 10^6$ • $1 \leq c_i \leq 10^6$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1e6 + 5, inf = 2e9, M = 1e9 + 7;
const int dado = 6;

int dp[maxn];
vector<int> coins;

void calc(){
    dp[0] = 1;

    for(int i = 0; i < maxn; i++){
        if(dp[i] == 0) continue;

        for(int coin : coins){
            if(i + coin >= maxn) continue;
```

```
            dp[i + coin] = (dp[i + coin] + dp[i]) % M;
        }
    }
}

void solve() {
    for(int i = 0; i < maxn; i++) dp[i] = 0;

    int n, x, c;
    cin >> n >> x;

    for(int i = 0; i < n; i++){
        cin >> c;
        coins.push_back(c);
    }

    calc();

    cout << dp[x] << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

2.4 Coin Combinations II

Consider a money system consisting of n coins. Each coin has a positive integer value. Your task is to calculate the number of distinct ordered ways you can produce a money sum x using the available coins. For example, if the coins are $\{2, 3, 5\}$ and the desired sum is 9, there are 3 ways:
• $2 + 2 + 5$ • $3 + 3 + 3$ • $2 + 2 + 2 + 3$

Input
The first input line has two integers n and x: the number of coins and the desired sum of money. The second line has n distinct integers $c_1, c_2, \ldots, c_n$: the value of each coin.

Output
Print one integer: the number of ways modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 100$ • $1 \leq x \leq 10^6$ • $1 \leq c_i \leq 10^6$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1e6 + 5, inf = 2e9, M = 1e9 + 7;
const int dado = 6;

int dp[maxn];
vector<int> coins;

void calc(){
    dp[0] = 1;

    for(int coin : coins){
        for(int i = 0; i < maxn; i++){
            if(i + coin >= maxn) continue;
            dp[i + coin] = (dp[i + coin] + dp[i]) % M;
        }
    }
}

void solve() {
    for(int i = 0; i < maxn; i++) dp[i] = 0;

    int n, x, c;
    cin >> n >> x;

    for(int i = 0; i < n; i++){
        cin >> c;
        coins.push_back(c);
    }

    calc();

    cout << dp[x] << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

2.5 Counting Numbers

Your task is to count the number of integers between a and b where no two adjacent digits are the same.

Input
The only input line has two integers a and b.

Output
Print one integer: the answer to the problem.

Constraints
• $0 \leq a \leq b \leq 10^{18}$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxbse = 9, maxexp = 18, maxm = 1005, inf = 2e9, M = 1e9 + 7;

// dp[a][b] = quantidade < a * 10 ^ b sem dig adjacentes iguais.
ll dp[maxbse + 1][maxexp + 1];

void calcdp(){
    for(int b = 1; b <= maxbse; b++) dp[b][0] = b - 1;
    dp[1][1] = 9;

    for(int e = 1; e <= maxexp; e++){
        // [1, 99999] = [1, 89999] + [90000, 99999]
        //                         + [0001, 8999]
        if(e == 2) dp[1][e] = dp[9][e - 1] + dp[9][e - 2] + 1; // adiciona o "90"
        if(e > 2)  dp[1][e] = dp[9][e - 1] + dp[9][e - 2] - dp[1][e - 3]; // remove "[90000, 90099]"

        for(int b = 2; b <= maxbse; b++){
            // [1, 69999] = [1, 59999] + [60000, 69999]
            //                         [0001, 9999] - [1, 6999] + [1, 5999]
            if(e == 1)
                dp[b][e] = dp[b - 1][e] + dp[1][e] - dp[b][e - 1] + dp[b - 1][e - 1] + 1; // adiciona o "60"
            else
                dp[b][e] = dp[b - 1][e] + dp[1][e] - dp[b][e - 1] + dp[b - 1][e - 1] - dp[1][e - 2]; // remove "[60000, 60099]"
        }
    }
}

// quantidade no range [0, x - 1]
ll ans(ll x){
    if(x == 0) return 0;
    // guarda todos os digitos
    vector<int> dig;
    // guarda o expoente
    int e = -1;

    while(x > 0){
        dig.pb(x % 10);
        x /= 10;
        e++;
    }
    reverse(all(dig));

    // pega os numeros em [0, d00000 - 1]
    ll tot = dp[dig[0]][e] + 1;

    for(int i = 1; i < dig.size(); i++){
        e--;
        // pega os numeros em [0, d0000 - 1]
        tot += dp[dig[i]][e];

        // remove os digitos que iniciam em 00
        if(e >= 2 && dig[i] > 0 && dig[i-1] > 0) tot -= dp[1][e - 1];
        // adiciona o 10
        if(e == 0 && dig[i] > 0 && dig[i-1] > 0) tot++;

        // se ha dois digitos iguais consecutivos, podemos parar
        if(dig[i] == dig[i-1]) break;

        // se o digito cresce, precisamos tirar os 'dd' contados
        if(dig[i] > dig[i-1]){
            tot += -dp[dig[i-1] + 1][e] + dp[dig[i-1]][e];
        }
    }

    return tot;
}

void solve() {
    calcdp();
```

```
    ll a, b; cin >> a >> b;
    cout << ans(b + 1) - ans(a) << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

2.6 Counting Tiling

Enunciado não encontrado no site. **Código-fonte:**

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 10, maxpotn = 1030, maxm = 1005, inf = 2e9, M = 1e9 + 7;

/*
complete[n] sao todos os possiveis bitsets de "proximos andares" dado que um andar tem n casas vazias
exemplo: se n = 2, podemos completar de 3 possibilidades, {1 1 1}, {1 0 0}, {0 0 1}:
| i | i | i |   | i |   |   |   | i |
| i | i | i |   | i | - | - |   | - | - | i |
*/
vector<int> complete[maxn + 1];

// dp[n][m] quantidade de formas de fazer m-1 andares completos e o do topo representado pelo bitset n
ll dp[maxpotn][maxm];

// a transicao[n] sao todos os possiveis andares depois de receber um topo com o bitset n
vector<int> transicao[maxpotn];
int n, m;

// calcula complete
void calcomplete(){
    complete[0].pb(0);
    complete[1].pb(1);
    for(int i = 2; i <= maxn; i++){
        // completa o com i-1 e coloca um domino em pe
        for(auto x : complete[i - 1]){
            complete[i].pb(x * 2 + 1);
        }
        // completa o com i-2 e coloca um domino deitado
        for(auto x : complete[i - 2]){
            complete[i].pb(x * 4 + 0);
        }
    }
}

// calcula transicao
void calctransicao(){
    for(int i = 0; i < (1 << n); i++){
        vector<int> v;
        int aux = i;
        int tot = 0;
        // salva seq de 0 e 1 de i
        while(aux != 0){
            int zero = 0;
            while(aux % 2 == 0){
                zero ++;
                aux /= 2;
            }
            v.pb(zero);

            int um = 0;
            while(aux % 2 == 1){
                um ++;
                aux /= 2;
            }
            v.pb(um);
            tot += zero + um;
        }
        // garante n bits
        if(tot != n){
            v.pb(n - tot);
            v.pb(0);
        }

        // fila que guarda as transicoes
        queue<int> fila;
        fila.push(0);

        while(!v.empty()){
            // se estamos em uma seq de 1's o proximo eh sempre 0
            int fsz = fila.size();
            for(int j = 0; j < fsz; j++){
                int at = fila.front();
                at = at << v.back();
                fila.push(at);
                fila.pop();
            }
            v.pop_back();

            // em sequencia de 0's, precisamos adicionar os complete
            fsz = fila.size();
            for(int j = 0; j < fsz; j++){
                int at = fila.front();
                at = at << v.back();
                for(auto x : complete[v.back()]){
                    fila.push(at + x);
                }
                fila.pop();
            }
            v.pop_back();
        }

        // salva a fila no vetor transicao
        while(!fila.empty()){
            transicao[i].pb(fila.front());
            fila.pop();
        }
    }
}

void solve() {
    cin >> n >> m;
    calcomplete();
    calctransicao();
    // uma forma do primeiro andar ter 0 dominos
    dp[0][1] = 1;

    // calcula dp
    for(int j = 1; j <= m; j++){
        for(int i = 0; i < (1 << n); i++){
            for(auto x : transicao[i]){
                dp[x][j + 1] = (dp[x][j + 1] + dp[i][j]) % M;
            }
        }
    }

    // quantidade de formas do andar m + 1 ter 0 pecas e o resto estar completo
    cout << dp[0][m + 1] << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

2.7 Counting Towers

Your task is to build a tower whose width is 2 and height is n. You have an unlimited supply of blocks whose width and height are integers. For example, here are some possible solutions for $n = 6$:
Given n, how many different towers can you build? Mirrored and rotated towers are counted separately if they look different.

Input
The first input line contains an integer t: the number of tests. After this, there are t lines, and each line contains an integer n: the height of the tower.

Output
For each test, print the number of towers modulo $10^9 + 7$.

Constraints
• $1 \le t \le 100$ • $1 \le n \le 10^6$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1e6 + 5, inf = 2e9, M = 1e9 + 7;

int f[maxn];

void calc(){
    f[0] = 1; f[1] = 2;
    int s = 3;
    for(int i = 2; i < maxn; i++){
        f[i] = (7LL * f[i - 1] + (ll)(M - 2) * s) % M;
        s = (s + f[i]) % M;
    }
}

void solve() {
    int n; cin >> n;
    cout << f[n] << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    calc();
    int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

2.8 Dice Combinations

Your task is to count the number of ways to construct sum n by throwing a dice one or more times. Each throw produces an outcome between 1 and 6. For example, if $n = 3$, there are 4 ways:
• $1+1+1$ • $1+2$ • $2+1$ • 3

Input
The only input line has an integer n.

Output
Print the number of ways modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 10^6$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1000005, inf = 2e9, M = 1e9 + 7;
const int dado = 6;

int dp[maxn];

int calc(int n){
    if(n < 0) return 0;
    if(dp[n] != 0) return dp[n];

    for(int i = 1; i <= dado; i++) dp[n] = (dp[n] + calc(n -
         i)) % M;

    return dp[n] ;
}

void solve() {
    int n; cin >> n;
    dp[0] = 1;
    cout << calc(n) << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

2.9 Edit Distance

The edit distance between two strings is the minimum number of operations required to transform one string into the other. The allowed operations are:
• Add one character to the string. • Remove one character from the string. • Replace one character in the string.
For example, the edit distance between LOVE and MOVIE is 2, because you can first replace L with M, and then add I. Your task is to calculate the edit distance between two strings.

Input
The first input line has a string that contains n characters between A–Z. The second input line has a string that contains m characters between A–Z.

Output
Print one integer: the edit distance between the strings.

Constraints
• $1 \leq n, m \leq 5000$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1e6 + 5, inf = 2e9, M = 1e9 + 7;

void solve() {
    string s, t;
    cin >> s >> t;

    int n = s.size(), m = t.size();

    int dp[n + 1][m + 1];

    for(int i = 0; i <= n; i++) dp[i][0] = i;
    for(int j = 0; j <= m; j++) dp[0][j] = j;

    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= m; j++){
            dp[i][j] = inf;
            if(s[i-1] == t[j-1])
                dp[i][j] = min(dp[i][j], dp[i - 1][j - 1]);
            dp[i][j] = min(dp[i][j], dp[i][j - 1] + 1);
            dp[i][j] = min(dp[i][j], dp[i - 1][j] + 1);
            dp[i][j] = min(dp[i][j], dp[i - 1][j - 1] + 1);
        }
    }

    cout << dp[n][m] << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

2.10 Elevator Rides

There are n people who want to get to the top of a building which has only one elevator. You know the weight of each person and the maximum allowed weight in the elevator. What is the minimum number of elevator rides?

Input
The first input line has two integers n and x: the number of people and the maximum allowed weight in the elevator. The second line has n integers $w_1, w_2, \ldots, w_n$: the weight of each person.

Output
Print one integer: the minimum number of rides.

Constraints
• $1 \le n \le 20$ • $1 \le x \le 10^9$ • $1 \le w_i \le x$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long int ll;

const int maxn = 1050000, maxm = 25, inf = 2e9, M = 1e9 + 7;

// dp[X] = {k, W} -> com o bitset X, k eh o menor numero de viagens, W eh o peso total da menor viagem
pair<int, int> dp[maxn];
bool tested[maxn];
int pot[maxm];

int cmp(int a, int b)
{
    int count1 = pc(a);
    int count2 = pc(b);

    if (count1 <= count2)
        return false;
    return true;
}

void solve() {
    int n, x; cin >> n >> x;
    vector<int> w(n);
    for(int i = 0; i < n; i++) cin >> w[i];
    pot[0] = 1;
    for(int i = 1; i < maxm; i++) pot[i] = pot[i - 1] * 2;
    for(int i = 0; i < maxn; i++) dp[i] = {inf, inf};
    dp[0] = {1, 0};

    queue<int> fila;
    fila.push(0);
    tested[0] = true;

    while(!fila.empty()){
        int X = fila.front();
        fila.pop();
        for(int i = 0; i < n; i++){
            int Y = (X | pot[i]);
            if(Y == X) continue;
            if(dp[X].S + w[i] <= x)
                dp[Y] = min(dp[Y], {dp[X].F, dp[X].S + w[i]});
            else
                dp[Y] = min(dp[Y], {dp[X].F + 1, w[i]});

            if(!tested[Y]){
                fila.push(Y);
                tested[Y] = true;
            }
        }
    }

    cout << dp[pot[n] - 1].F << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

2.11 Grid Paths I

Consider an $n \times n$ grid whose squares may have traps. It is not allowed to move to a square with a trap. Your task is to calculate the number of paths from the upper-left square to the lower-right square. You can only move right or down.

Input
The first input line has an integer n: the size of the grid. After this, there are n lines that describe the grid. Each line has n characters: . denotes an empty cell, and * denotes a trap.

Output
Print the number of paths modulo $10^9 + 7$.

Constraints
• $1 \le n \le 1000$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1e3 + 5, inf = 2e9, M = 1e9 + 7;

int dp[maxn][maxn];
char grid[maxn][maxn];
int n;

int calc(int a, int b){
    if(a < 0 || b < 0 || a >= n || b >= n) return 0;
    if(dp[a][b] >= 0) return dp[a][b];
    if(grid[a][b] == '*') return dp[a][b] = 0;

    return dp[a][b] = (calc(a + 1, b) + calc(a, b + 1)) % M;
}

void solve() {
    cin >> n;

    for(int i = 0; i < n; i++){
        for(int j = 0; j < n; j++){
            dp[i][j] = -1;
            cin >> grid[i][j];
        }
    }

    dp[n - 1][n - 1] = grid[n - 1][n - 1] == '*' ? 0 : 1;
    calc(0, 0);

    cout << dp[0][0] << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

2.12 Increasing Subsequence

You are given an array containing n integers. Your task is to determine the longest increasing subsequence in the array, i.e., the longest subsequence where every element is larger than the previous one. A subsequence is a sequence that can be derived from the array by deleting some elements without changing the order of the remaining elements.

Input
The first line contains an integer n: the size of the array. After this there are n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print the length of the longest increasing subsequence.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le x_i \le 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 5e2 + 5, inf = 2e9, M = 1e9 + 7;

template<typename T> int lis(vector<T> &v){
    vector<T> ans;
    for (T t : v){
        auto it = lower_bound(ans.begin(), ans.end(), t);
        if (it == ans.end()) ans.push_back(t);
        else *it = t;
    }
    return ans.size();
```

```
}

void solve() {
    int n; cin >> n;
    vector<int> v(n);

    for(int i = 0; i < n; i++) cin >> v[i];

    cout << lis<int> (v) << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

2.13 Increasing Subsequence II

Given an array of n integers, your task is to calculate the number of increasing subsequences it contains. If two subsequences have the same values but in different positions in the array, they are counted separately.

Input
The first input line has an integer n: the size of the array. The second line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print one integer: the number of increasing subsequences modulo $10^9 + 7$.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le x_i \le 10^9$

Código-fonte:

```
#include <bits/stdc++.h>
#define int long long

using namespace std;

const int M = 1e9 + 7;

struct BIT {
    vector<int> bit;
    int n;

    BIT(int n) {
        this->n = n;
        bit.assign(n, 0);
    }

    int sum(int r) {
        int ret = 0;
        for (; r >= 0; r = (r & (r + 1)) - 1)
            ret = (ret + bit[r]) % M;
        return ret;
    }

    void add(int idx, int delta) {
        for (; idx < n; idx = idx | (idx + 1))
            bit[idx] = (bit[idx] + delta) % M;
    }
};

void solve() {
    int n; cin >> n;
    int ans = 0;

    vector<int> a(n);
    for (int i = 0; i < n; i++) cin >> a[i];

    vector<int> aux = a;
    sort(aux.begin(), aux.end());
    aux.erase(unique(aux.begin(), aux.end()), aux.end());
    for (int i = 0; i < n; i++) {
        a[i] = lower_bound(aux.begin(), aux.end(), a[i]) -
               aux.begin() + 1;
    }

    BIT bit(n + 1); bit.add(0, 1);

    for(int i = 0; i < n; i++) {
        int soma = bit.sum(a[i] - 1);
        ans += soma; ans %= M;
        bit.add(a[i], soma);
    }

    cout << ans << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

2.14 Longest Common Subsequence

Given two arrays of integers, find their longest common subsequence. A subsequence is a sequence of array elements from left to right that can contain gaps. A common subsequence is a subsequence that appears in both arrays.

Input
The first line has two integers n and m: the sizes of the arrays. The second line has n integers $a_1, a_2, \ldots, a_n$: the contents of the first array. The third line has m integers $b_1, b_2, \ldots, b_m$: the contents of the second array.

Output
First print the length of the longest common subsequence. After that, print an example of such a sequence. If there are several solutions, you can print any of them.

Constraints
• $1 \le n, m \le 1000$ • $1 \le a_i, b_i \le 10^9$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

vector<vector<int>> dp;
vector<int> a, b;

int lcs(int n, int m) {
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= m; ++j)
            if (a[i - 1] == b[j - 1])
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);

    return dp[n][m];
}

void solve() {
    int n, m; cin >> n >> m;
    a.resize(n);
    b.resize(m);

    for(int i = 0; i < n; i++) cin >> a[i];
    for(int i = 0; i < m; i++) cin >> b[i];

    dp.resize(n + 1, vector<int>(m + 1, 0));

    int i = n, j = m, at = lcs(n, m);
    vector<int> ans;
    cout << at << '\n';

    while(i > 0 && j > 0)
    {
        if (a[i - 1] == b[j - 1]) {
            ans.push_back(a[i - 1]);
            i--; j--;
        }
        else if (dp[i - 1][j] >= dp[i][j - 1]) i--;
        else j--;
    }

    reverse(ans.begin(), ans.end());
    for(auto x : ans) cout << x << ' ';
    cout << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

2.15 Minimal Grid Path

You are given an $n \times n$ grid whose each square contains a letter. You should move from the upper-left square to the lower-right square. You can only move right or down. What is the lexicographically minimal string you can construct?

Input
The first line has an integer n: the size of the grid. After this, there are n lines that describe the grid. Each line has n letters between A and Z.

Output
Print the lexicographically minimal string.

Constraints
• $1 \leq n \leq 3000$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

vector<vector<char>> M;
int n;

void solve() {
    cin >> n;
    M.resize(n + 1, vector<char>(n + 1));

    for(int i = 0; i <= n; i++) M[i][0] = '$';
    for(int i = 0; i <= n; i++) M[0][i] = '$';

    for(int i = 1; i <= n; i++)
        for(int j = 1; j <= n; j++)
            cin >> M[i][j];

    string s; s.push_back(M[1][1]);
    for(int sum = 3; sum <= 2 * n; sum++)
    {
        char best = 'Z';
        for(int i = max(1, sum - n); i <= min(n, sum - 1); i
            ++)
        {
            int j = sum - i;
            if(M[i - 1][j] != '$') best = min(best, M[i][j]);
            if(M[i][j - 1] != '$') best = min(best, M[i][j]);
        }

        s.push_back(best);
        for(int i = max(1, sum - n); i <= min(n, sum - 1); i
            ++)
        {
            int j = sum - i;
            if(M[i - 1][j] == '$' && M[i][j - 1] == '$' || M[
                i][j] != best) M[i][j] = '$';
        }
    }

    cout << s << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

2.16 Minimizing Coins

Consider a money system consisting of n coins. Each coin has a positive integer value. Your task is to produce a sum of money x using the available coins in such a way that the number of coins is minimal. For example, if the coins are $\{1, 5, 7\}$ and the desired sum is 11, an optimal solution is $5 + 5 + 1$ which requires 3 coins.

Input
The first input line has two integers n and x: the number of coins and the desired sum of money. The second line has n distinct integers $c_1, c_2, \ldots, c_n$: the value of each coin.

Output
Print one integer: the minimum number of coins. If it is not possible to produce the desired sum, print -1.

Constraints
• $1 \leq n \leq 100$ • $1 \leq x \leq 10^6$ • $1 \leq c_i \leq 10^6$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1e6 + 5, inf = 2e9, M = 1e9 + 7;
const int dado = 6;

int dp[maxn];
vector<int> coins;

void calc(){
    dp[0] = 0;

    for(int i = 0; i < maxn; i++){
        if(dp[i] == inf) continue;

        for(int coin : coins){
            if(i + coin >= maxn) continue;
            dp[i + coin] = min(dp[i + coin], dp[i] + 1);
        }
    }
}

void solve() {
    for(int i = 0; i < maxn; i++) dp[i] = inf;

    int n, x, c;
    cin >> n >> x;

    for(int i = 0; i < n; i++){
        cin >> c;
        coins.push_back(c);
    }

    calc();

    if(dp[x] == inf) dp[x] = -1;
    cout << dp[x] << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

2.17 Money Sums

You have n coins with certain values. Your task is to find all money sums you can create using these coins.

Input
The first input line has an integer n: the number of coins. The next line has n integers $x_1, x_2, \ldots, x_n$: the values of the coins.

Output
First print an integer k: the number of distinct money sums. After this, print all possible sums in increasing order.

Constraints
• $1 \leq n \leq 100$ • $1 \leq x_i \leq 1000$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1e6 + 5, inf = 2e9, M = 1e9 + 7;

void solve() {
    int n; cin >> n;
    bitset<maxn> mark;
    mark[0] = 1;

    for(int i = 0; i < n; i++){
        int a; cin >> a;
        mark = (mark | mark << a);
    }

    cout << mark.count() - 1 << '\n';
    for(int i = 1; i < maxn; i++)
        if(mark[i] == 1)
            cout << i << ' ';
    cout << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

2.18 Mountain Range

There are n mountains in a row, each with a specific height. You begin your hang gliding route from some mountain. You can glide from mountain a to mountain b if mountain a is taller than mountain b and all mountains between a and b. What is the maximum number of mountains you can visit on your route?

Input
The first line has an integer n: the number of mountains. The next line has n integers $h_1, h_2, \ldots, h_n$: the heights of the mountains.

Output:
Print one integer: the maximum number of mountains.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq h_i \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
using namespace std;

const int inf = 2e9;
vector<int> lef, rig, a;
const int MAX = 2e5 + 5;

struct seg {
    int n, t[4*MAX];

    int query(int v, int tl, int tr, int l, int r) {
        if (l > r)
            return 0;
        if (l == tl && r == tr)
            return t[v];
        int tm = (tl + tr) / 2;

        int left = query(v*2, tl, tm, l, min(r, tm));
        int right = query(v*2+1, tm+1, tr, max(l, tm+1), r);

        return max(left, right);
    }

    void update(int v, int tl, int tr, int pos, int new_val) {
        if (tl == tr) {
            t[v] = new_val;
        } else {
            int tm = (tl + tr) / 2;
            if (pos <= tm)
                update(v*2, tl, tm, pos, new_val);
            else
                update(v*2+1, tm+1, tr, pos, new_val);
            t[v] = max(t[v*2], t[v*2+1]);
        }
    }
};

void calc() {
    int n = a.size();
    lef.resize(n), rig.resize(n);
    stack<int> st;
    for(int i = 0; i < n; i++) {
        while(!st.empty() && a[st.top()] < a[i]) st.pop();
        lef[i] = st.empty() ? -1 : st.top();
        st.push(i);
    }
    while(!st.empty()) st.pop();
    for(int i = n - 1; i >= 0; i--) {
        while(!st.empty() && a[st.top()] < a[i]) st.pop();
        rig[i] = st.empty() ? -1 : st.top();
        st.push(i);
    }
}

void solve() {
    int n; cin >> n; n += 2;
    a.resize(n); a[0] = inf;
    for(int i = 1; i <= n - 2; i++) cin >> a[i];
    a[n - 1] = inf;

    calc();

    vector<pair<int, int>> seq(n);
    for(int i = 0; i < n; i++) seq[i] = {a[i], i};
    sort(seq.begin(), seq.end());

    seg tree; tree.n = n;

    for(int i = 0; i < n - 2; i++) {
        int idx = seq[i].second;
        int L = lef[idx], R = rig[idx];

        int ans_left = tree.query(1, 0, n - 1, L + 1, idx);
        int ans_right = tree.query(1, 0, n - 1, idx, R - 1);

        tree.update(1, 0, n - 1, idx, max(ans_left, ans_right) + 1);
    }

    cout << tree.query(1, 0, n - 1, 0, n - 1) << "\n";

}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

2.19 Projects

There are n projects you can attend. For each project, you know its starting and ending days and the amount of money you would get as reward. You can only attend one project during a day. What is the maximum amount of money you can earn?

Input
The first input line contains an integer n: the number of projects. After this, there are n lines. Each such line has three integers a_i, b_i, and p_i: the starting day, the ending day, and the reward.

Output
Print one integer: the maximum amount of money you can earn.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq a_i \leq b_i \leq 10^9$ • $1 \leq p_i \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define f first
#define s second

using namespace std;

typedef long long int ll;
typedef pair<int, int> pii;

const int maxn = 5e2 + 5, inf = 2e9, M = 1e9 + 7;

void solve() {
    int n; cin >> n;
    vector<pair<pii, int>> v(n);

    for(int i = 0; i < n; i++) cin >> v[i].f.f >> v[i].f.s >> v[i].s;

    set<int> checkpoints;
    for(int i = 0; i < n; i++) {
        checkpoints.insert(v[i].f.f);
        checkpoints.insert(v[i].f.s);
    }

    unordered_map<int, int> comprime;
    int tot = 1;
    for(auto x : checkpoints){
        comprime[x] = tot;
        tot++;
    }

    sort(all(v));

    vector<ll> dp(tot + 1);
    int at = 0;

    for(int i = 1; i <= tot; i++){
        dp[i] = max(dp[i], dp[i - 1]);
        while(i == comprime[v[at].f.f]){
            dp[comprime[v[at].f.s]] = max(dp[comprime[v[at].f.s]], dp[i - 1] + v[at].s);
            at ++;
        }
    }

    cout << dp[tot] << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

2.20 Rectangle Cutting

Given an $a \times b$ rectangle, your task is to cut it into squares. On each move you can select a rectangle and cut it into two rectangles in such a way that all side lengths remain integers. What is the minimum possible number of moves?

Input
The only input line has two integers a and b.

Output
Print one integer: the minimum number of moves.

Constraints
• $1 \leq a, b \leq 500$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1e6 + 5, inf = 2e9, M = 1e9 + 7;

void solve() {
    int a, b;
    cin >> a >> b;

    int dp[a + 1][b + 1];

    for(int i = 1; i <= min(a, b); i++) dp[i][i] = 0;

    for(int i = 1; i <= a; i++){
        for(int j = 1; j <= b; j++){
            if(i == j) continue;
            dp[i][j] = inf;
            for(int k = 1; k < i; k++)
                dp[i][j] = min(dp[i][j], 1 + dp[i - k][j] +
                    dp[k][j]);
            for(int k = 1; k < j; k++)
                dp[i][j] = min(dp[i][j], 1 + dp[i][j - k] +
                    dp[i][k]);
        }
    }

    cout << dp[a][b] << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

2.21 Removal Game

There is a list of n numbers and two players who move alternately. On each move, a player removes either the first or last number from the list, and their score increases by that number. Both players try to maximize their scores. What is the maximum possible score for the first player when both players play optimally?

Input
The first input line contains an integer n: the size of the list. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the list.

Output
Print the maximum possible score for the first player.

Constraints
• $1 \le n \le 5000$ • $-10^9 \le x_i \le 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1e6 + 5, inf = 2e9, M = 1e9 + 7;

void solve() {
    int n; cin >> n;
    vector<int>   v(n + 1);
    vector<ll > acc(n + 1);

    for(int i = 1; i <= n; i++){
        cin >> v[i];
        acc[i] = acc[i-1] + v[i];
    }

    ll dp[n + 1][n + 1];

    for(int i = 0; i <= n; i++) dp[i][i] = v[i];

    for(int k = 1; k < n; k++){
        for(int i = 1; i <= n - k; i++){
            int j = i + k;
            dp[i][j] = (acc[j] - acc[i-1]) - min(dp[i + 1][j
                ], dp[i][j - 1]);
        }
    }

    cout << dp[1][n] << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

2.22 Removing Digits

You are given an integer n. On each step, you may subtract one of the digits from the number. How many steps are required to make the number equal to 0?

Input
The only input line has an integer n.

Output
Print one integer: the minimum number of steps.

Constraints
• $1 \le n \le 10^6$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1e6 + 5, inf = 2e9, M = 1e9 + 7;
const int dado = 6;

int dp[maxn];

vector<int> digits(int x){
    vector<int> v;
    while(x > 0){
        if(x % 10 > 0)
            v.pb(x % 10);
        x /= 10;
    }
    return v;
}

void calc(int n){
    dp[0] = 0;
    for(int i = 1; i <= n; i++){
        vector<int> v = digits(i);
        for(auto x : v)
            dp[i] = min(dp[i], dp[i - x] + 1);
    }
}

void solve() {
    for(int i = 0; i < maxn; i++) dp[i] = inf;

    int n;
    cin >> n;
    calc(n);

    cout << dp[n] << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

2.23 Two Sets II

Your task is to count the number of ways numbers $1, 2, \ldots, n$ can be divided into two sets of equal sum. For example, if $n = 7$, there are four solutions:
• $\{1, 3, 4, 6\}$ and $\{2, 5, 7\}$ • $\{1, 2, 5, 6\}$ and $\{3, 4, 7\}$ • $\{1, 2, 4, 7\}$ and $\{3, 5, 6\}$ • $\{1, 6, 7\}$ and $\{2, 3, 4, 5\}$

Input
The only input line contains an integer n.

Output
Print the answer modulo $10^9 + 7$.

Constraints
• $1 \le n \le 500$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
```

```cpp

using namespace std;

typedef long long int ll;

const int maxn = 5e2 + 5, inf = 2e9, M = 1e9 + 7;

// dp[i][j] eh o numero de sets de tamanho ate i cuja soma eh j
ll dp[maxn][(ll)maxn * (maxn + 1) / 2];

void solve() {
    int n; cin >> n;

    for(int i = 0; i <= n; i++) dp[i][0] = 0;
    for(int i = 0; i <= n * (n + 1) / 2; i++) dp[0][i] = 0;
    dp[0][0] = 1;

    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= n * (n + 1) / 2; j++){
            dp[i][j] = (dp[i][j] + dp[i - 1][j] + dp[i - 1][max(j - i, 0)]) % M;
        }
    }

    if(n * (n + 1) % 4 != 0){
        cout << "0\n";
    }
    else{
        cout << dp[n][n * (n + 1) / 4] << '\n';
    }

}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

3 Graph Algorithms

3.1 Building Roads

Byteland has n cities, and m roads between them. The goal is to construct new roads so that there is a route between any two cities. Your task is to find out the minimum number of roads required, and also determine which roads should be built.

Input
The first input line has two integers n and m: the number of cities and roads. The cities are numbered $1, 2, \ldots, n$. After that, there are m lines describing the roads. Each line has two integers a and b: there is a road between those cities. A road always connects two different cities, and there is at most one road between any two cities.

Output
First print an integer k: the number of required roads. Then, print k lines that describe the new roads. You can print any valid solution.

Constraints
• $1 \leq n \leq 10^5$ • $1 \leq m \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long int  ll;
typedef pair<int, int> pii;
typedef vector<int> vi;
typedef vector<vi> graph;

const int maxn = 1e3+5, inf = 2e9, M = 1e9 + 7;
const ll linf = 1e18;
int n, m;
vector<int> ans;
graph G;
vi check;

void dfs(int i){
    check[i] = 1;
    for(auto x : G[i]){
        if(check[x] == 0) dfs(x);
    }
}

void solve(){
    cin >> n >> m;
    G.resize(n + 1);
    check.resize(n + 1);

    for(int i = 1; i <= m; i++){
        int a, b; cin >> a >> b;
        G[a].pb(b); G[b].pb(a);
    }

    for(int i = 1; i <= n; i++){
        if(check[i] == 0){
            dfs(i);
            ans.pb(i);
        }
    }

    cout << ans.size() - 1 << '\n';
    for(int i = 1; i < ans.size(); i++){
        cout << ans[i - 1] << ' ' << ans[i] << '\n';
    }
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

3.2 Building Teams

There are n pupils in Uolevi's class, and m friendships between them. Your task is to divide the pupils into two teams in such a way that no two pupils in a team are friends. You can freely choose the sizes of the teams.

Input
The first input line has two integers n and m: the number of pupils and friendships. The pupils are numbered $1, 2, \ldots, n$. Then, there are m lines describing the friendships. Each line has two integers a and b: pupils a and b are friends. Every friendship is between two different pupils. You can assume that there is at most one friendship between any two pupils.

Output
Print an example of how to build the teams. For each pupil, print "1" or "2" depending on to which team the pupil will be assigned. You can print any valid team. If there are no solutions, print "IMPOSSIBLE".

Constraints
• $1 \leq n \leq 10^5$ • $1 \leq m \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long int  ll;
typedef pair<int, int> pii;
typedef vector<int> vi;
typedef vector<vi> graph;

const int maxn = 1e3+5, inf = 2e9, M = 1e9 + 7;
const ll linf = 1e18;
int n, m;
vector<int> ans;
graph G;
vi check;
bool flag = true;

void dfs(int i){
    for(auto x : G[i]){
        if(check[x] == 0){
            check[x] = 3 - check[i];
            dfs(x);
        }
        else if(check[x] == check[i]) flag = false;
    }
}

void solve(){
    cin >> n >> m;
    G.resize(n + 1);
    check.resize(n + 1);

    for(int i = 1; i <= m; i++){
        int a, b; cin >> a >> b;
        G[a].pb(b); G[b].pb(a);
    }

    for(int i = 1; i <= n; i++){
        if(check[i] == 0){
            check[i] = 1;
            dfs(i);
        }
    }

    if(!flag){
        cout << "IMPOSSIBLE\n";
        return;
    }

    for(int i = 1; i <= n; i++){
        cout << check[i] << ' ';
    }
    cout << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

3.3 Coin Collector

A game has n rooms and m tunnels between them. Each room has a certain number of coins. What is the maximum number of coins you can collect while moving through the tunnels when you can freely choose your starting and ending room?

Input
The first input line has two integers n and m: the number of rooms and tunnels. The rooms are numbered $1, 2, \ldots, n$. Then, there are n integers $k_1, k_2, \ldots, k_n$: the number of coins in each room. Finally, there are m lines describing the tunnels. Each line has two integers a and b: there is a tunnel from room a to room b. Each tunnel is a one-way tunnel.

Output

Print one integer: the maximum number of coins you can collect.

Constraints
• $1 \leq n \leq 10^5$ • $1 \leq m \leq 2 \cdot 10^5$ • $1 \leq k_i \leq 10^9$ • $1 \leq a, b \leq n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
using namespace std;

#define int long long

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 1e5 + 5;

int n, m;
vector<int> g[MAX];
vector<int> gi[MAX]; // grafo invertido
int vis[MAX];
stack<int> S;
int comp[MAX]; // componente conexo de cada vertice

void dfs(int k) {
    vis[k] = 1;
    for (int i = 0; i < (int) g[k].size(); i++)
        if (!vis[g[k][i]]) dfs(g[k][i]);

    S.push(k);
}

void scc(int k, int c) {
    vis[k] = 1;
    comp[k] = c;
    for (int i = 0; i < (int) gi[k].size(); i++)
        if (!vis[gi[k][i]]) scc(gi[k][i], c);
}

void kosaraju() {
    for (int i = 0; i < n; i++) vis[i] = 0;
    for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);

    for (int i = 0; i < n; i++) vis[i] = 0;
    while (S.size()) {
        int u = S.top();
        S.pop();
        if (!vis[u]) scc(u, u);
    }
}

vector<int> new_g[MAX];
int new_k[MAX];
int dp[MAX];

vector<int> topo_sort(int n) {
    vector<int> ret(n,-1), vis(n,0);

    int pos = n-1, dag = 1;
    function<void(int)> dfs = [&](int v) {
        vis[v] = 1;
        for (auto u : new_g[v]) {
            if (vis[u] == 1) dag = 0;
            else if (!vis[u]) dfs(u);
        }
        ret[pos--] = v, vis[v] = 2;
    };

    for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);

    if (!dag) ret.clear();
    return ret;
}

void solve(){
    cin >> n >> m;

    vector<int> k(n);
    for(int i = 0; i < n; i++) cin >> k[i];

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        g[a].push_back(b);
        gi[b].push_back(a);
    }

    kosaraju();

    for(int i = 0; i < n; i++) {
        for(auto x : g[i]) {
            if(comp[x] != comp[i]) new_g[comp[i]].push_back(
                comp[x]);
        }
        new_k[comp[i]] += k[i];
    }

    vi ret = topo_sort(n);

    int ans = 0;

    for(int i = ret.size() - 1; i >= 0; i--) {
        dp[ret[i]] = new_k[ret[i]];
        for(auto x : new_g[ret[i]]) {
            dp[ret[i]] = max(dp[x] + new_k[ret[i]], dp[ret[i
                ]]);
        }
        ans = max(ans, dp[ret[i]]);
    }

    cout << ans << '\n';

}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

3.4 Counting Rooms

You are given a map of a building, and your task is to count the number of its rooms. The size of the map is $n \times m$ squares, and each square is either floor or wall. You can walk left, right, up, and down through the floor squares.

Input
The first input line has two integers n and m: the height and width of the map. Then there are n lines of m characters describing the map. Each character is either . (floor) or # (wall).

Output

Print one integer: the number of rooms.

Constraints
• $1 \leq n, m \leq 1000$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1e3+5, inf = 2e9, M = 1e9 + 7;
const ll linf = 1e18;

char tab[maxn][maxn];
int check[maxn][maxn];
int n, m;

void dfs(int i, int j){
    if(check[i][j] == 0) return;
    check[i][j] = 0;
    dfs(i + 1, j);
    dfs(i, j + 1);
    dfs(i - 1, j);
    dfs(i, j - 1);
}

void solve(){
    cin >> n >> m;

    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= m; j++){
            cin >> tab[i][j];
            if(tab[i][j] == '.') check[i][j] = 1;
        }
    }

    int total = 0;

    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= m; j++){
            if(check[i][j] == 1){
                total++;
                dfs(i, j);
            }
        }
    }

    cout << total << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

3.5 Course Schedule

You have to complete n courses. There are m requirements of the form "course a has to be completed before course b". Your task is to find an order in which you can complete the courses.

Input
The first input line has two integers n and m: the number of courses and requirements. The courses are numbered $1, 2, \ldots, n$. After this, there are m lines describing the requirements. Each line has two integers a and b: course a has to be completed before course b.

Output
Print an order in which you can complete the courses. You can print any valid order that includes all the courses. If there are no solutions, print "IMPOSSIBLE".

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define int long long

using namespace std;

const int INF = 1000000000000000000;
const int MAX = 100010;

vector<int> g[MAX];

vector<int> topo_sort(int n) {
    vector<int> ret(n,-1), vis(n,0);

    int pos = n-1, dag = 1;
    function<void(int)> dfs = [&](int v) {
        vis[v] = 1;
        for (auto u : g[v]) {
            if (vis[u] == 1) dag = 0;
            else if (!vis[u]) dfs(u);
        }
        ret[pos--] = v, vis[v] = 2;
    };

    for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);

    if (!dag) ret.clear();
    return ret;
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int n, m; cin >> n >> m;

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        a--; b--;
        g[a].push_back(b);
    }

    vector<int> ans = topo_sort(n);

    if(ans.size() == 0)
        cout << "IMPOSSIBLE\n";
    else {
        for(auto x : ans) cout << x + 1 << ' ';
        cout << '\n';
    }

    return 0;
}
```

3.6 Cycle Finding

You are given a directed graph, and your task is to find out if it contains a negative cycle, and also give an example of such a cycle.

Input
The first input line has two integers n and m: the number of nodes and edges. The nodes are numbered $1, 2, \ldots, n$. After this, the input has m lines describing the edges. Each line has three integers a, b, and c: there is an edge from node a to node b whose length is c.

Output
If the graph contains a negative cycle, print first "YES", and then the nodes in the cycle in their correct order. If there are several negative cycles, you can print any of them. If there are no negative cycles, print "NO".

Constraints
• $1 \le n \le 2500$ • $1 \le m \le 5000$ • $1 \le a, b \le n$ • $-10^9 \le c \le 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define int long long

using namespace std;

const int MAX = 2505;
const int INF = 1000000000000000000;

struct Edge {
    int a, b, cost;
};

int n, m;
vector<int> g[MAX], gi[MAX];
vector<Edge> edges;
int vis[MAX];
stack<int> S;
int comp[MAX];

void dfs(int k) {
    vis[k] = 1;
    for (int i = 0; i < (int) g[k].size(); i++)
        if (!vis[g[k][i]]) dfs(g[k][i]);

    S.push(k);
}

void scc(int k, int c) {
    vis[k] = 1;
    comp[k] = c;
    for (int i = 0; i < (int) gi[k].size(); i++)
        if (!vis[gi[k][i]]) scc(gi[k][i], c);
}

void kosaraju() {
    for (int i = 0; i < n; i++) vis[i] = 0;
    for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);

    for (int i = 0; i < n; i++) vis[i] = 0;
    while (S.size()) {
        int u = S.top();
        S.pop();
        if (!vis[u]) scc(u, u);
    }
}

void solve()
{
    cin >> n >> m;
    for(int i = 0; i < m; i++) {
        int a, b, c; cin >> a >> b >> c;
        a--; b--;
        edges.push_back({a, b, c});
        if(a == b && c < 0) {
            cout << "YES\n" << a + 1 << ' ' << b + 1 << '\n';
            return;
        }
        g[a].push_back(b);
        gi[b].push_back(a);
    }

    kosaraju();

    int tot = 0;
    for(int i = 0; i < n; i++)
        tot = max(tot, comp[i]);

    vector<vector<int>> C(tot + 1);

    for(int i = 0; i < n; i++)
        C[comp[i]].push_back(i);

    vector<int> d(n, INF), p(n, -1);
    for(int i = 0; i <= tot; i++) {
        if(C[i].size() <= 1) continue;

        int v = C[i][0]; n = C[i].size();
        d[v] = 0;

        int x;
        for (int i = 0; i < n; ++i) {
            x = -1;
            for (Edge e : edges) {
                if (comp[e.a] != comp[v] || comp[e.b] != comp[v]) continue;
                if (d[e.a] < INF)
                    if (d[e.b] > d[e.a] + e.cost) {
                        d[e.b] = max(-INF, d[e.a] + e.cost);
                        p[e.b] = e.a;
                        x = e.b;
                    }
            }
        }

        if(x != -1) {
            int y = x;
            for (int i = 0; i < n; ++i)
```

```
                y = p[y];

            vector<int> path;
            for (int cur = y;; cur = p[cur]) {
                path.push_back(cur);
                if (cur == y && path.size() > 1)
                break;
            }
            reverse(path.begin(), path.end());

            cout << "YES\n";
            for (int u : path)
               cout << u + 1 << ' ';
            return;
        }

    }

    cout << "NO\n";
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

3.7 De Bruijn Sequence

Your task is to construct a minimum-length bit string that contains all possible substrings of length n. For example, when $n = 2$, the string 00110 is a valid solution, because its substrings of length 2 are 00, 01, 10 and 11.

Input
The only input line has an integer n.

Output
Print a minimum-length bit string that contains all substrings of length n. You can print any valid solution.

Constraints
• $1 \le n \le 15$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define int long long

using namespace std;

bool mark[1 << 15];
string s;
int mod;

void dfs(int u) {
    s += '0' + (u % 2);
    mark[u] = 1; int v = (u << 1) % mod;
    if(!mark[v + 1]) dfs(v + 1);
    else if(!mark[v]) dfs(v);
}

void solve(){
    int n; cin >> n;
    mod = 1 << n;
    s.resize(n - 1, '0');
    dfs(0);
    cout << s << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

3.8 Distinct Routes

A game consists of n rooms and m teleporters. At the beginning of each day, you start in room 1 and you have to reach room n. You can use each teleporter at most once during the game. How many days can you play if you choose your routes optimally?

Input
The first input line has two integers n and m: the number of rooms and teleporters. The rooms are numbered $1, 2, \ldots, n$. After this, there are m lines describing the teleporters. Each line has two integers a and b: there is a teleporter from room a to room b. There are no two teleporters whose starting and ending room are the same.

Output
First print an integer k: the maximum number of days you can play the game. Then, print k route descriptions according to the example. You can print any valid solution.

Constraints
• $2 \le n \le 500$ • $1 \le m \le 1000$ • $1 \le a, b \le n$

Código-fonte:

```
#include <bits/stdc++.h>
using namespace std;
#define pb push_back
#define pc __builtin_popcount
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

int n, m;

struct Dinic {
    struct Edge {
        int to, rev;
        ll c, oc;
        ll flow() { return max(oc - c, 0LL); } // if you need
            flows
    };
    vi lvl, ptr, q;
    vector<vector<Edge>> adj;
    Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
    void addEdge(int a, int b, ll c, ll rcap = 0) {
        adj[a].push_back({b, sz(adj[b]), c, c});
        adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
    }
    ll dfs(int v, int t, ll f) {
        if (v == t || !f) return f;
        for (int& i = ptr[v]; i < sz(adj[v]); i++) {
            Edge& e = adj[v][i];
            if (lvl[e.to] == lvl[v] + 1)
                if (ll p = dfs(e.to, t, min(f, e.c))) {
                    e.c -= p, adj[e.to][e.rev].c += p;
                    return p;
                }
        }
        return 0;
    }
    ll calc(int s, int t) {
        ll flow = 0; q[0] = s;
        rep(L,0,31) do { // 'int L=30' maybe faster for
            random data
            lvl = ptr = vi(sz(q));
            int qi = 0, qe = lvl[s] = 1;
            while (qi < qe && !lvl[t]) {
                int v = q[qi++];
                for (Edge e : adj[v])
                    if (!lvl[e.to] && e.c >> (30 - L))
                        q[qe++] = e.to, lvl[e.to] = lvl[v] +
                            1;
            }
            while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
        } while (lvl[t]);
        return flow;
    }
    bool leftOfMinCut(int a) { return lvl[a] != 0; }
};


void solve() {
    cin >> n >> m;
    Dinic G(n);
    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G.addEdge(a, b, 1);
    }

    cout << G.calc(0, n - 1) << '\n';
    int ini = 0;
    for(auto& e : G.adj[ini]) {
        if(e.flow() == 0) continue;
        e.c = e.oc;
        vector<int> pth; pth.push_back(0); pth.push_back(e.to
            );
        int at = e.to;
        while(at != n - 1) {
            for(auto& e1 : G.adj[at]) {
                if(e1.flow() == 0) continue;
                e1.c = e1.oc;
                at = e1.to;
                pth.push_back(at);
                break;
            }
        }
        cout << pth.size() << '\n';
        for(auto x : pth) cout << x + 1 << ' ';
        cout << '\n';
    }
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

3.9 Download Speed

Consider a network consisting of n computers and m connections. Each connection specifies how fast a computer can send data to another computer. Kotivalo wants to download some data from a server. What is the maximum speed he can do this, using the connections in the network?

Input
The first input line has two integers n and m: the number of computers and connections. The computers are numbered $1, 2, \ldots, n$. Computer 1 is the server and computer n is Kotivalo's computer. After this, there are m lines describing the connections. Each line has three integers a, b and c: computer a can send data to computer b at speed c.

Output
Print one integer: the maximum speed Kotivalo can download data.

Constraints
• $1 \le n \le 500$ • $1 \le m \le 1000$ • $1 \le a, b \le n$ • $1 \le c \le 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
using namespace std;
#define pb push_back
#define pc __builtin_popcount
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;


const int MAX = 1e5 + 5;
int n, m;

struct Dinic {
    struct Edge {
        int to, rev;
        ll c, oc;
        ll flow() { return max(oc - c, 0LL); } // if you need flows
    };
    vi lvl, ptr, q;
    vector<vector<Edge>> adj;
    Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
    void addEdge(int a, int b, ll c, ll rcap = 0) {
        adj[a].push_back({b, sz(adj[b]), c, c});
        adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
    }
    ll dfs(int v, int t, ll f) {
        if (v == t || !f) return f;
        for (int& i = ptr[v]; i < sz(adj[v]); i++) {
            Edge& e = adj[v][i];
            if (lvl[e.to] == lvl[v] + 1)
                if (ll p = dfs(e.to, t, min(f, e.c))) {
                    e.c -= p, adj[e.to][e.rev].c += p;
                    return p;
                }
        }
        return 0;
    }
    ll calc(int s, int t) {
        ll flow = 0; q[0] = s;
        rep(L,0,31) do { // 'int L=30' maybe faster for random data
            lvl = ptr = vi(sz(q));
            int qi = 0, qe = lvl[s] = 1;
            while (qi < qe && !lvl[t]) {
                int v = q[qi++];
                for (Edge e : adj[v])
                    if (!lvl[e.to] && e.c >> (30 - L))
                        q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
            }
            while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
        } while (lvl[t]);
        return flow;
    }
    bool leftOfMinCut(int a) { return lvl[a] != 0; }
};


void solve() {
    cin >> n >> m;
    Dinic G(n);
    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        int c; cin >> c;
        G.addEdge(a, b, c);
    }

    cout << G.calc(0, n - 1) << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

3.10 Flight Discount

Your task is to find a minimum-price flight route from Syrjälä to Metsälä. You have one discount coupon, using which you can halve the price of any single flight during the route. However, you can only use the coupon once. When you use the discount coupon for a flight whose price is x, its price becomes $\lfloor x/2 \rfloor$ (it is rounded down to an integer).

Input
The first input line has two integers n and m: the number of cities and flight connections. The cities are numbered $1, 2, \ldots, n$. City 1 is Syrjälä, and city n is Metsälä. After this there are m lines describing the flights. Each line has three integers a, b, and c: a flight begins at city a, ends at city b, and its price is c. Each flight is unidirectional. You can assume that it is always possible to get from Syrjälä to Metsälä.

Output
Print one integer: the price of the cheapest route from Syrjälä to Metsälä.

Constraints
• $2 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$ • $1 \le c \le 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define int long long

using namespace std;

const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

const int INF = 1000000000000000000;
vector<vector<pair<int, int>>> adj;

void dijkstra(int s, vector<int> & d) {
    int n = adj.size();
    d.assign(2 * n, INF);

    d[s] = 0;
    set<pair<int, int>> q;
    q.insert({0, s});
    while (!q.empty()) {
        int v = q.begin()->second;
        q.erase(q.begin());

        if(v >= n) {
            for (auto edge : adj[v - n]) {
                int to = edge.first + n;
                int len = edge.second;

                if (d[v] + len < d[to]) {
                    q.erase({d[to], to});
                    d[to] = d[v] + len;
                    q.insert({d[to], to});
                }
            }
        }
        else {
            for (auto edge : adj[v]) {
                int to = edge.first;
                int len = edge.second;
```

```
41
42                  if (d[v] + len < d[to]) {
43                      q.erase({d[to], to});
44                      d[to] = d[v] + len;
45                      q.insert({d[to], to});
46                  }
47                  to = to + n;
48                  len /= 2;
49                  if (d[v] + len < d[to]) {
50                      q.erase({d[to], to});
51                      d[to] = d[v] + len;
52                      q.insert({d[to], to});
53                  }
54              }
55          }
56      }
57  }
58
59  signed main() {
60      ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
61      int n, m; cin >> n >> m;
62      adj.resize(n);
63
64      for(int i = 0; i < m; i++){
65          int a, b, c; cin >> a >> b >> c;
66          adj[a - 1].push_back({b - 1, c});
67      }
68
69      vector<int> d;
70      dijkstra(0, d);
71
72      cout << d[2 * n - 1] << '\n';
73
74      return 0;
75  }
```

3.11 Flight Routes

Your task is to find the k shortest flight routes from Syrjälä to Metsälä. A route can visit the same city several times. Note that there can be several routes with the same price and each of them should be considered (see the example).

Input
The first input line has three integers n, m, and k: the number of cities, the number of flights, and the parameter k. The cities are numbered $1, 2, \ldots, n$. City 1 is Syrjälä, and city n is Metsälä. After this, the input has m lines describing the flights. Each line has three integers a, b, and c: a flight begins at city a, ends at city b, and its price is c. All flights are one-way flights. You may assume that there are at least k distinct routes from Syrjälä to Metsälä.

Output
Print k integers: the prices of the k cheapest routes sorted according to their prices.

Constraints
• $2 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$ • $1 \le c \le 10^9$ • $1 \le k \le 10$

Código-fonte:

```
1  #include <bits/stdc++.h>
2  #define pb push_back
3  #define all(x) x.begin(), x.end()
4  #define pc __builtin_popcount
5  #define int long long
6
7  using namespace std;
8
9  const int maxn = 1010, inf = 2e9, M = 1e9 + 7;
10
11 const int INF = 1000000000000000000;
12
13 vector<vector<pair<int, int>>> adj;
14 vector<priority_queue<int>> d;
15
16 void dijkstra(int s) {
17     int n = adj.size();
18
19     d.assign(n, priority_queue<int>());
20     d[s].push(0);
21
22     priority_queue<pair<int, int>, vector<pair<int, int>>,
           greater<pair<int, int>>> q;
23     q.push({0, s});
24
25     while (!q.empty()) {
26         int tot = q.top().first;
27         int v = q.top().second;
28         q.pop();
29
30         if (d[v].size() == 10 && tot > d[v].top()) continue;
31
32         for (auto edge : adj[v]) {
33             int to = edge.first;
34             int len = edge.second;
35             int new_dist = tot + len;
36
37             if (d[to].size() < 10) {
38                 d[to].push(new_dist);
39                 q.push({new_dist, to});
40             }
41             else if(d[to].top() > new_dist) {
42                 d[to].pop();
43                 d[to].push(new_dist);
44                 q.push({new_dist, to});
45             }
46         }
47     }
48 }
49
50 signed main() {
51     ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
52     int n, m, k; cin >> n >> m >> k;
53     adj.resize(n);
54
55     for(int i = 0; i < m; i++){
56         int a, b, c; cin >> a >> b >> c;
57         a--; b--;
58         adj[a].push_back({b, c});
59     }
60     dijkstra(0);
61
62     while(d[n - 1].size() > k) d[n - 1].pop();
63     vector<int> ans;
64     for(int i = 0; i < k; i++){
65         ans.push_back(d[n - 1].top());
66         d[n - 1].pop();
67     }
68     reverse(ans.begin(), ans.end());
69     for(auto x : ans) cout << x << ' ';
70     cout << '\n';
71
72     return 0;
73 }
```

3.12 Flight Routes Check

There are n cities and m flight connections. Your task is to check if you can travel from any city to any other city using the available flights.

Input
The first input line has two integers n and m: the number of cities and flights. The cities are numbered $1, 2, \ldots, n$. After this, there are m lines describing the flights. Each line has two integers a and b: there is a flight from city a to city b. All flights are one-way flights.

Output
Print "YES" if all routes are possible, and "NO" otherwise. In the latter case also print two cities a and b such that you cannot travel from city a to city b. If there are several possible solutions, you can print any of them.

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

Código-fonte:

```
1  #include <bits/stdc++.h>
2  #define pb push_back
3  #define all(x) x.begin(), x.end()
4  #define pc __builtin_popcount
5  #define F first
6  #define S second
7
8  #define int long long
9
10 using namespace std;
11
12 const int MAX = 1e5+5;
13
14 int n, m;
15
16 vector<int> g[MAX];
17 vector<int> gi[MAX]; // grafo invertido
18 int vis[MAX];
19 stack<int> S;
20 int comp[MAX]; // componente conexo de cada vertice
21
22 void dfs(int k) {
23     vis[k] = 1;
24     for (int i = 0; i < (int) g[k].size(); i++)
25         if (!vis[g[k][i]]) dfs(g[k][i]);
26
27     S.push(k);
28 }
29
30 void scc(int k, int c) {
31     vis[k] = 1;
32     comp[k] = c;
33     for (int i = 0; i < (int) gi[k].size(); i++)
34         if (!vis[gi[k][i]]) scc(gi[k][i], c);
35 }
36
37 void kosaraju() {
38     for (int i = 0; i < n; i++) vis[i] = 0;
39     for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);
40
41     for (int i = 0; i < n; i++) vis[i] = 0;
42     while (S.size()) {
```

```
        int u = S.top();
        S.pop();
        if (!vis[u]) scc(u, u);
    }
}

int vischeck[MAX];
void dfscheck(int k) {
    vischeck[k] = 1;
    for (int i = 0; i < (int) g[k].size(); i++)
        if (!vischeck[g[k][i]]) dfscheck(g[k][i]);
}

void solve(){
    cin >> n >> m;

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        g[a].push_back(b);
        gi[b].push_back(a);
    }

    kosaraju();

    bool flag = false;
    for(int i = 0; i < n; i++) {
        if(comp[i] != comp[0]) {
            cout << "NO\n";
            dfscheck(0);
            if(vischeck[i]) cout << i + 1 << ' ' << 1 << '\n'
                ;
            else cout << 1 << ' ' << i + 1 << '\n';

            flag = true;
            break;
        }
    }



    if(!flag) cout << "YES\n";

}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

3.13 Game Routes

A game has n levels, connected by m teleporters, and your task is to get from level 1 to level n. The game has been designed so that there are no directed cycles in the underlying graph. In how many ways can you complete the game?

Input
The first input line has two integers n and m: the number of levels and teleporters. The levels are numbered $1, 2, \ldots, n$. After this, there are m lines describing the teleporters. Each line has two integers a and b: there is a teleporter from level a to level b.

Output

Print one integer: the number of ways you can complete the game. Since the result may be large, print it modulo $10^9 + 7$.

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define int long long

using namespace std;

const int INF = 1000000000000000000;
const int MAX = 100010;
const int M  = 1e9 + 7;

vector<int> g[MAX];

vector<int> topo_sort(int n) {
    vector<int> ret(n,-1), vis(n,0);

    int pos = n-1, dag = 1;
    function<void(int)> dfs = [&](int v) {
        vis[v] = 1;
        for (auto u : g[v]) {
            if (vis[u] == 1) dag = 0;
            else if (!vis[u]) dfs(u);
        }
        ret[pos--] = v, vis[v] = 2;
    };

    for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);

    if (!dag) ret.clear();
    return ret;
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int n, m; cin >> n >> m;

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        a--; b--;
        g[a].push_back(b);
    }

    vector<int> ans = topo_sort(n);
    vector<int> tot(n, 0);

    int ini = 0; tot[0] = 1;

    int at = 0;
    for(int i = 0; i < ans.size(); i++)
        if(ans[i] == ini) at = i;

    for(int i = at; i < n; i++) {
        int at = ans[i];
        for(auto x : g[at]) {
            tot[x] = (tot[x] + tot[at]) % M;
        }
    }

    cout << tot[n - 1] << '\n';

    return 0;
}
```

3.14 Giant Pizza

Uolevi's family is going to order a large pizza and eat it together. A total of n family members will join the order, and there are m possible toppings. The pizza may have any number of toppings. Each family member gives two wishes concerning the toppings of the pizza. The wishes are of the form "topping x is good/bad". Your task is to choose the toppings so that at least one wish from everybody becomes true (a good topping is included in the pizza or a bad topping is not included).

Input
The first input line has two integers n and m: the number of family members and toppings. The toppings are numbered $1, 2, \ldots, m$. After this, there are n lines describing the wishes. Each line has two wishes of the form "+ x" (topping x is good) or "- x" (topping x is bad).

Output
Print a line with m symbols: for each topping "+" if it is included and "-" if it is not included. You can print any valid solution. If there are no valid solutions, print "IMPOSSIBLE".

Constraints
• $1 \le n, m \le 10^5$ • $1 \le x \le m$

Código-fonte:

```
#include <bits/stdc++.h>
using namespace std;

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

struct TwoSat {
    int N;
    vector<vi> gr;
    vi values; // 0 = false, 1 = true

    TwoSat(int n = 0) : N(n), gr(2*n) {}

    void either(int f, int j) {
        f = max(2*f, -1-2*f);
        j = max(2*j, -1-2*j);
        gr[f].push_back(j^1);
        gr[j].push_back(f^1);
    }
    void setValue(int x) { either(x, x); }

    vi val, comp, z; int time = 0;
    int dfs(int i) {
        int low = val[i] = ++time, x; z.push_back(i);
        for(int e : gr[i]) if (!comp[e])
            low = min(low, val[e] ?: dfs(e));
        if (low == val[i]) do {
            x = z.back(); z.pop_back();
            comp[x] = low;
            if (values[x>>1] == -1)
                values[x>>1] = x&1;
        } while (x != i);
```

```
        return val[i] = low;
    }

    bool solve() {
        values.assign(N, -1);
        val.assign(2*N, 0); comp = val;
        rep(i,0,2*N) if (!comp[i]) dfs(i);
        rep(i,0,N) if (comp[2*i] == comp[2*i+1]) return 0;
        return 1;
    }
};

void solve(){
    int n, m; cin >> n >> m;

    TwoSat ts(m);

    for(int i = 0; i < n; i++) {
        char c; int a; cin >> c >> a; a--;
        char d; int b; cin >> d >> b; b--;
        if(c == '-') a = ~a;
        if(d == '-') b = ~b;
        ts.either(a, b);
    }

    if(!ts.solve()) {
        cout << "IMPOSSIBLE\n";
        return;
    }

    for(int i = 0; i < m; i++) {
        if(ts.values[i]) cout << "+ ";
        else cout << "- ";
    }
    cout << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

3.15 Hamiltonian Flights

There are n cities and m flight connections between them. You want to travel from Syrjälä to Lehmälä so that you visit each city exactly once. How many possible routes are there?

Input
The first input line has two integers n and m: the number of cities and flights. The cities are numbered $1, 2, \ldots, n$. City 1 is Syrjälä, and city n is Lehmälä. Then, there are m lines describing the flights. Each line has two integers a and b: there is a flight from city a to city b. All flights are one-way flights.

Output
Print one integer: the number of routes modulo $10^9 + 7$.

Constraints
• $2 \le n \le 20$ • $1 \le m \le n^2$ • $1 \le a, b \le n$

Código-fonte:

```
#pragma GCC target ("avx2")
#pragma GCC optimize ("O3")
#pragma GCC optimize ("unroll-loops")

#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

typedef long long ll;

using namespace std;

const int MAX = 20;
const int MOD = 1e9 + 7;

int dp[MAX][1 << MAX];
int adj[MAX][MAX];

void calc(int n) {
    dp[0][1] = 1;
    for(int b = 0; b < (1 << (n - 1)); b++) {
        for(int at = 0; at < n - 1; at++) {
            int pot = 1 << at;
            if(!(pot & b)) continue;
            for(int nxt = 0; nxt < n - 1; nxt++) {
                int pot2 = 1 << nxt;
                if(pot2 & b || !adj[at][nxt]) continue;
                dp[nxt][b | pot2] = ((ll)dp[nxt][b | pot2] +
                    (ll) dp[at][b] * adj[at][nxt]) % MOD;
            }
        }
    }
}

void solve() {
    int n, m; cin >> n >> m;
    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        adj[a - 1][b - 1]++;
    }

    calc(n);

    int ans = 0;

    for(int at = 0; at < n - 1; at++) {
        if(!adj[at][n - 1]) continue;
        ans = ((ll)ans + (ll) dp[at][(1 << (n - 1)) - 1] *
            adj[at][n - 1]) % MOD;
    }

    cout << ans << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

3.16 High Score

You play a game consisting of n rooms and m tunnels. Your initial score is 0, and each tunnel increases your score by x where x may be both positive or negative. You may go through a tunnel several times. Your task is to walk from room 1 to room n. What is the maximum score you can get?

Input
The first input line has two integers n and m: the number of rooms and tunnels. The rooms are numbered $1, 2, \ldots, n$. Then, there are m lines describing the tunnels. Each line has three integers a, b and x: the tunnel starts at room a, ends at room b, and it increases your score by x. All tunnels are one-way tunnels. You can assume that it is possible to get from room 1 to room n.

Output
Print one integer: the maximum score you can get. However, if you can get an arbitrarily large score, print -1.

Constraints
• $1 \le n \le 2500$ • $1 \le m \le 5000$ • $1 \le a, b \le n$ • $-10^9 \le x \le 10^9$

Código-fonte:

```
#include <bits/stdc++.h>
using namespace std;

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int) (x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const ll inf=LLONG_MAX;
struct Ed{int a,b,w,s(){return a<b?a:-a;}};
struct Node{ll dist=inf; int prev=-1;};

void bellmanFord(vector<Node>&nodes,vector<Ed>&eds, int s){
    nodes[s].dist=0;
    sort(all(eds),[](Ed a,Ed b){return a.s()<b.s();});

    int lim=sz(nodes)/2+2;
    rep(i,0,lim) for (Ed ed:eds){
        Node cur=nodes[ed.a],&dest=nodes[ed.b];
        if (abs(cur.dist)==inf) continue;
        ll d=cur.dist+ed.w;
        if (d<dest.dist){
            dest.prev=ed.a;
            dest.dist=(i<lim-1?d:-inf);
        }
    }
    rep(i,0,lim) for(Ed e:eds){
        if (nodes[e.a].dist==-inf)
            nodes[e.b].dist=-inf;
    }
}

void solve() {
    int n, m; cin >> n >> m;
    vector<Node> nodes(n);
    vector<Ed> edges(m);
    for(int i = 0; i < m; i++) {
        int a, b, w; cin >> a >> b >> w;
        edges[i] = {a - 1, b - 1, -w};
    }
    bellmanFord(nodes, edges, 0);
    if(nodes[n - 1].dist == -inf) cout << -1 << '\n';
    else cout << -nodes[n - 1].dist << '\n';
}

signed main() {
```

```
    ios::sync_with_stdio(0); cin.tie(0);

    solve();

    return 0;
}
```

3.17 Investigation

You are going to travel from Syrjälä to Lehmälä by plane. You would like to find answers to the following questions:
• what is the minimum price of such a route? • how many minimum-price routes are there? (modulo $10^9 + 7$) • what is the minimum number of flights in a minimum-price route? • what is the maximum number of flights in a minimum-price route?

Input
The first input line contains two integers n and m: the number of cities and the number of flights. The cities are numbered $1, 2, \ldots, n$. City 1 is Syrjälä, and city n is Lehmälä. After this, there are m lines describing the flights. Each line has three integers a, b, and c: there is a flight from city a to city b with price c. All flights are one-way flights. You may assume that there is a route from Syrjälä to Lehmälä.

Output
Print four integers according to the problem statement.

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$ • $1 \le c \le 10^9$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define int long long

using namespace std;

const int INF = 1000000000000000000;
const int M = 1e9 + 7;

struct Node {
    int mn = INF;
    int cnt = 1;
    int mnsz = 1;
    int mxsz = 1;
};

vector<vector<pair<int, int>>> adj;
vector<Node> d;

void dijkstra(int s) {
    int n = adj.size();

    d.resize(n);
    d[s] = {0, 1, 1, 1};

    priority_queue<pair<int, int>, vector<pair<int, int>>,
        greater<pair<int, int>>> q;
    q.push({0, s});

    while (!q.empty()) {
        int len = q.top().first;
        int v = q.top().second;
        q.pop();

        if(len > d[v].mn) continue;

        for (auto edge : adj[v]) {
            int to = edge.first;
            int len = edge.second;
            int new_dist = d[v].mn + len;

            if(d[to].mn == new_dist) {
                d[to].cnt = (d[to].cnt + d[v].cnt) % M;
                d[to].mnsz = min(d[to].mnsz, d[v].mnsz + 1);
                d[to].mxsz = max(d[to].mxsz, d[v].mxsz + 1);
            }
            else if(d[to].mn > new_dist) {
                d[to].mn = new_dist;
                d[to].cnt = d[v].cnt;
                d[to].mnsz = d[v].mnsz + 1;
                d[to].mxsz = d[v].mxsz + 1;
                q.push({new_dist, to});
            }
        }
    }
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int n, m; cin >> n >> m;
    adj.resize(n);

    for(int i = 0; i < m; i++){
        int a, b, c; cin >> a >> b >> c;
        a--; b--;
        adj[a].push_back({b, c});
    }

    dijkstra(0);

    cout << d[n-1].mn << ' ' << d[n-1].cnt << ' ' << d[n-1].
         mnsz - 1 << ' ' << d[n-1].mxsz - 1 << '\n';

    return 0;
}
```

3.18 Knights Tour

Given a starting position of a knight on an 8×8 chessboard, your task is to find a sequence of moves such that it visits every square exactly once. On each move, the knight may either move two steps horizontally and one step vertically, or one step horizontally and two steps vertically.

Input
The only line has two integers x and y: the knight's starting position.

Output
Print a grid that shows how the knight moves (according to the example). You can print any valid solution.

Constraints
• $1 \le x, y \le 8$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

const int MAX = 8;

vector<vector<int>> ans = { {1, 16, 63, 22, 3, 18, 55, 46},
                            {40, 23, 2, 17, 58, 47, 4, 19},
                            {15, 64, 41, 62, 21, 54, 45, 56},
                            {24, 39, 32, 59, 48, 57, 20, 5},
                            {33, 14, 61, 42, 53, 30, 49, 44},
                            {38, 25, 36, 31, 60, 43, 6, 9},
                            {13, 34, 27, 52, 11, 8, 29, 50},
                            {26, 37, 12, 35, 28, 51, 10, 7}};

void solve() {
    int x, y; cin >> y >> x; int at = ans[x - 1][y - 1];

    for(int i = 0; i < MAX; i++) {
        for(int j = 0; j < MAX; j++) cout << (ans[i][j] - at
            + MAX * MAX) % (MAX * MAX) + 1 << ' ';
        cout << '\n';
    }

}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

3.19 Labyrinth

You are given a map of a labyrinth, and your task is to find a path from start to end. You can walk left, right, up and down.

Input
The first input line has two integers n and m: the height and width of the map. Then there are n lines of m characters describing the labyrinth. Each character is . (floor), # (wall), A (start), or B (end). There is exactly one A and one B in the input.

Output
First print "YES", if there is a path, and "NO" otherwise. If there is a path, print the length of the shortest such path and its description as a string consisting of characters L (left), R (right), U (up), and D (down). You can print any valid solution.

Constraints
• $1 \le n, m \le 1000$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long int  ll;
typedef pair<int, int> pii;

const int maxn = 1e3+5, inf = 2e9, M = 1e9 + 7;
const ll linf = 1e18;

vector<pii> macaco = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
vector<char> pos = {'D', 'U', 'R', 'L'};
vector<char> ans;
char tab[maxn][maxn];
int check[maxn][maxn];
pii pai[maxn][maxn];
int n, m;

template <typename T,typename U>
pair<T,U> operator+(const pair<T,U> & l,const pair<T,U> & r) {
    return {l.first+r.first,l.second+r.second};
}
template <typename T,typename U>
pair<T,U> operator-(const pair<T,U> & l,const pair<T,U> & r) {
    return {l.first-r.first,l.second-r.second};
}

bool bfs(pii A, pii B){
    queue<pii> fila;

    fila.push(A);

    while(!fila.empty()){
        pii X = fila.front();
        fila.pop();

        if(X == B) {
            break;
        }

        for(int i = 0; i < 4; i++){
            pii N = X + macaco[i];
            if(check[N.F][N.S] != 0){
                fila.push(N);
                check[N.F][N.S] = 0;
                pai[N.F][N.S] = X;
            }
        }
    }

    if(check[B.F][B.S] != 0) return false;

    pii X = B;
    while(X != A){
        for(int i = 0; i < 4; i++){
            if(macaco[i] == X - pai[X.F][X.S])
                ans.pb(pos[i]);
        }
        X = {pai[X.F][X.S]};
    }
    reverse(all(ans));
    return true;
}

void solve(){
    cin >> n >> m;
    pii A, B;

    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= m; j++){
            cin >> tab[i][j];
            if(tab[i][j] != '#') check[i][j] = 1;
            if(tab[i][j] == 'A') A = {i, j};
            if(tab[i][j] == 'B') B = {i, j};
        }
    }

    if(!bfs(A, B)) cout << "NO\n";
    else{
        cout << "YES\n";
        cout << ans.size() << '\n';
        for(char x : ans) cout << x;
        cout << '\n';
    }
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

3.20 Longest Flight Route

Uolevi has won a contest, and the prize is a free flight trip that can consist of one or more flights through cities. Of course, Uolevi wants to choose a trip that has as many cities as possible. Uolevi wants to fly from Syrjälä to Lehmälä so that he visits the maximum number of cities. You are given the list of possible flights, and you know that there are no directed cycles in the flight network.

Input
The first input line has two integers n and m: the number of cities and flights. The cities are numbered $1, 2, \ldots, n$. City 1 is Syrjälä, and city n is Lehmälä. After this, there are m lines describing the flights. Each line has two integers a and b: there is a flight from city a to city b. Each flight is a one-way flight.

Output
First print the maximum number of cities on the route. After this, print the cities in the order they will be visited. You can print any valid solution. If there are no solutions, print "IMPOSSIBLE".

Constraints
• $2 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define int long long

using namespace std;

const int INF = 1000000000000000000;
const int MAX = 100010;

vector<int> g[MAX];

vector<int> topo_sort(int n) {
    vector<int> ret(n,-1), vis(n,0);

    int pos = n-1, dag = 1;
    function<void(int)> dfs = [&](int v) {
        vis[v] = 1;
        for (auto u : g[v]) {
            if (vis[u] == 1) dag = 0;
            else if (!vis[u]) dfs(u);
        }
        ret[pos--] = v, vis[v] = 2;
    };

    for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);

    if (!dag) ret.clear();
    return ret;
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int n, m; cin >> n >> m;

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        a--; b--;
        g[a].push_back(b);
    }

    vector<int> ans = topo_sort(n);
    vector<int> dist(n, -1), pai(n, -1);

    int ini = 0;
    dist[0] = 1;

    int at = 0;
    for(int i = 0; i < ans.size(); i++)
        if(ans[i] == ini) at = i;

    for(int i = at; i < n; i++) {
        int at = ans[i];
        for(auto x : g[at]) {
            if(dist[x] < dist[at] + 1) {
                dist[x] = dist[at] + 1;
                pai[x] = at;
            }
        }
    }

    vector<int> res;

    if(dist[n - 1] == -1) {
        cout << "IMPOSSIBLE\n";
        return 0;
    }

    for(int i = n - 1; i != 0; i = pai[i]) {
        res.push_back(i + 1);
    }
    res.push_back(1); reverse(res.begin(), res.end());

    cout << res.size() << '\n';
    for(auto x : res) cout << x << ' ';
    cout << '\n';

    return 0;
}
```

3.21 Mail Delivery

Your task is to deliver mail to the inhabitants of a city. For this reason, you want to find a route whose starting and ending point are the post office, and that goes through every street exactly once.

Input
The first input line has two integers n and m: the number of crossings and streets. The crossings are numbered $1, 2, \ldots, n$, and the post office is located at crossing 1. After that, there are m lines describing the streets. Each line has two integers a and b: there is a street between crossings a and b. All streets are two-way streets. Every street is between two different crossings, and there is at most one street between two crossings.

Output
Print all the crossings on the route in the order you will visit them. You can print any valid solution. If there are no solutions, print "IMPOSSIBLE".

Constraints
$2 \le n \le 10^5$ $1 \le m \le 2 \cdot 10^5$ $1 \le a, b \le n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
using namespace std;

#define int long long

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 1e5 + 5;

template<bool directed=false> struct euler {
    int n;
    vector<vector<pair<int, int>>> g;
    vector<int> used;

    euler(int n_) : n(n_), g(n) {}
    void add(int a, int b) {
        int at = used.size();
        used.push_back(0);
        g[a].emplace_back(b, at);
        if (!directed) g[b].emplace_back(a, at);
    }
    pair<bool, vector<pair<int, int>>> get_path(int src) {
        if (!used.size()) return {true, {}};
        vector<int> beg(n, 0);
        for (int& i : used) i = 0;
        // {{vertice, anterior}, label}
        vector<pair<pair<int, int>, int>> ret, st = {{{src,
             -1}, -1}};
        while (st.size()) {
            int at = st.back().first.first;
            int& it = beg[at];
            while (it < g[at].size() and used[g[at][it].
                 second]) it++;
            if (it == g[at].size()) {
                if (ret.size() and ret.back().first.second !=
                     at)
                    return {false, {}};
                ret.push_back(st.back()), st.pop_back();
            } else {
                st.push_back({{g[at][it].first, at}, g[at][it
                    ].second});
                used[g[at][it].second] = 1;
            }
        }
        if (ret.size() != used.size()+1) return {false, {}};
        vector<pair<int, int>> ans;
        for (auto i : ret) ans.emplace_back(i.first.first, i.
            second);
        reverse(ans.begin(), ans.end());
        return {true, ans};
    }
    pair<bool, vector<pair<int, int>>> get_cycle() {
        if (!used.size()) return {true, {}};
        int src = 0;
        while (!g[src].size()) src++;
        auto ans = get_path(src);
        if (!ans.first or ans.second[0].first != ans.second.
            back().first)
            return {false, {}};
        ans.second[0].second = ans.second.back().second;
        ans.second.pop_back();
        return ans;
    }
};

void solve(){
    int n, m;
    cin >> n >> m;

    euler g(n);

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        g.add(a, b);
    }

    auto [flag, ans] = g.get_path(0);

    if(!flag || ans[0].first != ans.back().first) cout << "
        IMPOSSIBLE\n";
    else {
        for(auto x : ans) cout << x.first + 1 << ' ';
    }
    cout << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

3.22 Message Route

Syrjälä's network has n computers and m connections. Your task is to find out if Uolevi can send a message to Maija, and if it is possible, what is the minimum number of computers on such a route.

Input
The first input line has two integers n and m: the number of computers and connections. The computers are numbered $1, 2, \ldots, n$. Uolevi's computer is 1 and Maija's computer is n. Then, there are m lines describing the connections. Each line has two integers a and b: there is a connection between those computers. Every connection is between two different computers, and there is at most one connection between any two computers.

Output
If it is possible to send a message, first print k: the minimum number of computers on a valid route. After this, print an example of such a route. You can print any valid solution. If there are no routes, print "IMPOSSIBLE".

Constraints
• $2 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long int  ll;
typedef pair<int, int> pii;
typedef vector<int> vi;
typedef vector<vi> graph;

const int maxn = 1e3+5, inf = 2e9, M = 1e9 + 7;
const ll linf = 1e18;
int n, m;
vector<int> ans;
graph G;
vi check;
vi pai;

bool bfs(int a, int b){
    queue<int> fila;
    fila.push(a);
    check[a] = 1;
    pai[a] = 0;

    while(!fila.empty()){
        int x = fila.front();
        fila.pop();

        if(x == b) {
            break;
        }

        for(auto y : G[x]){
            if(check[y] != 1){
                fila.push(y);
                check[y] = 1;
                pai[y] = x;
            }
        }
    }

    if(check[b] != 1) return false;
    return true;
}
```

```

void solve(){
    cin >> n >> m;
    G.resize(n + 1);
    check.resize(n + 1);
    pai.resize(n + 1);

    for(int i = 1; i <= m; i++){
        int a, b; cin >> a >> b;
        G[a].pb(b); G[b].pb(a);
    }

    if(!bfs(1, n)){
        cout << "IMPOSSIBLE\n";
        return;
    }

    int x = n;
    ans.pb(n);

    while(pai[x] != 0){
        ans.pb(pai[x]);
        x = pai[x];
    }
    reverse(all(ans));

    cout << ans.size() << '\n';
    for(int i = 0; i < ans.size(); i++){
        cout << ans[i] << ' ';
    }
    cout << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

3.23 Monsters

You and some monsters are in a labyrinth. When taking a step to some direction in the labyrinth, each monster may simultaneously take one as well. Your goal is to reach one of the boundary squares without ever sharing a square with a monster. Your task is to find out if your goal is possible, and if it is, print a path that you can follow. Your plan has to work in any situation; even if the monsters know your path beforehand.

Input
The first input line has two integers n and m: the height and width of the map. After this there are n lines of m characters describing the map. Each character is . (floor), # (wall), A (start), or M (monster). There is exactly one A in the input.

Output
First print "YES" if your goal is possible, and "NO" otherwise. If your goal is possible, also print an example of a valid path (the length of the path and its description using characters D, U, L, and R). You can print any path, as long as its length is at most $n \cdot m$ steps.

Constraints

- $1 \le n, m \le 1000$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long int  ll;
typedef pair<int, int> pii;

const int maxn = 1e3+5, inf = 2e9, M = 1e9 + 7;
const ll linf = 1e18;

vector<pii> macaco = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
vector<char> pos = {'D', 'U', 'R', 'L'};
vector<pii> ans;
char tab[maxn][maxn];
pii pai[maxn][maxn];
int n, m;
pii A;
vector<pii> Monster;

template <typename T,typename U>
pair<T,U> operator+(const pair<T,U> & l,const pair<T,U> & r) {
    return {l.first+r.first,l.second+r.second};
}
template <typename T,typename U>
pair<T,U> operator-(const pair<T,U> & l,const pair<T,U> & r) {
    return {l.first-r.first,l.second-r.second};
}

pii bfs(){
    queue<pii> fila;

    for(auto x : Monster){
        fila.push(x);
        pai[x.F][x.S] = {0, 0};
    }
    fila.push(A);
    pai[A.F][A.S] = {0, 0};

    while(!fila.empty()){
        pii X = fila.front();
        fila.pop();

        for(int i = 0; i < 4; i++){
            pii N = X + macaco[i];
            if(tab[N.F][N.S] == '.'){
                fila.push(N);
                tab[N.F][N.S] = tab[X.F][X.S];
                pai[N.F][N.S] = X;
            }
        }
    }

    pii ans = {0, 0};

    for(int i = 1; i <= n; i++){
        if(tab[i][1] == 'A') ans = {i, 1};
        if(tab[i][m] == 'A') ans = {i, m};
    }
    for(int i = 1; i <= m; i++){
        if(tab[1][i] == 'A') ans = {1, i};
        if(tab[n][i] == 'A') ans = {n, i};
    }
    return ans;
}

void solve(){
    cin >> n >> m;

    for(int i = 1; i <= n; i++){
        for(int j = 1; j <= m; j++){
            cin >> tab[i][j];
            if(tab[i][j] == 'A') A = {i, j};
            if(tab[i][j] == 'M') Monster.pb({i, j});
        }
    }

    pii res = bfs();

    if(res.F == 0){
        cout << "NO\n";
        return;
    }

    while(pai[res.F][res.S].F != 0){
        ans.pb(res - pai[res.F][res.S]);
        res = pai[res.F][res.S];
    }

    reverse(all(ans));

    cout << "YES\n";
    cout << ans.size() << '\n';
    for(pii x : ans){
        for(int i = 0; i < 4; i++){
            if(x == macaco[i]) cout << pos[i];
        }
    }
    cout << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

3.24 Planets Cycles

You are playing a game consisting of n planets. Each planet has a teleporter to another planet (or the planet itself). You start on a planet and then travel through teleporters until you reach a planet that you have already visited before. Your task is to calculate for each planet the number of teleportations there would be if you started on that planet.

Input
The first input line has an integer n: the number of planets. The planets are numbered $1, 2, \ldots, n$. The second line has n integers $t_1, t_2, \ldots, t_n$: for each planet, the destination of the teleporter. It is possible that $t_i = i$.

Output
Print n integers according to the problem statement.

Constraints

• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq t_i \leq n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define int long long

using namespace std;

const int MAX = 2e5 + 5, LOG = 30;

int n, q;
vector<int> g[MAX], gi[MAX], comp_elem[MAX], comp_graph_inv[MAX];
// root, dist_root
tuple<int, int> dist[MAX];
stack<int> S;
int vis[MAX], comp[MAX];

void dfs(int k) {
    vis[k] = 1;
    for (int i = 0; i < (int) g[k].size(); i++)
        if (!vis[g[k][i]]) dfs(g[k][i]);

    S.push(k);
}

void scc(int k, int c) {
    vis[k] = 1;
    comp[k] = c;
    for (int i = 0; i < (int) gi[k].size(); i++)
        if (!vis[gi[k][i]]) scc(gi[k][i], c);
}

void kosaraju() {
    for (int i = 0; i < n; i++) vis[i] = 0;
    for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);

    for (int i = 0; i < n; i++) vis[i] = 0;
    while (S.size()) {
        int u = S.top();
        S.pop();
        if (!vis[u]) scc(u, u);
    }
}

void dfs_calc(int s, int p) {
    dist[s] = {get<0>(dist[p]), get<1>(dist[p]) + 1};
    if(get<1>(dist[s]) == 1) get<0>(dist[s]) = g[s][0];

    for(auto x : comp_graph_inv[s]) dfs_calc(x, s);
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin >> n;

    for(int i = 0; i < n; i++) {
        int a; cin >> a;
        g[i].push_back(a - 1);
        gi[a - 1].push_back(i);
    }

    kosaraju();

    for(int i = 0; i < n; i++) {
        if (comp[i] == i) {
            int at = i, pos = 0;
            do {
                comp_elem[i].push_back(at);
                at = g[at][0];
            } while (at != i && comp[at] == i);
        }
    }

    for(int i = 0; i < n; i++)
        for(auto x : comp_elem[i])
            for(auto u : g[x])
                if(comp[u] != i) comp_graph_inv[comp[u]].push_back(i);

    for(int i = 0; i < n; i++) dist[i] = {comp[i], 0};
    for(int i = 0; i < n; i++) {
        if((g[i].size() == 1 && g[i][0] == i) || (comp_elem[comp[i]].size() > 1 && comp[i] == i)) {
            dist[i] = {i, -1};
            dfs_calc(i, i);
        }
    }

    for(int i = 0; i < n; i++) {
        auto d = dist[i];
        cout << get<1>(d) + comp_elem[comp[get<0>(d)]].size() << ' ';
    }
    cout << '\n';

    return 0;
}
```

3.25 Planets Queries I

You are playing a game consisting of n planets. Each planet has a teleporter to another planet (or the planet itself). Your task is to process q queries of the form: when you begin on planet x and travel through k teleporters, which planet will you reach?

Input
The first input line has two integers n and q: the number of planets and queries. The planets are numbered $1, 2, \ldots, n$. The second line has n integers $t_1, t_2, \ldots, t_n$: for each planet, the destination of the teleporter. It is possible that $t_i = i$. Finally, there are q lines describing the queries. Each line has two integers x and k: you start on planet x and travel through k teleporters.

Output
Print the answer to each query.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq t_i \leq n$ • $1 \leq x \leq n$ • $0 \leq k \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define int long long

using namespace std;

const int MAX = 2e5 + 5, LOG = 30;

int n, q;
int bin_lift[MAX][LOG];

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin >> n >> q;

    for(int i = 1; i <= n; i++) cin >> bin_lift[i][0];

    for(int i = 1; i < LOG; i++)
        for(int s = 1; s <= n; s++)
            bin_lift[s][i] = bin_lift[bin_lift[s][i-1]][i-1];

    while(q--) {
        int u, k; cin >> u >> k;
        int at = 0, dest = u;
        while(k > 0) {
            if(k % 2) dest = bin_lift[dest][at];
            at++; k/=2;
        }

        cout << dest << '\n';
    }


    return 0;
}
```

3.26 Planets Queries II

You are playing a game consisting of n planets. Each planet has a teleporter to another planet (or the planet itself). You have to process q queries of the form: You are now on planet a and want to reach planet b. What is the minimum number of teleportations?

Input
The first input line contains two integers n and q: the number of planets and queries. The planets are numbered $1, 2, \ldots, n$. The second line contains n integers $t_1, t_2, \ldots, t_n$: for each planet, the destination of the teleporter. Finally, there are q lines describing the queries. Each line has two integers a and b: you are now on planet a and want to reach planet b.

Output
For each query, print the minimum number of teleportations. If it is not possible to reach the destination, print -1.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $1 \le a, b \le n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define int long long

using namespace std;

const int MAX = 2e5 + 5, LOG = 30;

int n, q;
vector<int> g[MAX], gi[MAX], comp_elem[MAX], comp_graph[MAX],
      comp_graph_inv[MAX];
// root, dist_root, tin, tout
tuple<int, int, int, int> dist[MAX];
stack<int> S;
int vis[MAX], comp[MAX], comp_place[MAX];

void dfs(int k) {
    vis[k] = 1;
    for (int i = 0; i < (int) g[k].size(); i++)
        if (!vis[g[k][i]]) dfs(g[k][i]);

    S.push(k);
}

void scc(int k, int c) {
    vis[k] = 1;
    comp[k] = c;
    for (int i = 0; i < (int) gi[k].size(); i++)
        if (!vis[gi[k][i]]) scc(gi[k][i], c);
}

void kosaraju() {
    for (int i = 0; i < n; i++) vis[i] = 0;
    for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);

    for (int i = 0; i < n; i++) vis[i] = 0;
    while (S.size()) {
        int u = S.top();
        S.pop();
        if (!vis[u]) scc(u, u);
    }
}

int t = 0;
void dfs_calc(int s, int p) {
    dist[s] = {get<0>(dist[p]), get<1>(dist[p]) + 1, t, -1};
          t++;
    if(get<1>(dist[s]) == 1) get<0>(dist[s]) = g[s][0];

    for(auto x : comp_graph_inv[s]) dfs_calc(x, s);

    get<3>(dist[s]) = t; t++;
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin >> n >> q;

    for(int i = 0; i < n; i++) {
        int a; cin >> a;
        g[i].push_back(a - 1);
        gi[a - 1].push_back(i);
    }

    kosaraju();

    for(int i = 0; i < n; i++) {
        if (comp[i] == i) {
            int at = i, pos = 0;
            do {
                comp_elem[i].push_back(at);
                comp_place[at] = pos++;
                at = g[at][0];
            } while (at != i && comp[at] == i);
        }
    }

    for(int i = 0; i < n; i++)
        for(auto x : comp_elem[i])
            for(auto u : g[x])
                if(comp[u] != i) {
                    comp_graph[i].push_back(comp[u]);
                    comp_graph_inv[comp[u]].push_back(i);
                }

    for(int i = 0; i < n; i++) {
        if((g[i].size() == 1 && g[i][0] == i) || (comp_elem[
             comp[i]].size() > 1 && comp[i] == i)) {
            dist[i] = {i, -1, 0, 0};
            dfs_calc(i, i);
        }
    }

    for(int i = 0; i < q; i++) {
        int a, b; cin >> a >> b; a--; b--;
        auto da = dist[a], db = dist[b];
        if(comp[a] == comp[b]) {
            int ans = comp_place[b] - comp_place[a];
            if(ans < 0) ans += comp_elem[comp[a]].size();
            cout << ans << '\n';
        }
        else if(comp[get<0>(da)] == comp[b]) {
            int ans = comp_place[b] - comp_place[get<0>(da)];
            if(ans < 0) ans += comp_elem[comp[get<0>(da)]].
                  size();
            ans += get<1>(da);
            cout << ans << '\n';
        }
        else if(get<0>(da) == get<0>(db) && get<2>(da) > get
             <2>(db) && get<3>(da) < get<3>(db)) {
            cout << get<1>(da) - get<1>(db) << '\n';
        }
        else {
            cout << -1 << '\n';
        }
    }

    return 0;
}
```

3.27 Planetsand Kingdoms

A game has n planets, connected by m teleporters. Two planets a and b belong to the same kingdom exactly when there is a route both from a to b and from b to a. Your task is to determine for each planet its kingdom.

Input
The first input line has two integers n and m: the number of planets and teleporters. The planets are numbered $1, 2, \ldots, n$. After this, there are m lines describing the teleporters. Each line has two integers a and b: you can travel from planet a to planet b through a teleporter.

Output
First print an integer k: the number of kingdoms. After this, print for each planet a kingdom label between 1 and k. You can print any valid solution.

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define F first
#define S second

#define int long long

using namespace std;

const int MAX = 1e5+5;

int n, m;

vector<int> g[MAX];
vector<int> gi[MAX]; // grafo invertido
int vis[MAX];
stack<int> S;
int comp[MAX]; // componente conexo de cada vertice

void dfs(int k) {
    vis[k] = 1;
    for (int i = 0; i < (int) g[k].size(); i++)
        if (!vis[g[k][i]]) dfs(g[k][i]);

    S.push(k);
}

void scc(int k, int c) {
    vis[k] = 1;
    comp[k] = c;
    for (int i = 0; i < (int) gi[k].size(); i++)
```

```
        if (!vis[gi[k][i]]) scc(gi[k][i], c);
}

void kosaraju() {
    for (int i = 0; i < n; i++) vis[i] = 0;
    for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);

    for (int i = 0; i < n; i++) vis[i] = 0;
    while (S.size()) {
        int u = S.top();
        S.pop();
        if (!vis[u]) scc(u, u);
    }
}

int vischeck[MAX];
void dfscheck(int k) {
    vischeck[k] = 1;
    for (int i = 0; i < (int) g[k].size(); i++)
        if (!vischeck[g[k][i]]) dfscheck(g[k][i]);
}

void solve(){
    cin >> n >> m;

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        g[a].push_back(b);
        gi[b].push_back(a);
    }

    kosaraju();

    vector<int> transf(n);
    int at = 1;

    for(int i = 0; i < n; i++) {
        if(transf[comp[i]] == 0) {
            transf[comp[i]] = at;
            at++;
        }
    }

    cout << at - 1 << '\n';
    for(int i = 0; i < n; i++) cout << transf[comp[i]] << ' '
        ;
    cout << '\n';


}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

3.28 Police Chase

Kaaleppi has just robbed a bank and is now heading to the harbor. However, the police wants to stop him by closing some streets of the city. What is the minimum number of streets that should be closed so that there is no route between the bank and the harbor?

Input
The first input line has two integers n and m: the number of crossings and streets. The crossings are numbered $1, 2, \ldots, n$. The bank is located at crossing 1, and the harbor is located at crossing n. After this, there are m lines that describing the streets. Each line has two integers a and b: there is a street between crossings a and b. All streets are two-way streets, and there is at most one street between two crossings.

Output
First print an integer k: the minimum number of streets that should be closed. After this, print k lines describing the streets. You can print any valid solution.

Constraints
• $2 \le n \le 500$ • $1 \le m \le 1000$ • $1 \le a, b \le n$

Código-fonte:

```
#include <bits/stdc++.h>
using namespace std;
#define pb push_back
#define pc __builtin_popcount
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

int n, m;

struct Dinic {
    struct Edge {
        int to, rev;
        ll c, oc;
        ll flow() { return max(oc - c, 0LL); } // if you need
            flows
    };
    vi lvl, ptr, q;
    vector<vector<Edge>> adj;
    Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
    void addEdge(int a, int b, ll c, ll rcap = 0) {
        adj[a].push_back({b, sz(adj[b]), c, c});
        adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
    }
    ll dfs(int v, int t, ll f) {
        if (v == t || !f) return f;
        for (int& i = ptr[v]; i < sz(adj[v]); i++) {
            Edge& e = adj[v][i];
            if (lvl[e.to] == lvl[v] + 1)
                if (ll p = dfs(e.to, t, min(f, e.c))) {
                    e.c -= p, adj[e.to][e.rev].c += p;
                    return p;
                }
        }
        return 0;
    }
    ll calc(int s, int t) {
        ll flow = 0; q[0] = s;
        rep(L,0,31) do { // 'int L=30' maybe faster for
            random data
            lvl = ptr = vi(sz(q));
            int qi = 0, qe = lvl[s] = 1;
            while (qi < qe && !lvl[t]) {
                int v = q[qi++];
                for (Edge e : adj[v])
                    if (!lvl[e.to] && e.c >> (30 - L))
                        q[qe++] = e.to, lvl[e.to] = lvl[v] +
                            1;
            }
            while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
        } while (lvl[t]);
        return flow;
    }
    bool leftOfMinCut(int a) { return lvl[a] != 0; }
};


void solve() {
    cin >> n >> m;
    Dinic G(n);
    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G.addEdge(a, b, 1);
        G.addEdge(b, a, 1);
    }

    cout << G.calc(0, n - 1) << '\n';
    for(int i = 0; i < n; i++)
        for(int j = 0; j < sz(G.adj[i]); j+=2) {
            auto e = G.adj[i][j];
            if(i < e.to && G.leftOfMinCut(i) ^ G.leftOfMinCut
                (e.to))
                cout << i + 1 << ' ' << e.to + 1 << '\n';
        }
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

3.29 Road Construction

There are n cities and initially no roads between them. However, every day a new road will be constructed, and there will be a total of m roads. A component is a group of cities where there is a route between any two cities using the roads. After each day, your task is to find the number of components and the size of the largest component.

Input
The first input line has two integers n and m: the number of cities and roads. The cities are numbered $1, 2, \ldots, n$. Then, there are m lines describing the new roads. Each line has two integers a and b: a new road is constructed between cities a and b. You may assume that every road will be constructed between two different cities.

Output
Print m lines: the required information after each day.

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define F first
```

```
#define S second

#define int long long

using namespace std;

int n, m;

struct dsu {
    vector<int> id, sz;

    dsu(int n) : id(n), sz(n, 1) { iota(id.begin(), id.end(),
          0); }

    int find(int a) { return a == id[a] ? a : id[a] = find(id
         [a]); }

    void unite(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return;
        if (sz[a] < sz[b]) swap(a, b);
        sz[a] += sz[b], id[b] = a;
    }
};

void solve(){
    cin >> n >> m;
    dsu D(n);

    int tot = n, maxi = 1;

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        if(D.find(a) != D.find(b)) tot--;
        D.unite(a, b);
        maxi = max(maxi, D.sz[D.find(a)]);
        cout << tot << ' ' << maxi << '\n';
    }
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

3.30 Road Reparation

There are n cities and m roads between them. Unfortunately, the condition of the roads is so poor that they cannot be used. Your task is to repair some of the roads so that there will be a decent route between any two cities. For each road, you know its reparation cost, and you should find a solution where the total cost is as small as possible.

Input
The first input line has two integers n and m: the number of cities and roads. The cities are numbered $1, 2, \ldots, n$. Then, there are m lines describing the roads. Each line has three integers a, b and c: there is a road between cities a and b, and its reparation cost is c. All roads are two-way roads. Every road is between two different cities, and there is at most one road between two cities.

Output
Print one integer: the minimum total reparation cost. However, if there are no solutions, print "IMPOSSIBLE".

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$ • $1 \le c \le 10^9$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define F first
#define S second

#define int long long

using namespace std;

int n, m;
vector<tuple<int, int, int>> edg; // {peso,[x,y]}

struct dsu {
    vector<int> id, sz;

    dsu(int n) : id(n), sz(n, 1) { iota(id.begin(), id.end(),
          0); }

    int find(int a) { return a == id[a] ? a : id[a] = find(id
         [a]); }

    void unite(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return;
        if (sz[a] < sz[b]) swap(a, b);
        sz[a] += sz[b], id[b] = a;
    }
};

pair<int, vector<tuple<int, int, int>>> kruskal(int n) {
    dsu D(n);
    sort(edg.begin(), edg.end());

    int cost = 0;
    vector<tuple<int, int, int>> mst;
    for (auto [w,x,y] : edg) if (D.find(x) != D.find(y)) {
        mst.emplace_back(w, x, y);
        cost += w;
        D.unite(x,y);
    }

    int root = D.find(0);
    for(int i = 0; i < n; i++)
        if (D.find(i) != root) return{-1, {}};

    return {cost, mst};
}

void solve(){
    cin >> n >> m;

    for(int i = 0; i < m; i++) {
        int a, b, c; cin >> a >> b >> c;
        edg.push_back({c, a - 1, b - 1});
    }

    auto ans = kruskal(n);
    if(ans.first == -1) cout << "IMPOSSIBLE\n";
    else cout << ans.first << '\n';
}
```

```

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

3.31 Round Trip

Byteland has n cities and m roads between them. Your task is to design a round trip that begins in a city, goes through two or more other cities, and finally returns to the starting city. Every intermediate city on the route has to be distinct.

Input
The first input line has two integers n and m: the number of cities and roads. The cities are numbered $1, 2, \ldots, n$. Then, there are m lines describing the roads. Each line has two integers a and b: there is a road between those cities. Every road is between two different cities, and there is at most one road between any two cities.

Output
First print an integer k: the number of cities on the route. Then print k cities in the order they will be visited. You can print any valid solution. If there are no solutions, print "IMPOSSIBLE".

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long int  ll;
typedef pair<int, int> pii;
typedef vector<int> vi;
typedef vector<vi> graph;

const int maxn = 1e3+5, inf = 2e9, M = 1e9 + 7;
const ll linf = 1e18;
int n, m;
graph G;
vi check;
vi pai;

pii bfs(int a){
    queue<int> fila;
    fila.push(a);
    check[a] = 1;
    pai[a] = 0;

    int p1 = 0, p2 = 0;

```

```
    while(!fila.empty()){
        int x = fila.front();
        fila.pop();

        for(auto y : G[x]){
            if(check[y] == 1 && y != pai[x]){
                p1 = x;
                p2 = y;
                break;
            }
            else if(check[y] != 1){
                fila.push(y);
                check[y] = 1;
                pai[y] = x;
            }
        }

        if(p1 != 0) break;
    }

    return {p1, p2};
}

void solve(){
    cin >> n >> m;
    G.resize(n + 1);
    check.resize(n + 1);
    pai.resize(n + 1);

    for(int i = 1; i <= m; i++){
        int a, b; cin >> a >> b;
        G[a].pb(b); G[b].pb(a);
    }

    pii p;

    for(int i = 1; i <= n; i++){
        if(check[i] == 0){
            p = bfs(i);
            if(p.F != 0) break;
        }
    }

    if(p.F == 0){
        cout << "IMPOSSIBLE\n";
        return;
    }

    vi ans1, ans2, ans;

    int x = p.F;
    ans1.pb(x);
    while(pai[x] != 0){
        ans1.pb(pai[x]);
        x = pai[x];
    }

    x = p.S;
    ans2.pb(x);
    while(pai[x] != 0){
        ans2.pb(pai[x]);
        x = pai[x];
    }

    int bck = 0;
    while(ans1.back() == ans2.back()){
        bck = ans1.back();
        ans1.pop_back(); ans2.pop_back();
    }

    cout << ans1.size() + ans2.size() + 2 << '\n';
    for(int i = 0; i < ans1.size(); i++){
        cout << ans1[i] << ' ';
    }
    cout << bck << ' ';
    for(int i = ans2.size() - 1; i >= 0; i--){
        cout << ans2[i] << ' ';
    }
    cout << ans1[0] << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

3.32 Round Trip II

Byteland has n cities and m flight connections. Your task is to design a round trip that begins in a city, goes through one or more other cities, and finally returns to the starting city. Every intermediate city on the route has to be distinct.

Input
The first input line has two integers n and m: the number of cities and flights. The cities are numbered $1, 2, \ldots, n$. Then, there are m lines describing the flights. Each line has two integers a and b: there is a flight connection from city a to city b. All connections are one-way flights from a city to another city.

Output
First print an integer k: the number of cities on the route. Then print k cities in the order they will be visited. You can print any valid solution. If there are no solutions, print "IMPOSSIBLE".

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define int long long

using namespace std;

const int INF = 1000000000000000000;

vector<vector<int>> adj;
vector<int> vis, pai;
vector<int> ans;
int cnt = 0;

bool dfs(int v) {
    vis[v] = 1;
    for(auto x : adj[v]) {
        if(vis[x] == 1) {
            ans.push_back(x);
            int at = v;
            while(at != x) {
                ans.push_back(at);
                at = pai[at];
            }
            ans.push_back(x);

            return 1;
        }
        else if (vis[x] == 0) {
            pai[x] = v;
            if(dfs(x)) return 1;
        }
    }
    vis[v] = 2;
    return 0;
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int n, m; cin >> n >> m;
    adj.resize(n); vis.resize(n); pai.resize(n);
    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        a--; b--;
        adj[a].push_back(b);
    }

    for(int i = 0; i < n; i++) {
        if(vis[i] == 0) {
            if(dfs(i)) {
                reverse(ans.begin(), ans.end());
                cout << ans.size() << '\n';
                for(auto x : ans) cout << x + 1 << ' ';
                cout << '\n';
                return 0;
            }
        }
    }

    cout << "IMPOSSIBLE\n";

    return 0;
}
```

3.33 School Dance

There are n boys and m girls in a school. Next week a school dance will be organized. A dance pair consists of a boy and a girl, and there are k potential pairs. Your task is to find out the maximum number of dance pairs and show how this number can be achieved.

Input
The first input line has three integers n, m and k: the number of boys, girls, and potential pairs. The boys are numbered $1, 2, \ldots, n$, and the girls are numbered $1, 2, \ldots, m$. After this, there are k lines describing the potential pairs. Each line has two integers a and b: boy a and girl b are willing to dance together.

Output
First print one integer r: the maximum number of dance pairs. After this, print r lines describing the pairs. You can print any valid solution.

Constraints
• $1 \le n, m \le 500$ • $1 \le k \le 1000$ • $1 \le a \le n$ • $1 \le b \le m$

Código-fonte:

```cpp
#include <bits/stdc++.h>
using namespace std;
#define pb push_back
#define pc __builtin_popcount
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

mt19937 rng((int) chrono::steady_clock::now().
      time_since_epoch().count());

struct kuhn {
    int n, m;
    vector<vector<int>> g;
    vector<int> vis, ma, mb;

    kuhn(int n_, int m_) : n(n_), m(m_), g(n),
        vis(n+m), ma(n, -1), mb(m, -1) {}

    void add(int a, int b) { g[a].push_back(b); }

    bool dfs(int i) {
        vis[i] = 1;
        for (int j : g[i]) if (!vis[n+j]) {
            vis[n+j] = 1;
            if (mb[j] == -1 or dfs(mb[j])) {
                ma[i] = j, mb[j] = i;
                return true;
            }
        }
        return false;
    }
    int matching() {
        int ret = 0, aum = 1;
        for (auto& i : g) shuffle(i.begin(), i.end(), rng);
        while (aum) {
            for (int j = 0; j < m; j++) vis[n+j] = 0;
            aum = 0;
            for (int i = 0; i < n; i++)
                if (ma[i] == -1 and dfs(i)) ret++, aum = 1;
        }
        return ret;
    }
};

pair<vector<int>, vector<int>> recover(kuhn& K) {
    K.matching();
    int n = K.n, m = K.m;
    for (int i = 0; i < n+m; i++) K.vis[i] = 0;
    for (int i = 0; i < n; i++) if (K.ma[i] == -1) K.dfs(i);
    vector<int> ca, cb;
    for (int i = 0; i < n; i++) if (!K.vis[i]) ca.push_back(i
        );
    for (int i = 0; i < m; i++) if (K.vis[n+i]) cb.push_back(
        i);
    return {ca, cb};
}

void solve() {
    int n, m, k; cin >> n >> m >> k;
    kuhn G(n, m);
    for(int i = 0; i < k; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G.add(a, b);
    }

    int t = G.matching();
    cout << t << '\n';
    for(int i = 0; i < n; i++) {
        if(G.ma[i] != -1) cout << i + 1 << ' ' << G.ma[i] + 1
            << '\n';
    }
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

3.34 Shortest Routes I

There are n cities and m flight connections between them. Your task is to determine the length of the shortest route from Syrjälä to every city.

Input
The first input line has two integers n and m: the number of cities and flight connections. The cities are numbered $1, 2, \ldots, n$, and city 1 is Syrjälä. After that, there are m lines describing the flight connections. Each line has three integers a, b and c: a flight begins at city a, ends at city b, and its length is c. Each flight is a one-way flight. You can assume that it is possible to travel from Syrjälä to all other cities.

Output
Print n integers: the shortest route lengths from Syrjälä to cities $1, 2, \ldots, n$.

Constraints
• $1 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$ • $1 \le c \le 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define int long long

using namespace std;

const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

const int INF = 1000000000000000000;
vector<vector<pair<int, int>>> adj;

void dijkstra(int s, vector<int> & d, vector<int> & p) {
    int n = adj.size();
    d.assign(n, INF);
    p.assign(n, -1);

    d[s] = 0;
    set<pair<int, int>> q;
    q.insert({0, s});
    while (!q.empty()) {
        int v = q.begin()->second;
        q.erase(q.begin());

        for (auto edge : adj[v]) {
            int to = edge.first;
            int len = edge.second;

            if (d[v] + len < d[to]) {
                q.erase({d[to], to});
                d[to] = d[v] + len;
                p[to] = v;
                q.insert({d[to], to});
            }
        }
    }
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int n, m; cin >> n >> m;
    adj.resize(n+1);

    for(int i = 0; i < m; i++){
        int a, b, c; cin >> a >> b >> c;
        adj[a].push_back({b, c});
    }

    vector<int> d, p;
    dijkstra(1, d, p);

    for(int i = 1; i <= n; i++){
        cout << d[i] << ' ';
    }
    cout << '\n';

    return 0;
}
```

3.35 Shortest Routes II

There are n cities and m roads between them. Your task is to process q queries where you have to determine the length of the shortest route between two given cities.

Input
The first input line has three integers n, m and q: the number of cities, roads, and queries. Then, there are m lines describing the roads. Each line has three integers a, b and c: there is a road between cities a and b whose length is c. All roads are two-way roads. Finally, there are q lines describing the queries. Each line has two integers a and b: determine the length of the shortest route between cities a and b.

Output
Print the length of the shortest route for each query. If there is no route, print -1 instead.

Constraints
• $1 \le n \le 500$ • $1 \le m \le n^2$ • $1 \le q \le 10^5$ • $1 \le a, b \le n$ • $1 \le c \le 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define int long long
using namespace std;

typedef vector<vector<int>> vii;
const int inf = LLONG_MAX;

void floydWarshall(vector<vector<int>> &dist) {
    int n = dist.size();
```

```
    for(int k = 0; k < n; k++)
    for(int j = 0; j < n; j++)
    for(int i = 0; i < n; i++)
    if(dist[i][k] != inf && dist[k][j] != inf) {
        dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
            ;
    }
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int n, m, q; cin >> n >> m >> q;
    vii dist(n, vector<int>(n, inf));

    for(int i = 0; i < m; i++){
        int a, b, c; cin >> a >> b >> c;
        dist[a-1][b-1] = min(dist[a-1][b-1], c);
        dist[b-1][a-1] = min(dist[b-1][a-1], c);
    }
    for(int i = 0; i < n; i++) dist[i][i] = 0;

    floydWarshall(dist);

    while(q--) {
        int a, b; cin >> a >> b;
        cout << (dist[a-1][b-1] == inf ? -1 : dist[a-1][b-1])
             << '\n';
    }

    return 0;
}
```

3.36 Teleporters Path

A game has n levels and m teleporters between them. You win the game if you move from level 1 to level n using every teleporter exactly once. Can you win the game, and what is a possible way to do it?

Input
The first input line has two integers n and m: the number of levels and teleporters. The levels are numbered $1, 2, \ldots, n$. Then, there are m lines describing the teleporters. Each line has two integers a and b: there is a teleporter from level a to level b. You can assume that each pair (a, b) in the input is distinct.

Output
Print $m+1$ integers: the sequence in which you visit the levels during the game. You can print any valid solution. If there are no solutions, print "IMPOSSIBLE".

Constraints
• $2 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define int long long

using namespace std;

const int MAX = 1e5 + 5;
int n, m;
vector<int> G[MAX];

template<bool directed=false> struct euler {
    int n;
    vector<vector<pair<int, int>>> g;
    vector<int> used;

    euler(int n_) : n(n_), g(n) {}
    void add(int a, int b) {
        int at = used.size();
        used.push_back(0);
        g[a].emplace_back(b, at);
        if (!directed) g[b].emplace_back(a, at);
    }

    pair<bool, vector<pair<int, int>>> get_path(int src) {
        if (!used.size()) return {true, {}};
        vector<int> beg(n, 0);
        for (int& i : used) i = 0;
        vector<pair<pair<int, int>, int>> ret, st = {{{src,
             -1}, -1}};
        while (st.size()) {
            int at = st.back().first.first;
            int& it = beg[at];
            while (it < g[at].size() and used[g[at][it].
                 second]) it++;
            if (it == g[at].size()) {
                if (ret.size() and ret.back().first.second !=
                     at)
                    return {false, {}};
                ret.push_back(st.back()), st.pop_back();
            } else {
                st.push_back({{g[at][it].first, at}, g[at][it
                     ].second});
                used[g[at][it].second] = 1;
            }
        }
        if (ret.size() != used.size()+1) return {false, {}};
        vector<pair<int, int>> ans;
        for (auto i : ret) ans.emplace_back(i.first.first, i.
             second);
        reverse(ans.begin(), ans.end());
        return {true, ans};
    }
};

void solve() {
    cin >> n >> m;
    euler<true> E(n);
    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        E.add(a, b);
    }

    auto x = E.get_path(0);
    if(!x.first || x.second.back().first != n - 1) cout << "
         IMPOSSIBLE\n";
    else {
        for(auto k : x.second) cout << k.first + 1 << ' ';
        cout << '\n';
    }
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

4 Range Queries

4.1 Distinct Values Queries

You are given an array of n integers and q queries of the form: how many distinct values are there in a range $[a, b]$?

Input
The first input line has two integers n and q: the array size and number of queries. The next line has n integers $x_1, x_2, \ldots, x_n$: the array values. Finally, there are q lines describing the queries. Each line has two integers a and b.

Output
For each query, print the number of distinct values in the range.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$ • $1 \leq a \leq b \leq n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) x.size()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

struct FT {
    vector<int> s;
    FT(int n) : s(n) {}
    void update(int pos, int dif) { // a[pos] += dif
        for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
    }
    int query(int pos) { // sum of values in [0, pos)
        int res = 0;
        for (; pos > 0; pos &= pos - 1) res += s[pos-1];
        return res;
    }
};

void solve(){
    int n, q; cin >> n >> q;
    vector<int> v(n), is_first(n), next_eq(n);
    map<int, int> M;
    for(int i = 0; i < n; i++) cin >> v[i];

    for(int i = n - 1; i >= 0; i--) {
        auto it = M.find(v[i]);
        if(it == M.end()) {
            is_first[i] = 1;
            next_eq[i] = n;
        }
        else {
            is_first[i] = 1;
            is_first[it -> second] = 0;
            next_eq[i] = it -> second;
        }
        M[v[i]] = i;
    }

    vector<pair<pair<int, int>, int>> query(q);
    for(int i = 0; i < q; i++) {
        int a, b; cin >> a >> b;
        query[i] = {{a - 1, b - 1}, i};
    }

    sort(all(query));
    vector<int> ans(q);

    FT bit(n);
    for(int i = 0; i < n; i++) if(is_first[i]) bit.update(i,
        1);

    int at = 0;
    for(int i = 0; i < q; i++) {
        int a = query[i].first.first, b = query[i].first.
            second;
        for(; at < a; at++) if(next_eq[at] <= n - 1) bit.
            update(next_eq[at], 1);
        ans[query[i].second] = bit.query(b + 1) - bit.query(a
            );
    }

    for(auto x : ans) cout << x << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

4.2 Distinct Values Queries II

Given an array of n integers, your task is to process q queries of the following types:
update the value at position k to u check if every value in range $[a, b]$ is distinct

Input
The first line has two integers n and q: the number of values and queries. The second line has n integers $x_1, x_2, \ldots, x_n$: the array values. Finally, there are q lines describing the queries. Each line has three integers: either "1 k u" or "2 a b".

Output
For each query of type 2, print YES if every value in the range is distinct and NO otherwise.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq x_i, u \leq 10^9$ • $1 \leq k \leq n$ • $1 \leq a \leq b \leq n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) x.size()
#define F first
#define S second
#define pc __builtin_popcount

using namespace std;

const int MAX = 2e5;

int n, q;
vector<int> v, next_eq, prev_eq;
set<int> seq[2 * MAX];

namespace seg {
    // o proximo que eh igual a alguem do node
    int seg[4*MAX];

    int build(int p=1, int l=0, int r=n-1) {
        if (l == r) {
            auto it = seq[v[l]].lower_bound(l + 1);
            if(it != seq[v[l]].end()) return seg[p] = *it;
            return seg[p] = MAX + 1;
        }
        int m = (l+r)/2;
        return seg[p] = min(build(2*p, l, m), build(2*p+1, m
            +1, r));
    }
    int query(int a, int b, int p=1, int l=0, int r=n-1) {
        if (a <= l and r <= b) return seg[p];
        if (b < l or r < a) return MAX + 1;
        int m = (l+r)/2;
        return min(query(a, b, 2*p, l, m), query(a, b, 2*p+1,
            m+1, r));
    }
    void update(int a, int p=1, int l=0, int r=n-1) {
        if (l == r) {
            auto it = seq[v[l]].lower_bound(l + 1);
            if(it != seq[v[l]].end()) seg[p] = *it;
            else seg[p] = MAX + 1;
        }
        else {
            int m = (l+r)/2;
            if (a <= m)
                update(a, p*2, l, m);
            else
                update(a, p*2+1, m+1, r);
            seg[p] = min(seg[p*2], seg[p*2 + 1]);
        }
    }
};

void solve(){
    cin >> n >> q; v.resize(n);
    vector<int> all_numbers;

    for(int i = 0; i < n; i++) {
        cin >> v[i];
        all_numbers.push_back(v[i]);
    }

    vector<pair<int, pair<int, int>>> query(q);
    for(int i = 0; i < q; i++) {
        int a, b, c; cin >> a >> b >> c;
        if(a == 1) {
            query[i] = {a, {b - 1, c}};
            all_numbers.push_back(c);
        }
        else {
            query[i] = {a, {b - 1, c - 1}};
        }
    }

    sort(all(all_numbers));
    all_numbers.erase(unique(all(all_numbers)), all_numbers.
        end());

    for(int i = 0; i < n; i++) v[i] = lower_bound(all(
        all_numbers), v[i]) - all_numbers.begin();

    for(int i = 0; i < n; i++) seq[v[i]].insert(i);

    seg::build();
```

```
82
83      for(int i = 0; i < q; i++) {
84          if(query[i].F == 1) {
85              query[i].S.S = lower_bound(all(all_numbers),
                    query[i].S.S) - all_numbers.begin();
86              auto [u, k] = query[i].S;
87
88              vector<int> alts;
89              alts.push_back(u);
90
91              auto it = seq[v[u]].find(u);
92              if(it != seq[v[u]].begin()) alts.push_back(*prev(
                    it));
93              seq[v[u]].erase(it);
94
95              v[u] = k;
96              seq[k].insert(u);
97
98              it = seq[v[u]].find(u);
99              if(it != seq[v[u]].begin()) alts.push_back(*prev(
                    it));
100
101             for(auto x : alts) seg::update(x);
102         }
103         else {
104             auto [a, b] = query[i].S;
105             cout << ((seg::query(a, b) <= b) ? "NO\n" : "YES\
                    n");
106         }
107     }
108 }
109
110 int main() {
111     ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
112     //int t; cin >> t; for(int i = 1; i <= t; i++)
113     solve();
114     return 0;
115 }
```

4.3 Dynamic Range Minimum Queries

Given an array of n integers, your task is to process q queries of the following types:
update the value at position k to u what is the minimum value in range $[a, b]$?

Input
The first input line has two integers n and q: the number of values and queries. The second line has n integers $x_1, x_2, \ldots, x_n$: the array values. Finally, there are q lines describing the queries. Each line has three integers: either "1 k u" or "2 a b".

Output
Print the result of each query of type 2.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq x_i, u \leq 10^9$ • $1 \leq k \leq n$ • $1 \leq a \leq b \leq n$

Código-fonte:

```
1 #include <bits/stdc++.h>
2 #define pb push_back
3 #define all(x) x.begin(), x.end()
4 #define sz(x) x.size()
5 #define pc __builtin_popcount
6
7 using namespace std;
8
9 const int MAX = 2e5 + 5;
10 const int INF = 2e9;
11 int v[MAX];
12
13 typedef long long int ll;
14
15 namespace seg {
16     ll seg[4*MAX];
17     int n, *v;
18
19     ll build(int p=1, int l=0, int r=n-1) {
20         if (l == r) return seg[p] = v[l];
21         int m = (l+r)/2;
22         return seg[p] = min(build(2*p, l, m), build(2*p+1, m
                +1, r));
23     }
24     void build(int n2, int* v2) {
25         n = n2, v = v2;
26         build();
27     }
28     ll query(int a, int b, int p=1, int l=0, int r=n-1) {
29         if (a <= l and r <= b) return seg[p];
30         if (b < l or r < a) return INF;
31         int m = (l+r)/2;
32         return min(query(a, b, 2*p, l, m), query(a, b, 2*p+1,
                m+1, r));
33     }
34     void update(int a, int x, int p=1, int l=0, int r=n-1) {
35         if (l == r) {
36             seg[p] = x;
37         }
38         else {
39             int m = (l+r)/2;
40             if (a <= m)
41                 update(a, x, p*2, l, m);
42             else
43                 update(a, x, p*2+1, m+1, r);
44             seg[p] = min(seg[p*2], seg[p*2+1]);
45         }
46     }
47 };
48
49 void solve(){
50     int n, q; cin >> n >> q;
51
52     for(int i = 0; i < n; i++) {
53         cin >> v[i];
54     }
55
56     seg::build(n, v);
57
58     for(int i = 0; i < q; i++) {
59         int qi; cin >> qi;
60         if(qi == 1) {
61             int k, u; cin >> k >> u; k--;
62             seg::update(k, u);
63         }
64         else {
65             int a, b; cin >> a >> b; a--; b--;
66             cout << seg::query(a, b) << '\n';
67         }
68     }
69 }
70
71 int main() {
72     ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
73     solve();
74     return 0;
75 }
```

4.4 Dynamic Range Sum Queries

Given an array of n integers, your task is to process q queries of the following types:
update the value at position k to u what is the sum of values in range $[a, b]$?

Input
The first input line has two integers n and q: the number of values and queries. The second line has n integers $x_1, x_2, \ldots, x_n$: the array values. Finally, there are q lines describing the queries. Each line has three integers: either "1 k u" or "2 a b".

Output
Print the result of each query of type 2.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq x_i, u \leq 10^9$ • $1 \leq k \leq n$ • $1 \leq a \leq b \leq n$

Código-fonte:

```
1  #include <bits/stdc++.h>
2  #define pb push_back
3  #define all(x) x.begin(), x.end()
4  #define sz(x) x.size()
5  #define pc __builtin_popcount
6
7  using namespace std;
8
9  typedef long long int ll;
10
11 struct FT {
12     vector<ll> s;
13     FT(int n) : s(n) {}
14     void update(int pos, ll dif) { // a[pos] += dif
15         for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
16     }
17     ll query(int pos) { // sum of values in [0, pos)
18         ll res = 0;
19         for (; pos > 0; pos &= pos - 1) res += s[pos-1];
20         return res;
21     }
22 };
23
24 void solve(){
25     int n, q; cin >> n >> q;
26     vector<int> v(n);
27     FT bit(n);
28     for(int i = 0; i < n; i++) {
29         cin >> v[i];
30         bit.update(i, v[i]);
31     }
32
33     for(int i = 0; i < q; i++) {
34         int a, b, c; cin >> a >> b >> c; b--;
35         if(a == 1) {
36             bit.update(b, c - v[b]);
37             v[b] = c;
38         }
39         else {
```

```
            c--;
            cout << bit.query(c + 1) - bit.query(b) << '\n';
        }
    }
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

4.5 Forest Queries

You are given an $n \times n$ grid representing the map of a forest. Each square is either empty or contains a tree. The upper-left square has coordinates $(1, 1)$, and the lower-right square has coordinates (n, n). Your task is to process q queries of the form: how many trees are inside a given rectangle in the forest?

Input
The first input line has two integers n and q: the size of the forest and the number of queries. Then, there are n lines describing the forest. Each line has n characters: . is an empty square and * is a tree. Finally, there are q lines describing the queries. Each line has four integers y_1, x_1, y_2, x_2 corresponding to the corners of a rectangle.

Output
Print the number of trees inside each rectangle.

Constraints
• $1 \leq n \leq 1000$ • $1 \leq q \leq 2 \cdot 10^5$ • $1 \leq y_1 \leq y_2 \leq n$ • $1 \leq x_1 \leq x_2 \leq n$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) x.size()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int MAX = 1005;

int pref[MAX][MAX];

void solve(){
    int n, q; cin >> n >> q;

    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= n; j++) {
            char c; cin >> c;
            pref[i][j] = (c == '*' ? 1 : 0);
        }
    }

    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= n; j++) {
            pref[i][j] += pref[i][j - 1];
        }
    }

    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= n; j++) {
            pref[i][j] += pref[i - 1][j];
        }
    }

    while(q--) {
        int x1, x2, y1, y2;
        cin >> y1 >> x1 >> y2 >> x2;

        int ans = pref[y2][x2] + pref[y1 - 1][x1 - 1] - pref[
            y2][x1 - 1] - pref[y1 - 1][x2];
        cout << ans << '\n';
    }
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

4.6 Hotel Queries

There are n hotels on a street. For each hotel you know the number of free rooms. Your task is to assign hotel rooms for groups of tourists. All members of a group want to stay in the same hotel. The groups will come to you one after another, and you know for each group the number of rooms it requires. You always assign a group to the first hotel having enough rooms. After this, the number of free rooms in the hotel decreases.

Input
The first input line contains two integers n and m: the number of hotels and the number of groups. The hotels are numbered $1, 2, \ldots, n$. The next line contains n integers $h_1, h_2, \ldots, h_n$: the number of free rooms in each hotel. The last line contains m integers $r_1, r_2, \ldots, r_m$: the number of rooms each group requires.

Output
Print the assigned hotel for each group. If a group cannot be assigned a hotel, print 0 instead.

Constraints
• $1 \leq n, m \leq 2 \cdot 10^5$ • $1 \leq h_i \leq 10^9$ • $1 \leq r_i \leq 10^9$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) x.size()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int MAX = 2e5+20;
const ll LINF = 1e18;

namespace seg {
    ll seg[4*MAX], lazy[4*MAX];
    int n, *v;

    ll build(int p=1, int l=0, int r=n-1) {
        lazy[p] = 0;
        if (l == r) return seg[p] = v[l];
        int m = (l+r)/2;
        return seg[p] = max(build(2*p, l, m), build(2*p+1, m
            +1, r));
    }
    void build(int n2, int* v2) {
        n = n2, v = v2;
        build();
    }
    ll query(int a, int p=1, int l=0, int r=n-1) {
        if(seg[p] < a) return -1;
        if(l == r) return l;
        int m = (l+r)/2;
        int pos = query(a, 2*p, l, m);
        if(pos != -1) return pos;
        return query(a, 2*p+1, m+1, r);
    }
    ll update(int a, int x, int p=1, int l=0, int r=n-1) {
        if (a < l || r < a) {
            return seg[p];
        }
        if(l == r) {
            return seg[p] = seg[p] + x;
        }
        int m = (l+r)/2;
        return seg[p] = max(update(a, x, 2*p, l, m), update(a
            , x, 2*p+1, m+1, r));
    }
};

void solve() {
    int n, m; cin >> n >> m;

    int h[n];
    for(int i = 0; i < n; i++) cin >> h[i];

    seg::build(n, h);

    for(int i = 0; i < m; i++) {
        int s; cin >> s;
        int ans = seg::query(s);
        cout << ans + 1 << ' ';
        if(ans != -1)
            seg::update(ans, -s);
    }
    cout << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

4.7 List Removals

You are given a list consisting of n integers. Your task is to remove elements from the list at given positions, and report the removed elements.

Input

The first input line has an integer n: the initial size of the list. During the process, the elements are numbered $1, 2, \ldots, k$ where k is the current size of the list. The second line has n integers $x_1, x_2, \ldots, x_n$: the contents of the list. The last line has n integers $p_1, p_2, \ldots, p_n$: the positions of the elements to be removed.

Output
Print the elements in the order they are removed.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le x_i \le 10^9$ • $1 \le p_i \le n - i + 1$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;
template <class T>
    using ord_set = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) x.size()
#define pc __builtin_popcount


void solve() {
    int n; cin >> n;
    vector<int> x(n);
    for(auto &a : x) cin >> a;

    ord_set<int> s;
    for(int i = 0; i < n; i++) s.insert(i);

    for(int i = 0; i < n; i++) {
        int p; cin >> p;
        auto pos = s.find_by_order(p - 1);
        cout << x[*pos] << ' ';
        s.erase(pos);
    }
    cout << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

4.8 Pizzeria Queries

There are n buildings on a street, numbered $1, 2, \ldots, n$. Each building has a pizzeria and an apartment. The pizza price in building k is p_k. If you order a pizza from building a to building b, its price (with delivery) is $p_a + |a - b|$. Your task is to process two types of queries:
The pizza price p_k in building k becomes x. You are in building k and want to order a pizza. What is the minimum price?

Input
The first input line has two integers n and q: the number of buildings and queries. The second line has n integers $p_1, p_2, \ldots, p_n$: the initial pizza price in each building. Finally, there are q lines that describe the queries. Each line is either "1 k x" or "2 k".

Output
Print the answer for each query of type 2.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $1 \le p_i, x \le 10^9$ • $1 \le k \le n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) x.size()
#define pc __builtin_popcount

using namespace std;

const int MAX = 2e5;
const int INF = 2e9;
int v[MAX], n, q;

typedef long long int ll;

namespace seg {
    pair<ll, ll> seg[4*MAX];

    pair<ll, ll> build(int p=1, int l=0, int r=n-1) {
        if (l == r) return seg[p] = {v[l] - l, v[l] + l};
        int m = (l+r)/2;
        auto esq = build(2*p, l, m), dir = build(2*p+1, m+1,
            r);
        return seg[p] = {min(esq.first, dir.first), min(esq.
            second, dir.second)};
    }
    pair<ll, ll> query(int a, int b, int p=1, int l=0, int r=
        n-1) {
        if (a <= l and r <= b) return seg[p];
        if (b < l or r < a) return {INF, INF};
        int m = (l+r)/2;
        auto esq = query(a, b, 2*p, l, m), dir = query(a, b,
            2*p+1, m+1, r);
        return {min(esq.first, dir.first), min(esq.second,
            dir.second)};
    }
    void update(int a, int x, int p=1, int l=0, int r=n-1) {
        if (l == r) {
            seg[p] = {x - l, x + l};
        }
        else {
            int m = (l+r)/2;
            if (a <= m)
                update(a, x, p*2, l, m);
```

```
            else
                update(a, x, p*2+1, m+1, r);
            auto esq = seg[p*2], dir = seg[p*2+1];
            seg[p] = {min(esq.first, dir.first), min(esq.
                second, dir.second)};
        }
    }
};

void solve(){
    cin >> n >> q;

    for(int i = 0; i < n; i++) {
        cin >> v[i];
    }

    seg::build();

    for(int i = 0; i < q; i++) {
        int qi; cin >> qi;
        if(qi == 1) {
            int k, x; cin >> k >> x; k--;
            seg::update(k, x);
        }
        else {
            int k; cin >> k; k--;
            int esq = seg::query(0, k).first + k;
            int dir = seg::query(k, n - 1).second - k;
            cout << min(esq, dir) << '\n';
        }
    }
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

4.9 Polynomial Queries

Your task is to maintain an array of n values and efficiently process the following types of queries:
Increase the first value in range $[a, b]$ by 1, the second value by 2, the third value by 3, and so on. Calculate the sum of values in range $[a, b]$.

Input
The first input line has two integers n and q: the size of the array and the number of queries. The next line has n values $t_1, t_2, \ldots, t_n$: the initial contents of the array. Finally, there are q lines describing the queries. The format of each line is either "1 a b" or "2 a b".

Output
Print the answer to each sum query.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $1 \le t_i \le 10^6$ • $1 \le a \le b \le n$

Código-fonte:

```
#include <bits/stdc++.h>
using namespace std;
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) (int) x.size()
#define pc __builtin_popcount

typedef long long ll;

const int MAX = 2e5 + 20;
const ll LINF = 4e18;

namespace seg {
    ll seg[4*MAX];
    pair<ll, ll> lazy[4*MAX];
    ll n, *v;

    ll build(int p=1, int l=0, int r=n-1) {
        lazy[p] = {0, 0};
        if (l == r) return seg[p] = v[l];
        int m = (l+r)/2;
        return seg[p] = build(2*p, l, m) + build(2*p+1, m+1,
            r);
    }
    void build(int n2, ll* v2) {
        n = n2, v = v2;
        build();
    }
    void prop(int p, int l, int r) {
        ll ini = lazy[p].first, pulo = lazy[p].second, len =
            r - l + 1;
        seg[p] += ini * len;
        seg[p] += pulo * (len - 1) * len / 2;
        if (l != r) {
            lazy[2*p].first += ini;
            lazy[2*p].second += pulo;

            int len_esq = (r - l) / 2 + 1;
            lazy[2*p + 1].first += ini + pulo * len_esq;
            lazy[2*p + 1].second += pulo;
        }
        lazy[p] = {0, 0};
    }
    ll query(int a, int b, int p=1, int l=0, int r=n-1) {
        prop(p, l, r);
        if (a <= l and r <= b) return seg[p];
        if (b < l or r < a) return 0;
        int m = (l+r)/2;
        return query(a, b, 2*p, l, m) + query(a, b, 2*p+1, m
            +1, r);
    }
    ll update(int a, int b, int p=1, int l=0, int r=n-1) {
        prop(p, l, r);
        if (a <= l and r <= b) {
            lazy[p].first += (l - a + 1);
            lazy[p].second += 1;
            prop(p, l, r);
            return seg[p];
        }
        if (b < l or r < a) return seg[p];
        int m = (l+r)/2;
        return seg[p] = update(a, b, 2*p, l, m) + update(a, b
            , 2*p+1, m+1, r);
    }
};

void solve() {
    int n, q; cin >> n >> q;
    ll t[n + 1]; t[0] = 0;
    for(int i = 1; i <= n; i++) {
        cin >> t[i];
    }

    seg::build(n + 1, t);

    for(int i = 0; i < q; i++) {
        int c; cin >> c;
        int a, b; cin >> a >> b;

        if(c == 1) {
            seg::update(a, b);
        }
        else {
            cout << seg::query(a, b) << '\n';
        }

    }

}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

4.10 Prefix Sum Queries

Given an array of n integers, your task is to process q queries of the following types:
update the value at position k to u what is the maximum prefix sum in range $[a, b]$?
Empty prefixes (with sum 0) are allowed.

Input
The first input line has two integers n and q: the number of values and queries. The second line has n integers $x_1, x_2, \ldots, x_n$: the array values. Finally, there are q lines describing the queries. Each line has three integers: either "1 k u" or "2 a b".

Output
Print the result of each query of type 2.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $-10^9 \le x_i, u \le 10^9$ • $1 \le k \le n$ • $1 \le a \le b \le n$

Código-fonte:

```
#include <bits/stdc++.h>
using namespace std;
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) (int) x.size()
#define pc __builtin_popcount

typedef long long ll;

const int MAX = 2e5+20;
const ll LINF = 4e18;

namespace seg {
    ll seg[4*MAX], lazy[4*MAX];
    ll n, *v;

    ll build(int p=1, int l=0, int r=n-1) {
        lazy[p] = 0;
        if (l == r) return seg[p] = v[l];
        int m = (l+r)/2;
```

```
        return seg[p] = max(build(2*p, l, m), build(2*p+1, m
            +1, r));
    }
    void build(int n2, ll* v2) {
        n = n2, v = v2;
        build();
    }
    void prop(int p, int l, int r) {
        seg[p] += lazy[p];
        if (l != r) lazy[2*p] += lazy[p], lazy[2*p+1] += lazy
            [p];
        lazy[p] = 0;
    }
    ll query(int a, int b, int p=1, int l=0, int r=n-1) {
        prop(p, l, r);
        if (a <= l and r <= b) return seg[p];
        if (b < l or r < a) return -LINF;
        int m = (l+r)/2;
        return max(query(a, b, 2*p, l, m), query(a, b, 2*p+1,
            m+1, r));
    }
    ll update(int a, int b, int x, int p=1, int l=0, int r=n
        -1) {
        prop(p, l, r);
        if (a <= l and r <= b) {
            lazy[p] += x;
            prop(p, l, r);
            return seg[p];
        }
        if (b < l or r < a) return seg[p];
        int m = (l+r)/2;
        return seg[p] = max(update(a, b, x, 2*p, l, m),
            update(a, b, x, 2*p+1, m+1, r));
    }
};

void solve() {
    int n, q; cin >> n >> q;
    ll x[n + 1], pref[n + 1]; x[0] = 0; pref[0] = 0;
    for(int i = 1; i <= n; i++) {
        cin >> x[i]; pref[i] = pref[i - 1] + x[i];
    }

    seg::build(n + 1, pref);

    for(int i = 0; i < q; i++) {
        int c; cin >> c;
        int a, b; cin >> a >> b;

        if(c == 1) {
            seg::update(a, n, b - x[a]);
            x[a] = b;
        }
        else {
            ll ans = seg::query(a, b) - seg::query(a - 1, a -
                1);
            cout << max(ans, 0LL) << '\n';
        }

    }

}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

4.11 Range Update Queries

Given an array of n integers, your task is to process q queries of the following types:
increase each value in range $[a, b]$ by u what is the value at position k?

Input
The first input line has two integers n and q: the number of values and queries. The second line has n integers $x_1, x_2, \ldots, x_n$: the array values. Finally, there are q lines describing the queries. Each line has three integers: either "1 a b u" or "2 k".

Output
Print the result of each query of type 2.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq x_i, u \leq 10^9$ • $1 \leq k \leq n$ • $1 \leq a \leq b \leq n$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) x.size()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

struct FT {
    vector<ll> s;
    FT(int n) : s(n) {}
    void update(int pos, ll dif) { // a[pos] += dif
        for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
    }
    ll query(int pos) { // sum of values in [0, pos)
        ll res = 0;
        for (; pos > 0; pos &= pos - 1) res += s[pos-1];
        return res;
    }
};

void solve(){
    int n, q; cin >> n >> q;
    FT bit(n + 2);

    for(int i = 1; i <= n; i++) {
        int a; cin >> a;
        bit.update(i, a);
        bit.update(i + 1, -a);
    }

    for(int i = 0; i < q; i++) {
        int c; cin >> c;
        if(c == 1) {
            int a, b, u; cin >> a >> b >> u;
            bit.update(a, u);
            bit.update(b + 1, -u);
        }
        else {
            int k; cin >> k;
            cout << bit.query(k + 1) << '\n';
        }
    }
}

```

```
48 int main() {
49     ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
50     solve();
51     return 0;
52 }
```

4.12 Range Xor Queries

Given an array of n integers, your task is to process q queries of the form: what is the xor sum of values in range $[a, b]$?

Input
The first input line has two integers n and q: the number of values and queries. The second line has n integers $x_1, x_2, \ldots, x_n$: the array values. Finally, there are q lines describing the queries. Each line has two integers a and b: what is the xor sum of values in range $[a, b]$?

Output
Print the result of each query.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$ • $1 \leq a \leq b \leq n$

Código-fonte:

```
1  #include <bits/stdc++.h>
2  #define pb push_back
3  #define all(x) x.begin(), x.end()
4  #define sz(x) x.size()
5  #define pc __builtin_popcount
6
7  using namespace std;
8
9  typedef long long int ll;
10
11 struct FT {
12     vector<ll> s;
13     FT(int n) : s(n) {}
14     void update(int pos, ll dif) { // a[pos] += dif
15         for (; pos < sz(s); pos |= pos + 1) s[pos] ^= dif;
16     }
17     ll query(int pos) { // sum of values in [0, pos)
18         ll res = 0;
19         for (; pos > 0; pos &= pos - 1) res ^= s[pos-1];
20         return res;
21     }
22 };
23
24 void solve(){
25     int n, q; cin >> n >> q;
26     vector<int> v(n);
27     FT bit(n);
28     for(int i = 0; i < n; i++) {
29         cin >> v[i];
30         bit.update(i, v[i]);
31     }
32
33     for(int i = 0; i < q; i++) {
34         int a, b; cin >> a >> b; a--; b--;
35         cout << (bit.query(b + 1) ^ bit.query(a)) << '\n';
36     }
37 }
38
39 int main() {
40     ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
41     solve();
42     return 0;
43 }
```

4.13 Salary Queries

A company has n employees with certain salaries. Your task is to keep track of the salaries and process queries.

Input
The first input line contains two integers n and q: the number of employees and queries. The employees are numbered $1, 2, \ldots, n$. The next line has n integers $p_1, p_2, \ldots, p_n$: each employee's salary. After this, there are q lines describing the queries. Each line has one of the following forms:
• ! k x: change the salary of employee k to x • ? a b: count the number of employees whose salary is between $a \ldots b$

Output
Print the answer to each ? query.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq p_i \leq 10^9$ • $1 \leq k \leq n$ • $1 \leq x \leq 10^9$
• $1 \leq a \leq b \leq 10^9$

Código-fonte:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  #define pb push_back
4  #define all(x) x.begin(), x.end()
5  #define sz(x) (int) x.size()
6  #define pc __builtin_popcount
7
8  typedef long long ll;
9
10 const int MAX = 6e5+20;
11
12 struct FT {
13     vector<ll> s;
14     FT(int n) : s(n) {}
15     void update(int pos, ll dif) { // a[pos] += dif
16         for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
17     }
18     ll query(int pos) { // sum of values in [0, pos)
19         ll res = 0;
20         for (; pos > 0; pos &= pos - 1) res += s[pos-1];
21         return res;
22     }
23     ll query(int a, int b) {
24         return query(b + 1) - query(a);
25     }
26 };
27
28 void solve() {
29     int n, q; cin >> n >> q;
30     vector<int> p(n);
31     vector<tuple<int, int, int>> queries(q);
32     for(int i = 0; i < n; i++) cin >> p[i];
33
34     for(int i = 0; i < q; i++) {
35         char c; cin >> c;
36         int a, b; cin >> a >> b;
37         queries[i] = {c, a, b};
38     }
39
40     vector<int> all_salaries;
41     for(auto x : p) all_salaries.push_back(x);
42     for(auto [c, a, b] : queries) {
43         all_salaries.push_back(b);
44         if(c == '?') all_salaries.push_back(a);
45     }
46
47     sort(all_salaries.begin(), all_salaries.end());
48     all_salaries.erase(unique(all_salaries.begin(),
           all_salaries.end()), all_salaries.end());
49
50     for(int i = 0; i < n; i++) p[i] = lower_bound(all(
           all_salaries), p[i]) - all_salaries.begin();
51     for(int i = 0; i < q; i++) {
52         get<2>(queries[i]) = lower_bound(all(all_salaries),
               get<2>(queries[i])) - all_salaries.begin();
53         if(get<0>(queries[i]) == '?')
54             get<1>(queries[i]) = lower_bound(all(all_salaries
                   ), get<1>(queries[i])) - all_salaries.begin
                   ();
55     }
56
57     FT bit(MAX);
58     for(auto x : p) bit.update(x, 1);
59
60     for(auto [c, a, b] : queries) {
61         if(c == '!') {
62             bit.update(b, 1);
63             bit.update(p[a - 1], -1);
64             p[a - 1] = b;
65         }
66         else {
67             int ans = bit.query(a, b);
68             cout << ans << '\n';
69         }
70     }
71 }
72
73 signed main() {
74     ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
75     solve();
76     return 0;
77 }
```

4.14 Static Range Minimum Queries

Given an array of n integers, your task is to process q queries of the form: what is the minimum value in range $[a, b]$?

Input
The first input line has two integers n and q: the number of values and queries. The second line has n integers $x_1, x_2, \ldots, x_n$: the array values. Finally, there are q lines describing the queries. Each line has two integers a and b: what is the minimum value in range $[a, b]$?

Output
Print the result of each query.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$ • $1 \leq a \leq b \leq n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
using namespace std;

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
#define pb push_back
#define pc __builtin_popcount

typedef long long int ll;

template<class T>
struct RMQ {
    vector<vector<T>> jmp;
    RMQ(const vector<T>& V) : jmp(1, V) {
        for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) {
            jmp.emplace_back(sz(V) - pw * 2 + 1);
            rep(j,0,sz(jmp[k]))
                jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
        }
    }
    T query(int a, int b) {
        assert(a < b); // or return inf if a == b
        int dep = 31 - __builtin_clz(b - a);
        return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
    }
};

void solve(){
    int n, q; cin >> n >> q;
    vector<int> v(n);
    for(int i = 0; i < n; i++) {
        cin >> v[i];
    }
    RMQ st(v);

    for(int i = 0; i < q; i++) {
        int a, b; cin >> a >> b;
        cout << st.query(a - 1, b) << '\n';
    }
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

4.15 Static Range Sum Queries

Given an array of n integers, your task is to process q queries of the form: what is the sum of values in range $[a, b]$?

Input
The first input line has two integers n and q: the number of values and queries. The second line has n integers $x_1, x_2, \ldots, x_n$: the array values. Finally, there are q lines describing the queries. Each line has two integers a and b: what is the sum of values in range $[a, b]$?

Output
Print the result of each query.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$ • $1 \leq a \leq b \leq n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) x.size()
#define pc __builtin_popcount


using namespace std;

typedef long long int ll;

void solve(){
    int n, q; cin >> n >> q;
    vector<ll> v(n + 1), pref(n + 1);
    for(int i = 1; i <= n; i++) {
        cin >> v[i];
        pref[i] = v[i] + pref[i - 1];
    }

    for(int i = 0; i < q; i++) {
        int a, b; cin >> a >> b;
        cout << pref[b] - pref[a - 1] << '\n';
    }
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

4.16 Subarray Sum Queries

There is an array consisting of n integers. Some values of the array will be updated, and after each update, your task is to report the maximum subarray sum in the array.

Input
The first input line contains integers n and m: the size of the array and the number of updates. The array is indexed $1, 2, \ldots, n$. The next line has n integers: $x_1, x_2, \ldots, x_n$: the initial contents of the array. Then there are m lines describing the changes. Each line has two integers k and x: the value at position k becomes x.

Output
After each update, print the maximum subarray sum. Empty subarrays (with sum 0) are allowed.

Constraints
• $1 \leq n, m \leq 2 \cdot 10^5$ • $-10^9 \leq x_i \leq 10^9$ • $1 \leq k \leq n$ • $-10^9 \leq x \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int MAXN = 2e5 + 5, INF = 2e9, M = 1e9 + 7;

struct node {
    ll best;
    ll bestprefix;
    ll bestsufix;
    ll sum;
};

node t[4*MAXN];

node combine(node a, node b) {
    node c;
    c.sum = a.sum + b.sum;
    c.best = max(max(a.best, b.best), a.bestsufix + b.bestprefix);
    c.bestprefix = max(a.bestprefix, a.sum + b.bestprefix);
    c.bestsufix = max(b.bestsufix, b.sum + a.bestsufix);
    return c;
}

void build(ll a[], ll v, ll tl, ll tr) {
    if (tl == tr) {
        t[v].sum = a[tl];
        t[v].best = max(a[tl], 0LL);
        t[v].bestprefix = max(a[tl], 0LL);
        t[v].bestsufix = max(a[tl], 0LL);
    } else {
        ll tm = (tl + tr) / 2;
        build(a, v*2, tl, tm);
        build(a, v*2+1, tm+1, tr);
        t[v] = combine(t[v*2], t[v*2+1]);
    }
}

void update(ll v, ll tl, ll tr, ll pos, ll new_val) {
    if (tl == tr) {
        t[v].sum = new_val;
        t[v].best = max(new_val, 0LL);
        t[v].bestprefix = max(new_val, 0LL);
        t[v].bestsufix = max(new_val, 0LL);
    } else {
        ll tm = (tl + tr) / 2;
        if (pos <= tm)
            update(v*2, tl, tm, pos, new_val);
        else
            update(v*2+1, tm+1, tr, pos, new_val);
        t[v] = combine(t[v*2], t[v*2+1]);
    }
}

void solve(){
    ll n, m; cin >> n >> m;
    ll x[n + 1];
    x[0] = 0;
    for(ll i = 1; i <= n; i++) cin >> x[i];

    build(x, 1, 0, n);

    for(ll i = 0; i < m; i++)
    {
        ll k, a; cin >> k >> a;
        update(1, 0, n, k, a);
        cout << t[1].best << '\n';
    }

```

```
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

4.17 Subarray Sum Queries II

You are given an array of n integers and q queries. In each query, your task is to calculate the maximum subarray sum in the range $[a, b]$. Empty subarrays (with sum 0) are allowed.

Input
The first line contains two integers n and q: the number of elements and the number of queries. Then there are n integers $x_1, x_2, \ldots, x_n$: the contents of the array. Finally there are q lines that describe the queries. Each line has two integers a and b.

Output
Print the answer for each query.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $-10^9 \leq x_i \leq 10^9$ • $1 \leq a \leq b \leq n$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int MAXN = 2e5 + 5, INF = 2e9, M = 1e9 + 7;

struct node {
    ll best;
    ll bestprefix;
    ll bestsufix;
    ll sum;
};

node t[4*MAXN];

node combine(node a, node b) {
    node c;
    c.sum = a.sum + b.sum;
    c.best = max(max(a.best, b.best), a.bestsufix + b.
         bestprefix);
    c.bestprefix = max(a.bestprefix, a.sum + b.bestprefix);
    c.bestsufix = max(b.bestsufix, b.sum + a.bestsufix);
    return c;
}

void build(ll a[], ll v, ll tl, ll tr) {
    if (tl == tr) {
        t[v].sum = a[tl];
        t[v].best = max(a[tl], 0LL);
        t[v].bestprefix = max(a[tl], 0LL);
        t[v].bestsufix = max(a[tl], 0LL);
    } else {
        ll tm = (tl + tr) / 2;
        build(a, v*2, tl, tm);
        build(a, v*2+1, tm+1, tr);
        t[v] = combine(t[v*2], t[v*2+1]);
    }
}

void update(ll v, ll tl, ll tr, ll pos, ll new_val) {
    if (tl == tr) {
        t[v].sum = new_val;
        t[v].best = max(new_val, 0LL);
        t[v].bestprefix = max(new_val, 0LL);
        t[v].bestsufix = max(new_val, 0LL);
    } else {
        ll tm = (tl + tr) / 2;
        if (pos <= tm)
            update(v*2, tl, tm, pos, new_val);
        else
            update(v*2+1, tm+1, tr, pos, new_val);
        t[v] = combine(t[v*2], t[v*2+1]);
    }
}

node query(int v, int tl, int tr, int l, int r) {
    if (l > r) return {0, 0, 0, 0};
    if (l == tl && r == tr) return t[v];
    int tm = (tl + tr) / 2;
    node left  = query(v*2, tl, tm, l, min(r, tm));
    node right = query(v*2+1, tm+1, tr, max(l, tm+1), r);
    return combine(left, right);
}

void solve(){
    ll n, m; cin >> n >> m;
    ll x[n + 1];
    x[0] = 0;
    for(ll i = 1; i <= n; i++) cin >> x[i];

    build(x, 1, 0, n);

    for(ll i = 0; i < m; i++)
    {
        ll a, b; cin >> a >> b;
        cout << query(1, 0, n, a, b).best << '\n';
    }
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

4.18 Visible Buildings Queries

There are n buildings in a row numbered $1, 2, \ldots, n$ from left to right. You are standing to the left of the first building. You can see a building if it is taller than all buildings to its left. Your task is to process q queries: If only buildings in range $[a, b]$ existed, how many buildings would you see?

Input
The first line has two integers n and q: the number of buildings and queries. The second line has n integers $h_1, h_2, \ldots, h_n$: the heights of the buildings. Finally, there are q lines describing the queries. Each line has two integers a and b.

Output
For each query, print one integer: the number of visible buildings.

Constraints
• $1 \leq n \leq 10^5$ • $1 \leq q \leq 2 \cdot 10^5$ • $1 \leq h_i \leq 10^9$ • $1 \leq a \leq b \leq n$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int MAX = 1e5, INF = 2e9;
int n, q;
vector<int> next_h, h;

void next_higher() {
    next_h.resize(n, n);
    stack<int> st;

    for (int i = n - 1; i >= 0; --i) {
        while (!st.empty() && h[st.top()] <= h[i])
            st.pop();
        if (!st.empty())
            next_h[i] = st.top();
        st.push(i);
    }
}

namespace seg {
    // (answer, pos of the biggest)
    pair<int, int> seg[4*MAX];

    pair<int, int> query(int a, int b, int p=1, int l=0, int
         r=n-1) {
        if (a <= l and r <= b) return seg[p];
        if (b < l or r < a) return {0, -1};
        int m = (l+r)/2;

        auto [esq_ans, esq_big] = query(a, b, 2*p, l, m);
        if (esq_big == -1) {
            return query(a, b, 2*p+1, m+1, r);
        }
        int nxt = next_h[esq_big];
        if(nxt > b || nxt > r) return {esq_ans, esq_big};

        auto [dir_ans, dir_big] = query(nxt, b, 2*p+1, m+1, r
             );
        return {dir_ans + esq_ans, dir_big};
    }

    pair<int, int> build(int p=1, int l=0, int r=n-1) {
        if (l == r) return seg[p] = {1, l};
        int m = (l+r)/2;

        auto [esq_ans, esq_big] = build(2*p, l, m); build(2*p
             +1, m+1, r);

        int nxt = next_h[esq_big];
        if(nxt > r) return seg[p] = {esq_ans, esq_big};

```

```
56          auto [dir_ans, dir_big] = query(nxt, r, 2*p+1, m+1, r
                );
57          return seg[p] = {dir_ans + esq_ans, dir_big};
58      }
59  };
60
61  void solve(){
62      cin >> n >> q;
63      h.resize(n);
64      for(int i = 0; i < n; i++) cin >> h[i];
65      next_higher();
66
67      seg::build();
68
69      for(int i = 0; i < q; i++)
70      {
71          int l, r; cin >> l >> r; l--; r--;
72          cout << seg::query(l, r).first << '\n';
73      }
74  }
75
76  int main() {
77      ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
78      //int t; cin >> t; for(int i = 1; i <= t; i++)
79      solve();
80      return 0;
81  }
```

5 Tree Algorithms

5.1 Company Queries I

A company has n employees, who form a tree hierarchy where each employee has a boss, except for the general director. Your task is to process q queries of the form: who is employee x's boss k levels higher up in the hierarchy?

Input
The first input line has two integers n and q: the number of employees and queries. The employees are numbered $1, 2, \ldots, n$, and employee 1 is the general director. The next line has $n-1$ integers $e_2, e_3, \ldots, e_n$: for each employee $2, 3, \ldots, n$ their boss. Finally, there are q lines describing the queries. Each line has two integers x and k: who is employee x's boss k levels higher up?

Output
Print the answer for each query. If such a boss does not exist, print -1.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $1 \le e_i \le i-1$ • $1 \le x \le n$ • $1 \le k \le n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 2e5 + 5;
const int LOG = 19;
const int INF = 1e9;

int n, q;
vi adj[MAX];

int up[MAX][LOG];
int depth[MAX];

void dfs(int s, int p) {
    up[s][0] = p;
    depth[s] = depth[p] + 1;
    for(int i = 1; i < LOG; i++) {
        up[s][i] = up[up[s][i-1]][i-1];
    }

    for(auto x : adj[s]) {
        if(x == p) continue;
        dfs(x, s);
    }
}

void solve(){
    cin >> n >> q;

    for(int i = 2; i <= n; i++) {
        int a; cin >> a;
        adj[a].push_back(i);
        adj[i].push_back(a);
    }

    dfs(1, 0);

    for(int i = 0; i < q; i++) {
        int x, k; cin >> x >> k;
        int at = 0;
        while(k) {
            if(k % 2 == 1) {
                x = up[x][at];
            }
            k >>= 1;
            at++;
        }
        if(x == 0) cout << -1 << '\n';
        else cout << x << '\n';
    }
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

5.2 Company Queries II

A company has n employees, who form a tree hierarchy where each employee has a boss, except for the general director. Your task is to process q queries of the form: who is the lowest common boss of employees a and b in the hierarchy?

Input
The first input line has two integers n and q: the number of employees and queries. The employees are numbered $1, 2, \ldots, n$, and employee 1 is the general director. The next line has $n-1$ integers $e_2, e_3, \ldots, e_n$: for each employee $2, 3, \ldots, n$ their boss. Finally, there are q lines describing the queries. Each line has two integers a and b: who is the lowest common boss of employees a and b?

Output
Print the answer for each query.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $1 \le e_i \le i-1$ • $1 \le a, b \le n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 2e5 + 5;
const int LOG = 19;
const int INF = 1e9;

int n, q;
vi adj[MAX];

int up[MAX][LOG];
int depth[MAX];

void dfs(int s, int p) {
    up[s][0] = p;
    depth[s] = depth[p] + 1;
    for(int i = 1; i < LOG; i++) {
        up[s][i] = up[up[s][i-1]][i-1];
    }

    for(auto x : adj[s]) {
        if(x == p) continue;
        dfs(x, s);
    }
}

int get_lca(int a, int b) {
    if(depth[a] < depth[b]) swap(a, b);

    int diff = depth[a] - depth[b];
    for(int i = 0; i < LOG; i++) {
        if((diff >> i) & 1) a = up[a][i];
    }

    if(a == b) return a;

    for(int i = LOG - 1; i >= 0; i--) {
        if(up[a][i] != up[b][i]) {
            a = up[a][i];
            b = up[b][i];
        }
    }
    return up[a][0];
}

void solve(){
    cin >> n >> q;

    for(int i = 2; i <= n; i++) {
        int a; cin >> a;
        adj[a].push_back(i);
        adj[i].push_back(a);
    }

    dfs(1, 0);

    for(int i = 0; i < q; i++) {
        int a, b; cin >> a >> b;
        cout << get_lca(a, b) << '\n';
    }
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

5.3 Counting Paths

You are given a tree consisting of n nodes, and m paths in the tree. Your task is to calculate for each node the number of paths containing that node.

Input
The first input line contains integers n and m: the number of nodes and paths. The nodes are numbered $1, 2, \ldots, n$. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b. Finally, there are m lines describing the paths. Each line contains two integers a and b: there is a path between nodes a and b.

Output
Print n integers: for each node $1, 2, \ldots, n$, the number of paths containing that node.

Constraints
• $1 \leq n, m \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
using namespace std;

#define rep(i, a, b) for(int i = a; i < (b); ++i)

#define int long long
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) x.size()
#define pc __builtin_popcount
#define F first
#define S second

typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 2e5 + 5;
vector<vi> G(MAX);
int ans[MAX];

template<class T>
struct RMQ {
    vector<vector<T>> jmp;
    RMQ(const vector<T>& V) : jmp(1, V) {
        for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) {
            jmp.emplace_back(sz(V) - pw * 2 + 1);
            rep(j,0,sz(jmp[k]))
                jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
        }
    }
    T query(int a, int b) {
        assert(a < b); // or return inf if a == b
        int dep = 31 - __builtin_clz(b - a);
        return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
    }
};

struct LCA {
    int T = 0;
    vi time, path, ret, pai;
    RMQ<int> rmq;

    LCA(vector<vi>& C) : time(sz(C)), pai(sz(C)), rmq((dfs(C,0,-1), ret)) {}
    void dfs(vector<vi>& C, int v, int par) {
        pai[v] = par;
        time[v] = T++;
        for (int y : C[v]) if (y != par) {
            path.push_back(v), ret.push_back(time[v]);
            dfs(C, y, v);
        }
    }

    int lca(int a, int b) {
        if (a == b) return a;
        tie(a, b) = minmax(time[a], time[b]);
        return path[rmq.query(a, b)];
    }
    //dist(a,b){return depth[a] + depth[b] - 2*depth[lca(a,b)];}
};

void dfs(int v, int p) {
    for(auto x : G[v]) {
        if(x == p) continue;
        dfs(x, v);
        ans[v] += ans[x];
    }
}

void solve(){
    int n, m; cin >> n >> m;

    for(int i = 0; i < n - 1; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G[a].pb(b); G[b].pb(a);
    }

    LCA lca(G);

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        int c = lca.lca(a, b);

        if(c == b) swap(a, b);
        if(c == a) {
            ans[b]++;
        }
        else {
            ans[a]++; ans[b]++; ans[c]--;
        }

        if(lca.pai[c] >= 0) ans[lca.pai[c]]--;
    }

    dfs(0, -1);

    for(int i = 0; i < n; i++) cout << ans[i] << ' ';
    cout << '\n';

}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

5.4 Distance Queries

You are given a tree consisting of n nodes. Your task is to process q queries of the form: what is the distance between nodes a and b?

Input
The first input line contains two integers n and q: the number of nodes and queries. The nodes are numbered $1, 2, \ldots, n$. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b. Finally, there are q lines describing the queries. Each line contains two integer a and b: what is the distance between nodes a and b?

Output
Print q integers: the answer to each query.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 2e5 + 5;
const int LOG = 19;
const int INF = 1e9;

int n, q;
vi adj[MAX];

int up[MAX][LOG];
int depth[MAX];

void dfs(int s, int p) {
    up[s][0] = p;
    depth[s] = depth[p] + 1;
    for(int i = 1; i < LOG; i++) {
        up[s][i] = up[up[s][i-1]][i-1];
    }

    for(auto x : adj[s]) {
        if(x == p) continue;
        dfs(x, s);
    }
}

int get_lca(int a, int b) {
    if(depth[a] < depth[b]) swap(a, b);

    int diff = depth[a] - depth[b];
    for(int i = 0; i < LOG; i++) {
        if((diff >> i) & 1) a = up[a][i];
    }

    if(a == b) return a;

    for(int i = LOG - 1; i >= 0; i--) {
        if(up[a][i] != up[b][i]) {
            a = up[a][i];
            b = up[b][i];
        }
    }
    return up[a][0];
}

int dist(int a, int b) {
    return depth[a] + depth[b] - 2 * depth[get_lca(a, b)];
}

void solve(){
    cin >> n >> q;

    for(int i = 2; i <= n; i++) {
        int a, b; cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }

    dfs(1, 0);

    for(int i = 0; i < q; i++) {
        int a, b; cin >> a >> b;
        cout << dist(a, b) << '\n';
    }
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

5.5 Distinct Colors

You are given a rooted tree consisting of n nodes. The nodes are numbered $1, 2, \ldots, n$, and node 1 is the root. Each node has a color. Your task is to determine for each node the number of distinct colors in the subtree of the node.

Input
The first input line contains an integer n: the number of nodes. The nodes are numbered $1, 2, \ldots, n$. The next line consists of n integers $c_1, c_2, \ldots, c_n$: the color of each node. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

Output
Print n integers: for each node $1, 2, \ldots, n$, the number of distinct colors.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$ • $1 \leq c_i \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) x.size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 2e5 + 5;
vi G[MAX];
int v[2 * MAX];
pair<pii, int> query[MAX];
int at = -1;

struct FT {
    vector<int> s;
    FT(int n) : s(n) {}
    void update(int pos, int dif) { // a[pos] += dif
        for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
    }
    int query(int pos) { // sum of values in [0, pos)
        int res = 0;
        for (; pos > 0; pos &= pos - 1) res += s[pos-1];
        return res;
    }
};

void solve_DistinctValuesQueries(int n, int q) {
    vector<int> is_first(n), next_eq(n);
    map<int, int> M;

    for(int i = n - 1; i >= 0; i--) {
        auto it = M.find(v[i]);
        if(it == M.end()) {
            is_first[i] = 1;
            next_eq[i] = n;
        }
        else {
            is_first[i] = 1;
            is_first[it -> second] = 0;
            next_eq[i] = it -> second;
        }
        M[v[i]] = i;
    }

    sort(query, query + q);
    vector<int> ans(q);

    FT bit(n);
    for(int i = 0; i < n; i++) if(is_first[i]) bit.update(i,
        1);

    int at = 0;
    for(int i = 0; i < q; i++) {
        int a = query[i].F.F, b = query[i].F.S;
        for(; at < a; at++) if(next_eq[at] <= n - 1) bit.
            update(next_eq[at], 1);
        ans[query[i].S] = bit.query(b + 1) - bit.query(a);
    }

    for(auto x : ans) cout << x << ' ';
    cout << '\n';
}

void dfs(int s, int p) {
    at++;
    query[s].S = s;
    query[s].F.F = at; v[at] = s;
    for(auto x : G[s]) {
        if(x == p) continue;
        dfs(x, s);
```

```
    }
    at++;
    query[s].F.S = at; v[at] = s;
}

void solve(){
    int n; cin >> n;
    vector<int> c(n);
    for(int i = 0; i < n; i++) cin >> c[i];

    for(int i = 0; i < n - 1; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G[a].pb(b); G[b].pb(a);
    }

    dfs(0, -1);

    for(int i = 0; i < 2 * n; i++) v[i] = c[v[i]];

    solve_DistinctValuesQueries(2 * n, n);
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

5.6 Findinga Centroid

Given a tree of n nodes, your task is to find a centroid, i.e., a node such that when it is appointed the root of the tree, each subtree has at most $\lfloor n/2 \rfloor$ nodes.

Input
The first input line contains an integer n: the number of nodes. The nodes are numbered $1, 2, \ldots, n$. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

Output
Print one integer: a centroid node. If there are several possibilities, you can choose any of them.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 2e5 + 5;
const int LOG = 19;
const int INF = 1e9;

int n, subsize[MAX];
vector<int> g[MAX];

void dfs(int k, int p=-1) {
    subsize[k] = 1;
    for (int i : g[k]) if (i != p) {
        dfs(i, k);
        subsize[k] += subsize[i];
    }
}

int centroid(int k, int p=-1, int size=-1) {
    if (size == -1) size = subsize[k];
    for (int i : g[k]) if (i != p) if (subsize[i] > size/2)
        return centroid(i, k, size);
    return k;
}

pair<int, int> centroids(int k=0) {
    dfs(k);
    int i = centroid(k), i2 = i;
    for (int j : g[i]) if (2*subsize[j] == subsize[k]) i2 = j
        ;
    return {i, i2};
}

void solve(){
    cin >> n;
    for(int i = 0; i < n - 1; i++) {
        int a, b; cin >> a >> b; a--; b--;
        g[a].push_back(b);
        g[b].push_back(a);
    }

    auto ans = centroids();
    cout << ans.first + 1 << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

5.7 Fixed-Length Paths I

Given a tree of n nodes, your task is to count the number of distinct paths that consist of exactly k edges.

Input
The first input line contains two integers n and k: the number of nodes and the path length. The nodes are numbered $1, 2, \ldots, n$. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

Output
Print one integer: the number of paths.

Constraints
• $1 \leq k \leq n \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 2e5 + 5;
const int LOG = 19;
const int INF = 1e9;

int n, subsize[MAX];
vector<int> g[MAX];

void dfs(int k, int p=-1) {
    subsize[k] = 1;
    for (int i : g[k]) if (i != p) {
        dfs(i, k);
        subsize[k] += subsize[i];
    }
}

int centroid(int k, int p=-1, int size=-1) {
    if (size == -1) size = subsize[k];
    for (int i : g[k]) if (i != p) if (subsize[i] > size/2)
        return centroid(i, k, size);
    return k;
}

pair<int, int> centroids(int k=0) {
    dfs(k);
    int i = centroid(k), i2 = i;
    for (int j : g[i]) if (2*subsize[j] == subsize[k]) i2 = j
        ;
    return {i, i2};
}

void solve(){
    cin >> n;
    for(int i = 0; i < n - 1; i++) {
        int a, b; cin >> a >> b; a--; b--;
        g[a].push_back(b);
        g[b].push_back(a);
    }

    auto ans = centroids();
    cout << ans.first + 1 << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

5.8 Path Queries

You are given a rooted tree consisting of n nodes. The nodes are numbered $1, 2, \ldots, n$, and node 1 is the root. Each node has a value. Your task is to process following types of queries: change the value of node s to x calculate the sum of values on the path from the root to node s

Input
The first input line contains two integers n and q: the number of nodes and queries. The nodes are numbered $1, 2, \ldots, n$. The next line has n integers $v_1, v_2, \ldots, v_n$: the value of each node. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b. Finally, there are q lines describing the queries. Each query is either of the form "1 s x" or "2 s".

Output
Print the answer to each query of type 2.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq a, b, s \leq n$ • $1 \leq v_i, x \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define int long long
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) x.size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 2e5 + 5;
vi G[MAX];
int v[2 * MAX];
pii tempo[MAX];
int at = -1;

struct FT {
    vector<int> s;
    FT(int n) : s(n) {}
    void update(int pos, int dif) { // a[pos] += dif
        for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
    }
    int query(int pos) { // sum of values in [0, pos)
        int res = 0;
        for (; pos > 0; pos &= pos - 1) res += s[pos-1];
        return res;
    }
};

void dfs(int s, int p, const vector<int> &t) {
    at++;
    tempo[s].F = at; v[at] = t[s];
    for(auto x : G[s]) {
        if(x == p) continue;
        dfs(x, s, t);
    }
    at++;
    tempo[s].S = at; v[at] = -t[s];
}

void solve(){
    int n, q; cin >> n >> q;
    vector<int> t(n);
    for(int i = 0; i < n; i++) cin >> t[i];

    for(int i = 0; i < n - 1; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G[a].pb(b); G[b].pb(a);
    }

    dfs(0, -1, t);

    FT bit(2 * n);
    for(int i = 0; i < 2 * n; i++) bit.update(i, v[i]);

    for(int i = 0; i < q; i++) {
        int a; cin >> a;
        if(a == 1) {
            int s, x; cin >> s >> x; s--;
            auto [tin, tout] = tempo[s];
            bit.update(tin, x - v[tin]);
            bit.update(tout, v[tin] - x);
            v[tin] = x; v[tout] = -x;
        }
        else {
            int s; cin >> s; s--;
            auto [tin, tout] = tempo[s];
            cout << bit.query(tout) << '\n';
        }
    }
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

5.9 Path Queries II

You are given a tree consisting of n nodes. The nodes are numbered $1, 2, \ldots, n$. Each node has a value. Your task is to process following types of queries:
change the value of node s to x find the maximum value on the path between nodes a and b.

Input
The first input line contains two integers n and q: the number of nodes and queries. The nodes are numbered $1, 2, \ldots, n$. The next line has n integers $v_1, v_2, \ldots, v_n$: the value of each node. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b. Finally, there are q lines describing the queries. Each query is either of the form "1 s x" or "2 a b".

Output
Print the answer to each query of type 2.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq a, b, s \leq n$ • $1 \leq v_i, x \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define int long long
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) x.size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 2e5 + 5;
int v[MAX];
vector<vi> G;

namespace seg {
    int tree[2 * MAX];
    int n;

    void build(int n2) {
        n = n2;
        for(int i = 0; i <= 2 * n; i++) tree[i] = 0;
    }

    void update(int pos, int x) {
        for (tree[pos += n] = x; pos > 1; pos >>= 1)
            tree[pos >> 1] = max(tree[pos], tree[pos ^ 1]);
    }

    int query(int l, int r) {
        int res = 0;
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) res = max(res, tree[l++]);
            if (r & 1) res = max(res, tree[--r]);
        }
        return res;
    }
}

template <bool VALS_EDGES> struct HLD {
    int N, tim = 0;
    vector<vi> adj;
    vi par, siz, rt, pos;
    HLD(vector<vi> adj_)
        : N(sz(adj_)), adj(adj_), par(N, -1), siz(N, 1),
          rt(N),pos(N){ dfsSz(0); dfsHld(0); seg::build(N);}
    void dfsSz(int v) {
        for (int& u : adj[v]) {
            adj[u].erase(find(all(adj[u]), v));
            par[u] = v;
            dfsSz(u);
            siz[v] += siz[u];
            if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
        }
    }
    void dfsHld(int v) {
        pos[v] = tim++;
        for (int u : adj[v]) {
            rt[u] = (u == adj[v][0] ? rt[v] : u);
            dfsHld(u);
        }
    }
    template <class B> void process(int u, int v, B op) {
        for (;; v = par[rt[v]]) {
            if (pos[u] > pos[v]) swap(u, v);
            if (rt[u] == rt[v]) break;
            op(pos[rt[v]], pos[v] + 1);
        }
        op(pos[u] + VALS_EDGES, pos[v] + 1);
    }
    void updateNode(int u, int val) {
        seg::update(pos[u], val);
    }

    int queryPath(int u, int v_node) {
        int res = 0;
        process(u, v_node, [&](int l, int r) {
            res = max(res, seg::query(l, r));
        });
        return res;
    }
};

void solve(){
    int n, q; cin >> n >> q;
    G.resize(n);
    for(int i = 0; i < n; i++) cin >> v[i];

    for(int i = 0; i < n - 1; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G[a].pb(b); G[b].pb(a);
    }

    HLD<false> hld(G);
    for(int i = 0; i < n; i++) {
        hld.updateNode(i, v[i]);
    }

    for(int i = 0; i < q; i++) {
        int c; cin >> c;
        if(c == 1) {
            int s, x; cin >> s >> x; s--;
            hld.updateNode(s, x);
            v[s] = x;
        }
        else {
            int a, b; cin >> a >> b; a--; b--;
            cout << hld.queryPath(a, b) << '\n';
        }
    }
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

5.10 Subordinates

Given the structure of a company, your task is to calculate for each employee the number of their subordinates.

Input
The first input line has an integer n: the number of employees. The employees are numbered $1, 2, \ldots, n$, and employee 1 is the general director of the company. After this, there are $n-1$ integers: for each employee $2, 3, \ldots, n$ their direct boss in the company.

Output
Print n integers: for each employee $1, 2, \ldots, n$ the number of their subordinates.

Constraints

- $1 \le n \le 2 \cdot 10^5$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 2e5 + 5;
const int LOG = 19;
const int INF = 1e9;

int subtree[MAX];
vi adj[MAX];

void dfs(int s, int p) {
    subtree[s] = 1;
    for(auto x : adj[s]) {
        if(x == p) continue;
        dfs(x, s);
        subtree[s] += subtree[x];
    }
}

void solve(){
    int n; cin >> n;
    for(int i = 2; i <= n; i++) {
        int a; cin >> a;
        adj[a].push_back(i);
        adj[i].push_back(a);
    }

    dfs(1, 0);
    for(int i = 1; i <= n; i++) cout << subtree[i] - 1 << ' ';
    cout << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

5.11 Subtree Queries

You are given a rooted tree consisting of n nodes. The nodes are numbered $1, 2, \ldots, n$, and node 1 is the root. Each node has a value. Your task is to process following types of queries: change the value of node s to x calculate the sum of values in the subtree of node s

Input
The first input line contains two integers n and q: the number

of nodes and queries. The nodes are numbered $1, 2, \ldots, n$. The next line has n integers $v_1, v_2, \ldots, v_n$: the value of each node. Then there are $n-1$ lines describing the edges. Each line contans two integers a and b: there is an edge between nodes a and b. Finally, there are q lines describing the queries. Each query is either of the form "1 s x" or "2 s".

Output
Print the answer to each query of type 2.

Constraints
• $1 \leq n, q \leq 2 \cdot 10^5$ • $1 \leq a, b, s \leq n$ • $1 \leq v_i, x \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define int long long
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) x.size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 2e5 + 5;
vi G[MAX];
int v[2 * MAX];
pii tempo[MAX];
int at = -1;

struct FT {
    vector<int> s;
    FT(int n) : s(n) {}
    void update(int pos, int dif) { // a[pos] += dif
        for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
    }
    int query(int pos) { // sum of values in [0, pos)
        int res = 0;
        for (; pos > 0; pos &= pos - 1) res += s[pos-1];
        return res;
    }
};

void dfs(int s, int p, const vector<int> &t) {
    at++;
    tempo[s].F = at; v[at] = t[s];
    for(auto x : G[s]) {
        if(x == p) continue;
        dfs(x, s, t);
    }
    at++;
    tempo[s].S = at; v[at] = t[s];
}

void solve(){
    int n, q; cin >> n >> q;
    vector<int> t(n);
    for(int i = 0; i < n; i++) cin >> t[i];

    for(int i = 0; i < n - 1; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G[a].pb(b); G[b].pb(a);
    }

    dfs(0, -1, t);

    FT bit(2 * n);
    for(int i = 0; i < 2 * n; i++) bit.update(i, v[i]);

    for(int i = 0; i < q; i++) {
        int a; cin >> a;
        if(a == 1) {
            int s, x; cin >> s >> x; s--;
            auto [tin, tout] = tempo[s];
            bit.update(tin, x - v[tin]);
            bit.update(tout, x - v[tin]);
            v[tin] = x; v[tout] = x;
        }
        else {
            int s; cin >> s; s--;
            auto [tin, tout] = tempo[s];
            cout << (bit.query(tout + 1) - bit.query(tin)) /
                 2 << '\n';
        }
    }
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

5.12 Tree Diameter

You are given a tree consisting of n nodes. The diameter of a tree is the maximum distance between two nodes. Your task is to determine the diameter of the tree.

Input
The first input line contains an integer n: the number of nodes. The nodes are numbered $1, 2, \ldots, n$. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

Output
Print one integer: the diameter of the tree.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;


const int MAX = 2e5 + 5;
const int LOG = 19;
const int INF = 1e9;

int dist[MAX];
vi adj[MAX];

void dfs(int s, int p) {
    dist[s] = dist[p] + 1;
    for(auto x : adj[s]) {
        if(x == p) continue;
        dfs(x, s);
    }
}

void solve(){
    int n; cin >> n;
    for(int i = 1; i < n; i++) {
        int a, b; cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }

    dist[0] = -1;
    dfs(1, 0);
    int maxi = -1, at = 0;
    for(int i = 1; i <= n; i++) {
        if(maxi < dist[i]) {
            at = i;
            maxi = dist[i];
        }
    }

    dfs(at, 0);
    maxi = -1;
    for(int i = 1; i <= n; i++) {
        if(maxi < dist[i]) maxi = dist[i];
    }

    cout << maxi << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

5.13 Tree Distances I

You are given a tree consisting of n nodes. Your task is to determine for each node the maximum distance to another node.

Input
The first input line contains an integer n: the number of nodes. The nodes are numbered $1, 2, \ldots, n$. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

Output
Print n integers: for each node $1, 2, \ldots, n$, the maximum distance to another node.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 2e5 + 5;

int n;
vi G[MAX];
int d[MAX], par[MAX];
int df, c1, c2 = -1;

// queremos contar pra cima e pra baixo

void center(){
    int f;
    function<void(int)> dfs = [&] (int v) {
        if(d[v] > df) f = v, df = d[v];
        for (int u : G[v]) if (u != par[v])
            d[u] = d[v] + 1, par[u] = v, dfs(u);
    };

    f = df = par[0] = -1, d[0] = 0;
    dfs(0);

    int root = f;
    f = df = par[root] = -1, d[root] = 0;
    dfs(root);

    vector<int> c;
    while(f != -1) {
        if(d[f] == df / 2 or d[f] == (df + 1) / 2) c.
            push_back(f);
        f = par[f];
    }

    c1 = c[0];
    if(c.size() > 1) c2 = c[1];
    d[c1] = 0; par[c1] = -1;
    dfs(c1);
}

int check[MAX], ans[MAX];

void dfs(int s, int p) {
    if(s == c2) check[s] = 1;
    for(auto x : G[s]) {
        if(x == p) continue;
        if(check[s]) check[x] = 1;
        dfs(x, s);
    }

    if(check[s]) ans[s] = df / 2 + d[s];
    else ans[s] = (df + 1) / 2 + d[s];
}

void solve(){
    cin >> n;

    for(int i = 0; i < n - 1; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G[a].pb(b); G[b].pb(a);
    }
    center();
    dfs(c1, -1);

    for(int i = 0; i < n; i++) {
        cout << ans[i] << ' ';
    }
    cout << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

5.14 Tree Distances II

You are given a tree consisting of n nodes. Your task is to determine for each node the sum of the distances from the node to all other nodes.

Input
The first input line contains an integer n: the number of nodes. The nodes are numbered $1, 2, \ldots, n$. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

Output
Print n integers: for each node $1, 2, \ldots, n$, the sum of the distances.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define F first
#define S second

#define int long long

using namespace std;

typedef pair<int, int> pii;
typedef vector<int> vi;
typedef vector<vi> graph;

int n;
graph G;
vi sz, down, up;
int ans = 0;

// queremos contar pra cima e pra baixo

void dfs_down(int s, int p){
    sz[s] = 1;
    down[s] = 0;
    for(auto x : G[s]){
        if(x == p) continue;
        dfs_down(x, s);
        down[s] += down[x];
        sz[s] += sz[x];
    }
    down[s] += sz[s] - 1;
}

void dfs_up(int s, int p){
    if(p != -1)
        up[s] = up[p] + (down[p] - down[s] - sz[s]) + (n - sz[s]);

    for(auto x : G[s]){
        if(x == p) continue;
        dfs_up(x, s);
    }
}

void solve(){
    cin >> n;

    G.resize(n + 1);
    sz.resize(n + 1);
    down.resize(n + 1);
    up.resize(n + 1);

    for(int i = 0; i < n - 1; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G[a].pb(b); G[b].pb(a);
    }

    dfs_down(0, -1);
    dfs_up(0, -1);

    for(int i = 0; i < n; i++) {
        cout << up[i] + down[i] << ' ';
    }
    cout << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

5.15 Tree Matching

You are given a tree consisting of n nodes. A matching is a set of edges where each node is an endpoint of at most one edge. What is the maximum number of edges in a matching?

Input
The first input line contains an integer n: the number of nodes. The nodes are numbered $1, 2, \ldots, n$. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b.

Output
Print one integer: the maximum number of pairs.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define int long long
```

```cpp
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) x.size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 2e5 + 5;
vi G[MAX];
int dp[MAX][2];

void dfs(int s, int p) {
    for(auto x : G[s]) {
        if(x == p) continue;
        dfs(x, s);
    }
    for(auto x : G[s]) {
        if(x == p) continue;
        dp[s][0] += max(dp[x][0], dp[x][1]);
    }
    for(auto x : G[s]) {
        if(x == p) continue;
        dp[s][1] = max(dp[s][1], 1 + dp[s][0] - max(dp[x][0],
              dp[x][1]) + dp[x][0]);
    }
}

void solve(){
    int n; cin >> n;

    for(int i = 0; i < n - 1; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G[a].pb(b); G[b].pb(a);
    }

    dfs(0, -1);

    cout << max(dp[0][0], dp[0][1]) << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

6 Mathematics

6.1 Candy Lottery

There are n children, and each of them independently gets a random integer number of candies between 1 and k. What is the expected maximum number of candies a child gets?

Input
The only input line contains two integers n and k.

Output
Print the expected number rounded to six decimal places (rounding half to even).

Constraints
• $1 \leq n \leq 100$ • $1 \leq k \leq 100$

Código-fonte:

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long double ld;

int main() {
    ios::sync_with_stdio(0), cout.tie(0);
    int n, k; cin >> n >> k;
    ld total = 0;

    for(int i = 1; i <= k; i++){
        total += 1 - pow((i-1) / (ld)k, (ld)n);
    }

    cout << setprecision(6) << fixed;
    cout << total << '\n';

    return 0;
}
```

7 String Algorithms

7.1 Counting Patterns

Given a string and patterns, count for each pattern the number of positions where it appears in the string.

Input
The first input line has a string of length n. The next input line has an integer k: the number of patterns. Finally, there are k lines that describe the patterns. The string and the patterns consist of characters a–z.

Output
For each pattern, print the number of positions.

Constraints
• $1 \leq n \leq 10^5$ • $1 \leq k \leq 5 \cdot 10^5$ • the total length of the patterns is at most $5 \cdot 10^5$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) (int)x.size()
#define pc __builtin_popcount

#define int long long

using namespace std;

typedef long long ll;

vector<int> suffix_array(string s) {
    s += "$";
    int n = s.size(), N = max(n, 260LL);
    vector<int> sa(n), ra(n);
    for(int i = 0; i < n; i++) sa[i] = i, ra[i] = s[i];

    for(int k = 0; k < n; k ? k *= 2 : k++) {
        vector<int> nsa(sa), nra(n), cnt(N);

        for(int i = 0; i < n; i++) nsa[i] = (nsa[i]-k+n)%n,
             cnt[ra[i]]++;
        for(int i = 1; i < N; i++) cnt[i] += cnt[i-1];
        for(int i = n-1; i+1; i--) sa[--cnt[ra[nsa[i]]]] =
             nsa[i];

        for(int i = 1, r = 0; i < n; i++) nra[sa[i]] = r +=
             ra[sa[i]] !=
            ra[sa[i-1]] or ra[(sa[i]+k)%n] != ra[(sa[i-1]+k)%
                 n];
        ra = nra;
        if (ra[sa[n-1]] == n-1) break;
    }
    return vector<int>(sa.begin()+1, sa.end());
}

string s;
bool complb(int p, const string& t) {
    return s.compare(p, t.size(), t) < 0;
}
bool compup(const string& t, int p) {
    return s.compare(p, t.size(), t) > 0;
}

void solve() {
    cin >> s;
    int k; cin >> k;

    vector<int> sa = suffix_array(s);

    for(int i = 0; i < k; i++) {
        string t; cin >> t;
        auto ub = upper_bound(sa.begin(), sa.end(), t, compup
             );
        auto lb = lower_bound(sa.begin(), sa.end(), t, complb
             );
        cout << ub - lb << '\n';
    }

}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

7.2 Distinct Substrings

Count the number of distinct substrings that appear in a string.

Input
The only input line has a string of length n that consists of characters a–z.

Output
Print one integer: the number of substrings.

Constraints
• $1 \leq n \leq 10^5$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

#define int long long

using namespace std;

typedef long long ll;

// Suffix Array - O(n log n)
//
// kasai recebe o suffix array e calcula lcp[i],
// o lcp entre s[sa[i],...,n-1] e s[sa[i+1],..,n-1]
//
// Complexidades:
// suffix_array - O(n log(n))
// kasai - O(n)

vector<int> suffix_array(string s) {
    s += "$";
    int n = s.size(), N = max(n, 260LL);
    vector<int> sa(n), ra(n);
    for(int i = 0; i < n; i++) sa[i] = i, ra[i] = s[i];

    for(int k = 0; k < n; k ? k *= 2 : k++) {
        vector<int> nsa(sa), nra(n), cnt(N);

        for(int i = 0; i < n; i++) nsa[i] = (nsa[i]-k+n)%n,
             cnt[ra[i]]++;
        for(int i = 1; i < N; i++) cnt[i] += cnt[i-1];
        for(int i = n-1; i+1; i--) sa[--cnt[ra[nsa[i]]]] =
             nsa[i];

        for(int i = 1, r = 0; i < n; i++) nra[sa[i]] = r +=
             ra[sa[i]] !=
            ra[sa[i-1]] or ra[(sa[i]+k)%n] != ra[(sa[i-1]+k)%
                 n];
        ra = nra;
        if (ra[sa[n-1]] == n-1) break;
    }
    return vector<int>(sa.begin()+1, sa.end());
}

vector<int> kasai(string s, vector<int> sa) {
    int n = s.size(), k = 0;
    vector<int> ra(n), lcp(n);
    for (int i = 0; i < n; i++) ra[sa[i]] = i;

    for (int i = 0; i < n; i++, k -= !!k) {
        if (ra[i] == n-1) { k = 0; continue; }
        int j = sa[ra[i]+1];
        while (i+k < n and j+k < n and s[i+k] == s[j+k]) k++;
        lcp[ra[i]] = k;
    }
    return lcp;
}

void solve() {
    string s; cin >> s;
    int n = s.size();

    vector<int> sa = suffix_array(s);
    vector<int> lcp = kasai(s, sa);

    int ans = 0;
    for(auto x : lcp) ans += x;
    cout << n * (n + 1) / 2 - ans << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

7.3 Finding Borders

A border of a string is a prefix that is also a suffix of the string but not the whole string. For example, the borders of abcababcab are ab and abcab. Your task is to find all border lengths of a given string.

Input
The only input line has a string of length n consisting of characters a–z.

Output
Print all border lengths of the string in increasing order.

Constraints
- $1 \le n \le 10^6$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

#define int long long

using namespace std;

typedef long long ll;

// Z
//
// z[i] = lcp(s, s[i..n))
//
// Complexidades:
// z - O(|s|)
// match - O(|s| + |p|)

vector<int> get_z(string s) {
    int n = s.size();
    vector<int> z(n, 0);

    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n and s[z[i]] == s[i + z[i]]) z[i
            ]++;
        if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
    }

    return z;
}

void solve() {
    string s; cin >> s;
    int n; n = s.size();

    auto v = get_z(s);
    for(int i = n - 1; i >= 0; i--) {
        if(n - i == v[i]) cout << v[i] << '␣';
    }
    cout << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

7.4 Repeating Substring

A repeating substring is a substring that occurs in two (or more) locations in the string. Your task is to find the longest repeating substring in a given string.

Input
The only input line has a string of length n that consists of characters a–z.

Output
Print the longest repeating substring. If there are several possibilities, you can print any of them. If there is no repeating substring, print -1.

Constraints
- $1 \le n \le 10^5$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
#define pc __builtin_popcount

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

struct SuffixArray {
    vi sa, lcp;
    SuffixArray(string s, int lim=256) {
        s.push_back(0); int n = sz(s), k = 0, a, b;
        vi x(all(s)), y(n), ws(max(n, lim));
        sa = lcp = y, iota(all(sa), 0);
        for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim
             = p) {
            p = j, iota(all(y), n - j);
            rep(i,0,n) if (sa[i] >= j) y[p++] = sa[i] - j;
            fill(all(ws), 0);
            rep(i,0,n) ws[x[i]]++;
            rep(i,1,lim) ws[i] += ws[i - 1];
            for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
            swap(x, y), p = 1, x[sa[0]] = 0;
            rep(i,1,n) a = sa[i - 1], b = sa[i], x[b] =
                (y[a] == y[b] && y[a + j] == y[b + j]) ? p -
                    1 : p++;
        }
        for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
            for (k && k--, j = sa[x[i] - 1];
                    s[i + k] == s[j + k]; k++);
    }
};

void solve() {
    string s; cin >> s;
    SuffixArray sa(s);

    int tot = 0; int loc = 0;
    for(int i = 0; i < sz(sa.lcp); i++) {
        tot = max(tot, sa.lcp[i]);
        if(tot == sa.lcp[i]) loc = sa.sa[i];
    }

    if(tot == 0) {
        cout << -1 << '\n';
        return;
    }

    for(int i = 0; i < tot; i++) {
        cout << s[i + loc];
    }
    cout << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

7.5 String Functions

We consider a string of n characters, indexed $1, 2, \ldots, n$. Your task is to calculate all values of the following functions:
- $z(i)$ denotes the maximum length of a substring that begins at position i and is a prefix of the string. In addition, $z(1) = 0$.
- $\pi(i)$ denotes the maximum length of a substring that ends at position i, is a prefix of the string, and whose length is at most $i - 1$.

Note that the function z is used in the Z-algorithm, and the function π is used in the KMP algorithm.

Input
The only input line has a string of length n. Each character is between a–z.

Output
Print two lines: first the values of the z function, and then the values of the π function.

Constraints
- $1 \le n \le 10^6$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) (int)x.size()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;
typedef vector<int> vi;

void pi(const string& s) {
    vi p(sz(s));
    for(int i = 1; i < sz(s); i++) {
        int g = p[i - 1];
        while(g && s[i] != s[g]) g = p[g - 1];
        p[i] = g + (s[i] == s[g]);
    }

    for(auto x : p) cout << x << '␣';
```

```
    cout << '\n';
}

void Z(const string& s) {
    vi z(sz(s));
    int l = -1, r = -1;
    for(int i = 1; i < sz(s); i++) {
        z[i] = i >= r ? 0 : min(r - i, z[i - l]);
        while(i + z[i] < sz(s) && s[i + z[i]] == s[z[i]]) z[i
            ]++;
        if(i + z[i] > r) l = i, r = i + z[i];
    }
    for(auto x : z) cout << x << '␣';
    cout << '\n';
}

void solve() {
    string s; cin >> s;
    Z(s); pi(s);
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

7.6 String Matching

Given a string and a pattern, your task is to count the number of positions where the pattern occurs in the string.

Input
The first input line has a string of length n, and the second input line has a pattern of length m. Both of them consist of characters a–z.

Output
Print one integer: the number of occurrences.

Constraints
• $1 \le n, m \le 10^6$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1e6 + 5, inf = 2e9, P = 1e9 + 7;
const ll  linf = 4e18, M = 2305843009213693951;

ll pot[maxn];

ll hsh(string s, int n){
    ll h = 0;
    for(int i = 0; i < n; i++){
        h = (h + (__int128 (pot[n - 1 - i]) * s[i]) % M) % M;
    }

    return h;
}

void calcpot(){
    pot[0] = 1;
    for(int i = 1; i < maxn; i++){
        pot[i] = (__int128 (pot[i - 1]) * P) % M;
    }
}

void solve() {
    string s, t; cin >> s >> t;
    int tot = 0;

    if(t.size() > s.size()){
        cout << "0\n";
        return;
    }

    ll at = hsh(s, t.size()), h = hsh(t, t.size());

    if(at == h) tot++;
    for(int i = 0; i < (int)s.size() - (int)t.size(); i++){
        at = ((__int128 (at) * pot[1] - __int128 (s[i]) * pot
            [t.size()] + s[i + t.size()]) % M + M) % M;
        if(at == h) tot++;
    }

    cout << tot << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    calcpot();
    solve();
    return 0;
}
```

7.7 String Transform

Consider the following string transformation:
append the character # to the string (we assume that # is lexicographically smaller than all other characters of the string) generate all rotations of the string sort the rotations in increasing order based on this order, construct a new string that contains the last character of each rotation
For example, the string babc becomes babc#. Then, the sorted list of rotations is #babc, abc#b, babc#, bc#ba, and c#bab. This yields a string cb#ab.

Input
The only input line contains the transformed string of length $n + 1$. Each character of the original string is one of a–z.

Output
Print the original string of length n.

Constraints
• $1 \le n \le 10^6$

Código-fonte:

```
#include <bits/stdc++.h>
using namespace std;

#define rep(i, from, to) for (int i = from; i < (to); ++i)
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) (int)x.size()
#define pc __builtin_popcount

#define int long long
typedef pair<int, int> pii;
typedef vector<int> vi;

void solve() {
    string s;
    vector<pair<char, int>> t;
    cin >> s;

    for(int i = 0; i < sz(s); i++) {
        t.push_back({s[i], i});
    }
    sort(t.begin(), t.end());

    char c; int p = 0;
    for(int i = 0; i < sz(s); i++) {
        tie(c, p) = t[p];
        if(c != '#') cout << c;
    }
    cout << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

7.8 Substring Order I

You are given a string of length n. If all of its distinct substrings are ordered lexicographically, what is the kth smallest of them?

Input
The first input line has a string of length n that consists of characters a–z. The second input line has an integer k.

Output
Print the kth smallest distinct substring in lexicographical order.

Constraints
• $1 \le n \le 10^5$ • $1 \le k \le \frac{n(n+1)}{2}$ • It is guaranteed that k does not exceed the number of distinct substrings.

Código-fonte:

```
#include <bits/stdc++.h>
using namespace std;

#define rep(i, from, to) for (int i = from; i < (to); ++i)
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) (int)x.size()
#define pc __builtin_popcount

typedef pair<int, int> pii;
typedef vector<int> vi;
```

```cpp
typedef long long ll;

struct SuffixArray {
    vi sa, lcp;
    SuffixArray(string s, int lim=256) { // or vector<int>
        s.push_back(0); int n = sz(s), k = 0, a, b;
        vi x(all(s)), y(n), ws(max(n, lim));
        sa = lcp = y, iota(all(sa), 0);
        for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
            p = j, iota(all(y), n - j);
            rep(i,0,n) if (sa[i] >= j) y[p++] = sa[i] - j;
            fill(all(ws), 0);
            rep(i,0,n) ws[x[i]]++;
            rep(i,1,lim) ws[i] += ws[i - 1];
            for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
            swap(x, y), p = 1, x[sa[0]] = 0;
            rep(i,1,n) a = sa[i - 1], b = sa[i], x[b] =
                (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
        }
        for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
            for (k && k--, j = sa[x[i] - 1];
                    s[i + k] == s[j + k]; k++);
    }
};

void solve() {
    string s; cin >> s;
    int n = sz(s);
    ll k; cin >> k;

    SuffixArray SA(s);
    ll cnt = 0;

    string ans;

    for(int i = 0; i <= n; i++) {
        if(cnt + (n - SA.sa[i]) - SA.lcp[i] < k) {
            cnt += (n - SA.sa[i]) - SA.lcp[i];
            continue;
        }

        ll posfim = SA.lcp[i] + k - cnt;

        for(int j = SA.sa[i]; j < SA.sa[i] + posfim; j++) ans += s[j];
        break;
    }

    cout << ans << '\n';

}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

7.9 Substring Order II

You are given a string of length n. If all of its substrings (not necessarily distinct) are ordered lexicographically, what is the kth smallest of them?

Input
The first input line has a string of length n that consists of characters a–z. The second input line has an integer k.

Output
Print the kth smallest substring in lexicographical order.

Constraints
• $1 \le n \le 10^5$ • $1 \le k \le \frac{n(n+1)}{2}$

Código-fonte:

```cpp
#include <bits/stdc++.h>
using namespace std;

#define rep(i, from, to) for (int i = from; i < (to); ++i)
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) (int)x.size()
#define pc __builtin_popcount

typedef pair<int, int> pii;
typedef vector<int> vi;
typedef long long ll;

const int MAX = 1e5 + 10;

namespace sam {
    int cur, sz, len[2*MAX], link[2*MAX], acc[2*MAX];
    ll cnt[2 * MAX];
    int nxt[2*MAX][26];

    void add(int c) {
        int at = cur;
        cnt[sz] = 1;
        len[sz] = len[cur]+1, cur = sz++;
        while (at != -1 and !nxt[at][c]) nxt[at][c] = cur, at = link[at];
        if (at == -1) { link[cur] = 0; return; }
        int q = nxt[at][c];
        if (len[q] == len[at]+1) { link[cur] = q; return; }
        int qq = sz++;
        cnt[qq] = 0;
        len[qq] = len[at]+1, link[qq] = link[q];
        for (int i = 0; i < 26; i++) nxt[qq][i] = nxt[q][i];
        while (at != -1 and nxt[at][c] == q) nxt[at][c] = qq, at = link[at];
        link[cur] = link[q] = qq;
    }
    void build(string& s) {
        cur = 0, sz = 0, len[0] = 0, link[0] = -1, sz++;
        for (auto i : s) add(i-'a');
        int at = cur;
        while (at) acc[at] = 1, at = link[at];
    }

    void calc_rep() {
        vector<pair<int, int>> v;
        for (int i = 1; i < sz; i++) {
            v.push_back({len[i], i});
        }
        sort(v.rbegin(), v.rend());

        for (auto [l, u] : v) {
            if (link[u] != -1) {
                cnt[link[u]] += cnt[u];
            }
        }
        cnt[0] = 0;
    }

    ll dp[2*MAX];
    ll paths(int i) {
        auto& x = dp[i];
        if (x) return x;
        x = cnt[i];
        for (int j = 0; j < 26; j++) if (nxt[i][j]) x += paths(nxt[i][j]);
        return x;
    }
    void kth_substring(ll k, int at=0) { // k=1 : menor substring lexicog.
        if(at == 0) calc_rep();
        if(k <= cnt[at]) {
            cout << '\n';
            return;
        }
        k -= cnt[at];
        for (int i = 0; i < 26; i++) if (k and nxt[at][i]) {
            if (paths(nxt[at][i]) >= k) {
                cout << char('a'+i);
                kth_substring(k, nxt[at][i]);
                return;
            }
            k -= paths(nxt[at][i]);
        }
    }
};

void solve() {
    string s; cin >> s;
    int n = sz(s);
    ll k; cin >> k;

    sam::build(s);
    sam::kth_substring(k);
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

8 Advanced Techniques

8.1 Distinct Routes II

A game consists of n rooms and m teleporters. At the beginning of each day, you start in room 1 and you have to reach room n. You can use each teleporter at most once during the game. You want to play the game for exactly k days. Every time you use any teleporter you have to pay one coin. What is the minimum number of coins you have to pay during k days if you play optimally?

Input
The first input line has three integers n, m and k: the number of rooms, the number of teleporters and the number of days you play the game. The rooms are numbered $1, 2, \ldots, n$. After this, there are m lines describing the teleporters. Each line has two integers a and b: there is a teleporter from room a to room b. There are no two teleporters whose starting and ending room are the same.

Output
First print one integer: the minimum number of coins you have to pay if you play optimally. Then, print k route descriptions according to the example. You can print any valid solution. If it is not possible to play the game for k days, print only -1.

Constraints
• $2 \le n \le 500$ • $1 \le m \le 1000$ • $1 \le k \le n-1$ • $1 \le a, b \le n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#include <bits/extc++.h>

#define int long long
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const ll INF = numeric_limits<ll>::max() / 4;

struct MCMF {
    struct edge {
        int from, to, rev;
        ll cap, cost, flow;
    };
    int N;
    vector<vector<edge>> ed;
    vi seen;
    vector<ll> dist, pi;
    vector<edge*> par;

    MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}

    void addEdge(int from, int to, ll cap, ll cost) {
        if (from == to) return;
        ed[from].push_back(edge{ from,to,sz(ed[to]),cap,cost,0 });
        ed[to].push_back(edge{ to,from,sz(ed[from])-1,0,-cost,0 });
    }

    void path(int s) {
        fill(all(seen), 0);
        fill(all(dist), INF);
        dist[s] = 0; ll di;

        __gnu_pbds::priority_queue<pair<ll, int>> q;
        vector<decltype(q)::point_iterator> its(N);
        q.push({ 0, s });

        while (!q.empty()) {
            s = q.top().second; q.pop();
            seen[s] = 1; di = dist[s] + pi[s];
            for (edge& e : ed[s]) if (!seen[e.to]) {
                ll val = di - pi[e.to] + e.cost;
                if (e.cap - e.flow > 0 && val < dist[e.to]) {
                    dist[e.to] = val;
                    par[e.to] = &e;
                    if (its[e.to] == q.end())
                        its[e.to] = q.push({ -dist[e.to], e.to });
                    else
                        q.modify(its[e.to], { -dist[e.to], e.to });
                }
            }
        }
        rep(i,0,N) pi[i] = min(pi[i] + dist[i], INF);
    }

    pair<ll, ll> maxflow(int s, int t) {
        ll totflow = 0, totcost = 0;
        while (path(s), seen[t]) {
            ll fl = INF;
            for (edge* x = par[t]; x; x = par[x->from])
                fl = min(fl, x->cap - x->flow);

            totflow += fl;
            for (edge* x = par[t]; x; x = par[x->from]) {
                x->flow += fl;
                ed[x->to][x->rev].flow -= fl;
            }
        }
        rep(i,0,N) for(edge& e : ed[i]) totcost += e.cost * e.flow;
        return {totflow, totcost/2};
    }
};

void solve(){
    int n, m, k; cin >> n >> m >> k;

    MCMF g(n + 1);

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        g.addEdge(a, b, 1, 1);
    }

    g.addEdge(0, 1, k, 0);
    auto ans = g.maxflow(0, n);

    if(ans.first == k) {
        cout << ans.second << '\n';

        for(int i = 0; i < k; i++) {
            vector<int> path;
            int curr = 1;
            path.push_back(curr);

            while(curr != n) {
                for(auto& e : g.ed[curr]) {
                    if(e.cap == 1 && e.flow == 1) {
                        e.flow = 0;
                        curr = e.to;
                        path.push_back(curr);
                        break;
                    }
                }
            }

            cout << path.size() << '\n';
            for(int node : path) {
                cout << node << " ";
            }
            cout << '\n';
        }

    }
    else {
        cout << -1 << '\n';
    }
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

8.2 Eulerian Subgraphs

You are given an undirected graph that has n nodes and m edges. We consider subgraphs that have all nodes of the original graph and some of its edges. A subgraph is called Eulerian if each node has even degree. Your task is to count the number of Eulerian subgraphs modulo $10^9 + 7$.

Input
The first input line has two integers n and m: the number of nodes and edges. The nodes are numbered $1, 2, \ldots, n$. After this, there are m lines that describe the edges. Each line has two integers a and b: there is an edge between nodes a and b. There is at most one edge between two nodes, and each edge connects two distinct nodes.

Output
Print the number of Eulerian subgraphs modulo $10^9 + 7$.

Constraints
• $1 \le n \le 10^5$ • $0 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

Código-fonte:

```cpp
#include <bits/stdc++.h>

#define int long long
```

```cpp
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 1e5+5, MOD = 1e9+7;
vector<int> G[MAX];
bool vis[MAX];

void dfs(int s) {
    vis[s] = 1;
    for(auto x : G[s]) if(!vis[x]) dfs(x);
}

ll fexp(ll b, ll e) {
    ll ans = 1;
    for (; e; b = b * b % MOD, e /= 2)
        if (e & 1) ans = ans * b % MOD;
    return ans;
}

void solve(){
    int n, m; cin >> n >> m;
    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        G[a].push_back(b);
        G[b].push_back(a);
    }

    int comp = 0;
    for(int i = 1; i <= n; i++) {
        if(!vis[i]) {
            dfs(i);
            comp++;
        }
    }

    int exp = m - (n - comp);
    cout << fexp(2, exp) << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

8.3 Housesand Schools

There are n houses on a street, numbered $1, 2, \ldots, n$. The distance of houses a and b is $|a - b|$. You know the number of children in each house. Your task is to establish k schools in such a way that each school is in some house. Then, each child goes to the nearest school. What is the minimum total walking distance of the children if you act optimally?

Input
The first input line has two integers n and k: the number of houses and the number of schools. The houses are numbered $1, 2 \ldots, n$. After this, there are n integers $c_1, c_2, \ldots, c_n$: the number of children in each house.

Output
Print the minimum total distance.

Constraints
• $1 \leq k \leq n \leq 3000$ • $1 \leq c_i \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>

#define int long long
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 3e3 + 3;
const int LINF = 4000000000000000000;

int dp[MAX][MAX];

int n, K;
int cur_j;
int c[MAX], pref[MAX], pref1[MAX];

int cost(int i, int k) {
    int m = (k + i) / 2;
    int p1 = (pref1[m] - pref1[k - 1]) - k * (pref[m] - pref[k - 1]);
    int p2 = i * (pref[i] - pref[m]) - (pref1[i] - pref1[m]);
    return p1 + p2;
}

struct DP {
    int lo(int ind) { return cur_j - 1; }
    int hi(int ind) { return ind; }
    ll f(int ind, int k) { return dp[k][cur_j - 1] + cost(ind, k); }
    void store(int ind, int k, ll v) { dp[ind][cur_j] = v; }

    void rec(int L, int R, int LO, int HI) {
        if (L >= R) return;
        int mid = (L + R) >> 1;
        pair<ll, int> best(LLONG_MAX, LO);
        rep(k, max(LO,lo(mid)), min(HI,hi(mid)))
            best = min(best, make_pair(f(mid, k), k));
        store(mid, best.second, best.first);
        rec(L, mid, LO, best.second+1);
        rec(mid+1, R, best.second, HI);
    }
    void solve(int L, int R) { rec(L, R, 0, n + 1); }
};

void solve(){
    cin >> n >> K;
    for(int i = 1; i <= n; i++) {
        cin >> c[i];
        pref[i] = pref[i - 1] + c[i];
        pref1[i] = pref1[i - 1] + c[i] * i;
    }

    DP dcdp;

    for(int i = 1; i <= n; i++) dp[i][1] = pref[i] * i - pref1[i];

    for(int j = 2; j <= K; j++) {
        cur_j = j;
        dcdp.solve(1, n + 1);
    }

    int ans = LINF;

    for(int i = K; i <= n; i++) {
        ans = min(ans, dp[i][K] + (pref1[n] - pref1[i - 1]) - i * (pref[n] - pref[i - 1]));
    }

    cout << ans << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

8.4 Longest Palindrome

Given a string, your task is to determine the longest palindromic substring of the string. For example, the longest palindrome in aybabtu is bab.

Input
The only input line contains a string of length n. Each character is one of a–z.

Output
Print the longest palindrome in the string. If there are several solutions, you may print any of them.

Constraints
• $1 \leq n \leq 10^6$

Código-fonte:

```cpp
#include <bits/stdc++.h>

#define int long long
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
```

```
typedef pair<int, int> pii;
typedef vector<int> vi;
typedef vector<ll> vl;

array<vi, 2> manacher(const string& s) {
    int n = sz(s);
    array<vi,2> p = {vi(n+1), vi(n)};
    rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) {
        int t = r-i+!z;
        if (i<r) p[z][i] = min(t, p[z][l+t]);
        int L = i-p[z][i], R = i+p[z][i]-!z;
        while (L>=1 && R+1<n && s[L-1] == s[R+1])
            p[z][i]++, L--, R++;
        if (R>r) l=L, r=R;
    }
    return p;
}

void solve(){
    string s; cin >> s;
    auto ans = manacher(s);

    int q[2] = {-1, -1}, pos[2] = {-1, -1};
    for(int j = 0; j < 2; j++) {
        for(int i = 0; i < ans[j].size(); i++) {
            if(q[j] < ans[j][i]) {
                q[j] = ans[j][i];
                pos[j] = i;
            }
        }
    }

    int bst = 0;
    if(q[0] * 2 < q[1] * 2 + 1) bst = 1;

    if(pos[bst] == 0) {
        cout << s[0] << '\n';
        return;
    }

    for(int i = pos[bst] - q[bst]; i < pos[bst] + q[bst] +
         bst; i++) {
        cout << s[i];
    }
    cout << '\n';

}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

8.5 Monster Game

Enunciado não encontrado no site. **Código-fonte:**

```
#include <bits/stdc++.h>

#define int long long
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef pair<ll, ll> pll;

const int MAX = 2e5+1;

// matar custa s_i * f tempo e vc recebe a forca f_i
// dp[i] = resposta matando o monstro i
// dp[n] = 0
// dp[k] = min(s[l] * f[k] + dp[l]); k + 1 <= l <= n.
// CHT em s[l] * x + dp[l]

int dp[MAX], s[MAX], f[MAX];
int n, x;

struct Line {
    mutable ll k, m, p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
    // (for doubles, use inf = 1/.0, div(a,b) = a/b)
    static const ll inf = LLONG_MAX;
    ll div(ll a, ll b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b); }
    bool isect(iterator x, iterator y) {
        if (y == end()) return x->p = inf, 0;
        if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
        else x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
    void add(ll k, ll m) {
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y = erase
            (y));
        while ((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
    }
    ll query(ll x) {
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};

void solve(){
    cin >> n >> x; dp[n] = 0;
    for(int i = 1; i <= n; i++) cin >> s[i]; s[0] = 0;
    for(int i = 1; i <= n; i++) cin >> f[i]; f[0] = x;

    LineContainer cht;
    cht.add(-s[n], -dp[n]);

    for(int at = n - 1; at >= 0; at--) {
        dp[at] = -cht.query(f[at]);
        cht.add(-s[at], -dp[at]);
    }

    cout << dp[0] << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

8.6 Necessary Cities

There are n cities and m roads between them. There is a route between any two cities. A city is called necessary if there is no route between some other two cities after removing that city (and adjacent roads). Your task is to find all necessary cities.

Input
The first input line has two integers n and m: the number of cities and roads. The cities are numbered $1, 2, \ldots, n$. After this, there are m lines that describe the roads. Each line has two integers a and b: there is a road between cities a and b. There is at most one road between two cities, and every road connects two distinct cities.

Output
First print an integer k: the number of necessary cities. After that, print a list of k cities. You may print the cities in any order.

Constraints
• $2 \le n \le 10^5$ • $1 \le m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

vector<vi> G;

struct block_cut_tree {
    vector<vector<int>> g, blocks, tree;
    vector<vector<pair<int, int>>> edgblocks;
    stack<int> s;
    stack<pair<int, int>> s2;
    vector<int> id, art, pos;

    block_cut_tree(vector<vector<int>> g_) : g(g_) {
        int n = g.size();
        id.resize(n, -1), art.resize(n), pos.resize(n);
        build();
    }

    int dfs(int i, int& t, int p = -1) {
        int lo = id[i] = t++;
        s.push(i);

        if (p != -1) s2.emplace(i, p);
        for (int j : g[i]) if (j != p and id[j] != -1) s2.
            emplace(i, j);

        for (int j : g[i]) if (j != p) {
```

```
            if (id[j] == -1) {
                int val = dfs(j, t, i);
                lo = min(lo, val);

                if (val >= id[i]) {
                    art[i]++;
                    blocks.emplace_back(1, i);
                    while (blocks.back().back() != j)
                        blocks.back().push_back(s.top()), s.
                            pop();

                    edgblocks.emplace_back(1, s2.top()), s2.
                        pop();
                    while (edgblocks.back().back() != pair(j,
                        i))
                        edgblocks.back().push_back(s2.top()),
                            s2.pop();
                }
                // if (val > id[i]) aresta i-j eh ponte
            }
            else lo = min(lo, id[j]);
        }

        if (p == -1 and art[i]) art[i]--;
        return lo;
    }

    void build() {
        int t = 0;
        for (int i = 0; i < g.size(); i++) if (id[i] == -1)
            dfs(i, t, -1);

        tree.resize(blocks.size());
        for (int i = 0; i < g.size(); i++) if (art[i])
            pos[i] = tree.size(), tree.emplace_back();

        for (int i = 0; i < blocks.size(); i++) for (int j :
            blocks[i]) {
            if (!art[j]) pos[j] = i;
            else tree[i].push_back(pos[j]), tree[pos[j]].
                push_back(i);
        }
    }
};

void solve(){
    int n, m; cin >> n >> m;

    G.resize(n);

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G[a].push_back(b);
        G[b].push_back(a);
    }

    block_cut_tree BCT(G);

    vector<int> ans;
    for(int i = 0; i < n; i++) if(BCT.art[i] >= 1) ans.
        push_back(i + 1);
    cout << ans.size() << '\n';
    for(auto x : ans) cout << x << ' ';
    cout << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

8.7 Necessary Roads

There are n cities and m roads between them. There is a route between any two cities. A road is called necessary if there is no route between some two cities after removing that road. Your task is to find all necessary roads.

Input
The first input line has two integers n and m: the number of cities and roads. The cities are numbered $1, 2, \ldots, n$. After this, there are m lines that describe the roads. Each line has two integers a and b: there is a road between cities a and b. There is at most one road between two cities, and every road connects two distinct cities.

Output
First print an integer k: the number of necessary roads. After that, print k lines that describe the roads. You may print the roads in any order.

Constraints
• $2 \leq n \leq 10^5$ • $1 \leq m \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

vector<vi> G;

struct block_cut_tree {
    vector<vector<int>> g, blocks, tree;
    vector<vector<pair<int, int>>> edgblocks;
    stack<int> s;
    stack<pair<int, int>> s2;
    vector<int> id, art, pos;

    block_cut_tree(vector<vector<int>> g_) : g(g_) {
        int n = g.size();
        id.resize(n, -1), art.resize(n), pos.resize(n);
        build();
    }

    int dfs(int i, int& t, int p = -1) {
        int lo = id[i] = t++;
        s.push(i);

        if (p != -1) s2.emplace(i, p);
        for (int j : g[i]) if (j != p and id[j] != -1) s2.
            emplace(i, j);

        for (int j : g[i]) if (j != p) {
            if (id[j] == -1) {
                int val = dfs(j, t, i);
                lo = min(lo, val);

                if (val >= id[i]) {
                    art[i]++;
                    blocks.emplace_back(1, i);
                    while (blocks.back().back() != j)
                        blocks.back().push_back(s.top()), s.
                            pop();

                    edgblocks.emplace_back(1, s2.top()), s2.
                        pop();
                    while (edgblocks.back().back() != pair(j,
                        i))
                        edgblocks.back().push_back(s2.top()),
                            s2.pop();
                }
                // if (val > id[i]) aresta i-j eh ponte
            }
            else lo = min(lo, id[j]);
        }

        if (p == -1 and art[i]) art[i]--;
        return lo;
    }

    void build() {
        int t = 0;
        for (int i = 0; i < g.size(); i++) if (id[i] == -1)
            dfs(i, t, -1);

        tree.resize(blocks.size());
        for (int i = 0; i < g.size(); i++) if (art[i])
            pos[i] = tree.size(), tree.emplace_back();

        for (int i = 0; i < blocks.size(); i++) for (int j :
            blocks[i]) {
            if (!art[j]) pos[j] = i;
            else tree[i].push_back(pos[j]), tree[pos[j]].
                push_back(i);
        }
    }
};

void solve(){
    int n, m; cin >> n >> m;

    G.resize(n);

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        G[a].push_back(b);
        G[b].push_back(a);
    }

    block_cut_tree BCT(G);

    vector<pii> ans;

    for(auto& edge_block : BCT.edgblocks) {
        if(edge_block.size() == 1) {
            ans.push_back(edge_block[0]);
        }
    }

    cout << ans.size() << '\n';

    for(auto x : ans) {
        cout << x.first + 1 << ' ' << x.second + 1 << '\n';
    }
}

signed main() {
```

```
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

8.8 Parcel Delivery

There are n cities and m routes through which parcels can be carried from one city to another city. For each route, you know the maximum number of parcels and the cost of a single parcel. You want to send k parcels from Syrjälä to Lehmälä. What is the cheapest way to do that?

Input
The first input line has three integers n, m and k: the number of cities, routes and parcels. The cities are numbered $1, 2, \ldots, n$. City 1 is Syrjälä and city n is Lehmälä. After this, there are m lines that describe the routes. Each line has four integers a, b, r and c: there is a route from city a to city b, at most r parcels can be carried through the route, and the cost of each parcel is c.

Output
Print one integer: the minimum total cost or -1 if there are no solutions.

Constraints
• $2 \le n \le 500$ • $1 \le m \le 1000$ • $1 \le k \le 100$ • $1 \le a, b \le n$ • $1 \le r, c \le 1000$

Código-fonte:

```
#include <bits/stdc++.h>
#include <bits/extc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const ll INF = numeric_limits<ll>::max() / 4;

struct MCMF {
    struct edge {
        int from, to, rev;
        ll cap, cost, flow;
    };
    int N;
    vector<vector<edge>> ed;
    vi seen;
    vector<ll> dist, pi;
    vector<edge*> par;

    MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}

    void addEdge(int from, int to, ll cap, ll cost) {
        if (from == to) return;
        ed[from].push_back(edge{ from,to,sz(ed[to]),cap,cost,0 });
        ed[to].push_back(edge{ to,from,sz(ed[from])-1,0,-cost,0 });
    }

    void path(int s) {
        fill(all(seen), 0);
        fill(all(dist), INF);
        dist[s] = 0; ll di;

        __gnu_pbds::priority_queue<pair<ll, int>> q;
        vector<decltype(q)::point_iterator> its(N);
        q.push({ 0, s });

        while (!q.empty()) {
            s = q.top().second; q.pop();
            seen[s] = 1; di = dist[s] + pi[s];
            for (edge& e : ed[s]) if (!seen[e.to]) {
                ll val = di - pi[e.to] + e.cost;
                if (e.cap - e.flow > 0 && val < dist[e.to]) {
                    dist[e.to] = val;
                    par[e.to] = &e;
                    if (its[e.to] == q.end())
                        its[e.to] = q.push({ -dist[e.to], e.to });
                    else
                        q.modify(its[e.to], { -dist[e.to], e.to });
                }
            }
        }
        rep(i,0,N) pi[i] = min(pi[i] + dist[i], INF);
    }

    pair<ll, ll> maxflow(int s, int t) {
        ll totflow = 0, totcost = 0;
        while (path(s), seen[t]) {
            ll fl = INF;
            for (edge* x = par[t]; x; x = par[x->from])
                fl = min(fl, x->cap - x->flow);

            totflow += fl;
            for (edge* x = par[t]; x; x = par[x->from]) {
                x->flow += fl;
                ed[x->to][x->rev].flow -= fl;
            }
        }
        rep(i,0,N) for(edge& e : ed[i]) totcost += e.cost * e.flow;
        return {totflow, totcost/2};
    }
};

void solve(){
    int n, m, k; cin >> n >> m >> k;

    MCMF G(n + 1);
    G.addEdge(0, 1, k, 0);

    for(int i = 0; i < m; i++) {
        int a, b, r, c; cin >> a >> b >> r >> c;
        G.addEdge(a, b, r, c);
    }

    auto ans = G.maxflow(0, n);
    if(ans.first == k) cout << ans.second << '\n';
    else cout << -1 << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

8.9 Signal Processing

You are given two integer sequences: a signal and a mask. Your task is to process the signal by moving the mask through the signal from left to right. At each mask position calculate the sum of products of aligned signal and mask values in the part where the signal and the mask overlap.

Input
The first input line consists of two integers n and m: the length of the signal and the length of the mask. The next line consists of n integers $a_1, a_2, \ldots, a_n$ defining the signal. The last line consists of m integers $b_1, b_2, \ldots, b_m$ defining the mask.

Output
Print $n + m - 1$ integers: the sum of products of aligned values at each mask position from left to right.

Constraints
• $1 \le n, m \le 2 \cdot 10^5$ • $1 \le a_i, b_i \le 100$

Código-fonte:

```
#include <bits/stdc++.h>

#define int long long
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;
typedef vector<ll> vl;

const ll mod = (17LL << 27) + 1, root = 3; // = 2281701377
// const ll mod = (119 << 23) + 1, root = 62; // = 998244353
// For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 << 21
// and 483 << 21 (same root). The last two are > 10^9.

ll modpow(ll b, ll e) {
    ll ans = 1;
    for (; e; b = b * b % mod, e /= 2)
        if (e & 1) ans = ans * b % mod;
    return ans;
}

typedef vector<ll> vl;
```

```cpp
void ntt(vl &a) {
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vl rt(2, 1);
    for (static int k = 2, s = 2; k < n; k *= 2, s++) {
        rt.resize(n);
        ll z[] = {1, modpow(root, mod >> s)};
        rep(i,k,2*k) rt[i] = rt[i / 2] * z[i & 1] % mod;
    }
    vi rev(n);
    rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
            ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i
                + j];
            a[i + j + k] = ai - z + (z > ai ? mod : 0);
            ai += (ai + z >= mod ? z - mod : z);
        }
}
vl conv(const vl &a, const vl &b) {
    if (a.empty() || b.empty()) return {};
    int s = sz(a) + sz(b) - 1, B = 32 - __builtin_clz(s),
        n = 1 << B;
    int inv = modpow(n, mod - 2);
    vl L(a), R(b), out(n);
    L.resize(n), R.resize(n);
    ntt(L), ntt(R);
    rep(i,0,n)
        out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod
            ;
    ntt(out);
    return {out.begin(), out.begin() + s};
}

void solve(){
    int n, m; cin >> n >> m;
    vector<int> a(n), b(m);
    for(auto &x : a) cin >> x;
    for(auto &x : b) cin >> x;
    reverse(all(b));

    vl ans = conv(a, b);

    for(auto x : ans) cout << x << ' ';
    cout << endl;
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

8.10 Subarray Squares

Given an array of n elements, your task is to divide into k subarrays. The cost of each subarray is the square of the sum of the values in the subarray. What is the minimum total cost if you act optimally?

Input
The first input line has two integers n and k: the array elements and the number of subarrays. The array elements are numbered $1, 2, \ldots, n$. The second line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print one integer: the minimum total cost.

Constraints
• $1 \le k \le n \le 3000$ • $1 \le x_i \le 10^5$

Código-fonte:

```cpp
#include <bits/stdc++.h>

#define int long long
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef pair<ll, ll> pll;

const int MAX = 3001;

int n, k;
ll dp[MAX], x[MAX], psum[MAX];
ll res[MAX];

struct DP { // Modify at will:
    int lo(int ind) { return 0; }
    int hi(int ind) { return ind; }
    ll f(int ind, int fim) {
        ll at = psum[ind] - psum[fim];
        return at * at + dp[fim];
    }
    void store(int ind, int k, ll v) { res[ind] = v; }
    void rec(int L, int R, int LO, int HI) {
        if (L >= R) return;
        int mid = (L + R) >> 1;
        pair<ll, int> best(LLONG_MAX, LO);
        rep(k, max(LO,lo(mid)), min(HI,hi(mid)))
            best = min(best, make_pair(f(mid, k), k));
        store(mid, best.second, best.first);
        rec(L, mid, LO, best.second+1);
        rec(mid+1, R, best.second, HI);
    }
    void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
};

void solve(){
    cin >> n >> k;
    for(int i = 1; i <= n; i++) {
        cin >> x[i];
        psum[i] = psum[i - 1] + x[i];
        dp[i] = psum[i] * psum[i];
    }

    DP dcdp;

    for(int j = 1; j < k; j++) {
        dcdp.solve(1, n + 1);
        for(int i = 0; i <= n; i++) dp[i] = res[i];
    }

    cout << dp[n] << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

8.11 Substring Reversals

Given a string, your task is to process operations where you reverse a substring of the string. What is the final string after all the operations?

Input
The first input line has two integers n and m: the length of the string and the number of operations. The characters of the string are numbered $1, 2, \ldots, n$. The next line has a string of length n that consists of characters A–Z. Finally, there are m lines that describe the operations. Each line has two integers a and b: you reverse a substring from position a to position b.

Output
Print the final string after all the operations.

Constraints
• $1 \le n, m \le 2 \cdot 10^5$ • $1 \le a \le b \le n$

Código-fonte:

```cpp
#include <bits/stdc++.h>

#define int long long
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef pair<ll, ll> pll;

/**
 * Author: someone on Codeforces
 * Date: 2017-03-14
 * Source: folklore
 * Description: A short self-balancing tree. It acts as a
 *  sequential container with log-time splits/joins, and
 *  is easy to augment with additional data.
 * Time: $O(\log N)$
 * Status: stress-tested
 */

struct Node {
    Node *l = 0, *r = 0;
    int val, y, c = 1;
```

```
35      Node(int val) : val(val), y(rand()) {}
36      void recalc();
37  };
38
39  int cnt(Node* n) { return n ? n->c : 0; }
40  void Node::recalc() { c = cnt(l) + cnt(r) + 1; }
41
42  template<class F> void each(Node* n, F f) {
43      if (n) { each(n->l, f); f(n->val); each(n->r, f); }
44  }
45
46  pair<Node*, Node*> split(Node* n, int k) {
47      if (!n) return {};
48      if (cnt(n->l) >= k) { // "n->val >= k" for lower_bound(k)
49          auto [L,R] = split(n->l, k);
50          n->l = R;
51          n->recalc();
52          return {L, n};
53      } else {
54          auto [L,R] = split(n->r,k - cnt(n->l) - 1); // and
                just "k"
55          n->r = L;
56          n->recalc();
57          return {n, R};
58      }
59  }
60
61  Node* merge(Node* l, Node* r) {
62      if (!l) return r;
63      if (!r) return l;
64      if (l->y > r->y) {
65          l->r = merge(l->r, r);
66          return l->recalc(), l;
67      } else {
68          r->l = merge(l, r->l);
69          return r->recalc(), r;
70      }
71  }
72
73  Node* ins(Node* t, Node* n, int pos) {
74      auto [l,r] = split(t, pos);
75      return merge(merge(l, n), r);
76  }
77
78  // Example application: move the range [l, r) to index k
79  void move(Node*& t, int l, int r, int k) {
80      Node *a, *b, *c;
81      tie(a,b) = split(t, l); tie(b,c) = split(b, r - l);
82      if (k <= l) t = merge(ins(a, b, k), c);
83      else t = merge(a, ins(c, b, k - r));
84  }
85
86  void solve(){
87
88  }
89
90  signed main() {
91      ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
92      solve();
93      return 0;
94  }
```

9 Additional Problems I

9.1 Beautiful Permutation II

A permutation of integers $1, 2, \ldots, n$ is called beautiful if there are no adjacent elements whose difference is 1. Given n, construct the lexicographically minimal beautiful permutation if such a permutation exists.

Input
The only line contains an integer n.

Output
Print the lexicographically minimal beautiful permutation of integers $1, 2, \ldots, n$. If there is no such permutation, print "NO SOLUTION".

Constraints
- $1 \leq n \leq 10^6$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

#define int long long

using namespace std;

void solve() {
    int n; cin >> n;
    vector<int> v(n);

    if(n == 1) cout << "1\n";
    else if(n <= 3) cout << "NO SOLUTION\n";
    else {
        int i = 0;
        for(i = 0; 5 * i + 8 < n; i++) {
            v[5 * i + 0] = 1 + 5 * i;
            v[5 * i + 1] = 3 + 5 * i;
            v[5 * i + 2] = 5 + 5 * i;
            v[5 * i + 3] = 2 + 5 * i;
            v[5 * i + 4] = 4 + 5 * i;
        }

        if(n % 5 == 4) {
            v[5 * i + 0] = 5 * i + 2;
            v[5 * i + 1] = 5 * i + 4;
            v[5 * i + 2] = 5 * i + 1;
            v[5 * i + 3] = 5 * i + 3;
        }

        if(n % 5 == 0) {
            v[5 * i + 0] = 5 * i + 1;
            v[5 * i + 1] = 5 * i + 3;
            v[5 * i + 2] = 5 * i + 5;
            v[5 * i + 3] = 5 * i + 2;
            v[5 * i + 4] = 5 * i + 4;
        }

        if(n % 5 == 1) {
            v[5 * i + 0] = 5 * i + 1;
            v[5 * i + 1] = 5 * i + 3;
            v[5 * i + 2] = 5 * i + 5;
            v[5 * i + 3] = 5 * i + 2;
            v[5 * i + 4] = 5 * i + 4;
            v[5 * i + 5] = 5 * i + 6;
        }

        if(n % 5 == 2) {
            v[5 * i + 0] = 5 * i + 1;
            v[5 * i + 1] = 5 * i + 3;
            v[5 * i + 2] = 5 * i + 5;
            v[5 * i + 3] = 5 * i + 2;
            v[5 * i + 4] = 5 * i + 6;
            v[5 * i + 5] = 5 * i + 4;
            v[5 * i + 6] = 5 * i + 7;
        }

        if(n % 5 == 3) {
            v[5 * i + 0] = 5 * i + 1;
            v[5 * i + 1] = 5 * i + 3;
            v[5 * i + 2] = 5 * i + 5;
            v[5 * i + 3] = 5 * i + 2;
            v[5 * i + 4] = 5 * i + 6;
            v[5 * i + 5] = 5 * i + 8;
            v[5 * i + 6] = 5 * i + 4;
            v[5 * i + 7] = 5 * i + 7;
        }

        for(auto x : v) cout << x << ' ';
        cout << '\n';
    }
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

9.2 Distinct Values Sum

You are given an array $x_1, x_2, \ldots, x_n$. Let $d(a, b)$ denote the number of distinct values in the subarray $x_a, x_{a+1}, \ldots, x_b$. Your task is to calculate the sum $\sum_{a=1}^{n} \sum_{b=a}^{n} d(a, b)$, i.e., the sum of $d(a, b)$ for all subarrays.

Input
The first line has an integer n: the array size. The next line has n integers $x_1, x_2, \ldots, x_n$: the array contents.

Output
Print one integer: the required sum.

Constraints
- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq x_i \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long double ld;
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

void solve(){
    int n; cin >> n;
    vector<int> v(n);
    for(auto &x : v) cin >> x;
    auto aux = v;

    sort(all(aux));
    aux.erase(unique(all(aux)), aux.end());

    int max_el = 0;

    for(int i = 0; i < n; i++) {
        v[i] = lower_bound(all(aux), v[i]) - aux.begin();
        max_el = max(max_el, v[i]);
    }

    vector<vi> pos(max_el + 1, vi(1, -1));

    for(int i = 0; i < n; i++) {
        int aux = i - pos[v[i]].back(); pos[v[i]].pop_back();
        pos[v[i]].push_back(aux);
        pos[v[i]].push_back(i);
    }

    for(int i = 0; i <= max_el; i++) {
        if(pos[i].size() == 1) pos[i].pop_back();
        else {
            int aux = n - pos[i].back(); pos[i].pop_back();
            pos[i].push_back(aux);
        }
    }

    ll tot = 0;

    for(int i = 0; i <= max_el; i++) {
        if(pos[i].size() == 0) continue;
        int m = pos[i].size();
        vector<ll> pref_sum(m, 0), pref_sq(m, 0);

        pref_sum[0] = pos[i][0];
        pref_sq[0] = (ll)pos[i][0] * pos[i][0];

        for(int j = 1; j < m; j++) {
            pref_sum[j] = pref_sum[j - 1] + pos[i][j];
            pref_sq[j] = pref_sq[j - 1] + pos[i][j] * pos[i][j];
        }

        ll aux = 0;

        for(int j = 0; j < m; j++) {
            int at = pos[i][j];
            // a_j * a_k * (j + 1), j < k
            aux -= at * (j + 1) * (pref_sum[m - 1] - pref_sum[j]);
            // a_j * a_k * (j), j > k
            if(j > 0) aux += at * (j) * (pref_sum[j - 1]);
        }

        tot += aux;
    }

    cout << ((ll) n ) * (n + 1) * (n + 2) / 6 - tot << endl;


}

signed main() {
```

```
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

9.3 Maximum Average Subarrays

You are given an array of n integers. For each $i = 1, 2, \ldots, n$, your task is to find the subarray ending at index i with the largest average. If there are multiple subarrays with the largest average, you should find the longest one.

Input
The first line has an integer n: the size of the array. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print n integers: the length of the subarray ending at index i with the largest average for each $i = 1, 2, \ldots, n$.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^6$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long double ld;
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

struct Line {
    mutable ll k, m;
    mutable ld p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(ld x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
    static constexpr ld inf = 1e18;
    bool isect(iterator x, iterator y) {
        if (y == end()) return x->p = inf, 0;
        if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
        else x->p = ((ld)y->m - x->m) / ((ld)x->k - y->k);
        return x->p >= y->p;
    }
    void add(ll k, ll m) {
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y = erase
            (y));
        while ((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
    }
};

void solve(){
    int n; cin >> n;
    LineContainer cht;
    cht.add(0, 0);
    ll pref = 0;
    for(int i = 1; i <= n; i++) {
        int v; cin >> v; pref += v;
        cht.add(i, -pref);
        int bst = next(cht.rbegin()) -> k;
        cout << i - bst << ' ';
    }
    cout << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

9.4 Maximum Building I

You are given a map of a forest where some squares are empty and some squares have trees. What is the maximum area of a rectangular building that can be placed in the forest so that no trees must be cut down?

Input
The first input line contains integers n and m: the size of the forest. After this, the forest is described. Each square is empty (.) or has trees (*).

Input
Print the maximum area of a rectangular building.

Constraints
• $1 \leq n, m \leq 1000$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long double ld;
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 1e3 + 5;

bool M[MAX][MAX];
int v[MAX];

vector<int> next_smaller(int n) {
    vector<int> ans(n + 1, -1), st;

    for (int i = 0; i <= n; i++) {
        while (!st.empty() && v[i] < v[st.back()]) {
            ans[st.back()] = i;
            st.pop_back();
        }
        st.push_back(i);
    }
    return ans;
}

vector<int> prev_smaller(int n) {
    vector<int> ans(n + 1, -1), st;

    for (int i = n; i >= 0; i--) {
        while (!st.empty() && v[i] < v[st.back()]) {
            ans[st.back()] = i;
            st.pop_back();
        }
        st.push_back(i);
    }
    return ans;
}

void solve(){
    int n, m; cin >> n >> m;
    for(int i = 0; i <= n + 1; i++) {
        M[i][0] = 1;
        M[i][m + 1] = 1;
    }
    for(int j = 0; j <= m + 1; j++) {
        M[0][j] = 1;
        M[n + 1][j] = 1;
    }
    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= m; j++) {
            char c; cin >> c;
            if(c == '*') M[i][j] = 1;
        }
    }

    int ans = 0;

    for(int i = 1; i <= n; i++) {
        v[0] = -1; v[m + 1] = -1;
        for(int j = 1; j <= m; j++) {
            if(M[i][j]) v[j] = 0;
            else v[j]++;
        }

        auto nxt = next_smaller(m + 1);
        auto prev = prev_smaller(m + 1);

        for(int j = 1; j <= m; j++) {
            int h = v[j], w = nxt[j] - prev[j] - 1;
            ans = max(ans, h * w);
        }
    }

    cout << ans << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

9.5 Nearest Campsites I

A campground is represented as a grid where each square can contain a campsite that is either reserved or free. The distance between two squares (x_1, y_1) and (x_2, y_2) is the Manhattan distance $|x_1 - x_2| + |y_1 - y_2|$. Your task is to find the maximum distance from a free campsite to the nearest reserved campsite.

Input
The first line has two integers n and m: the number of reserved and free campsites. The next n lines describe the locations of the reserved campsites. Each line has two integers x and y. The next m lines describe the locations of the free campsites. Each line has two integers x and y. You can assume that each square contains at most one campsite.

Output
Print one integer: the longest distance to the nearest reserved campsite.

Constraints
• $1 \leq n, m \leq 10^5$ • $1 \leq x, y \leq 10^6$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount

#define int long long
using namespace std;

typedef long long ll;
typedef pair<int, int> pii;

const int MAX = 2e6+10;
const int INF = 1e9;

struct FT {
    vector<ll> pref;
    vector<ll> suff;

    FT(int n) : pref(n, INF), suff(n, INF) {}

    void update(int pos, ll dif) {
        for (int i = pos; i < sz(pref); i |= i + 1) {
            pref[i] = min(pref[i], dif);
        }
        for (int i = pos; i >= 0; i = (i & (i + 1)) - 1) {
            suff[i] = min(suff[i], dif);
        }
    }

    ll query_pref(int pos) { // [0, pos - 1]
        ll res = INF;
        for (int i = pos; i > 0; i &= i - 1) {
            res = min(res, pref[i-1]);
        }
        return res;
    }

    ll query_suff(int pos) { // [pos, MAX - 1]
        ll res = INF;
        for (int i = pos; i < sz(suff); i |= i + 1) {
            res = min(res, suff[i]);
        }
        return res;
    }
};

void solve() {
    int n, m; cin >> n >> m;
    vector<tuple<int, int, int>> p1(n), p2(m);
    vector<int> ans(m);

    for(int i = 0; i < n; i++) {
        int a, b; cin >> a >> b;
        p1[i] = {a + MAX / 2, b + MAX / 2, i};
    }
    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        p2[i] = {a + MAX / 2, b + MAX / 2, i};
    }
    sort(all(p1)); sort(all(p2));
    auto aux1 = p1, aux2 = p2;

    reverse(all(p1));

    FT nw(MAX), sw(MAX);

    for(int i = 0; i < p2.size(); i++) {
        auto [x, y, ind] = p2[i];
        while(p1.size() > 0) {
            auto [a, b, ii] = p1.back();

            if(a <= x) {
                nw.update(b, -a + b);
                sw.update(b, -a - b);
                p1.pop_back();
            }
            else break;
        }

        int ans_nw = nw.query_suff(y) + x - y;
        int ans_sw = sw.query_pref(y) + x + y;

        ans[ind] = min(ans_sw, ans_nw);
    }

    p1 = aux1; p2 = aux2;
    reverse(all(p2));
    FT ne(MAX), se(MAX);

    for(int i = 0; i < p2.size(); i++) {
        auto [x, y, ind] = p2[i];
        while(p1.size() > 0) {
            auto [a, b, ii] = p1.back();

            if(a >= x) {
                ne.update(b, a + b);
                se.update(b, a - b);
                p1.pop_back();
            }
            else break;
        }

        int ans_ne = ne.query_suff(y) - x - y;
        int ans_se = se.query_pref(y) - x + y;

        ans[ind] = min({ans[ind], ans_se, ans_ne});
    }

    int res = -1;
    for(auto x : ans) res = max(res, x);
    cout << res << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
}
```

```
    return 0;
}
```

9.6 Nearest Campsites II

A campground is represented as a grid where each square can contain a campsite that is either reserved or free. The distance between two squares (x_1, y_1) and (x_2, y_2) is the Manhattan distance $|x_1 - x_2| + |y_1 - y_2|$. Your task is to find the distance from each free campsite to the nearest reserved campsite.

Input
The first line has two integers n and m: the number of reserved and free campsites. The next n lines describe the locations of the reserved campsites. Each line has two integers x and y. The next m lines describe the locations of the free campsites. Each line has two integers x and y. You can assume that each square contains at most one campsite.

Output
Print m integers: the distances from each free campsite to the nearest reserved campsite, in the input order.

Constraints
• $1 \le n, m \le 10^5$ • $1 \le x, y \le 10^6$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount

#define int long long
using namespace std;

typedef long long ll;
typedef pair<int, int> pii;

const int MAX = 2e6+10;
const int INF = 1e9;

struct FT {
    vector<ll> pref;
    vector<ll> suff;

    FT(int n) : pref(n, INF), suff(n, INF) {}

    void update(int pos, ll dif) {
        for (int i = pos; i < sz(pref); i |= i + 1) {
            pref[i] = min(pref[i], dif);
        }
        for (int i = pos; i >= 0; i = (i & (i + 1)) - 1) {
            suff[i] = min(suff[i], dif);
        }
    }

    ll query_pref(int pos) { // [0, pos - 1]
        ll res = INF;
        for (int i = pos; i > 0; i &= i - 1) {
            res = min(res, pref[i-1]);
        }
        return res;
    }

    ll query_suff(int pos) { // [pos, MAX - 1]
        ll res = INF;
        for (int i = pos; i < sz(suff); i |= i + 1) {
            res = min(res, suff[i]);
        }
        return res;
    }
};

void solve() {
    int n, m; cin >> n >> m;
    vector<tuple<int, int, int>> p1(n), p2(m);
    vector<int> ans(m);

    for(int i = 0; i < n; i++) {
        int a, b; cin >> a >> b;
        p1[i] = {a + MAX / 2, b + MAX / 2, i};
    }
    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        p2[i] = {a + MAX / 2, b + MAX / 2, i};
    }
    sort(all(p1)); sort(all(p2));
    auto aux1 = p1, aux2 = p2;

    reverse(all(p1));

    FT nw(MAX), sw(MAX);

    for(int i = 0; i < p2.size(); i++) {
        auto [x, y, ind] = p2[i];
        while(p1.size() > 0) {
            auto [a, b, ii] = p1.back();

            if(a <= x) {
                nw.update(b, -a + b);
                sw.update(b, -a - b);
                p1.pop_back();
            }
            else break;
        }

        int ans_nw = nw.query_suff(y) + x - y;
        int ans_sw = sw.query_pref(y) + x + y;

        ans[ind] = min(ans_sw, ans_nw);
    }

    p1 = aux1; p2 = aux2;
    reverse(all(p2));
    FT ne(MAX), se(MAX);

    for(int i = 0; i < p2.size(); i++) {
        auto [x, y, ind] = p2[i];
        while(p1.size() > 0) {
            auto [a, b, ii] = p1.back();

            if(a >= x) {
                ne.update(b, a + b);
                se.update(b, a - b);
                p1.pop_back();
            }
            else break;
        }

        int ans_ne = ne.query_suff(y) - x - y;
        int ans_se = se.query_pref(y) - x + y;

        ans[ind] = min({ans[ind], ans_se, ans_ne});
    }

    for(auto res : ans) cout << res << ' ';
    cout << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

9.7 Sorting Methods

Here are some possible methods using which we can sort the elements of an array in increasing order:
At each step, choose two adjacent elements and swap them. At each step, choose any two elements and swap them. At each step, choose any element and move it to another position. At each step, choose any element and move it to the front of the array.
Given a permutation of numbers $1, 2, \ldots, n$, calculate the minimum number of steps to sort the array using the above methods.

Input
The first input line contains an integer n. The second line contains n integers describing the permutation.

Output
Print four numbers: the minimum number of steps using each method.

Constraints
• $1 \le n \le 2 \cdot 10^5$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

#define int long long

typedef long long ll;

using namespace std;

// inversion count
int sort_swap_adj(int n, vector<int> l) {
    vector<int> r = l; sort(r.begin(), r.end());
    vector<int> v(n), bit(n);

    vector<pair<int, int>> w;
    for (int i = 0; i < n; i++) w.push_back({r[i], i+1});
    sort(w.begin(), w.end());

    for (int i = 0; i < n; i++) {
        auto it = lower_bound(w.begin(), w.end(), make_pair(l
            [i], 0LL));
        if (it == w.end() or it->first != l[i]) return -1;
        v[i] = it->second;
        it->second = -1;
    }
```

```
27
28      ll ans = 0;
29      for (int i = n-1; i >= 0; i--) {
30          for (int j = v[i]-1; j; j -= j&-j) ans += bit[j];
31          for (int j = v[i]; j < n; j += j&-j) bit[j]++;
32      }
33      return ans;
34  }
35
36  int sort_swap(int n, vector<int> v) {
37      vector<int> find(n);
38      int tot = 0;
39      for(int i = 0; i < n; i++) find[v[i]] = i;
40      for(int i = 0; i < n; i++) {
41          if(v[i] != i) {
42              int x = v[i];
43              swap(v[i], v[find[i]]);
44              swap(find[i], find[x]);
45              tot++;
46          }
47      }
48      return tot;
49  }
50
51  // n - lis
52  int sort_move(int n, vector<int> v) {
53      vector<int> ans;
54      for (auto t : v){
55          auto it = lower_bound(ans.begin(), ans.end(), t);
56          if (it == ans.end()) ans.push_back(t);
57          else *it = t;
58      }
59      return n - ans.size();
60  }
61
62  int sort_front(int n, vector<int> v) {
63      int at = n - 1;
64      for(int i = n - 1; i >= 0; i--) {
65          if(v[i] == at) at--;
66      }
67      return at + 1;
68  }
69
70  void solve() {
71      int n; cin >> n;
72      vector<int> v(n);
73      for(int i = 0; i < n; i++) {
74          cin >> v[i]; v[i]--;
75      }
76
77      cout << sort_swap_adj(n, v) << ' ' << sort_swap(n, v) <<
            ' ' << sort_move(n, v) << ' ' << sort_front(n, v)
            << '\n';
78  }
79
80  signed main() {
81      ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
82      solve();
83      return 0;
84  }
```

9.8 Swap Game

You are given a 3×3 grid containing the numbers $1, 2, \ldots, 9$. Your task is to perform a sequence of moves so that the grid will look like this:

1&2&3
4&5&6
7&8&9

On each move, you can swap the numbers in any two adjacent squares (horizontally or vertically). What is the minimum number of moves required?

Input
The input has three lines, and each of them has three integers.

Output
Print one integer: the minimum number of moves.

Código-fonte:

```
1   #include <bits/stdc++.h>
2   #define pb push_back
3   #define all(x) x.begin(), x.end()
4   #define pc __builtin_popcount
5
6   using namespace std;
7
8   vector<int> adj[9];
9   unordered_map<int, int> dp;
10  vector<pair<int, int>> macaco = {{1, 0}, {0, 1}};
11
12  int p[9] = {100000000, 10000000, 1000000, 100000, 10000,
        1000, 100, 10, 1};
13
14  int troca(int at, int i, int j) {
15      int d1 = (at / p[i]) % 10;
16      int d2 = (at / p[j]) % 10;
17      return at + (d2 - d1) * p[i] + (d1 - d2) * p[j];
18  }
19
20  void solve() {
21      for(int i = 0; i < 3; i++) {
22          for(int j = 0; j < 3; j++) {
23              for(auto a : macaco) {
24                  if(a.first + i < 0 || a.first + i > 2 || a.
                        second + j < 0 || a.second + j > 2)
                        continue;
25                  adj[i * 3 + j].emplace_back((i + a.first) * 3
                        + (j + a.second));
26              }
27          }
28      }
29
30      int b = 123'456'789; dp[b] = 0;
31      queue<int> q; q.push(b);
32
33      int target = 0;
34      for(int i = 0; i < 9; i++) {
35          int a; cin >> a;
36          target = 10 * target + a;
37      }
38
39      if(target == b) {
40          cout << "0\n";
41          return;
42      }
43
44      while(!q.empty()) {
45          int at = q.front(); q.pop();
46          int ans = dp[at];
47          for(int k = 0; k < 9; k++) {
48              for(auto x : adj[k]) {
49                  int u = troca(at, k, x);
50                  if (!dp.count(u)) {
51                      dp[u] = ans + 1;
52                      if(u == target) {
53                          cout << ans + 1 << '\n';
54                          return;
55                      }
56                      q.push(u);
57                  }
58              }
59          }
60      }
61  }
62
63  signed main() {
64      ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
65      solve();
66      return 0;
67  }
```

9.9 Water Containers Moves

There are two water containers: container A has volume a and container B has volume b. Your want to measure x units of water using the containers. Initially both containers are empty. On each move, you can fill a container, empty a container or move water from a container to another container. When you move water, you must always fill or empty at least one container. After the moves, container A must have x units of water. Find a sequence of moves where the total amount of moved water is minimum or state that it is impossible to measure the water.

Input
The only line has three integers a, b and x.

Output
First print two integers n and m: the number of moves and the total amount of water moved. After that print a sequence of n moves. Each move must move at least one unit of water and be one of the following:
• FILL A: fill container A • FILL B: fill container B • EMPTY A: empty container A • EMPTY B: empty container B • MOVE A B: move water from container A to container B • MOVE B A: move water from container B to container A
If it is not possible to measure the water, only print -1.

Constraints
• $1 \le a, b, x \le 1000$

Código-fonte:

```
1   #include <bits/stdc++.h>
2   using namespace std;
3   #define pb push_back
4   #define all(x) x.begin(), x.end()
5   #define pc __builtin_popcount
6
7   #define int long long
8
9   const int MAX = 1005;
10  const int INF = 1e18;
11  // A_cheio = (0, 1); B_vazio = (1, 0), etc
12  vector<vector<vector<int>>> dp(2, vector<vector<int>>(2,
        vector<int>(MAX, INF)));
13
14
15  void solve() {
```

```
    int a, b, x; cin >> a >> b >> x;
    dp[0][0][0] = 0; dp[0][0][b] = b;
    dp[1][0][0] = 0; dp[1][0][a] = a;
    dp[0][1][0] = a; dp[0][1][b] = a + b;
    dp[1][1][0] = b; dp[1][1][a] = a + b;


    for(int j = 0; j < 4 * MAX; j++)
    for(int i = 1; i < MAX; i++) {
        // FILL A
        dp[0][1][i] = min(dp[0][1][i], dp[0][0][i] + a);
        // FILL B
        dp[1][1][i] = min(dp[1][1][i], dp[1][0][i] + b);

        // EMPTY A
        dp[0][0][i] = min(dp[0][0][i], dp[0][1][i] + a);
        // EMPTY B
        dp[1][0][i] = min(dp[1][0][i], dp[1][1][i] + b);

        // MOVE A B
        if(i <= b) dp[0][0][i] = min(dp[1][0][i] + i, dp
            [0][0][i]);
        if(i >= a && i <= b) dp[0][0][i] = min(dp[0][1][i - a
            ] + a, dp[0][0][i]);

        if(b + i <= a) dp[1][1][i] = min(dp[1][0][b + i] + b,
             dp[1][1][i]);
        if(a - i >= 0 && b >= a - i) dp[1][1][i] = min(dp
            [1][1][i], dp[0][1][b - (a - i)] + (a - i));

        // MOVE B A
        if(i <= a) dp[1][0][i] = min(dp[0][0][i] + i, dp
             [1][0][i]);
        if(i >= b && i <= a) dp[1][0][i] = min(dp[1][1][i - b
            ] + b, dp[1][0][i]);

        if(a + i <= b) dp[0][1][i] = min(dp[0][0][a + i] + a,
             dp[0][1][i]);
        if(b - i >= 0 && a >= b - i) dp[0][1][i] = min(dp
            [0][1][i], dp[1][1][a - (b - i)] + (b - i));
    }

    vector<string> ans;
    int min_cost = min(dp[1][0][x], dp[1][1][x]);

    if(min_cost >= INF) {
        cout << "-1\n";
        return;
    }

    bool A = 1, c = 1;
    if(dp[1][0][x] < dp[1][1][x]) c = 0;

    while(true) {

        if((A == 0 || A == 1) && c == 0 && x == 0) break;
        if(A == 0 && c == 0 && x == b) {
            ans.push_back("FILL B");
            break;
        }
        if(A == 1 && c == 0 && x == a) {
            ans.push_back("FILL A");
            break;
        }
        if(A == 0 && c == 1 && x == 0) {
            ans.push_back("FILL A");
            break;
        }
        if(A == 1 && c == 1 && x == b) {
            ans.push_back("FILL B");
            break;
        }
        if(A == 0 && c == 1 && x == b) {
            ans.push_back("FILL A");
            ans.push_back("FILL B");
            break;
        }
        if(A == 1 && c == 1 && x == a) {
            ans.push_back("FILL A");
            ans.push_back("FILL B");
            break;
        }

        if(A == 0 && c == 1) {
            // FILL A
            if(dp[0][1][x] == dp[0][0][x] + a) {
                ans.push_back("FILL A");
                c = 0;
            }
            // MOVE B A
            else if(b >= x && a >= b - x && dp[0][1][x] == dp
                 [1][1][a - (b - x)] + (b - x)) {
                ans.push_back("MOVE B A");
                A = 1;
                x = a - (b - x);
            }
            else if(a + x <= b && dp[0][1][x] == dp[0][0][a +
                 x] + a) {
                ans.push_back("MOVE B A");
                c = 0;
                x = a + x;
            }
        }
        else if(A == 1 && c == 1) {
            // FILL B
            if(dp[1][1][x] == dp[1][0][x] + b) {
                ans.push_back("FILL B");
                c = 0;
            }
            // MOVE A B
            else if(a >= x && b >= a - x && dp[1][1][x] == dp
                 [0][1][b - (a - x)] + (a - x)) {
                ans.push_back("MOVE A B");
                A = 0;
                x = b - (a - x);
            }
            else if(b + x <= a && dp[1][1][x] == dp[1][0][b +
                 x] + b) {
                ans.push_back("MOVE A B");
                c = 0;
                x = b + x;
            }
        }
        else if(A == 0 && c == 0) {
            // EMPTY A
            if(dp[0][0][x] == dp[0][1][x] + a) {
                ans.push_back("EMPTY A");
                c = 1;
            }
            else if(x <= b && dp[0][0][x] == dp[1][0][x] + x)
                 {
                ans.push_back("MOVE A B");
                A = 1;
            }
            else if(x >= a && x <= b && dp[0][0][x] == dp
                 [0][1][x - a] + a){
                ans.push_back("MOVE A B");
                c = 1;
                x = x - a;
            }
        }
        else if(A == 1 && c == 0) {
            // EMPTY B
            if(dp[1][0][x] == dp[1][1][x] + b) {
                ans.push_back("EMPTY B");
                c = 1;
            }
            else if(x <= a && dp[1][0][x] == dp[0][0][x] + x)
                 {
                ans.push_back("MOVE B A");
                A = 0;
            }
            else if(x >= b && x <= a && dp[1][0][x] == dp
                 [1][1][x - b] + b){
                ans.push_back("MOVE B A");
                c = 1;
                x = x - b;
            }
        }
    }

    cout << ans.size() << ' ';
    cout << min_cost << '\n';

    reverse(ans.begin(), ans.end());

    for(auto x : ans) cout << x << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

9.10 Writing Numbers

You would like to write a list of positive integers $1, 2, 3, \ldots$ using your computer. However, you can press each key 0–9 at most n times during the process. What is the last number you can write?

Input
The only input line contains the value of n.

Output
Print the last number you can write.

Constraints
- $1 \leq n \leq 10^{18}$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long double ld;
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

ll pot[19], memo[19];
const ll INF = 1e18;

ll ans(ll n) {
```

```
22      if(n == 0) return 0;
23      if(n < 10) return 1;
24      if(n == 10) return 2;
25      ll a = n, k = 0;
26      while(a > 9) {
27          a /= 10;
28          k++;
29      }
30
31      if(a > 1) {
32          return ans(n - pot[k] * a) + memo[k] * a + pot[k];
33      }
34      else {
35          return ans(n - pot[k] * a) + (n - pot[k] * a + 1) +
                memo[k];
36      }
37  }
38
39  void solve(){
40      ll n; cin >> n;
41      ll l = 0, r = INF;
42
43      while(r - l > 1) {
44          ll m = (l + r) / 2;
45          if(ans(m) <= n) l = m;
46          else r = m;
47      }
48
49      cout << l << '\n';
50  }
51
52  signed main() {
53      ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
54      pot[0] = 1;
55      for(int i = 1; i <= 18; i++) {
56          pot[i] = 10 * pot[i - 1];
57      }
58      for(int i = 0; i <= 18; i++) {
59          memo[i] = i * pot[i - 1];
60      }
61      solve();
62      return 0;
63  }
```

10 Additional Problems II

10.1 Book Shop II

You are in a book shop which sells n different books. You know the price, the number of pages and the number of copies of each book. You have decided that the total price of your purchases will be at most x. What is the maximum number of pages you can buy? You can buy several copies of the same book.

Input
The first input line contains two integers n and x: the number of book and the maximum total price. The next line contains n integers $h_1, h_2, \ldots, h_n$: the price of each book. The next line contains n integers $s_1, s_2, \ldots, s_n$: the number of pages of each book. The last line contains n integers $k_1, k_2, \ldots, k_n$: the number of copies of each book.

Output
Print one integer: the maximum number of pages.

Constraints
• $1 \le n \le 100$ • $1 \le x \le 10^5$ • $1 \le h_i, s_i, k_i \le 1000$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long double ld;
typedef long long ll;
typedef pair<ll, ll> pll;
typedef tuple<ll, ll, ll, ll> tll;
typedef pair<int, int> pii;
typedef vector<int> vi;

void solve(){
    int n, x; cin >> n >> x;
    vector<pll> list;

    vector<ll> p(n), h(n), k(n);
    for(int i = 0; i < n; i++) cin >> p[i];
    for(int i = 0; i < n; i++) cin >> h[i];
    for(int i = 0; i < n; i++) cin >> k[i];

    for (int i = 0; i < n; i++) {
        ll c = 1;
        while (k[i] > c) {
            k[i] -= c;
            list.push_back({c * p[i], c * h[i]});
            c *= 2;
        }
        list.push_back({k[i] * p[i], k[i] * h[i]});
    }

    vector<ll> dp(x + 1, 0);

    for(auto item : list) {
        ll peso = item.first, valor = item.second;

        for(int j = x; j >= peso; j--) {
            dp[j] = max(dp[j], dp[j - peso] + valor);
        }
    }

    cout << dp[x] << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

10.2 Food Division

There are n children around a round table. For each child, you know the amount of food they currently have and the amount of food they want. The total amount of food in the table is correct. At each step, a child can give one unit of food to his or her neighbour. What is the minimum number of steps needed?

Input
The first input line contains an integer n: the number of children. The next line has n integers $a_1, a_2, \ldots, a_n$: the current amount of food for each child. The last line has n integers $b_1, b_2, \ldots, b_n$: the required amount of food for each child.

Output
Print one integer: the minimum number of steps.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $0 \le a_i, b_i \le 10^6$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define int long long
using namespace std;

void solve(){
    int n; cin >> n;
    vector<int> x(n), y(n), a(n), S(n);
    for(int i = 0; i < n; i++) cin >> x[i];
    for(int i = 0; i < n; i++) cin >> y[i];

    for(int i = 1; i < n; i++)
    {
        S[i] = S[i - 1] + y[i] - x[i];
    }

    int ans = 0;
    sort(S.begin(), S.end());
    int median = S[n/2];

    for(int i = 0; i < n; i++)
    {
        ans += abs(median - S[i]);
    }

    cout << ans << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

10.3 GCDSubsets

You are given an array of n integers. Your task is to calculate the number of non-empty subsets whose elements' greatest common divisor is equal to k for each $k = 1, \ldots, n$.

Input
The first line has an integer n: the size of the array. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print n integers as specified above modulo $10^9 + 7$.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le x_i \le n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

void solve(){
    int n; cin >> n;
    vector<int> x(n);
    for(int i = 0; i < n; i++) cin >> x[i];
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int t; cin >> t; while(t--)
    solve();
    return 0;
}
```

10.4 Grid Coloring II

You are given an $n \times m$ grid where each cell contains one character A, B or C. For each cell, you must change the character to A, B or C. The new character must be different from the old one. Your task is to change the characters in every cell such that no two adjacent cells have the same character.

Input
The first line has two integers n and m: the number of rows

and columns. The next n lines each have m characters: the description of the grid.

Output
Print n lines each with m characters: the description of the final grid. You may print any valid solution. If no solution exists, just print IMPOSSIBLE.

Constraints
- $1 \leq n, m \leq 500$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef vector<int> vi;
typedef long long int ll;

const int maxn = 2e5 + 5, inf = 2e9, M = 1e9 + 7;

struct TwoSat {
    int N;
    vector<vi> gr;
    vi values; // 0 = false, 1 = true

    TwoSat(int n = 0) : N(n), gr(2*n) {}

    int addVar() { // (optional)
        gr.emplace_back();
        gr.emplace_back();
        return N++;
    }

    void either(int f, int j) {
        f = max(2*f, -1-2*f);
        j = max(2*j, -1-2*j);
        gr[f].push_back(j^1);
        gr[j].push_back(f^1);
    }
    void setValue(int x) { either(x, x); }

    void atMostOne(const vi& li) { // (optional)
        if (li.size() <= 1) return;
        int cur = ~li[0];
        for(int i = 2; i < li.size(); i++) {
            int next = addVar();
            either(cur, ~li[i]);
            either(cur, next);
            either(~li[i], next);
            cur = ~next;
        }
        either(cur, ~li[1]);
    }

    vi val, comp, z; int time = 0;
    int dfs(int i) {
        int low = val[i] = ++time, x; z.push_back(i);
        for(int e : gr[i]) if (!comp[e])
            low = min(low, val[e] ?: dfs(e));
        if (low == val[i]) do {
            x = z.back(); z.pop_back();
            comp[x] = low;
            if (values[x>>1] == -1)
                values[x>>1] = x&1;
        } while (x != i);
        return val[i] = low;
    }

    bool solve() {
        values.assign(N, -1);
        val.assign(2*N, 0); comp = val;
        for(int i = 0; i < 2 * N; i++) if (!comp[i]) dfs(i);
        for(int i = 0; i < 1 * N; i++) if (comp[2*i] == comp
            [2*i+1]) return 0;
        return 1;
    }
};

void solve(){
    int n, m; cin >> n >> m;
    char M[n][m];

    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            cin >> M[i][j];

    TwoSat ts(n * m);
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < m; j++)
        {
            if(i + 1 < n)
            {
                if(M[i][j] == M[i + 1][j])
                {
                    ts.either((i * m + j), ((i + 1) * m + j))
                        ;
                    ts.either(~(i * m + j), ~((i + 1) * m + j
                        ));
                }
                else if(M[i][j] + M[i + 1][j] == 'B' + 'C')
                {
                    ts.either((i * m + j), ((i + 1) * m + j))
                        ;
                }
                else if(M[i][j] + M[i + 1][j] == 'A' + 'B')
                {
                    ts.either(~(i * m + j), ~((i + 1) * m + j
                        ));
                }
                else if(M[i][j] == 'A' && M[i + 1][j] == 'C')
                {
                    ts.either((i * m + j), ~((i + 1) * m + j)
                        );
                }
                else
                {
                    ts.either(~(i * m + j), ((i + 1) * m + j)
                        );
                }
            }

            if(j + 1 < m)
            {
                if(M[i][j] == M[i][j + 1])
                {
                    ts.either((i * m + j), (i * m + (j + 1)))
                        ;
                    ts.either(~(i * m + j), ~(i * m + (j + 1)
                        ));
                }
                else if(M[i][j] + M[i][j + 1] == 'B' + 'C')
                {
                    ts.either((i * m + j), (i * m + (j + 1)))
                        ;
                }
                else if(M[i][j] + M[i][j + 1] == 'A' + 'B')
                {
                    ts.either(~(i * m + j), ~(i * m + (j + 1)
                        ));
                }
                else if(M[i][j] == 'A' && M[i][j + 1] == 'C')
                {
                    ts.either((i * m + j), ~(i * m + (j + 1)
                        ));
                }
                else if(M[i][j] == 'B' && M[i][j + 1] == 'B')
                {
                    ts.either((i * m + j), (i * m + (j + 1)
                        ));
                }
                else
                {
                    ts.either(~(i * m + j), (i * m + (j + 1)
                        ));
                }
            }
        }
    }

    if(!ts.solve())
    {
        cout << "IMPOSSIBLE\n";
        return;
    }

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < m; j++)
        {
            int ans = ts.values[i * m + j];
            if(M[i][j] == 'A') cout << "BC"[ans];
            if(M[i][j] == 'B') cout << "AC"[ans];
            if(M[i][j] == 'C') cout << "AB"[ans];
        }
        cout << '\n';
    }
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

10.5 Increasing Array II

You are given an array of n integers. You want to modify the array so that it is increasing, i.e., every element is at least as large as the previous element. On each move, you can increase or decrease the value of any element by one. What is the minimum number of moves required?

Input
The first input line contains an integer n: the size of the array. Then, the second line contains n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print the minimum number of moves.

Constraints
- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq x_i \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
```

```cpp
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define int long long

using namespace std;

void solve(){
    int n; cin >> n;
    priority_queue<int> pq;
    int tot = 0;
    for(int i = 0; i < n; i++) {
        int x; cin >> x;
        // slope aumenta em 2
        pq.push(x); pq.push(x);
        tot += pq.top() - x;
        pq.pop();
    }
    cout << tot << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

10.6 KSubset Sums I

You are given an array of n integers. Consider the sums of all 2^n subsets of the given array (including the empty subset with sum equal to zero). Your task is to find the k smallest subset sums.

Input
The first line has two integers n and k: the size of the array and the number of subset sums k. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print k integers: the k smallest subset sums in increasing order.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq k \leq \min(2^n, 2 \cdot 10^5)$ • $-10^9 \leq x_i \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long double ld;
typedef long long ll;
typedef pair<ll, ll> pll;
typedef pair<int, int> pii;
typedef vector<int> vi;

void solve(){
    int n, k; cin >> n >> k;
    vector<ll> v(n);
    ll base = 0;

    for(auto &x : v) {
        cin >> x;
        if(x < 0) base += x;
        x = abs(x);
    }
    sort(all(v));

    cout << base << ' '; k--;
    // (minimo, posicao minima)
    priority_queue<pll, vector<pll>, greater<pll>> pq;
    pq.push({base + v[0], 0});

    while(!pq.empty() && k--) {
        auto [at, pos] = pq.top(); pq.pop();
        cout << at << ' ';
        if(pos == n - 1) continue;
        pq.push({at - v[pos] + v[pos + 1], pos + 1});
        pq.push({at + v[pos + 1], pos + 1});
    }
    cout << '\n';

}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

10.7 KSubset Sums II

You are given an array of n integers. Consider the sums of all $\binom{n}{m}$ subsets of the given array with exactly m elements. Your task is to find the k smallest subset sums.

Input
The first line has three integers n, m and k: the size of the array, the size of the subsets and the number of subset sums k. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print k integers: the k smallest subset sums in increasing order.

Constraints
• $1 \leq m < n \leq 2 \cdot 10^5$ • $1 \leq k \leq \min\left(\binom{n}{m}, 2 \cdot 10^5\right)$ • $-10^9 \leq x_i \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long double ld;
typedef long long ll;
typedef pair<ll, ll> pll;
typedef tuple<ll, ll, ll, ll> tll;
typedef pair<int, int> pii;
typedef vector<int> vi;

void solve(){
    int n, m, k; cin >> n >> m >> k;
    vector<ll> v(n);
    ll base = 0;

    for(auto &x : v) {
        cin >> x;
    }
    sort(all(v));

    for(int i = 0; i < m; i++) base += v[i];

    // (minimo possivel, ficha que ta mexendo, pos_atual, pos_final)
    priority_queue<tll, vector<tll>, greater<tll>> pq;

    pq.push({base, m - 1, m - 1, n - 1});
    vector<ll> ans;

    while(!pq.empty() && ans.size() < k) {
        auto [mini, at, pos, fim] = pq.top(); pq.pop();
        ans.push_back(mini);

        if(pos + 1 <= fim)
            pq.push({mini - v[pos] + v[pos + 1], at, pos + 1, fim});

        if(at >= 1 && at <= pos - 1)
            pq.push({mini - v[at - 1] + v[at], at - 1, at, pos - 1});
    }

    for(auto x : ans) cout << x << ' ';
    cout << '\n';

}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

10.8 Knight Moves Queries

There is a knight on an infinite chessboard. The rows and columns are 1-indexed. Your task is to efficiently process queries of the form: when the knight starts at position (x, y), what is the minimum number of moves the knight needs to do to reach the top-left corner.

Input
The first line has an integer n: the number of queries. After this, there are n lines. Each line has two integers x and y: the knight position.

Output
For each query, print the minimum number of moves.

Constraints
• $1 \leq n \leq 10^5$ • $1 \leq x, y \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

void solve(){
    int i, j; cin >> i >> j;
    i--; j--;

    int ans = 0;
    if(j <= i / 2)
    {
        ans = (i / 2);
        if(i == 1 && j == 0)
            ans = (3);

        else if((i % 4 == 1 || i % 4 == 2) && j % 2 == 0)
            ans = (i / 2 + 1);

        else if((i % 4 == 3 || i % 4 == 0) && j % 2 == 1)
            ans = (i / 2 + 1);

        else if(i % 4 == 3 && j % 2 == 0)
            ans = (i / 2 + 2);

        else if(i % 4 == 1 && j % 2 == 1)
            ans = (i / 2 + 2);
    }

    else if(i <= j / 2)
    {
        ans = (j / 2);
        if(j == 1 && i == 0)
            ans = (3);

        else if((j % 4 == 1 || j % 4 == 2) && i % 2 == 0)
            ans = (j / 2 + 1);

        else if((j % 4 == 3 || j % 4 == 0) && i % 2 == 1)
            ans = (j / 2 + 1);

        else if(j % 4 == 3 && i % 2 == 0)
            ans = (j / 2 + 2);

        else if(j % 4 == 1 && i % 2 == 1)
            ans = (j / 2 + 2);
    }

    else
    {
        ans = ((i + j) / 3);
        if(j == 1 && i == 1)
            ans = (4);

        else if(j == 2 && i == 2)
            ans = (4);

        else if((i + j) % 3 == 1)
            ans = ((i + j) / 3 + 1);

        else if((i + j) % 3 == 2)
            ans = ((i + j) / 3 + 2);
    }

    cout << ans << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int t; cin >> t; while(t--)
    solve();
    return 0;
}
```

10.9 Maximum Building II

You are given a map of a forest where some squares are empty and some squares have trees. You want to place a rectangular building in the forest so that no trees need to be cut down. For each building size, your task is to calculate the number of ways you can do this.

Input
The first input line contains integers n and m: the size of the forest. After this, the forest is described. Each square is empty (.) or has trees (*).

Output
Print n lines each containing m integers.

Constraints
• $1 \leq n, m \leq 1000$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long double ld;
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 1e3 + 5;

bool M[MAX][MAX];
bool mark[MAX][MAX];
int pref[MAX][MAX];
int v[MAX];

vector<int> next_smaller(int n) {
    vector<int> ans(n + 1, -1), st;

    for (int i = 0; i <= n; i++) {
        while (!st.empty() && v[i] < v[st.back()]) {
            ans[st.back()] = i;
            st.pop_back();
        }
        st.push_back(i);
    }
    return ans;
}

vector<int> prev_smaller(int n) {
    vector<int> ans(n + 1, -1), st;

    for (int i = n; i >= 0; i--) {
        while (!st.empty() && v[i] < v[st.back()]) {
            ans[st.back()] = i;
            st.pop_back();
        }
        st.push_back(i);
    }
    return ans;
}

void solve(){
    int n, m; cin >> n >> m;
    for(int i = 0; i <= n + 1; i++) {
        M[i][0] = 1;
        M[i][m + 1] = 1;
    }
    for(int j = 0; j <= m + 1; j++) {
        M[0][j] = 1;
        M[n + 1][j] = 1;
    }
    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= m; j++) {
            char c; cin >> c;
            if(c == '*') M[i][j] = 1;
        }
    }

    int ans = 0;

    for(int i = 1; i <= n; i++) {
        v[0] = -1; v[m + 1] = -1;
        for(int j = 1; j <= m; j++) {
            if(M[i][j]) v[j] = 0;
            else v[j]++;
        }

        auto nxt = next_smaller(m + 1);
        auto prev = prev_smaller(m + 1);

        vector<pair<int, int>> unmark;

        for(int j = 1; j <= m; j++) {
            if(mark[nxt[j]][prev[j]]) continue;
            mark[nxt[j]][prev[j]] = 1; unmark.push_back({nxt[
                j], prev[j]});

            int h1 = max(1, max(v[nxt[j]], v[prev[j]]) + 1);
            int h = v[j], w = nxt[j] - prev[j] - 1;

            if (h1 <= h) {
                pref[h1][w]++;
                pref[h + 1][w]--;
            }
        }

        for(auto [x, y] : unmark) mark[x][y] = 0;
    }

    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= m; j++) {
            pref[i][j] += pref[i - 1][j];
        }
    }

    for(int i = 1; i <= n; i++) {
        for(int j = m - 1; j >= 1; j--) {
            pref[i][j] += pref[i][j + 1];
```

```
        }
    }

    for(int i = 1; i <= n; i++) {
        for(int j = m - 1; j >= 1; j--) {
            pref[i][j] += pref[i][j + 1];
        }
    }

    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= m; j++) {
            cout << pref[i][j] << ' ';
        }
        cout << '\n';
    }

}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

10.10 Mex Grid Queries

Consider a two-dimensional grid whose rows and columns are 1-indexed. Each square contains the smallest nonnegative integer that does not appear to the left on the same row or above on the same column. Your task is to calculate the value at square (y, x).

Input
The only input line contains two integers y and x.

Output
Print one integer: the value at square (y, x).

Constraints
• $1 \le y, x \le 10^9$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long double ld;
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

void solve(){
    int x, y; cin >> x >> y;
    cout << ((x - 1) ^ (y - 1)) << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

10.11 Minimum Cost Pairs

Given an array of n integers, consider pairings of k pairs. Each number can appear in at most one pair, and the cost of a pair (a, b) is $|a - b|$. The cost of a pairing is the sum of all costs of the pairs. Calculate the minimum costs of pairings for $k = 1, 2, \ldots, \lfloor n/2 \rfloor$.

Input
The first line has an integer: the array size. The next line has n integers $x_1, x_2, \ldots, x_n$: the array contents.

Output
Print $\lfloor n/2 \rfloor$ integers: the minimum costs of pairings.

Constraints
• $2 \le n \le 2 \cdot 10^5$ • $1 \le x_i \le 10^9$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define int long long

using namespace std;

const int INF = 10000000000000000;

void solve(){
    int n; cin >> n;
    vector<int> x(n);

    // diferenca, L, R
    set<tuple<int, int, int>> d;
    // elementos com L = i, R = i
    vector<tuple<int, int, int>> L(n - 1), R(n - 1);

    for(int i = 0; i < n; i++) cin >> x[i];
    sort(all(x));

    for(int i = 0; i < n - 1; i++) {
        d.insert({x[i + 1] - x[i], i, i});
        L[i] = {x[i + 1] - x[i], i, i};
        R[i] = {x[i + 1] - x[i], i, i};
    }

    int tot = 0;

    for(int k = 1; k <= n / 2; k++) {
        auto top = *d.begin(); d.erase(d.begin());
        tot += get<0>(top);

        tuple<int, int, int> a, b;
        // elemento tq R = top.L - 1
        if(get<1>(top) == 0) a = {INF, 0, 2 * n}; // INF pq
            trocar as pontas so piora
        else a = R[get<1>(top) - 1];

        // elemento tq L = top.R + 1
        if(get<2>(top) == n - 2) b = {INF, 2 * n, n - 2};
        else b = L[get<2>(top) + 1];

        d.insert({get<0>(a) + get<0>(b) - get<0>(top), get
            <1>(a), get<2>(b)});
        L[get<1>(a)] = {get<0>(a) + get<0>(b) - get<0>(top),
            get<1>(a), get<2>(b)};
        R[get<2>(b)] = {get<0>(a) + get<0>(b) - get<0>(top),
            get<1>(a), get<2>(b)};
        d.erase(a); d.erase(b);

        cout << tot << ' ';
    }

    cout << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

10.12 Programmersand Artists

A company wants to hire a programmers and b artists. There are a total of n applicants, and each applicant can become either a programmer or an artist. You know each applicant's programming and artistic skills. Your task is to select the new employees so that the sum of their skills is maximum.

Input
The first input line has three integers a, b and n: the required number of programmers and artists, and the total number of applicants. After this, there are n lines that describe the applicants. Each line has two integers x and y: the applicant's programming and artistic skills.

Output
Print one integer: the maximum sum of skills.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $0 \le a, b \le n$ • $a + b \le n$ • $1 \le x, y \le 10^9$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;
const ll LINF = 4e18;

vector<pair<int, int>> v;
set<pair<int, int>> A, B, difA, difB;

int rem(char x, int pos){
    int s = 0;
    if(x == 'A') {
```

```
        A.erase({v[pos].first, pos});
        difA.erase({v[pos].first - v[pos].second, pos});
        s -= v[pos].first;
    }
    if(x == 'B') {
        B.erase({v[pos].second, pos});
        difB.erase({v[pos].second - v[pos].first, pos});
        s -= v[pos].second;
    }
    return s;
}

int add(char x, int pos){
    int s = 0;
    if(x == 'A') {
        A.insert({v[pos].first, pos});
        difA.insert({v[pos].first - v[pos].second, pos});
        s += v[pos].first;
    }
    if(x == 'B') {
        B.insert({v[pos].second, pos});
        difB.insert({v[pos].second - v[pos].first, pos});
        s += v[pos].second;
    }
    return s;
}

void solve(){
    int n, a, b; cin >> a >> b >> n;
    v.resize(n);
    ll s = 0;

    for(int i = 0; i < n; i++) cin >> v[i].first >> v[i].
         second;

    for(int i = 0; i < a + b; i++) {
        s += add('A', i);
    }

    for(int i = 0; i < b; i++) {
        auto it = difA.begin();
        int pos = it -> second;
        s += rem('A', pos);
        s += add('B', pos);
    }

    for(int i = a + b; i < n; i++) {
        ll maxs = 0, s1 = -LINF, s2 = -LINF, s3 = -LINF, s4 =
             -LINF;
        int posA_rem = -1, posB_rem = -1, posA_swap = -1,
            posB_swap = -1;

        // add i as a and remove other from a
        if (!A.empty()) {
            posA_rem = A.begin()->second;
            s1 = (ll)v[i].first - v[posA_rem].first;
        }

        // add i as b and remove other from b
        if (!B.empty()) {
            posB_rem = B.begin()->second;
            s2 = (ll)v[i].second - v[posB_rem].second;
        }

        // add i as a , swap some a to b and remove other
             from b
        if (!A.empty() && !B.empty()) {
            posA_swap = difA.begin()->second;
            posB_rem = B.begin()->second;
            s3 = (ll)v[i].first - v[posA_swap].first + v[
                 posA_swap].second - v[posB_rem].second;
        }

        // add i as b , swap some b to a and remove other
             from a
        if (!A.empty() && !B.empty()) {
            posB_swap = difB.begin()->second;
            posA_rem = A.begin()->second;
            s4 = (ll)v[i].second - v[posB_swap].second + v[
                 posB_swap].first - v[posA_rem].first;
        }

        maxs = max({0LL, s1, s2, s3, s4});
        s += maxs;

        if(maxs == 0) continue;
        else if(maxs == s1) {
            rem('A', posA_rem);
            add('A', i);
        }else if(maxs == s2) {
            rem('B', posB_rem);
            add('B', i);
        }else if(maxs == s3) {
            rem('A', posA_swap);
            rem('B', posB_rem);
            add('A', i);
            add('B', posA_swap);
        }else if(maxs == s4) {
            rem('B', posB_swap);
            rem('A', posA_rem);
            add('B', i);
            add('A', posB_swap);
        }
    }

    cout << s << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    // int t; cin >> t; while(t--)
    solve();
    return 0;
}
```

10.13 School Excursion

A group of n children are coming to Helsinki. There are two possible attractions: a child can visit either Korkeasaari (zoo) or Linnanmäki (amusement park). There are m pairs of children who want to visit the same attraction. Your task is to find all possible alternatives for the number of children that will visit Korkeasaari. The children's wishes have to be taken into account.

Input
The first input line has two integers n and m: the number of children and their wishes. The children are numbered $1, 2, \ldots, n$. After this, there are m lines describing the children's wishes. Each line has two integers a and b: children a and b want to visit the same attraction.

Output
Print a bit string of length n where a one-bit at index i indicates that it is possible that exactly i children visit Korkeasaari (the bit string is to be considered one-indexed).

Constraints
• $1 \le n \le 10^5$ • $0 \le m \le 10^5$ • $1 \le a, b \le n$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long double ld;
typedef long long ll;
typedef pair<ll, ll> pll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 1e5 + 5;
vi g[MAX];
int vis[MAX];

int dfs(int v) {
    vis[v] = 1;
    int at = 1;

    for(auto x : g[v]) {
        if(vis[x]) continue;
        at += dfs(x);
    }

    return at;
}

void solve() {
    int n, m; cin >> n >> m;
    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--; b--;
        g[a].pb(b);
        g[b].pb(a);
```

```
    }

    vector<int> sizes;
    for(int i = 0; i < n; i++) {
        if(!vis[i]) sizes.push_back(dfs(i));
    }
    sort(all(sizes));

    bitset<MAX> dp = 1;

    for(auto x : sizes) {
        dp |= (dp << x);
    }

    for(int i = n - 1; i >= 0; i--) cout << dp[i];
    cout << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

11 Advanced Graph Problems

11.1 Acyclic Graph Edges

Given an undirected graph, your task is to choose a direction for each edge so that the resulting directed graph is acyclic.

Input
The first input line has two integers n and m: the number of nodes and edges. The nodes are numbered $1, 2, \ldots, n$. After this, there are m lines describing the edges. Each line has two distinct integers a and b: there is an edge between nodes a and b.

Output
Print m lines describing the directions of the edges. Each line has two integers a and b: there is an edge from node a to node b. You can print any valid solution.

Constraints
• $1 \leq n \leq 10^5$ • $1 \leq m \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

void solve(){
    int n, m; cin >> n >> m;
    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        cout << min(a, b) << ' ' << max(a, b) << '\n';
    }
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

11.2 Course Schedule II

You want to complete n courses that have requirements of the form "course a has to be completed before course b". You want to complete course 1 as soon as possible. If there are several ways to do this, you want then to complete course 2 as soon as possible, and so on. Your task is to determine the order in which you complete the courses.

Input
The first input line has two integers n and m: the number of courses and requirements. The courses are numbered $1, 2, \ldots, n$. Then, there are m lines describing the requirements. Each line has two integers a and b: course a has to be completed before course b. You can assume that there is at least one valid schedule.

Output
Print one line having n integers: the order in which you complete the courses.

Constraints
• $1 \leq n \leq 10^5$ • $1 \leq m \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 1e5 + 10;
vector<int> G[MAX], ans;
int deg_in[MAX];

void solve(){
    int n, m; cin >> n >> m;
    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        G[b].push_back(a);
        deg_in[a]++;
    }

    priority_queue<int> pq;

    for(int i = n; i >= 1; i--) {
        if(!deg_in[i]) pq.push(i);
    }

    while(!pq.empty()) {
        int at = pq.top(); pq.pop();
        ans.push_back(at);
        for(auto x : G[at]) {
            deg_in[x]--;
            if(deg_in[x] == 0) pq.push(x);
        }
    }

    reverse(all(ans));
    for(auto x : ans) cout << x << ' ';
    cout << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

11.3 Even Outdegree Edges

Given an undirected graph, your task is to choose a direction for each edge so that in the resulting directed graph each node has an even outdegree. The outdegree of a node is the number of edges coming out of that node.

Input
The first input line has two integers n and m: the number of nodes and edges. The nodes are numbered $1, 2, \ldots, n$. After this, there are m lines describing the edges. Each line has two integers a and b: there is an edge between nodes a and b. You may assume that the graph is simple, i.e., there is at most one edge between any two nodes and every edge connects two distinct nodes.

Output
Print m lines describing the directions of the edges. Each line has two integers a and b: there is an edge from node a to node b. You can print any valid solution. If there are no solutions, only print IMPOSSIBLE.

Constraints
• $1 \leq n \leq 10^5$ • $1 \leq m \leq 2 \cdot 10^5$ • $1 \leq a, b \leq n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 1e5 + 1;

vector<pii> G[MAX];
vector<pair<pii, bool>> edg;
bool vis[MAX];

int deg = 0;

void dfs(int at, int pai) {
    vis[at] = 1;
    for(auto [x, e] : G[at]) {
```

```
        if(!vis[x]) {
            dfs(x, at);
        }
    }
    deg = 0;
    int e_pai = -1;
    for(auto [x, e] : G[at]) {
        if(!edg[e].second && x != pai) {
            edg[e].second = 1;
            edg[e].first = {at, x};
            deg++;
        }
        else if(edg[e].second) {
            deg += (edg[e].first.first == at);
        }
        else {
            e_pai = e;
        }
    }

    if(e_pai == -1) return;

    if(deg % 2 == 0) edg[e_pai].first = {pai, at};
    else edg[e_pai].first = {at, pai};

    edg[e_pai].second = 1;
}

void solve(){
    int n, m; cin >> n >> m;

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        G[a].push_back({b, i});
        G[b].push_back({a, i});
        edg.push_back({{a, b}, 0});
    }

    for(int i = 1; i <= n; i++) {
        if(!vis[i]) dfs(i, 0);
        if(deg % 2) {
            cout << "IMPOSSIBLE\n";
            return;
        }
    }

    for(auto [par, def] : edg) {
        cout << par.first << ' ' << par.second << '\n';
    }

}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

11.4 Graph Coloring

You are given a simple graph with n nodes and m edges. Your task is to use the minimum possible number of colors to color each node such that no edge connects two nodes of the same color.

Input
The first line has two integers n and m: the number of nodes and edges. The nodes are numbered $1, 2, \ldots, n$. Then there are m lines describing the edges. Each line has two integers a and b: there is an edge connecting nodes a and b.

Output
First, print an integer k: the minimum number of colors. After this, print n integers $c_1, c_2, \ldots, c_n$: the colors of the nodes. The colors should satisfy $1 \le c_i \le k$. You may print any valid solution.

Constraints
• $1 \le n \le 16$ • $0 \le m \le \frac{n(n-1)}{2}$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 16;

vi G[MAX];
pii dp[1 << MAX];

bool check(int mask) {
    for(int i = 0; i < MAX; i++) {
        if(!((1 << i) & mask)) continue;
        for(auto x : G[i])
        if((1 << x) & mask)
            return false;
    }
    return true;
}

void solve(){
    int n, m; cin >> n >> m;

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        G[a - 1].push_back(b - 1);
        G[b - 1].push_back(a - 1);
    }

    for(int mask = 1; mask < (1 << n); mask++) {
        if(check(mask)) {
            dp[mask] = {1, mask};
            continue;
        }

        for (int x = mask; x; ) {
            --x &= mask;
            if(dp[x].first != 1) continue;
            int novo = dp[x].first + dp[mask ^ x].first;
            if(dp[mask].first == 0 || dp[mask].first > novo)
                {
                dp[mask].first = novo;
                dp[mask].second = x;
            }
        }
    }

    int k = (1 << n) - 1;
    cout << dp[k].first << '\n';
    vector<int> ans(MAX, 0);
    int c = 1;
    while(k != 0) {
        for(int i = 0; i < MAX; i++) {
            if((1 << i) & dp[k].second) ans[i] = c;
        }
        k = dp[k].second ^ k;
        c++;
    }
    for(int i = 0; i < n; i++) {
        cout << ans[i] << ' ';
    }
    cout << '\n';

}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

11.5 Split Into Two Paths

You are given an acyclic directed graph with n nodes and m edges. Determine whether two paths can be formed in the graph such that each node of the graph appears in exactly one of the paths. Note that all edges of the graph do not need to appear in the paths.

Input
The first line has two integers n and m: the number of nodes and the number of edges. The nodes are numbered $1, 2, \ldots, n$. After this, there are m lines which describe the edges. Each line has two integers a and b: there is an edge in the graph from node a to node b.

Output
First print the line YES if the paths can be formed, or NO otherwise. If the paths can be formed, print them on the following two lines. At the beginning of both lines, print the amount of nodes in the path and then the nodes of the path in order. There must be an edge in the graph between subsequent nodes. One of the paths may contain zero nodes. If there exist multiple solutions, you can print any solution.

Constraints
• $2 \le n \le 2 \cdot 10^5$ • $0 \le m \le 5 \cdot 10^5$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back

```

```cpp
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;


vi topoSort(const vector<vi>& gr) {
    vi indeg(sz(gr)), q;
    for (auto& li : gr) for (int x : li) indeg[x]++;
    rep(i,0,sz(gr)) if (indeg[i] == 0) q.push_back(i);
    rep(j,0,sz(q)) for (int x : gr[q[j]])
        if (--indeg[x] == 0) q.push_back(x);
    return q;
}

void solve(){
    int n, m; cin >> n >> m;
    vector<vi> G(n+1), Grev(n+1);

    for(int i = 1; i <= n; i++) {
        G[0].push_back(i);
        Grev[i].push_back(0);
    }

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        G[a].push_back(b);
        Grev[b].push_back(a);
    }

    auto top = topoSort(G);
    vi rec(n+1);
    set<int> S = {0};

    vector<bool> liga(n + 1);
    for(int i = 1; i <= n; i++) {
        int v = top[i], u = top[i-1];
        for(auto x : Grev[v]) {
            if(x == u) {
                liga[i] = true;
                break;
            }
        }
    }

    for(int i = 2; i <= n; i++) {
        int v = top[i], u = top[i-1];
        int q = -1;

        for(auto x : Grev[v]) {
            auto it = S.find(x);
            if(it != S.end()) {
                q = *it;
                break;
            }
        }

        if(!liga[i]) {
            if(q != -1) {
                rec[i] = q;
                S.clear();
                S.insert(u);
            }
            else {
                cout << "NO\n";
                return;
            }
        }
        else {
            if(q != -1) {
                rec[i] = q;
                S.insert(u);
            }
        }
    }

    cout << "YES\n";
    vi color(n + 1);
    color[top[n]] = 1;
    for(int i = n; i >= 1; i--) {
        int v = top[i];
        int u = top[i-1];

        if(color[u] == 0 && liga[i]) {
            color[u] = color[v];
        }
        else if(color[u] == 0 && !liga[i]) {
            color[u] = 3 - color[v];
        }

        if(color[u] != color[v]) {
            color[rec[i]] = color[v];
        }
    }

    vi c[2];
    for(int i = 1; i <= n; i++) {
        c[color[top[i]] - 1].push_back(top[i]);
    }

    for(int q = 0; q <= 1; q++){
        cout << c[q].size() << ' ';
        for(auto x : c[q]) cout << x << ' '; cout << '\n';
    }
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

11.6 Strongly Connected Edges

Given an undirected graph, your task is to choose a direction for each edge so that the resulting directed graph is strongly connected.

Input
The first input line has two integers n and m: the number of nodes and edges. The nodes are numbered $1, 2, \ldots, n$. After this, there are m lines describing the edges. Each line has two integers a and b: there is an edge between nodes a and b. You may assume that the graph is simple, i.e., there are at most one edge between two nodes and every edge connects two distinct nodes.

Output
Print m lines describing the directions of the edges. Each line has two integers a and b: there is an edge from node a to node b. You can print any valid solution. If there are no solutions, only print IMPOSSIBLE.

Constraints
- $1 \le n \le 10^5$
- $1 \le m \le 2 \cdot 10^5$
- $1 \le a, b \le n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 1e5 + 1;

vi G[MAX];
int vis[MAX];
int pai[MAX];

int cnt = 0;

void dfs(int at) {
    cnt++; vis[at] = cnt;
    for(auto x : G[at]) {
        if(!vis[x]) {
            pai[x] = at;
            dfs(x);
        }
    }
}

vi val, comp, z, cont;
int Time, ncomps;
template<class G, class F> int dfs(int j, G& g, F& f) {
    int low = val[j] = ++Time, x; z.push_back(j);
    for (auto e : g[j]) if (comp[e] < 0)
        low = min(low, val[e] ?: dfs(e,g,f));
```

```
    if (low == val[j]) {
        do {
            x = z.back(); z.pop_back();
            comp[x] = ncomps;
            cont.push_back(x);
        } while (x != j);
        f(cont); cont.clear();
        ncomps++;
    }
    return val[j] = low;
}
template<class G, class F> void scc(G& g, F f) {
    int n = sz(g);
    val.assign(n, 0); comp.assign(n, -1);
    Time = ncomps = 0;
    rep(i,0,n) if (comp[i] < 0) dfs(i, g, f);
}

void solve(){
    int n, m; cin >> n >> m;
    vector<pii> edg(m);

    for(int i = 0; i < m; i++) {
        int a, b; cin >> a >> b;
        G[a].push_back(b);
        G[b].push_back(a);
        edg[i] = {a, b};
    }

    dfs(1);
    if(cnt != n) {
        cout << "IMPOSSIBLE\n";
        return;
    }

    vector<vi> G_aux(n + 1);

    for(auto [a, b] : edg) {
        if(pai[a] == b) {
            G_aux[b].push_back(a);
            continue;
        }
        if(pai[b] == a) {
            G_aux[a].push_back(b);
            continue;
        }
        if(vis[a] > vis[b]) swap(a, b);
        G_aux[b].push_back(a);
    }

    int res = 0;

    scc(G_aux, [&](vi& c) {
        res = max(res, sz(c));
    });

    if (res == n) {
        for(int i = 1; i <= n; i++) {
            for(auto x : G_aux[i]) {
                cout << i << ' ' << x << '\n';
            }
        }
    } else {
        cout << "IMPOSSIBLE\n";
    }

}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

11.7 Tree Coin Collecting I

You are given a tree with n nodes. Some nodes contain a coin. Your task is to answer q queries of the form: what is the shortest length of a path from node a to node b that visits a node with a coin?

Input
The first line contains two integers n and q: the number of nodes and queries. The nodes are numbered $1, 2, \ldots, n$. The second line contains n integers $c_1, c_2, \ldots, c_n$. If $c_i = 1$, node i has a coin. If $c_i = 0$, node i doesn't have a coin. You can assume at least one node has a coin. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b. Finally, there are q lines describing the queries. Each line contains two integers a and b: the start and end nodes.

Output
Print q integers: the answers to the queries.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $1 \le a, b \le n$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 2e5 + 5;
const int LOG = 19;
const int INF = 1e9;

int n, q;
bool coin[MAX];
int tam = 0;
vi adj[MAX];

int up[MAX][LOG];
int dist_to_coin[MAX][LOG];
int depth[MAX];

void calc_dist_to_coin() {
    queue<int> q;

    for(int i = 1; i <= n; i++) {
        dist_to_coin[i][0] = INF;
        if(coin[i]) {
            q.push(i);
            dist_to_coin[i][0] = 0;
        }
    }

    while(!q.empty()) {
        int at = q.front(); q.pop();
        for(auto x : adj[at]) {
            if(dist_to_coin[x][0] == INF) {
                dist_to_coin[x][0] = dist_to_coin[at][0] + 1;
                q.push(x);
            }
        }
    }
}

void dfs(int s, int p) {
    up[s][0] = p;
    depth[s] = depth[p] + 1;
    for(int i = 1; i < LOG; i++) {
        up[s][i] = up[up[s][i-1]][i-1];
        dist_to_coin[s][i] = min(dist_to_coin[s][i-1],
             dist_to_coin[up[s][i-1]][i-1]);
    }

    for(auto x : adj[s]) {
        if(x == p) continue;
        dfs(x, s);
    }
}

int get_lca(int a, int b) {
    if(depth[a] < depth[b]) swap(a, b);

    int diff = depth[a] - depth[b];
    for(int i = 0; i < LOG; i++) {
        if((diff >> i) & 1) a = up[a][i];
    }

    if(a == b) return a;

    for(int i = LOG - 1; i >= 0; i--) {
        if(up[a][i] != up[b][i]) {
            a = up[a][i];
            b = up[b][i];
        }
    }
    return up[a][0];
}

int dist(int a, int b) {
    return depth[a] + depth[b] - 2 * depth[get_lca(a, b)];
}

int min_dist(int a, int b) {
    int ans = INF;
    if(depth[a] < depth[b]) swap(a, b);

    int diff = depth[a] - depth[b];
    for(int i = 0; i < LOG; i++) {
        if((diff >> i) & 1) {
            ans = min(ans, dist_to_coin[a][i]);
            a = up[a][i];
        }
    }

    if(a == b) {
        return min(ans, dist_to_coin[a][0]);
    }

    for(int i = LOG - 1; i >= 0; i--) {
        if(up[a][i] != up[b][i]) {
            ans = min(ans, dist_to_coin[a][i]);
            ans = min(ans, dist_to_coin[b][i]);
            a = up[a][i];
            b = up[b][i];
        }
    }
```

```

    ans = min({ans, dist_to_coin[a][0], dist_to_coin[b][0],
         dist_to_coin[up[a][0]][0]});

    return ans;
}

void solve(){
    cin >> n >> q;
    int root = 0;
    for(int i = 1; i <= n; i++) {
        cin >> coin[i];
        if(coin[i] == 1) root = i;
    }

    for(int i = 0; i < n - 1; i++) {
        int a, b; cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }

    calc_dist_to_coin();
    dfs(root, 0);

    for(int i = 0; i < q; i++) {
        int a, b; cin >> a >> b;
        int ans = dist(a, b);
        ans += 2 * min_dist(a, b);
        cout << ans << '\n';
    }
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

11.8 Tree Coin Collecting II

You are given a tree with n nodes. Some nodes contain a coin. Your task is to answer q queries of the form: what is the shortest length of a path from node a to node b that visits all nodes with coins?

Input
The first line contains two integers n and q: the number of nodes and queries. The nodes are numbered $1, 2, \ldots, n$. The second line contains n integers $c_1, c_2, \ldots, c_n$. If $c_i = 1$, node i has a coin. If $c_i = 0$, node i doesn't have a coin. You can assume at least one node has a coin. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b. Finally, there are q lines describing the queries. Each line contains two integers a and b: the start and end nodes.

Output
Print q integers: the answers to the queries.

Constraints
• $1 \le n, q \le 2 \cdot 10^5$ • $1 \le a, b \le n$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 2e5 + 5;
const int LOG = 19;

bool coin[MAX];
int pai[MAX];
int subtree[MAX];
int tam = 0;
vi adj[MAX];

int up[MAX][LOG];
int depth[MAX];

int calc_subtree(int s, int p) {
    subtree[s] = coin[s];

    for(auto x : adj[s]) {
        if(x == p) continue;
        subtree[s] += calc_subtree(x, s);
    }

    if(subtree[s]) tam++;
    return subtree[s];
}

void dfs(int s, int p) {
    if(subtree[s]) pai[s] = s;
    else pai[s] = pai[p];

    up[s][0] = p;
    depth[s] = depth[p] + 1;
    for(int i = 1; i < LOG; i++) {
        up[s][i] = up[up[s][i-1]][i-1];
    }

    for(auto x : adj[s]) {
        if(x == p) continue;
        dfs(x, s);
    }
}

int get_lca(int a, int b) {
    if(depth[a] < depth[b]) swap(a, b);

    int diff = depth[a] - depth[b];
    for(int i = 0; i < LOG; i++) {
        if((diff >> i) & 1) a = up[a][i];
    }

    if(a == b) return a;

    for(int i = LOG - 1; i >= 0; i--) {
        if(up[a][i] != up[b][i]) {
            a = up[a][i];
            b = up[b][i];
        }
    }
    return up[a][0];
}

int dist(int a, int b) {
    return depth[a] + depth[b] - 2 * depth[get_lca(a, b)];
}

void solve(){
    int n, q; cin >> n >> q;
    int root = 0;
    for(int i = 1; i <= n; i++) {
        cin >> coin[i];
        if(coin[i] == 1) root = i;
    }

    for(int i = 0; i < n - 1; i++) {
        int a, b; cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }

    calc_subtree(root, 0);
    dfs(root, 0);

    for(int i = 0; i < q; i++) {
        int a, b; cin >> a >> b;
        int ans = 2 * (tam - 1);
        ans += dist(a, pai[a]);
        ans += dist(b, pai[b]);
        ans -= dist(pai[a], pai[b]);
        cout << ans << '\n';
    }
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

11.9 Tree Traversals

There are three common ways to traverse the nodes of a binary tree:
• Preorder: First process the root, then the left subtree, and finally the right subtree. • Inorder: First process the left subtree, then the root, and finally the right subtree. • Postorder: First process the left subtree, then the right subtree, and finally the root.
There is a binary tree of n nodes with distinct labels. You are given the preorder and inorder traversals of the tree, and your task is to determine its postorder traversal.

Input
The first input line has an integer n: the number of nodes. The nodes are numbered $1, 2, \ldots, n$. After this, there are two lines describing the preorder and inorder traversals of the tree. Both lines consist of n integers. You can assume that the input corresponds to a binary tree.

Output
Print the postorder traversal of the tree.

Constraints
• $1 \le n \le 10^5$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) x.begin(), x.end()
#define sz(x) (int)(x).size()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

struct node {
    int id = -1;
    node *left, *right;
};

vector<int> preord, inord, inord_rev;
int n;
int preord_pos = 0;

node* process(int left, int right) {
    if(left > right) return nullptr;
    int at = preord[preord_pos++];
    int pos = inord_rev[at];

    node* novo = new node();
    novo->id = at;
    novo->left = process(left, pos - 1);
    novo->right = process(pos + 1, right);

    return novo;
}

void answer(node* tree) {
    if(tree == nullptr) return;
    answer(tree->left);
    answer(tree->right);
    cout << tree->id << ' ';
}

void solve(){
    cin >> n;
    preord.resize(n); inord.resize(n); inord_rev.resize(n +
        1);

    for(int i = 0; i < n; i++) cin >> preord[i];
    for(int i = 0; i < n; i++) {
        cin >> inord[i];
        inord_rev[inord[i]] = i;
    }

    int root = preord[0];
    node* tree = process(0, n-1);

    answer(tree);
    cout << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

12 Bitwise Operations

12.1 All Subarray Xors

Given an array of n integers, your task is to find all integers that are the xor sum in some subarray.

Input
The first line has an integer n: the size of the array. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
First print an integer k: the number of distinct integers that are the xor sum in some subarray. After this print k integers: the xor sums in increasing order.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $0 \leq x_i \leq 10^6$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define int long long

using namespace std;

const int maxn = 1048576;
const int mod = 998244353;

int inverse(int x, int mod) {
    return x == 1 ? 1 : mod - mod / x * inverse(mod % x, mod)
        % mod;
}

void xormul(vector<int> a, vector<int> b, vector<int> &c) {
    int m = (int) a.size();
    c.resize(m);
    for (int n = m / 2; n > 0; n /= 2)
        for (int i = 0; i < m; i += 2 * n)
            for (int j = 0; j < n; j++) {
                int x = a[i + j], y = a[i + j + n];
                a[i + j] = (x + y) % mod;
                a[i + j + n] = (x - y + mod) % mod;
            }
    for (int n = m / 2; n > 0; n /= 2)
        for (int i = 0; i < m; i += 2 * n)
            for (int j = 0; j < n; j++) {
                int x = b[i + j], y = b[i + j + n];
                b[i + j] = (x + y) % mod;
                b[i + j + n] = (x - y + mod) % mod;
            }
    for (int i = 0; i < m; i++)
        c[i] = a[i] * b[i] % mod;
    for (int n = 1; n < m; n *= 2)
        for (int i = 0; i < m; i += 2 * n)
            for (int j = 0; j < n; j++) {
                int x = c[i + j], y = c[i + j + n];
                c[i + j] = (x + y) % mod;
                c[i + j + n] = (x - y + mod) % mod;
            }
    int mrev = inverse(m, mod);
    for (int i = 0; i < m; i++)
        c[i] = c[i] * mrev % mod;
}

void solve(){
    int n; cin >> n;

    vector<int> A(maxn, 0);
    vector<int> pref(n + 1, 0);
    A[0]++;
    for(int i = 1; i <= n; i++)
    {
        int a; cin >> a;
        pref[i] = a ^ pref[i - 1];
        A[pref[i]]++;
    }

    vector<int> B = A, C;
    xormul(A, B, C);

    vector<int> ans;

    if(C[0] > n + 1) ans.push_back(0);
    for(int i = 1; i < maxn; i++)
    {
        if(C[i] > 0) ans.push_back(i);
    }

    cout << ans.size() << '\n';
    for(auto x : ans) cout << x << ' ';
    cout << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

12.2 And Subset Count

You are given an array of n integers. Your task is to calculate the number of non-empty subsets whose elements' bitwise and is equal to k for each $k = 0, 1, \ldots, n$.

Input
The first line has an integer n: the size of the array. The next line has n integers $a_1, a_2, \ldots, a_n$: the contents of the array.

Output
Print $n + 1$ integers as specified above modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $0 \leq a_i \leq n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
using namespace std;

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define sz(x) (int)(x).size()

typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const int LMAX = 18, MOD = 1e9 + 7;
vector<int> sub(1 << LMAX);

void FST(vi& a, bool inv) {
    for (int n = sz(a), step = 1; step < n; step *= 2) {
        for (int i = 0; i < n; i += 2 * step) rep(j,i,i+step)
            {
            int &u = a[j], &v = a[j + step]; tie(u, v) =
            inv ? pii((v - u + MOD) % MOD, u) : pii(v, (u + v
                ) % MOD); // AND
        }
    }
}

ll pow(ll x, ll y, ll m) {
    ll ret = 1;
    while (y) {
        if (y & 1) ret = (ret * x) % m;
        y >>= 1;
        x = (x * x) % m;
    }
    return ret;
}

/*
    a ideia pro FST eh pensar no polinomio prod(1 + x^a_i),
        mas com AND
    fica muito lento fazer os FST's, entao tem que ajeitar
        isso
    testando, da pra perceber que os FST's ficam iguais aos
        andares do triangulo de pascal em mod 2 (quem eh 0
        fica 1, quem eh 1 fica 2)
    ai eh so calcular o produto com SOS dp e destransformar o
         FST
*/

void solve(){
    int n; cin >> n;
    vi a(n), sub(1 << LMAX);

    for(int i = 0; i < n; i++) {
        cin >> a[i]; sub[a[i]]++;
    }

    for (int i = 0; i < LMAX; i++) for (int mask = 0; mask <
        (1 << LMAX); mask++) {
        if (~mask>>i&1) sub[mask] += sub[mask^(1<<i)];
    }

    vector<int> ans(1 << LMAX);
    for(int i = 0; i < sz(ans); i++) {
        ans[i] = pow(2, sub[i], MOD);
    }
    reverse(all(ans));
    FST(ans, 1);

    for(int i = 0; i <= n; i++) cout << ans[i] << ' ';
    cout << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

12.3 Counting Bits

Your task is to count the number of one bits in the binary representations of integers between 1 and n.

Input
The only input line has an integer n.

Output
Print the number of one bits in the binary representations of integers between 1 and n.

Constraints
- $1 \leq n \leq 10^{15}$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const ll INF = 1e15, logINF = 50;

vector<ll> a(50);

void solve(){
    ll n; cin >> n;
    ll tot = 0;

    a[0] = 1;
    for(int i = 1; i < logINF; i++)
        a[i] = a[i - 1] * 2 + (1LL << i);

    ll aux = 0;

    for(int bit = 0; bit <= logINF; bit++) {
        if((n & (1LL << bit)) == 0) continue;

        if(bit == 0) {
            aux += (1LL << bit);
            tot = 1;
            continue;
        }

        tot += (aux + 1) + a[bit - 1];
        aux += (1LL << bit);
    }

    cout << tot << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

12.4 KSubset Xors

You are given an array of n integers. Consider the xors of all 2^n subsets of the array (including the empty subset with xor equal to zero). Your task is to find the k smallest subset xors.

Input
The first line has two integers n and k: the size of the array and the number of subset xors k. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print k integers: the k smallest subset xors in increasing order.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq k \leq \min(2^n, 2 \cdot 10^5)$ • $0 \leq x_i \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int INF = 1e9 + 10, logINF = 30, logmax = 18;

// Gauss - Z2
//
// D eh dimensao do espaco vetorial
// add(v) - adiciona o vetor v na base (retorna se ele jah pertencia ao span da base)
// coord(v) - retorna as coordenadas (c) de v na base atual (basis^T.c = v)
// recover(v) - retorna as coordenadas de v nos vetores na ordem em que foram inseridos
// coord(v).first e recover(v).first - se v pertence ao span
//
// Complexidade:
// add, coord, recover: O(D^2 / 64)

template<int D> struct gauss_z2 {
    bitset<D> basis[D], keep[D];
    int rk, in;
    vector<int> id;

    gauss_z2 () : rk(0), in(-1), id(D, -1) {};

    bool add(bitset<D> v) {
        in++;
        bitset<D> k;
        for (int i = D - 1; i >= 0; i--) if (v[i]) {
            if (basis[i][i]) v ^= basis[i], k ^= keep[i];
            else {
                k[i] = true, id[i] = in, keep[i] = k;
                basis[i] = v, rk++;
                return true;
            }
        }
        return false;
    }
    pair<bool, bitset<D>> coord(bitset<D> v) {
        bitset<D> c;
        for (int i = D - 1; i >= 0; i--) if (v[i]) {
            if (basis[i][i]) v ^= basis[i], c[i] = true;
            else return {false, bitset<D>()};
        }
        return {true, c};
    }
    pair<bool, vector<int>> recover(bitset<D> v) {
        auto [span, bc] = coord(v);
        if (not span) return {false, {}};
        bitset<D> aux;
        for (int i = D - 1; i >= 0; i--) if (bc[i]) aux ^= keep[i];
        vector<int> oc;
        for (int i = D - 1; i >= 0; i--) if (aux[i]) oc.push_back(id[i]);
        return {true, oc};
    }
};

void solve(){
    int n, k; cin >> n >> k;
    gauss_z2<logINF> Gauss;

    vector<int> a(n);
    for(int i = 0; i < n; i++)
    {
        cin >> a[i];
        bitset<logINF> v(a[i]);
        Gauss.add(v);
    }

    int tot = 0;

    vector<int> ans = {0};

    for (int i = 0; i < logINF; i++) {
        if (Gauss.basis[i].any()) {
            tot++;

            if(tot <= logmax)
            {
                int B = Gauss.basis[i].to_ullong();
                int sz = ans.size();
                for(int i = 0; i < sz; i++)
                    ans.push_back(ans[i] ^ B);
            }
        }
    }

    sort(ans.begin(), ans.end());
    int nul = n - tot;
    if(nul > 25) nul = 25;
    int mul = 1 << nul;
    for(auto x : ans)
    {
        for(int i = 0; i < mul; i++)
        {
            cout << x << ' ';
            k--;
            if(k == 0) {
                cout << '\n';
                return;
            }
        }
    }
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

12.5 Maximum Xor Subarray

Given an array of n integers, your task is to find the maximum xor sum of a subarray.

Input
The first line has an integer n: the size of the array. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print one integer: the maximum xor sum in a subarray.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $0 \leq x_i \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>

using namespace std;

#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

int find_ans(vector<int>&v0, vector<int>&v1, int mxbit) {
    if(v0.size() == 0 || v1.size() == 0) return 0;
    if(mxbit == 0) {
        return (v0.back() ^ v1.back());
    }

    int mxbit_new = 0;
    for(int i = 0; i < mxbit; i++) {
        for(auto x : v0) if(x & (1 << i)) mxbit_new = i;
        for(auto x : v1) if(x & (1 << i)) mxbit_new = i;
    }

    vector<int> v00, v01, v10, v11;

    for(auto x : v0) {
        if(x & (1 << mxbit_new)) v01.push_back(x);
        else v00.push_back(x);
    }
    for(auto x : v1) {
        if(x & (1 << mxbit_new)) v11.push_back(x);
        else v10.push_back(x);
    }
    if((v00.size() == 0 || v11.size() == 0) && (v01.size() == 0 || v10.size() == 0)) {
        return find_ans(v0, v1, mxbit_new);
    }

    return max(find_ans(v00, v11, mxbit_new), find_ans(v01, v10, mxbit_new));
}

void solve(){
    int n; cin >> n;
    vector<int> v(n + 1);
    for(int i = 1; i <= n; i++) {
        cin >> v[i];
        v[i] ^= v[i - 1];
    }

    sort(v.begin(), v.end());

    int lst = v.back(), mxbit = 0;
    for(int i = 0; i <= 30; i++) {
        if(lst & (1 << i)) mxbit = i;
    }

    vector<int> v0, v1;

    for(auto x : v) {
        if(x & (1 << mxbit)) v1.push_back(x);
        else v0.push_back(x);
    }

    int ans = find_ans(v0, v1, mxbit);

    cout << ans << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

12.6 Maximum Xor Subset

Given an array of n integers, your task is to find the maximum xor sum of a subset.

Input
The first line has an integer n: the size of the array. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print one integer: the maximum xor sum of a subset.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $0 \leq x_i \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int INF = 1e9 + 10, logINF = 30;

// Gauss - Z2
//
// D eh dimensao do espaco vetorial
// add(v) - adiciona o vetor v na base (retorna se ele jah pertencia ao span da base)
// coord(v) - retorna as coordenadas (c) de v na base atual (basis^T.c = v)
// recover(v) - retorna as coordenadas de v nos vetores na ordem em que foram inseridos
// coord(v).first e recover(v).first - se v pertence ao span
//
// Complexidade:
// add, coord, recover: O(D^2 / 64)

template<int D> struct gauss_z2 {
    bitset<D> basis[D], keep[D];
    int rk, in;
    vector<int> id;

    gauss_z2 () : rk(0), in(-1), id(D, -1) {};

    bool add(bitset<D> v) {
        in++;
        bitset<D> k;
        for (int i = D - 1; i >= 0; i--) if (v[i]) {
            if (basis[i][i]) v ^= basis[i], k ^= keep[i];
            else {
                k[i] = true, id[i] = in, keep[i] = k;
                basis[i] = v, rk++;
                return true;
            }
        }
        return false;
    }
    pair<bool, bitset<D>> coord(bitset<D> v) {
        bitset<D> c;
        for (int i = D - 1; i >= 0; i--) if (v[i]) {
            if (basis[i][i]) v ^= basis[i], c[i] = true;
            else return {false, bitset<D>()};
        }
        return {true, c};
    }
    pair<bool, vector<int>> recover(bitset<D> v) {
        auto [span, bc] = coord(v);
        if (not span) return {false, {}};
        bitset<D> aux;
        for (int i = D - 1; i >= 0; i--) if (bc[i]) aux ^= keep[i];
        vector<int> oc;
        for (int i = D - 1; i >= 0; i--) if (aux[i]) oc.push_back(id[i]);
        return {true, oc};
    }
};

void solve(){
    int n; cin >> n;
    gauss_z2<logINF> Gauss;

    vector<int> a(n);
    for(int i = 0; i < n; i++)
    {
        cin >> a[i];
        bitset<logINF> v(a[i]);
        Gauss.add(v);
    }

    bitset<logINF> res;
    for (int i = logINF - 1; i >= 0; i--) {
        if (!res[i] && Gauss.basis[i][i]) {
            res ^= Gauss.basis[i];
        }
    }

    int ans = (int)(res.to_ulong());
    cout << ans << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

12.7 Numberof Subset Xors

Given an array of n integers, your task is to find the number of different subset xors.

Input
The first line has an integer n: the size of the array. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print one integer: the number of different subset xors.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $0 \leq x_i \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int INF = 1e9 + 10, logINF = 30;

// Gauss - Z2
//
// D eh dimensao do espaco vetorial
// add(v) - adiciona o vetor v na base (retorna se ele jah pertencia ao span da base)
// coord(v) - retorna as coordenadas (c) de v na base atual ( basis^T.c = v)
// recover(v) - retorna as coordenadas de v nos vetores na ordem em que foram inseridos
// coord(v).first e recover(v).first - se v pertence ao span
//
// Complexidade:
// add, coord, recover: O(D^2 / 64)

template<int D> struct gauss_z2 {
    bitset<D> basis[D], keep[D];
    int rk, in;
    vector<int> id;

    gauss_z2 () : rk(0), in(-1), id(D, -1) {};

    bool add(bitset<D> v) {
        in++;
        bitset<D> k;
        for (int i = D - 1; i >= 0; i--) if (v[i]) {
            if (basis[i][i]) v ^= basis[i], k ^= keep[i];
            else {
                k[i] = true, id[i] = in, keep[i] = k;
                basis[i] = v, rk++;
                return true;
            }
        }
        return false;
    }
    pair<bool, bitset<D>> coord(bitset<D> v) {
        bitset<D> c;
        for (int i = D - 1; i >= 0; i--) if (v[i]) {
            if (basis[i][i]) v ^= basis[i], c[i] = true;
            else return {false, bitset<D>()};
        }
        return {true, c};
    }
    pair<bool, vector<int>> recover(bitset<D> v) {
        auto [span, bc] = coord(v);
        if (not span) return {false, {}};
        bitset<D> aux;
        for (int i = D - 1; i >= 0; i--) if (bc[i]) aux ^= keep[i];
        vector<int> oc;
        for (int i = D - 1; i >= 0; i--) if (aux[i]) oc.push_back(id[i]);
        return {true, oc};
    }
};

void solve(){
    int n; cin >> n;
    gauss_z2<logINF> Gauss;

    vector<int> a(n);
    for(int i = 0; i < n; i++)
    {
        cin >> a[i];
        bitset<logINF> v(a[i]);
        Gauss.add(v);
    }

    int ans = 1;
    for (int i = logINF - 1; i >= 0; i--) {
        if (Gauss.basis[i].any()) {
            ans *= 2;
        }
    }

    cout << ans << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

12.8 SOSBit Problem

Given a list of n integers, your task is to calculate for each element x:
the number of elements y such that $x \mid y = x$ the number of elements y such that $x \,\&\, y = x$ the number of elements y such that $x \,\&\, y \neq 0$

Input
The first line has an integer n: the size of the list. The next line has n integers $x_1, x_2, \ldots, x_n$: the elements of the list.

Output
Print n lines: for each element the required values.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^6$

Código-fonte:

```cpp
#include <bits/stdc++.h>

using namespace std;

#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

const int LMAX = 20;
vector<int> sub(1 << LMAX);
vector<int> sup(1 << LMAX);

void solve(){
    int n; cin >> n; vector<int> v(n);
    for(int i = 0; i < n; i++) {
        cin >> v[i];
        sub[v[i]]++; sup[v[i]]++;
    }

    for (int i = 0; i < LMAX; i++) for (int mask = 0; mask < (1 << LMAX); mask++) {
        if (mask>>i&1)  sub[mask] += sub[mask^(1<<i)];
        if (~mask>>i&1) sup[mask] += sup[mask^(1<<i)];
    }

    int msk = (1 << LMAX) - 1;

    for(int i = 0; i < n; i++) {
        cout << sub[v[i]] << ' ' << sup[v[i]] << ' ' << n - sub[msk - v[i]] << '\n';
    }
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

12.9 Xor Pyramid Diagonal

Consider a xor pyramid where each number is the xor of lower-left and lower-right numbers. Here is an example pyramid:
Given the bottom row of the pyramid, your task is to find the leftmost number of each row.

Input
The first line has an integer n: the size of the pyramid. The next line has n integers $a_1, a_2, \ldots, a_n$: the bottom row of the pyramid.

Output
Print n integers: the leftmost numbers of the rows from bottom to top.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq a_i \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>

using namespace std;

#define pb push_back
```

```
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

const int LMAX = 18;
vector<int> v(1 << LMAX);

void solve(){
    int n; cin >> n;
    for(int i = 0; i < n; i++) cin >> v[i];

    for (int i = 0; i < LMAX; i++) for (int mask = 0; mask < (1 << LMAX); mask++)
        if (mask>>i&1) v[mask] ^= v[mask^(1<<i)];

    for(int i = 0; i < n; i++) cout << v[i] << ' '; cout << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

12.10 Xor Pyramid Peak

Consider a xor pyramid where each number is the xor of lower-left and lower-right numbers. Here is an example pyramid: Given the bottom row of the pyramid, your task is to find the topmost number.

Input
The first line has an integer n: the size of the pyramid. The next line has n integers $a_1, a_2, \ldots, a_n$: the bottom row of the pyramid.

Output
Print one integer: the topmost number.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le a_i \le 10^9$

Código-fonte:

```
#include <bits/stdc++.h>

using namespace std;

#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

int bin_v2(int n, int k) {
    return pc(k) + pc(n - k) - pc(n);
}

void solve(){
    int n; cin >> n;
    int ans = 0;
    for(int i = 0; i < n; i++) {
        int a; cin >> a;
        if(bin_v2(n - 1, i) == 0) ans ^= a;
    }

    cout << ans << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

12.11 Xor Pyramid Row

Consider a xor pyramid where each number is the xor of lower-left and lower-right numbers. Here is an example pyramid: Given the bottom row of the pyramid, your task is to find the numbers on the k-th row from the top.

Input
The first line has two integers n and k: the size of the pyramid and the given row. The next line has n integers $a_1, a_2, \ldots, a_n$: the bottom row of the pyramid.

Output
Print k integers: the numbers on the k-th row from the top.

Constraints
• $1 \le k \le n \le 2 \cdot 10^5$ • $1 \le a_i \le 10^9$

Código-fonte:

```
#include <bits/stdc++.h>

using namespace std;

#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

int bin_v2(int n, int k) {
    return pc(k) + pc(n - k) - pc(n);
}

void solve(){
    int n, k; cin >> n >> k;
    int shift = n - k;
    vector<int> v(n);
    for(int i = 0; i < n; i++) cin >> v[i];

    for(int i = 1 << 30; i > 0; i >>= 1) {
        if((i & shift) == 0) continue;

        vector<int> aux(n - i);
        for(int j = 0; j < n - i; j++) {
            aux[j] = v[j] ^ v[j + i];
        }

        v = aux;
        n = aux.size();
    }

    for(auto x : v) cout << x << ' '; cout << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

13 Construction Problems

13.1 Chess Tournament

There will be a chess tournament of n players. Each player has announced the number of games they want to play. Each pair of players can play at most one game. Your task is to determine which games will be played so that everybody will be happy.

Input
The first input line has an integer n: the number of players. The players are numbered $1, 2, \ldots, n$. The next line has n integers $x_1, x_2, \ldots, x_n$: for each player, the number of games they want to play.

Output
First print an integer k: the number of games. Then, print k lines describing the games. You can print any valid solution. If there are no solutions, print "IMPOSSIBLE".

Constraints
• $1 \leq n \leq 10^5$ • $\sum_{i=1}^{n} x_i \leq 2 \cdot 10^5$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long int  ll;
typedef pair<int, int> pii;
typedef vector<int> vi;
typedef vector<vi> graph;

void solve(){
    int n; cin >> n;
    priority_queue<pii> pq;
    for(int i = 0; i < n; i++) {
        int d; cin >> d;
        if(d != 0) pq.push({d, i + 1});
    }

    vector<pii> ans;

    while(pq.size() > 0) {
        vector<pii> aux;
        auto [at, pos] = pq.top(); pq.pop();

        while(at > 0) {
            if(pq.size() == 0) {
                cout << "IMPOSSIBLE\n";
                return;
            }

            auto [nxt, nxt_pos] = pq.top(); pq.pop();
            aux.push_back({nxt - 1, nxt_pos});
            ans.push_back({pos, nxt_pos});
            at--;
        }

        for(auto [x, y] : aux) if(x > 0) pq.push({x, y});
    }

    cout << ans.size() << '\n';
    for(auto [x, y] : ans) cout << x << ' ' << y << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

13.2 Inverse Inversions

Your task is to create a permutation of numbers $1, 2, \ldots, n$ that has exactly k inversions. An inversion is a pair (a, b) where $a<b$ and $p_a>p_b$ where p_i denotes the number at position i in the permutation.

Input
The only input line has two integers n and k.

Output
Print a line that contains the permutation. You can print any valid solution.

Constraints
• $1 \leq n \leq 10^6$ • $0 \leq k \leq \frac{n(n-1)}{2}$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define F first
#define S second

using namespace std;

typedef long long int  ll;
typedef pair<int, int> pii;
typedef vector<int> vi;
typedef vector<vi> graph;

void solve(){
    ll n, m; cin >> n >> m;
    vi ans;
    deque<int> list;

    for(int i = 1; i <= n; i++) list.push_back(i);

    for(int i = n; i >= 1; i--) {
        int add = 0;
        if(m >= i - 1) {
            add = list.back(); list.pop_back();
            m -= i - 1;
        }
        else {
            add = list.front(); list.pop_front();
        }
        ans.push_back(add);
    }

    for(auto x : ans) cout << x << ' ';
    cout << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

14 Counting Problems

14.1 All Letter Subgrid Count I

You are given a grid of letters. Your task is to calculate the number of square subgrids that contain all the letters.

Input
The first line has two integers n and k: the size of the grid and the number of letters. The letters are the first k uppercase letters. After this, there are n lines that describe the grid. Each line has n letters.

Output
Print the number of subgrids.

Constraints
• $1 \leq n \leq 3000$ • $1 \leq k \leq 26$

Código-fonte:

```cpp
#pragma GCC optimize("O3,unroll-loops")

#include <bits/stdc++.h>
using namespace std;

#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define sz(x) (int)(x).size()

typedef vector<int> vi;
typedef long long ll;

const int MAXN = 3005;
const int MAXK = 26;

// maior quad com canto em (i, j) sem letra X
int dp[2][MAXN][MAXK];
int mat[MAXN][MAXN];

void solve() {
    int n, k; cin >> n >> k;

    if(k == 1) {
        cout << (ll)n * (n + 1LL) * (2 * n + 1LL) / 6 << '\n';
        return;
    }

    for(int i = 0; i < n; i++) {
        string s; cin >> s;
        for(int j = 0; j < n; j++) {
            mat[i][j] = s[j] - 'A';
        }
    }

    for(int j = 0; j < n; j++) {
        for(int c = 0; c < k; c++) {
            if(mat[0][j] != c) dp[0][j][c] = 1;
        }
    }

    ll tot = 0;

    for(int i = 1; i < n; i++) {
        for(int c = 0; c < k; c++)
            if(mat[i][0] != c) dp[1][0][c] = 1;

        for(int j = 1; j < n; j++) {
            int aux = 0;
            for(int c = 0; c < k; c++) {
                if(mat[i][j] == c) dp[1][j][c] = 0;
                else dp[1][j][c] = min({dp[1][j - 1][c], dp[0][j][c], dp[0][j - 1][c]}) + 1;
                aux = max(aux, dp[1][j][c]);
            }
            tot += min(i, j) - aux + 1;
        }

        for(int j = 0; j < n; j++) {
            for(int c = 0; c < k; c++) {
                dp[0][j][c] = dp[1][j][c];
                dp[1][j][c] = 0;
            }
        }
    }

    cout << tot << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

14.2 All Letter Subgrid Count II

You are given a grid of letters. Your task is to calculate the number of rectangle subgrids that contain all the letters.

Input
The first line has two integers n and k: the size of the grid and the number of letters. The letters are the first k uppercase letters. After this, there are n lines that describe the grid. Each line has n letters.

Output
Print the number of subgrids.

Constraints
• $1 \leq n \leq 500$ • $1 \leq k \leq 26$

Código-fonte:

```cpp
#pragma GCC optimize("O3")
#pragma GCC optimize("unroll-loops")

#include <bits/stdc++.h>
#pragma GCC target("avx2")
using namespace std;

#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define sz(x) (int)(x).size()

typedef vector<int> vi;
typedef long long ll;

const int MAXN = 505;
const int MAXLG = 9;

int jmp[MAXLG][MAXLG][MAXN][MAXN];
int lg[MAXN];
int pw[MAXLG];

inline int query(int r1, int c1, int r2, int c2) __attribute__((always_inline));
inline int query(int r1, int c1, int r2, int c2) {
    int depR = lg[r2 - r1 + 1];
    int depC = lg[c2 - c1 + 1];
    int pwR = pw[depR];
    int pwC = pw[depC];

    return jmp[depR][depC][r1][c1] |
           jmp[depR][depC][r1][c2 - pwC + 1] |
           jmp[depR][depC][r2 - pwR + 1][c1] |
           jmp[depR][depC][r2 - pwR + 1][c2 - pwC + 1];
}

void solve() {
    int n, k; cin >> n >> k;
    int bst = (1 << k) - 1;

    if(k == 1) {
        cout << ((ll)n * (n + 1LL) / 2) * ((ll)n * (n + 1LL) / 2) << '\n';
        return;
    }

    for(int i = 0; i < n; i++) {
        string s; cin >> s;
        for(int j = 0; j < n; j++) {
            jmp[0][0][i][j] = (1 << (s[j] - 'A'));
        }
    }

    // build
    for (int kc = 1; kc < MAXLG; ++kc) {
        int lenC = pw[kc-1];
        if(lenC > n) break;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j <= n - pw[kc]; j++) {
                jmp[0][kc][i][j] = (jmp[0][kc - 1][i][j] | jmp[0][kc - 1][i][j + lenC]);
            }
        }
    }

    for (int kr = 1; kr < MAXLG; ++kr) {
        int lenR = pw[kr-1];
        if(lenR > n) break;
        for (int kc = 0; kc < MAXLG; ++kc) {
            if(pw[kc] > n) break;
            for (int i = 0; i <= n - pw[kr]; i++) {
                for (int j = 0; j <= n - pw[kc]; j++) {
                    jmp[kr][kc][i][j] = (jmp[kr - 1][kc][i][j] | jmp[kr - 1][kc][i + lenR][j]);
                }
            }
        }
    }
    // end build

    ll tot = 0;

    for(int i = 0; i < n; i++) {
        ll aux = 0;
        for(int j = 0; j < n; j++) {
            int r = n - 1, c = j;

            for(; r >= i; r--) {
```

```
                for(; c < n; c++)
                    if(query(i, j, r, c) == bst) break;
                if(c == n) break;
                aux += n - c;
            }
        }
        tot += aux;
    }

    cout << tot << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);

    lg[1] = 0;
    for(int i = 2; i < MAXN; i++) lg[i] = lg[i/2] + 1;
    for(int i = 0; i < MAXLG; i++) pw[i] = 1 << i;

    solve();
    return 0;
}
```

14.3 Counting Sequences

Your task is to count the number of sequences of length n where each element is an integer between $1 \dots k$ and each integer between $1 \dots k$ appears at least once in the sequence. For example, when $n = 6$ and $k = 4$, some valid sequences are $[1, 3, 1, 4, 3, 2]$ and $[2, 2, 1, 3, 4, 2]$.

Input
The only input line has two integers n and k.

Output
Print one integer: the number of sequences modulo $10^9 + 7$.

Constraints
- $1 \le k \le n \le 10^6$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

#define int long long

using namespace std;

const int M = 1e9 + 7;
const int MAX = 1e6 + 10;
int fat[MAX], invfat[MAX];

int fexp(int b, int e) {
    int ans = 1;
    for (; e; b = b * b % M, e /= 2)
        if (e & 1) ans = ans * b % M;
    return ans;
}

int inv(int a) {
    return fexp(a, M - 2);
}
```

```

int calc(int a, int b) {
    if(b < 0 || b > a) return 0;
    return fat[a] * (invfat[b] * invfat[a - b] % M) % M;
}

// inclusao - exclusao em Ai = {seq's tais que i nao aparece}
void solve() {
    int n, k; cin >> n >> k;

    int sgn = 1, tot = 0;

    for(int i = 1; i < k; i++) {
        int pot = (fexp(k - i, n) * calc(k, i)) % M;
        pot = (pot * sgn + M) % M;

        tot = (tot + pot) % M;
        sgn = -sgn;
    }

    cout << (fexp(k, n) - tot + M) % M << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);

    fat[0] = 1;
    for(int i = 1; i < MAX; i++) fat[i] = (fat[i - 1] * i) % M;
    invfat[MAX - 1] = inv(fat[MAX - 1]);
    for(int i = MAX - 2; i >= 0; i--) invfat[i] = (invfat[i + 1] * (i + 1)) % M;

    solve();
    return 0;
}
```

14.4 Empty String

You are given a string consisting of n characters between a and z. On each turn, you may remove any two adjacent characters that are equal. Your goal is to construct an empty string by removing all the characters. In how many ways can you do this?

Input
The only input line has a string of length n.

Output
Print one integer: the number of ways modulo $10^9 + 7$.

Constraints
- $1 \le n \le 500$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

#define int long long

using namespace std;

```

```
const int M = 1e9 + 7;
const int MAX = 505;
int dp[MAX][MAX]; //dp[a][b] = formas de deletar os caracteres de a ate b
int fat[MAX], fatinv[MAX];

int fexp(int b, int e) {
    int ans = 1;
    for (; e; b = b * b % M, e /= 2)
        if (e & 1) ans = ans * b % M;
    return ans;
}

void calcfat() {
    fat[0] = 1;
    for(int i = 1; i < MAX; i++) fat[i] = (i * fat[i - 1]) % M;
    fatinv[MAX - 1] = fexp(fat[MAX - 1], M - 2);
    for(int i = MAX - 2; i >= 0; i--) fatinv[i] = ((i + 1) * fatinv[i + 1]) % M;
}

int choose(int a, int b) {
    return ((fat[a] * fatinv[b] % M) * fatinv[a - b]) % M;
}

void calc(const string& s) {
    for(int i = 0; i + 1 < s.size(); i++) if(s[i] == s[i+1]) dp[i][i+1] = 1;

    for(int tam = 4; tam <= s.size(); tam+=2)
    for(int i = 0, j = i + tam - 1; j < s.size(); i++, j++) {
        // s[i] opera com s[k]

        if(s[i] == s[i + 1])
            dp[i][j] = (dp[i][j] + dp[i + 2][j] * (j - i + 1) / 2) % M;

        for(int k = i + 3; k < j; k += 2) {
            if(s[i] == s[k]) dp[i][j] = (dp[i][j] + (dp[i + 1][k - 1] * dp[k + 1][j] % M) * choose((j - i + 1) / 2, (j - k) / 2)) % M;
        }

        if(s[i] == s[j]) dp[i][j] = (dp[i][j] + dp[i + 1][j - 1]) % M;
    }
}

void solve() {
    string s; cin >> s;
    calc(s);
    cout << dp[0][(int)s.size() - 1] << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    calcfat();
    solve();
    return 0;
}
```

14.5 Grid Paths II

Consider an $n \times n$ grid whose top-left square is $(1, 1)$ and bottom-right square is (n, n). Your task is to move from the top-left square to the bottom-right square. On each step you may move one square right or down. In addition, there are m traps in the grid. You cannot move to a square with a trap. What is the total number of possible paths?

Input
The first input line contains two integers n and m: the size of the grid and the number of traps. After this, there are m lines describing the traps. Each such line contains two integers y and x: the location of a trap. You can assume that there are no traps in the top-left and bottom-right square. You can also assume that each square has at most one trap.

Output
Print the number of paths modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 10^6$ • $1 \leq m \leq 1000$ • $1 \leq y, x \leq n$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

#define int long long

using namespace std;

const int M = 1e9 + 7;
const int MAX = 2e6 + 10;
int fat[MAX];

int fexp(int b, int e) {
    int ans = 1;
    for (; e; b = b * b % M, e /= 2)
        if (e & 1) ans = ans * b % M;
    return ans;
}

int inv(int a) {
    return fexp(a, M - 2);
}

int calc(int a, int b) {
    return fat[a + b] * (inv(fat[a]) * inv(fat[b]) % M) % M;
}

void solve() {
    int n; cin >> n;
    int m; cin >> m;

    vector<int> dp(m + 1);
    vector<pair<int, int>> trap(m + 1);

    for(int i = 0; i < m; i++) {
        int x, y; cin >> x >> y;
        trap[i] = {x - 1, y - 1};
        if(x == n && y == n) {
            cout << 0 << '\n';
            return;
        }
    }
    trap[m] = {n - 1, n - 1};

    sort(trap.begin(), trap.end());

    for(int i = 0; i <= m; i++) {
        dp[i] = calc(trap[i].first, trap[i].second);
        for(int j = 0; j < i; j++) {
            if(trap[i].second < trap[j].second) continue;
            dp[i] = (dp[i] - dp[j] * calc(trap[i].first -
                trap[j].first, trap[i].second - trap[j].
                second)) % M;
            dp[i] = (M + dp[i]) % M;
        }
    }

    cout << dp[m] << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);

    fat[0] = 1;
    for(int i = 1; i < MAX; i++) fat[i] = (fat[i - 1] * i) %
        M;

    solve();
    return 0;
}
```

14.6 Permutation Inversions

Your task is to count the number of permutations of $1, 2, \ldots, n$ that have exactly k inversions (i.e., pairs of elements in the wrong order). For example, when $n = 4$ and $k = 3$, there are 6 such permutations:
• $[1, 4, 3, 2]$ • $[2, 3, 4, 1]$ • $[2, 4, 1, 3]$ • $[3, 1, 4, 2]$ • $[3, 2, 1, 4]$ • $[4, 1, 2, 3]$

Input
The only input line has two integers n and k.

Output
Print the answer modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 500$ • $0 \leq k \leq \frac{n(n-1)}{2}$

Código-fonte:

```cpp
#include <bits/stdc++.h>
using namespace std;

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

const int MAX = 501;
const int MOD = 1e9 + 7;

// dp[t] = quantidade de perm com t invercoes
int dp[MAX * MAX / 2];

void solve() {
    int n, k; cin >> n >> k;

    dp[0] = 1;

    for(int i = 2; i <= n; i++) {
        int maxj = i * (i - 1) / 2;
        int window = 0;

        for(int j = maxj - (i - 1); j <= maxj; j++) {
            window = (window + dp[j]) % MOD;
        }

        for(int j = maxj; j >= 0; j--) {
            int ant = dp[j];
            dp[j] = window;
            if(j >= i) window = ((ll)window - ant + dp[j - i]
                + MOD) % MOD;
            else window = (window - ant + MOD) % MOD;
        }
    }

    cout << dp[k] << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    solve();
    return 0;
}
```

15 Sliding Window Problems

15.1 Sliding Window Cost

You are given an array of n integers. Your task is to calculate for each window of k elements, from left to right, the minimum total cost of making all elements equal. You can increase or decrease each element with cost x where x is the difference between the new and the original value. The total cost is the sum of such costs.

Input
The first line contains two integers n and k: the number of elements and the size of the window. Then there are n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Output $n - k + 1$ values: the costs.

Constraints
• $1 \leq k \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

void fix(multiset<int>* m, multiset<int>* M, ll* sm, ll* sM){
    if((*m).size() < (*M).size()){
        int x = *(*M).begin();
        (*m).insert(x);
        (*M).erase((*M).begin());
        *sm += x;
        *sM -= x;
    }
    else if((*m).size() >= (*M).size() + 2){
        int x = *(*m).rbegin();
        (*M).insert(x);
        (*m).erase((*m).find(x));
        *sM += x;
        *sm -= x;
    }
}

// Sum of Two Values nos pares (i, j) do vetor original
void solve() {
    int n, k; cin >> n >> k;
    vector<int> v(n);

    for(int i = 0; i < n; i++) cin >> v[i];

    multiset<int> m, M;
    ll sm = 0, sM = 0;

    vector<int> aux(k);
    for(int i = 0; i < k; i++) aux[i] = v[i];
    sort(all(aux));

    for(int i = 0; i < (k + 1) / 2; i++) {
        m.insert(aux[i]);
        sm += aux[i];
    }
    for(int i = (k + 1) / 2; i < k; i++) {
        M.insert(aux[i]);
        sM += aux[i];
    }

    for(int i = 0; i < n - k + 1; i++){
        int rem = v[i];
        int add = v[i + k];
        int a = *m.rbegin();
        if(k % 2 == 1)
            cout << sM  - sm + a << ' ';
        else
            cout << sM  - sm     << ' ';
        if(rem <= a) {
            m.erase(m.find(rem));
            sm -= rem;
        }
        else {
            M.erase(M.find(rem));
            sM -= rem;
        }
        fix(&m, &M, &sm, &sM);

        if(M.empty()) {
            m.insert(add);
            sm += add;
        }
        else {
            a = *M.begin();
            if(add >= a) {
                M.insert(add);
                sM += add;
            }
            else {
                m.insert(add);
                sm += add;
            }
        }
        fix(&m, &M, &sm, &sM);

    }
    cout << '\n';

}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

15.2 Sliding Window Median

You are given an array of n integers. Your task is to calculate the median of each window of k elements, from left to right. The median is the middle element when the elements are sorted. If the number of elements is even, there are two possible medians and we assume that the median is the smaller of them.

Input
The first line contains two integers n and k: the number of elements and the size of the window. Then there are n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print $n - k + 1$ values: the medians.

Constraints
• $1 \leq k \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

void fix(multiset<int>* m, multiset<int>* M){
    if((*m).size() < (*M).size()){
        (*m).insert(*(*M).begin());
        (*M).erase((*M).begin());
    }
    else if((*m).size() >= (*M).size() + 2){
        (*M).insert(*(*m).rbegin());
        (*m).erase((*m).find(*(*m).rbegin()));
    }
}

// Sum of Two Values nos pares (i, j) do vetor original
void solve() {
    int n, k; cin >> n >> k;
    vector<int> v(n);

    for(int i = 0; i < n; i++) cin >> v[i];

    multiset<int> m, M;

    vector<int> aux(k);
    for(int i = 0; i < k; i++) aux[i] = v[i];
    sort(all(aux));

    for(int i = 0; i < (k + 1) / 2; i++) m.insert(aux[i]);
    for(int i = (k + 1) / 2; i < k; i++) M.insert(aux[i]);

    for(int i = 0; i < n - k + 1; i++){
        int rem = v[i];
        int add = v[i + k];
        int a = *m.rbegin();
        cout << a << ' ';

        if(rem <= a) m.erase(m.find(rem));
        else M.erase(M.find(rem));
        fix(&m, &M);

        if(M.empty()) m.insert(add);
        else{
            a = *M.begin();
            if(add >= a) M.insert(add);
            else m.insert(add);
        }
        fix(&m, &M);

    }
    cout << '\n';

}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

```
60 }
61
62 int main() {
63     ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
64     //int t; cin >> t; while (t--)
65     solve();
66     return 0;
67 }
```

16 Sorting And Searching

16.1 Apartments

There are n applicants and m free apartments. Your task is to distribute the apartments so that as many applicants as possible will get an apartment. Each applicant has a desired apartment size, and they will accept any apartment whose size is close enough to the desired size.

Input
The first input line has three integers n, m, and k: the number of applicants, the number of apartments, and the maximum allowed difference. The next line contains n integers $a_1, a_2, \ldots, a_n$: the desired apartment size of each applicant. If the desired size of an applicant is x, they will accept any apartment whose size is between $x-k$ and $x+k$. The last line contains m integers $b_1, b_2, \ldots, b_m$: the size of each apartment.

Output
Print one integer: the number of applicants who will get an apartment.

Constraints
• $1 \le n, m \le 2 \cdot 10^5$ • $0 \le k \le 10^9$ • $1 \le a_i, b_i \le 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1e5+5, inf = 2e9, M = 1e9 + 7;

void solve(){
    int n, m, k;
    cin >> n >> m >> k;
    int a[n], b[m];

    for(int i=0; i<n; i++) cin >> a[i];
    for(int i=0; i<m; i++) cin >> b[i];

    sort(a, a+n);
    sort(b, b+m);

    int atual = 0, total = 0;

    for(int i=0; i<m; i++){
        if(atual >= n) break;

        if(abs(b[i] - a[atual]) <= k){
            atual ++;
            total ++;
        }
        else if(b[i] - a[atual] > k){
            atual ++;
            i --;
        }
    }

    cout << total << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

16.2 Array Division

You are given an array containing n positive integers. Your task is to divide the array into k subarrays so that the maximum sum in a subarray is as small as possible.

Input
The first input line contains two integers n and k: the size of the array and the number of subarrays in the division. The next line contains n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print one integer: the maximum sum in a subarray in the optimal division.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le k \le n$ • $1 \le x_i \le 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

bool ans(vector<int> v, ll m, int k){
    int n = v.size();
    ll at = 0;
    int cont = 0;

    for(int i = 0; i < n; i++){
        if(cont > k) break;
        if(at + v[i] > m){
            at = v[i];
            if(at > m){
                cont = k + 1;
                break;
            }
            cont++;
        }
        else{
            at += v[i];
        }
    }
    if(at > 0) cont++;

    return cont <= k;
}

void solve() {
    int n, k; cin >> n >> k;
    vector<int> v(n);

    ll l = 1, r = 1;
    for(int i = 0; i < n; i++){
        cin >> v[i];
        r   += v[i];
    }

    while(r - l > 1){
        ll m = (l + r) / 2;

        if(ans(v, m, k)) r = m;
        else l = m;
    }

    cout << r << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

16.3 Collecting Numbers

You are given an array that contains each number between $1 \ldots n$ exactly once. Your task is to collect the numbers from 1 to n in increasing order. On each round, you go through the array from left to right and collect as many numbers as possible. What will be the total number of rounds?

Input
The first line has an integer n: the array size. The next line has n integers $x_1, x_2, \ldots, x_n$: the numbers in the array.

Output
Print one integer: the number of rounds.

Constraints
• $1 \le n \le 2 \cdot 10^5$

Código-fonte:

```cpp
#include <bits/stdc++.h>

using namespace std;

#define all(x) x.begin(), x.end()
#define pb push_back
#define ll long long int
#define int long long
const int MOD = 1e9 + 7, MAX = 1e6 + 5, n = 10;

int32_t main(){
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int n; cin >> n;
    pair<int, int> p[n + 1];
    for(int i = 0; i < n; i++){
        cin >> p[i].first;
```

```
        p[i].second = i;
    }
    sort(p, p + n);
    p[n] = {-1, -1};

    int at = 0;

    for(int i = 0; i < n; i++){
        if(p[i].second > p[i+1].second) at++;
    }

    cout << at << '\n';
}
```

16.4 Collecting Numbers II

You are given an array that contains each number between $1 \dots n$ exactly once. Your task is to collect the numbers from 1 to n in increasing order. On each round, you go through the array from left to right and collect as many numbers as possible. Given m operations that swap two numbers in the array, your task is to report the number of rounds after each operation.

Input
The first line has two integers n and m: the array size and the number of operations. The next line has n integers $x_1, x_2, \dots, x_n$: the numbers in the array. Finally, there are m lines that describe the operations. Each line has two integers a and b: the numbers at positions a and b are swapped.

Output
Print m integers: the number of rounds after each swap.

Constraints
• $1 \le n, m \le 2 \cdot 10^5$ • $1 \le a, b \le n$

Código-fonte:

```
#include <bits/stdc++.h>
#define _ ios_base::sync_with_stdio(0); cin.tie(0); cout.tie
    (0);
#define MAXN 200100
#define INF 100000000
#define pb push_back
#define F first
#define S second

using namespace std;
typedef long long ll;
typedef pair<int, int> pii;
const int M = 1e9 + 7;

void printv (vector<int> v){
    for(int x : v) cout << x << ' ';
    cout << '\n';
}
void printvpii (vector<pii> v){
    for(pii x : v) cout << x.second << " ";
    cout << '\n';
}

int main () { _
    int n, m; cin >> n >> m;
    vector<int> v1(n);
    vector<pii> v2(n);
    for(int i = 0; i < n; i++){
        cin >> v1[i];
        v2[i] = {v1[i], i+1};
    }
    sort(v2.begin(), v2.end());
    v2.pb({-1, -1});
    int total = 0;
    for(int i = 0; i < n; i++)
        if(v2[i+1].S < v2[i].S) total++;

    for(int i = 0; i < m; i++){
        int a, b; cin >> a >> b; a--; b--;
        int x = v1[a]-1, y = v1[b]-1;
        swap(v1[a], v1[b]);
        if(x > y) swap(x, y);
        total -= (v2[x].second > v2[x+1].second) + (v2[y].
            second > v2[y+1].second) + (v2[y-1].second > v2
            [y].second);
        if(x > 0) total -= (v2[x-1].second > v2[x].second);
        if(y == x + 1) total += v2[x].second > v2[y].second;
        swap(v2[x].second, v2[y].second);
        total += (v2[x].second > v2[x+1].second) + (v2[y].
            second > v2[y+1].second) + (v2[y-1].second > v2
            [y].second);
        if(x > 0) total += (v2[x-1].second > v2[x].second);
        if(y == x + 1) total -= v2[x].second > v2[y].second;
        cout << total << '\n';
    }

    return 0;
}
```

16.5 Concert Tickets

There are n concert tickets available, each with a certain price. Then, m customers arrive, one after another. Each customer announces the maximum price they are willing to pay for a ticket, and after this, they will get a ticket with the nearest possible price such that it does not exceed the maximum price.

Input
The first input line contains integers n and m: the number of tickets and the number of customers. The next line contains n integers $h_1, h_2, \dots, h_n$: the price of each ticket. The last line contains m integers $t_1, t_2, \dots, t_m$: the maximum price for each customer in the order they arrive.

Output
Print, for each customer, the price that they will pay for their ticket. After this, the ticket cannot be purchased again. If a customer cannot get any ticket, print -1.

Constraints
• $1 \le n, m \le 2 \cdot 10^5$ • $1 \le h_i, t_i \le 10^9$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1010, inf = 2e9, M = 1e9 + 7;


void solve() {
    int n, m; cin >> n >> m;
    multiset<int> h;

    for(int i=0; i<n; i++){
        int a; cin >> a;
        h.insert(-a);
    }

    // for(auto x : h) cout << -x << ' ';
    // cout << '\n';

    for(int i=0; i<m; i++){
        int t; cin >> t;
        auto lwb = h.lower_bound(-t);

        if(lwb == h.end()) cout << "-1\n";
        else{
            cout << -*lwb << '\n';
            h.erase(lwb);
        }

        // for(auto x : h) cout << -x << ' ';
        // cout << '\n';

    }
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

16.6 Distinct Numbers

You are given a list of n integers, and your task is to calculate the number of distinct values in the list.

Input
The first input line has an integer n: the number of values. The second line has n integers $x_1, x_2, \dots, x_n$.

Output
Print one integers: the number of distinct values.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le x_i \le 10^9$

Código-fonte:

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    int n, x;
    set<int> s;
    scanf("%d", &n);
    for(int i=0; i<n; i++){
```

```
        scanf("%d", &x);
        s.insert(x);
    }
    printf("%d\n", s.size());
}
```

16.7 Distinct Values Subarrays

Given an array of n integers, count the number of subarrays where each element is distinct.

Input
The first line has an integer n: the array size. The second line has n integers $x_1, x_2, \ldots, x_n$: the array contents.

Output
Print the number of subarrays with distinct elements.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1e5+5, inf = 2e9, M = 1e9 + 7;

void solve(){
    int n; cin >> n;
    vector<int> x(n);

    for(int i = 0; i < n; i++)
        cin >> x[i];

    set<int> at;
    ll p1 = 0, p2 = 0;

    ll ans = 0;

    for(p1 = 0; p1 < n; p1++)
    {
        while(p2 < n && at.find(x[p2]) == at.end())
        {
            at.insert(x[p2]);
            p2++;
        }

        ans += (p2 - p1);
        at.erase(x[p1]);
    }

    cout << ans << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

16.8 Distinct Values Subarrays II

Given an array of n integers, your task is to calculate the number of subarrays that have at most k distinct values.

Input
The first input line has two integers n and k. The next line has n integers $x_1, x_2, \ldots, x_n$: the contents of the array.

Output
Print one integer: the number of subarrays.

Constraints
• $1 \leq k \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1010, inf = 2e9, M = 1e9 + 7;


void solve() {
    int n, k; cin >> n >> k;
    vector<int> v;

    for(int i = 0; i < n; i++){
        int a; cin >> a;
        v.pb(a);
    }

    map<int, int> rep;

    int p1 = 0, p2 = 0;
    ll cont = 0;
    int at = 0;
    int dis = 0;

    while(p1 < n || p2 < n){
        // cout << p1 << ' ' << p2 << '\n';
        if(p2 == n || (rep[v[p2]] == 0 && dis >= k)){
            // cout << "A\n";
            rep[v[p1]] --;
            if(rep[v[p1]] == 0) dis --;
            cont += at;
            at --;
            p1 ++;
        }
        else if(rep[v[p2]] == 0 && dis < k){
            // cout << "B\n";
            rep[v[p2]] = 1;
            dis ++;
            at ++;
            p2 ++;
        }
        else{
            // cout << "C\n";
            rep[v[p2]] ++;
            at ++;
            p2 ++;
        }
    }

    cout << cont << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

16.9 Distinct Values Subsequences

Given an array of n integers, count the number of subsequences where each element is distinct. A subsequence is a sequence of array elements from left to right that may have gaps.

Input
The first line has an integer n: the array size. The second line has n integers $x_1, x_2, \ldots, x_n$: the array contents.

Output
Print the number of subsequences with distinct elements. The answer can be large, so print it modulo $10^9 + 7$.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1e5+5, inf = 2e9, M = 1e9 + 7;

void solve(){
    int n; cin >> n;
    map<int, int> mapa;

    for(int i = 0; i < n; i++)
    {
        int a;
        cin >> a;
        mapa[a]++;
    }

    ll ans = 1;
    for(auto x : mapa)
        ans = (ans * (x.second + 1)) % M;

    cout << ans - 1 << '\n';
}

int main() {
```

```
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

16.10 Factory Machines

A factory has n machines which can be used to make products. Your goal is to make a total of t products. For each machine, you know the number of seconds it needs to make a single product. The machines can work simultaneously, and you can freely decide their schedule. What is the shortest time needed to make t products?

Input
The first input line has two integers n and t: the number of machines and products. The next line has n integers $k_1, k_2, \ldots, k_n$: the time needed to make a product using each machine.

Output
Print one integer: the minimum time needed to make t products.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq t \leq 10^9$ • $1 \leq k_i \leq 10^9$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1e5+5, inf = 2e9, M = 1e9 + 7;
const ll linf = 1e18;

__int128_t ans(vector<ll> k, ll m){
    __int128_t sum = 0;
    for(auto x : k) sum += m / x;
    return sum;
}

void solve(){
    ll n, t; cin >> n >> t;
    vector<ll> k(n);

    for(int i = 0; i < n; i++){
        cin >> k[i];
    }

    ll l = 0, r = linf, m;

    while(r - l > 1){
        m = (l + r) / 2;
        __int128_t sum = ans(k, m);
        if(sum >= t) r = m;
        else l = m;
    }
```

```
    cout << r << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

16.11 Ferris Wheel

There are n children who want to go to a Ferris wheel, and your task is to find a gondola for each child. Each gondola may have one or two children in it, and in addition, the total weight in a gondola may not exceed x. You know the weight of every child. What is the minimum number of gondolas needed for the children?

Input
The first input line contains two integers n and x: the number of children and the maximum allowed weight. The next line contains n integers $p_1, p_2, \ldots, p_n$: the weight of each child.

Output
Print one integer: the minimum number of gondolas.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x \leq 10^9$ • $1 \leq p_i \leq x$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1e5+5, inf = 2e9, M = 1e9 + 7;

void solve(){
    int n, x;
    cin >> n >> x;
    int p[n];

    for(int i=0; i<n; i++) cin >> p[i];
    sort(p, p+n);

    int fim = n-1, inicio = 0, total = n;

    while(fim > inicio){
        if(p[fim] + p[inicio] <= x){
            fim --;
            inicio ++;
            total --;
        }
        else{
            fim --;
        }
    }
```

```
    cout << total << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

16.12 Josephus Problem I

Consider a game where there are n children (numbered $1, 2, \ldots, n$) in a circle. During the game, every other child is removed from the circle until there are no children left. In which order will the children be removed?

Input
The only input line has an integer n.

Output
Print n integers: the removal order.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$

Código-fonte:

```
#include <bits/stdc++.h>

using namespace std;

int main(){
    int n;
    vector<int> a, b, resp;
    cin >> n;

    for(int i=0; i<n; i++){
        a.push_back(i+1);
    }

    int i=1;
    while(a.size()>0){
        if(a.size() == 1){
            resp.push_back(a[0]);
            break;
        }
        for(; i<a.size(); i+=2){
            if(i != 0) b.push_back(a[i-1]);
            resp.push_back(a[i]);
        }

        if(i == a.size()){
            b.push_back(a[a.size()-1]);
            i = 0;
        }else{
            i = 1;
        }
        a = b;
        b.clear();
    }

    for(int i=0; i<n; i++){
        cout << resp[i] << ' ';
    }
    cout << '\n';
```

```
39
40      return 0;
41  }
```

16.13 Josephus Problem II

Consider a game where there are n children (numbered $1, 2, \ldots, n$) in a circle. During the game, repeatedly k children are skipped and one child is removed from the circle. In which order will the children be removed?

Input
The only input line has two integers n and k.

Output
Print n integers: the removal order.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $0 \leq k \leq 10^9$

Código-fonte:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  #include <ext/pb_ds/assoc_container.hpp>
5  #include <ext/pb_ds/tree_policy.hpp>
6  using namespace __gnu_pbds;
7
8  #define ordered_set tree<int, null_type,less<int>,
       rb_tree_tag,tree_order_statistics_node_update>
9
10 ordered_set s;
11
12 int main() {
13   int n, k;
14   cin >> n >> k;
15   for(int i = 1; i <= n; i++) s.insert(i);
16   int at = 0;
17   vector<int> v;
18
19   while(s.size() != 0){
20     at = (at + k) % s.size();
21     auto it = s.find_by_order(at);
22     v.push_back(*it);
23     s.erase(it);
24   }
25
26   for(auto x : v) cout << x << ' ';
27   cout << '\n';
28
29 }
```

16.14 Maximum Subarray Sum

Given an array of n integers, your task is to find the maximum sum of values in a contiguous, nonempty subarray.

Input
The first input line has an integer n: the size of the array. The second line has n integers $x_1, x_2, \ldots, x_n$: the array values.

Output
Print one integer: the maximum subarray sum.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $-10^9 \leq x_i \leq 10^9$

Código-fonte:

```
1  #include <bits/stdc++.h>
2  #define pb push_back
3  #define all(x) x.begin(), x.end()
4  #define pc __builtin_popcount
5
6  using namespace std;
7
8  const int maxn = 1010, inf = 2e9, M = 1e9 + 7;
9
10 typedef long long int ll;
11
12 void solve() {
13     int n; cin >> n;
14     ll prefix = 0;
15     ll minimo = 0;
16     ll resp = -inf;
17
18     for(int i = 0; i < n; i ++){
19         int a; cin >> a;
20         prefix += a;
21         resp = max(resp, prefix - minimo);
22         minimo = min(minimo, prefix);
23     }
24
25     cout << resp << '\n';
26 }
27
28 int32_t main() {
29     ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
30     //int t; cin >> t; while (t--)
31     solve();
32     return 0;
33 }
```

16.15 Maximum Subarray Sum II

Given an array of n integers, your task is to find the maximum sum of values in a contiguous subarray with length between a and b.

Input
The first input line has three integers n, a and b: the size of the array and the minimum and maximum subarray length. The second line has n integers $x_1, x_2, \ldots, x_n$: the array values.

Output
Print one integer: the maximum subarray sum.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq a \leq b \leq n$ • $-10^9 \leq x_i \leq 10^9$

Código-fonte:

```
1  #include <bits/stdc++.h>
2  #define pb push_back
3  #define all(x) x.begin(), x.end()
4  #define pc __builtin_popcount
5
6  using namespace std;
7  typedef long long int ll;
8
9  const int maxn = 1010, inf = 2e9, M = 1e9 + 7;
10 const ll linf = 4e18;
11
12
13 void solve() {
14     int n, a, b; cin >> n >> a >> b;
15     vector<ll> prefix(n + 1); prefix[0] = 0;
16
17     for(int i = 1; i <= n; i++) {
18         cin >> prefix[i];
19         prefix[i] += prefix[i - 1];
20     }
21
22     multiset<ll> minimo;
23     ll resp = -linf;
24
25     for(int i = a; i <= n; i++){
26         ll at = prefix[i];
27
28         minimo.insert(prefix[i - a]);
29         if(i > b) minimo.erase(minimo.find(prefix[i - b - 1])
               );
30
31         resp = max(resp, at - *minimo.begin());
32     }
33
34     cout << resp << '\n';
35 }
36
37 int32_t main() {
38     ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
39     //int t; cin >> t; while (t--)
40     solve();
41     return 0;
42 }
```

16.16 Missing Coin Sum

You have n coins with positive integer values. What is the smallest sum you cannot create using a subset of the coins?

Input
The first line has an integer n: the number of coins. The second line has n integers $x_1, x_2, \ldots, x_n$: the value of each coin.

Output
Print one integer: the smallest coin sum.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$

Código-fonte:

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
```

```
4
5  #define all(x) x.begin(), x.end()
6  #define pb push_back
7  #define ll long long int
8  #define int long long
9  const int MOD = 1e9 + 7, MAX = 1e6 + 5, n = 10;
10
11 int32_t main(){
12     ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
13     int n; cin >> n;
14     int p[n];
15     for(int i = 0; i < n; i++){
16         cin >> p[i];
17     }
18     sort(p, p + n);
19
20     int at = 1;
21
22     for(int i = 0; i < n; i++){
23         if(at >= p[i]) at += p[i];
24         else break;
25     }
26
27     cout << at << '\n';
28 }
```

16.17 Movie Festival

In a movie festival n movies will be shown. You know the starting and ending time of each movie. What is the maximum number of movies you can watch entirely?

Input
The first input line has an integer n: the number of movies. After this, there are n lines that describe the movies. Each line has two integers a and b: the starting and ending times of a movie.

Output
Print one integer: the maximum number of movies.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq a < b \leq 10^9$

Código-fonte:

```
1  #include <bits/stdc++.h>
2  #define pb push_back
3  #define all(x) x.begin(), x.end()
4  #define pc __builtin_popcount
5
6  using namespace std;
7
8  typedef long long int ll;
9
10 const int maxn = 1010, inf = 2e9, M = 1e9 + 7;
11
12
13 void solve() {
14     int n; cin >> n;
15     vector<pair<int, int>> v;
16
17     for(int i=0; i<n; i++){
18         int a, b;
19         cin >> a >> b;
20
21         v.pb({b, a});
22     }
23
24     sort(v.begin(), v.end());
25
26     int total = 0, atual = 0;
27
28     for(auto x : v){
29         int ini = x.second, ed = x.first;
30         if(ini >= atual){
31             atual = ed;
32             total++;
33         }
34     }
35
36     cout << total << '\n';
37 }
38
39 int main() {
40     ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
41     //int t; cin >> t; while (t--)
42     solve();
43     return 0;
44 }
```

16.18 Movie Festival II

In a movie festival, n movies will be shown. Syrjälä's movie club consists of k members, who will be all attending the festival. You know the starting and ending time of each movie. What is the maximum total number of movies the club members can watch entirely if they act optimally?

Input
The first input line has two integers n and k: the number of movies and club members. After this, there are n lines that describe the movies. Each line has two integers a and b: the starting and ending time of a movie.

Output
Print one integer: the maximum total number of movies.

Constraints
• $1 \leq k \leq n \leq 2 \cdot 10^5$ • $1 \leq a < b \leq 10^9$

Código-fonte:

```
1  #include <bits/stdc++.h>
2  #define pb push_back
3  #define all(x) x.begin(), x.end()
4  #define pc __builtin_popcount
5
6  using namespace std;
7
8  typedef long long int ll;
9
10 const int maxn = 1010, inf = 2e9, M = 1e9 + 7;
11
12 void solve() {
13     int n, k; cin >> n >> k;
14     vector<pair<int, int>> v;
15
16     for(int i=0; i<n; i++){
17         int a, b;
18         cin >> a >> b;
19
20         v.pb({b, a});
21     }
22
23     sort(v.begin(), v.end());
24
25     int total = 0;
26     multiset<int> atual;
27
28     for(auto x : v){
29         int ini = x.second, ed = x.first;
30
31         if(atual.empty()){
32             atual.insert(ed);
33             total ++;
34             continue;
35         }
36
37         if(ini >= *atual.begin()){
38             auto it = atual.lower_bound(ini);
39             if(it == atual.end() || *it > ini) it = prev(it);
40             atual.erase(it);
41             atual.insert(ed);
42             total++;
43         }
44         else if(atual.size() < k) {
45             atual.insert(ed);
46             total++;
47         }
48     }
49
50     cout << total << '\n';
51 }
52
53 int main() {
54     ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
55     //int t; cin >> t; while (t--)
56     solve();
57     return 0;
58 }
```

16.19 Nearest Smaller Values

Given an array of n integers, your task is to find for each array position the nearest position to its left having a smaller value.

Input
The first input line has an integer n: the size of the array. The second line has n integers $x_1, x_2, \ldots, x_n$: the array values.

Output
Print n integers: for each array position the nearest position with a smaller value. If there is no such position, print 0.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x_i \leq 10^9$

Código-fonte:

```
1  #include <bits/stdc++.h>
2  #define pb push_back
3  #define all(x) x.begin(), x.end()
4  #define pc __builtin_popcount
5
6  using namespace std;
```

```

typedef long long int ll;

const int maxn = 1e5+5, inf = 2e9, M = 1e9 + 7;
const ll linf = 1e18;

bool cmp(pair<int, int> a, pair<int, int> b){
    if(a.first < b.first) return true;
    else if(a.first == b.first){
        return a.second > b.second;
    }
    return false;
}

void solve(){
    int n; cin >> n;
    vector<pair<int, int>> a(n + 1);
    a[0] = {0, 0};
    for(int i = 1; i <= n; i++){
        cin >> a[i].first;
        a[i].second = i;
    }

    sort(a.begin(), a.end(), cmp);

    set<int> s;
    s.insert(0);

    vector<int> res(n + 1);

    for(int i = 1; i <= n; i++){
        auto it = s.lower_bound(a[i].second);
        it--;
        res[a[i].second] = *it;
        s.insert(a[i].second);
    }

    for(int i = 1; i <= n; i++) cout << res[i] << ' ';
    cout << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

16.20 Nested Ranges Check

Given n ranges, your task is to determine for each range if it contains some other range and if some other range contains it. Range $[a, b]$ contains range $[c, d]$ if $a \leq c$ and $d \leq b$.

Input
The first input line has an integer n: the number of ranges. After this, there are n lines that describe the ranges. Each line has two integers x and y: the range is $[x, y]$. You may assume that no range appears more than once in the input.

Output
First print a line that describes for each range (in the input order) if it contains some other range (1) or not (0). Then print a line that describes for each range (in the input order) if some other range contains it (1) or not (0).

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x<y \leq 10^9$

Código-fonte:

```
#include <bits/stdc++.h>

using namespace std;

#define INF 100000000
#define all(x) x.begin(), x.end()
#define pb push_back
#define F first
#define S second

typedef long long ll;
typedef pair<int, int> pii;


#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

#define ordered_set tree<int, null_type,less_equal<int>,
     rb_tree_tag,tree_order_statistics_node_update>

bool comp(pii a, pii b){
    if(a.F < b.F) return 1;
    else if(a.F == b.F){
        return a.S > b.S;
    }
    return 0;
}

void solve() {
    int n; cin >> n;
    pair<int, int> v[n];
    map<pii, int> mapa;
    for(int i = 0; i < n; i++){
        int a, b; cin >> a >> b;
        v[i] = {a, b};
        mapa[v[i]] = i;
    }
    sort(v, v + n, comp);

    ordered_set s1;
    int contido[n];

    for(int i = 0; i < n; i++){
        contido[i] =  i - s1.order_of_key(v[i].second);
        s1.insert(v[i].second);
    }

    ordered_set s2;
    int contem[n];

    for(int i = n - 1; i >= 0; i--){
        contem[i] =  s2.order_of_key(v[i].second + 1);
        s2.insert(v[i].second);
    }

    int transf[n];

    for(int i = 0; i < n; i++){
        transf[mapa[v[i]]] = i;
    }

    for(int i = 0; i < n; i++) contem[transf[i]] == 0 ?  cout
          << 0 << ' ' : cout <<  1 << ' ';
    cout << '\n';
    for(int i = 0; i < n; i++) contido[transf[i]] == 0 ? cout
          << 0 << ' ' : cout << 1 << ' ';
    cout << '\n';

}

int main() {
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    // int t; cin >> t; while(t--)
    solve();

    return 0;
}
```

16.21 Nested Ranges Count

Given n ranges, your task is to count for each range how many other ranges it contains and how many other ranges contain it. Range $[a, b]$ contains range $[c, d]$ if $a \leq c$ and $d \leq b$.

Input
The first input line has an integer n: the number of ranges. After this, there are n lines that describe the ranges. Each line has two integers x and y: the range is $[x, y]$. You may assume that no range appears more than once in the input.

Output
First print a line that describes for each range (in the input order) how many other ranges it contains. Then print a line that describes for each range (in the input order) how many other ranges contain it.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x<y \leq 10^9$

Código-fonte:

```
#include <bits/stdc++.h>

using namespace std;

#define INF 100000000
#define all(x) x.begin(), x.end()
#define pb push_back
#define F first
#define S second

typedef long long ll;
typedef pair<int, int> pii;


#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

#define ordered_set tree<int, null_type,less_equal<int>,
     rb_tree_tag,tree_order_statistics_node_update>

bool comp(pii a, pii b){
    if(a.F < b.F) return 1;
    else if(a.F == b.F){
        return a.S > b.S;
    }
    return 0;
}
```

```cpp
void solve() {
    int n; cin >> n;
    pair<int, int> v[n];
    map<pii, int> mapa;
    for(int i = 0; i < n; i++){
        int a, b; cin >> a >> b;
        v[i] = {a, b};
        mapa[v[i]] = i;
    }
    sort(v, v + n, comp);

    ordered_set s1;
    int contido[n];

    for(int i = 0; i < n; i++){
        contido[i] =  i - s1.order_of_key(v[i].second);
        s1.insert(v[i].second);
    }

    ordered_set s2;
    int contem[n];

    for(int i = n - 1; i >= 0; i--){
        contem[i] =  s2.order_of_key(v[i].second + 1);
        s2.insert(v[i].second);
    }

    int transf[n];

    for(int i = 0; i < n; i++){
        transf[mapa[v[i]]] = i;
    }

    for(int i = 0; i < n; i++) cout << contem[transf[i]] << ' ';
    cout << '\n';
    for(int i = 0; i < n; i++) cout << contido[transf[i]] << ' ';
    cout << '\n';

}

int main() {
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    // int t; cin >> t; while(t--)
    solve();

    return 0;
}
```

16.22 Playlist

You are given a playlist of a radio station since its establishment. The playlist has a total of n songs. What is the longest sequence of successive songs where each song is unique?

Input
The first input line contains an integer n: the number of songs. The next line has n integers $k_1, k_2, \ldots, k_n$: the id number of each song.

Output
Print the length of the longest sequence of unique songs.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le k_i \le 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define _ ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
#define MAXN 200100
#define INF 100000000
#define pb push_back
#define F first
#define S second

using namespace std;
typedef long long ll;
typedef pair<int, int> pii;
const int M = 1e9 + 7;

int main () { _
    int n; cin >> n;
    vector<int> k(n);

    int cnt = 0;
    map<int, int> mp;

    for(int i = 0; i < n; i++){
        int x; cin >> x;
        if(mp.find(x) == mp.end()){
            cnt++;
            mp[x] = cnt;
        }
        k[i] = mp[x];
    }

    // for(int x : k) cout << x << ' '; cout << '\n';

    vector<int> f(cnt+1);
    for(int i=0; i<=cnt; i++){
        f[i] = -1;
    }

    int ini = 0, fim = 0;
    int M = 0;

    while(ini <= n-1 && fim <= n-1){
        if(ini <= f[k[fim]] && f[k[fim]] <= fim){
            ini = f[k[fim]] + 1;
        }
        f[k[fim]] = fim;
        M = max(M, fim - ini + 1);
        fim++;
    }

    cout << M << '\n';

    return 0;
}
```

16.23 Reading Books

There are n books, and Kotivalo and Justiina are going to read them all. For each book, you know the time it takes to read it. They both read each book from beginning to end, and they cannot read a book at the same time. What is the minimum total time required?

Input
The first input line has an integer n: the number of books. The second line has n integers $t_1, t_2, \ldots, t_n$: the time required to read each book.

Output
Print one integer: the minimum total time.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le t_i \le 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1e5+5, inf = 2e9, M = 1e9 + 7;
const ll linf = 1e18;

void solve(){
    int n; cin >> n;
    vector<int> a(n);

    for(int i = 0; i < n; i++){
        cin >> a[i];
    }

    sort(a.begin(), a.end());
    reverse(a.begin(), a.end());

    ll soma = 0;

    // se a soma > 2 * maior, tem como reorganizar para todo mundo ler o mais rapido possivel

    for(int i = 0; i < n; i++){
        soma += a[i];
    }

    if(soma >= 2 * a[0]) cout << soma << '\n';
    else cout << 2 * a[0] << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    // int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

16.24 Restaurant Customers

You are given the arrival and leaving times of n customers in a restaurant. What was the maximum number of customers in the restaurant at any time?

Input
The first input line has an integer n: the number of customers. After this, there are n lines that describe the customers. Each line has two integers a and b: the arrival and leaving times of

a customer. You may assume that all arrival and leaving times are distinct.

Output
Print one integer: the maximum number of customers.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le a < b \le 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1010, inf = 2e9, M = 1e9 + 7;


void solve() {
    int n; cin >> n;
    vector<pair<int, int>> v;

    for(int i=0; i<n; i++){
        int a, b;
        cin >> a >> b;

        v.pb({a, 1});
        v.pb({b, -1});
    }

    sort(v.begin(), v.end());

    int maxi = 0;
    int total = 0;

    for(auto p : v){
        total += p.second;
        maxi = max(maxi, total);
    }

    cout << maxi << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

16.25 Room Alocation

Enunciado não encontrado no site. **Código-fonte:**

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1e5+5, inf = 2e9, M = 1e9 + 7;

void solve(){
    int n, a, b; cin >> n;
    vector<tuple<int, int, int>> v;
    for(int i = 1; i <= n; i++){
        cin >> a >> b;
        v.push_back({a, 0, i});
        v.push_back({b, 1, i});
    }
    sort(all(v));
    queue<int> fila;
    vector<int> result(n + 1);
    int cont = 0;

    for(auto x : v){
        if(get<1>(x) == 0){
            if(fila.empty()){
                cont++;
                result[get<2>(x)] = cont;
            }
            else{
                result[get<2>(x)] = fila.front();
                fila.pop();
            }
        }
        else{
            fila.push(result[get<2>(x)]);
        }
    }

    cout << cont << '\n';
    for(int i = 1; i <=n; i++) cout << result[i] << ' ';
    cout << '\n';

}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

16.26 Stick Lengths

There are n sticks with some lengths. Your task is to modify the sticks so that each stick has the same length. You can either lengthen and shorten each stick. Both operations cost x where x is the difference between the new and original length. What is the minimum total cost?

Input
The first input line contains an integer n: the number of sticks. Then there are n integers: $p_1, p_2, \ldots, p_n$: the lengths of the sticks.

Output
Print one integer: the minimum total cost.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le p_i \le 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>

using namespace std;

#define all(x) x.begin(), x.end()
#define pb push_back
#define ll long long int
#define int long long
const int MOD = 1e9 + 7, MAX = 1e6 + 5, n = 10;

int32_t main(){
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int n; cin >> n;
    if(n == 1){
        cout << "0\n";
        return 0;
    }
    int p[n], soma = 0;
    for(int i = 0; i < n; i++){
        cin >> p[i];
        soma += p[i];
    }
    sort(p, p+n);
    int minimo = soma;
    for(int i = 0; i <= n/2; i++){
        soma -= 2*p[i];
        minimo = min(soma - (n - 2*(i + 1)) * p[i + 1],
              minimo);
    }

    for(int i = n/2 + 1; i < n - 1; i++){
        soma -= 2*p[i];
        minimo = min(soma - (n - 2*(i + 1)) * p[i], minimo);
    }

    cout << minimo << '\n';
}
```

16.27 Subarray Divisibility

Given an array of n integers, your task is to count the number of subarrays where the sum of values is divisible by n.

Input
The first input line has an integer n: the size of the array. The next line has n integers $a_1, a_2, \ldots, a_n$: the contents of the array.

Output
Print one integer: the required number of subarrays.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $-10^9 \le a_i \le 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1010, inf = 2e9, M = 1e9 + 7;
```

```

void solve() {
    int n; cin >> n;
    ll sum = 0;
    vector<ll> v;
    v.pb(sum);

    for(int i = 0; i < n; i++){
        int a; cin >> a;
        a = (a % n + n) % n;
        sum = (sum + a) % n;
        v.pb(sum);
    }

    ll cont = 0;

    map<ll, ll> rep;

    for(auto a : v){
        ll k = rep[a];
        rep[a] ++;
        cont += k;
    }

    cout << cont << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

16.28 Subarray Sums I

Given an array of n positive integers, your task is to count the number of subarrays having sum x.

Input
The first input line has two integers n and x: the size of the array and the target sum x. The next line has n integers $a_1, a_2, \ldots, a_n$: the contents of the array.

Output
Print one integer: the required number of subarrays.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $1 \le x, a_i \le 10^9$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1010, inf = 2e9, M = 1e9 + 7;


void solve() {
    int n, x; cin >> n >> x;
    ll sum = 0;
    set<ll> v;
    v.insert(sum);

    for(int i = 0; i < n; i++){
        int a; cin >> a;
        sum += a;
        v.insert(sum);
    }

    int cont = 0;

    for(auto a : v){
        auto it = v.find(x + a);
        if(it != v.end())
            cont++;
    }

    cout << cont << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

16.29 Subarray Sums II

Given an array of n integers, your task is to count the number of subarrays having sum x.

Input
The first input line has two integers n and x: the size of the array and the target sum x. The next line has n integers $a_1, a_2, \ldots, a_n$: the contents of the array.

Output
Print one integer: the required number of subarrays.

Constraints
• $1 \le n \le 2 \cdot 10^5$ • $-10^9 \le x, a_i \le 10^9$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1010, inf = 2e9, M = 1e9 + 7;


void solve() {
    int n, x; cin >> n >> x;
    ll sum = 0;
    vector<ll> v;
    v.pb(sum);

    for(int i = 0; i < n; i++){
        int a; cin >> a;
        sum += a;
        v.pb(sum);
    }

    ll cont = 0;

    map<ll, ll> rep;

    for(auto a : v){
        ll k = rep[a - x];
        rep[a] ++;
        cont += k;
    }

    cout << cont << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

16.30 Sumof Four Values

You are given an array of n integers, and your task is to find four values (at distinct positions) whose sum is x.

Input
The first input line has two integers n and x: the array size and the target sum. The second line has n integers $a_1, a_2, \ldots, a_n$: the array values.

Output
Print four integers: the positions of the values. If there are several solutions, you may print any of them. If there are no solutions, print IMPOSSIBLE.

Constraints
• $1 \le n \le 1000$ • $1 \le x, a_i \le 10^9$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1010, inf = 2e9, M = 1e9 + 7;

// Sum of Two Values nos pares (i, j) do vetor original
void solve() {
    int n, x; cin >> n >> x;
    vector<pair<int, int>> v;

    for(int i = 0; i < n; i++){
        int a; cin >> a;
        v.pb({a, i});
    }
```

```

    if(n < 4){
        cout << "IMPOSSIBLE\n";
        return;
    }

    vector<pair<int, pair<int, int>>> pares;

    for(int i = 0; i < n; i++){
        for(int j = i + 1; j < n; j++){
            pares.push_back({v[i].first + v[j].first, {i, j
                }});
        }
    }

    sort(pares.begin(), pares.end());

    int fim = pares.size() - 1, ini = 0;
    int soma = 0;

    while(soma != x && fim != ini){
        soma = pares[fim].first + pares[ini].first;
        if(soma < x) ini++;
        else if(soma > x) fim--;
        else{
            set<int> s;
            s.insert(pares[ini].second.first);
            s.insert(pares[ini].second.second);
            s.insert(pares[fim].second.first);
            s.insert(pares[fim].second.second);
            if(s.size() != 4){
                if(pares[ini].first == pares[ini + 1].first){
                    ini ++;
                }
                else{
                    fim --;
                }
                soma = -1;
            }
        }
    }

    if(soma == x) cout << pares[ini].second.first + 1 << ' ' 
        << pares[ini].second.second + 1 << ' ' <<
                        pares[fim].second.first + 1 << ' '
                            << pares[fim].second.second +
                             1 << '\n';
    else cout << "IMPOSSIBLE\n";
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

16.31 Sumof Three Values

You are given an array of n integers, and your task is to find three values (at distinct positions) whose sum is x.

Input
The first input line has two integers n and x: the array size and the target sum. The second line has n integers $a_1, a_2, \ldots, a_n$: the array values.

Output
Print three integers: the positions of the values. If there are several solutions, you may print any of them. If there are no solutions, print IMPOSSIBLE.

Constraints
• $1 \leq n \leq 5000$ • $1 \leq x, a_i \leq 10^9$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1010, inf = 2e9, M = 1e9 + 7;
vector<pair<int, int>> v;

pair<int, int> two_values(int ini, int fim, int x) {
    int soma = 0;

    do{
        soma = v[fim].first + v[ini].first;
        if(soma < x) ini++;
        else if(soma > x) fim--;
    } while(soma != x && fim != ini);

    if(soma == x){
        return {v[ini].second + 1, v[fim].second + 1};
    }

    else return {-1, -1};
}

void solve() {
    int n, x; cin >> n >> x;

    for(int i=0; i<n; i++){
        int a; cin >> a;
        v.pb({a, i});
    }

    if(n < 3){
        cout << "IMPOSSIBLE\n";
        return;
    }

    sort(v.begin(), v.end());

    for(int i = 0; i < n - 2; i++){
        pair<int, int> p = two_values(i + 1, n - 1, x - v[i].
            first);
        if(p.first != -1){
            cout << v[i].second + 1 << ' ' << p.first << ' '
                << p.second << '\n';
            return;
        }
    }

    cout << "IMPOSSIBLE\n";
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

16.32 Sumof Two Values

You are given an array of n integers, and your task is to find two values (at distinct positions) whose sum is x.

Input
The first input line has two integers n and x: the array size and the target sum. The second line has n integers $a_1, a_2, \ldots, a_n$: the array values.

Output
Print two integers: the positions of the values. If there are several solutions, you may print any of them. If there are no solutions, print IMPOSSIBLE.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq x, a_i \leq 10^9$

Código-fonte:

```
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1010, inf = 2e9, M = 1e9 + 7;


void solve() {
    int n, x; cin >> n >> x;
    vector<pair<int, int>> v;

    for(int i=0; i<n; i++){
        int a; cin >> a;
        v.pb({a, i});
    }

    sort(v.begin(), v.end());

    int fim = n-1, ini = 0;
    int soma = 0;

    while(soma != x && fim != ini){
        soma = v[fim].first + v[ini].first;
        if(soma < x) ini++;
        else if(soma > x) fim--;
    }

    if(soma == x) cout << v[ini].second + 1 << ' ' << v[fim].
        second + 1 << '\n';
    else cout << "IMPOSSIBLE\n";
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; while (t--)
    solve();
    return 0;
}
```

16.33 Tasksand Deadlines

You have to process n tasks. Each task has a duration and a deadline, and you will process the tasks in some order one after another. Your reward for a task is $d - f$ where d is its deadline and f is your finishing time. (The starting time is 0, and you have to process all tasks even if a task would yield negative reward.) What is your maximum reward if you act optimally?

Input
The first input line has an integer n: the number of tasks. After this, there are n lines that describe the tasks. Each line has two integers a and d: the duration and deadline of the task.

Output
Print one integer: the maximum reward.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq a, d \leq 10^6$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define pb push_back
#define all(x) x.begin(), x.end()
#define pc __builtin_popcount

using namespace std;

typedef long long int ll;

const int maxn = 1e5+5, inf = 2e9, M = 1e9 + 7;
const ll linf = 1e18;

void solve(){
    int n; cin >> n;
    int d;
    ll soma = 0;
    vector<int> a(n + 1);
    for(int i = 1; i <= n; i++){
        cin >> a[i] >> d;
        soma += d;
    }

    sort(a.begin() + 1, a.end());

    for(int i = 1; i <= n; i++){
        soma -= (ll)a[i] * (n - i + 1);
    }

    cout << soma << '\n';
}

int main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    //int t; cin >> t; for(int i = 1; i <= t; i++)
    solve();
    return 0;
}
```

16.34 Towers

You are given n cubes in a certain order, and your task is to build towers using them. Whenever two cubes are one on top of the other, the upper cube must be smaller than the lower cube. You must process the cubes in the given order. You can always either place the cube on top of an existing tower, or begin a new tower. What is the minimum possible number of towers?

Input
The first input line contains an integer n: the number of cubes. The next line contains n integers $k_1, k_2, \ldots, k_n$: the sizes of the cubes.

Output
Print one integer: the minimum number of towers.

Constraints
• $1 \leq n \leq 2 \cdot 10^5$ • $1 \leq k_i \leq 10^9$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define _ ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
#define MAXN 200100
#define INF 100000000
#define pb push_back
#define F first
#define S second

using namespace std;
typedef long long ll;
typedef pair<int, int> pii;
const int M = 1e9 + 7;

int main () { _
    int n; cin >> n;

    multiset<int> s;

    for(int i = 0; i < n; i++){
        int x; cin >> x;
        if(s.upper_bound(x) == s.end()) s.insert(x);
        else{
            s.erase(s.upper_bound(x));
            s.insert(x);
        }
    }
        cout << s.size() << '\n';

    return 0;
}
```

16.35 Traffic Lights

There is a street of length x whose positions are numbered $0, 1, \ldots, x$. Initially there are no traffic lights, but n sets of traffic lights are added to the street one after another. Your task is to calculate the length of the longest passage without traffic lights after each addition.

Input
The first input line contains two integers x and n: the length of the street and the number of sets of traffic lights. Then, the next line contains n integers $p_1, p_2, \ldots, p_n$: the position of each set of traffic lights. Each position is distinct.

Output
Print the length of the longest passage without traffic lights after each addition.

Constraints
• $1 \leq x \leq 10^9$ • $1 \leq n \leq 2 \cdot 10^5$ • $0<p_i<x$

Código-fonte:

```cpp
#include <bits/stdc++.h>
#define _ ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
#define MAXN 200100
#define INF 100000000
#define pb push_back
#define F first
#define S second

using namespace std;
typedef long long ll;
typedef pair<int, int> pii;
const int M = 1e9 + 7;

int main () { _
    int x, n; cin >> x >> n;

    set<int> p; p.insert(0); p.insert(x);
    multiset<int> dist; dist.insert(x);

    for(int i = 0; i < n; i++){
        int y; cin >> y;
        auto it = p.upper_bound(y);
        int M = *it;
        it --;
        int m = *it;


        dist.erase(dist.lower_bound(M-m));

        dist.insert(M-y);
        dist.insert(y-m);

        p.insert(y);
        // for(auto x : dist) cout << x << ' ';
        // cout << '\n';
        cout << *dist.crbegin() << '\n';
    }

    return 0;
}
```